Linear Algebra for Computer Science



Essential Mathematics for Computer and Data Scientists

Manoj Thulasidas

Buy it
Scan or Tap

Linear Algebra for Computer Science



Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Linear Algebra for Computer Science

Essential Mathematics for Computer Scientists

Manoj Thulasidas

ASIAN BOOKS
Singapore

ASIAN BOOKS

Manoj Thulasidas
Associate Professor of Computer Science (Education)
80 Stamford Road, School of Computing and Information Systems,
Singapore Management University, Singapore 178902

https://www.thulasidas.com
https://smu.sg/manoj
https://LA4CS.com

Copyright © Manoj Thulasidas, 2021.

The notices on page 305 constitute an extension of this copyright page.

All rights reserved. No part of this Book may be reproduced or transmitted in any form, or by any means (electronic or mechanical, including photocopying, recording or taping on information storage and retrieval systems), without the prior written permission of the copyright owner.

ISBN: 978-981-18-2045-8 (ebook)



Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Published in Singapore

"You can't learn too much linear algebra."

—Prof. Benedict Gross, Harvard

"Matrices act. They don't just sit there."

—Prof. Gilbert Strang, MIT

Contents

Acknowledgments 1

Preface 2

Introduction 4

I.1 Why Learn Linear Algebra? 5

I.2 Learning Objectives and Competencies 6

I.3 Organization 7

Part I Numerical Computations

1. Expressions, Functions, Equations and Linearity 11

1.1 Variables and Expressions 11

1.2 Equations and Equality 14

1.3 Multivariable Functions 15

1.4 Vectors and Matrices 16

1.5 The Big Picture 19

2. Vectors and Matrices 21

2.1 Vectors 21

2.2 Advanced Vector Concepts 29

2.3 Matrices 35
2.4 Matrix Multiplication 38

3. Transposes and Determinants 48
3.1 Transpose of a Matrix 48
3.2 Definitions and Matrices with Special Properties 53
3.3 Determinant of a Matrix 56
3.4 Properties of Determinants 63
3.5 Summary: Numerical Computations 65

Part II Algebraic View

4. Systems of Linear Equations 68
4.1 When Can We Solve Linear Equations? 68
4.2 Gaussian Elimination 75
4.3 Applications of Gaussian Elimination 79
4.4 More Examples 92
4.5 Gaussian Elimination: A Foundational Algorithm 94

5. Ranks and Inverses of Matrices 97
5.1 The Concept of Rank 98
5.2 Gauss-Jordan Elimination and Canonical Forms 101
5.3 Matrix Inverses and Invertibility 107
5.4 Computational Methods for Inverses and Systems 110
5.5 Summary: Algebraic View 115

Part III Geometric View

6. Vector Spaces, Bases and Dimensions 117
6.1 Linear Combinations 117
6.2 Vector Spaces and Subspaces 124
6.3 Bases and Dimensions 127
6.4 Geometry of Linear Equations 130

7. Bases, Orthogonality, and the Gram-Schmidt Process 137
7.1 Bases and Components 138
7.2 Orthogonality and Projections 144
7.3 The Gram-Schmidt Process 148

8. Review and Recap 158
8.1 Formalizing the Abstraction: Vector Spaces 158
8.2 The Two Geometries: Row and Column Pictures 160
8.3 Algebraic Properties: Transposes and Inverses 163
8.4 Algorithms and Matrix Shapes 164

9. The Four Fundamental Spaces 170
9.1 Column Space 171
9.2 Null Space 174
9.3 Row Space 178
9.4 Left Null Space 181
9.5 Synthesis and Review 182

10. Projection, Least Squares and Linear Regression 191
10.1 Projection Revisited 192
10.2 Projection to Subspace 194
10.3 Linear Regression 199
10.4 Summary: Geometric View 204

Part IV Advanced Topics

11. Eigenvalue Decomposition and Diagonalization 207
11.1 Fundamentals and Computation 208
11.2 Mathematical Properties and Geometry 216
11.3 Diagonalization and Matrix Powers 223
Appendix: Complex Numbers and Eigenvalue Properties 229

R. Recap: The Story So Far 233

12. Special Matrices, Similarity, and Algorithms 245
12.1 Symmetric and Hermitian Matrices 245
12.2 Positive Definite and Gram Matrices 250
12.3 Markov Matrices and State Transitions 258
12.4 Matrix Similarity and Canonical Forms 262
12.5 Computational Algorithms 270

13. Singular Value Decomposition 276
13.1 The Mechanics of SVD: Intuition and Computation 277
13.2 Applications: Low-Rank Approximation and Data Compression 289
13.3 Theoretical Connections: Inverses and Subspaces 295

Summing Up... 300

Glossary 302

Credits 305

List of Figures

2.1	Example of Scalar Multiplication	24
2.2	Addition of Vectors	26
2.3	Two Linearly Independent Vectors	28
2.4	Two Linearly Dependent Vectors	30
2.5	Dot Product and the Angle Between Vectors	33
2.6	Matrix Multiplication: Illustrated	40
2.7	Column and Row Pictures of Matrix Multiplication	45
3.1	How a Matrix Transforms Unit Vectors	58
3.2	Determinant and Area: Proof Without Words	59
3.3	Determinants as Positive and Negative Areas	59
3.4	Minors and Cofactors	62
4.1	Visualizing Equations	71
4.2	Gaussian Elimination on a Simple Matrix	80
4.3	Gaussian Elimination: Elementary Matrix	87
5.1	Gauss-Jordan Elimination	102
6.1	Two Independent Vectors in $\mathbb{R}^2$	118
6.2	Two Dependent Vectors in $\mathbb{R}^2$	119
6.3	Two Vectors on the xy plane in $\mathbb{R}^3$	120

6.4	Two General Vectors in $\mathbb{R}^3$	120
6.5	Geometric View of Solvable Equations	131
6.6	Geometric View of Two Inconsistent Equations	133
6.7	Geometric View of Three Inconsistent Equations	134
7.1	Visualization of Change of Basis	142
7.2	Rotation Matrix in $\mathbb{R}^2$	147
7.3	Dot Product as Projection	150
7.4	The Gram-Schmidt Process	152
8.1	Matrix Shapes, RREF and Solvability	168
9.1	The Four Fundamental Subspaces	183
10.1	Projection of a Vector onto Another	192
10.2	Projection to a Subspace	194
10.3	Example of Simple Linear Regression	200
10.4	Multiple Linear Regression: Data Matrix	202
10.5	Multiple Linear Regression: Notations	203
10.6	Multiple Linear Regression: Visualization	204
11.1	The Shear Matrix	211
11.2	Eigenanalysis: Visualization	222
R.1	Recap: Four Fundamental Spaces	234
R.2	Recap: Full-Rank Square Matrix	236
R.3	Recap: Full-Column-Rank Tall Matrix	237
R.4	Recap: Full-Row-Rank Wide Matrix	239
R.5	Recap: General Matrix	243
13.1	SVD Input Matrix	278
13.2	SVD interpretation: $\boldsymbol{V}^\mathsf{T}$	279
13.3	SVD interpretation: $\boldsymbol{\Sigma}$	280
13.4	SVD interpretation: $\boldsymbol{U}$ and summary	281
13.5	SVD interpretation: summary	281
13.6	Shapes of $\hat{\boldsymbol{U}}, \hat{\boldsymbol{\Sigma}}$ and $\boldsymbol{V}$ in the SVD of a full-column-rank $\boldsymbol{A}$.	285
13.7	Shapes of $\boldsymbol{U}, \boldsymbol{\Sigma}$ and $\boldsymbol{V}$ in the SVD of a general $\boldsymbol{A}$.	286
13.8	Examples of PCA	294
13.9	Pseudo-Inverse and the Elegant Symmetry of Linear Algebra	299

List of Tables

4.1	Various permutations of simultaneous equations in two variables, showing solvability	69
4.2	Properties and behavior of linear equations and solutions	73
4.3	Illustration of solvability conditions based on the characteristics of REF	81
4.4	$\boldsymbol{A = LU}$ Decomposition	87
4.5	$\boldsymbol{A = PLU}$ Decomposition	89
4.6	Example 1: Gaussian elimination and solvability: Unique solution	92
4.7	Example 2: Gaussian elimination and solvability: No solutions	93
4.8	Example 3: More equations than unknowns, but solvable	94
4.9	Example 4: More equations than unknowns, with no solutions	95
4.10	Example 5: Gaussian elimination and solvability: Infinity of solutions	95
5.1	Gauss-Jordan on a full-rank, square coefficient matrix, giving us the unique solution	104

5.2 Gauss-Jordan on a rank-deficient coefficient matrix, resulting in an infinity of solutions 105

5.3 Gauss-Jordan on a rank-deficient "wide" coefficient matrix 106

7.1 Examples of change of basis. 142

8.1 Named algorithms and their uses 165

9.1 Summary of the four fundamental spaces 184

9.2 The fundamental spaces of matrices of different shapes 185

11.1 Fibonacci numbers (f_k) vs. the eigenvalue approximation ($f_k^{(\text{approx})}$) 229

12.1 Migration probabilities 261

12.2 Jordan Normal Forms $\boldsymbol{J}$ of $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$ with $\boldsymbol{A}\boldsymbol{s_i} = \lambda_i \boldsymbol{s_i}$ 268

12.3 Multiplicities of eigenvalues from Jordan blocks 270

Acknowledgments

This book is the final metamorphosis of a collection of lecture notes, written specifically to help my students in *Linear Algebra for Machine Learning* at Singapore Management University. It would not have been possible without the opportunity to teach that course and the encouragement of my friends, colleagues, and students. I am deeply grateful for their support, and owe a special thanks to those who generously proofread the drafts and suggested edits.

M.T.

Preface

This book has its origin in my experience teaching linear algebra to Computer Science students at Singapore Management University. Traditionally, Linear Algebra is taught as a pure mathematics course, almost as an afterthought, not fully integrated with any other applied curriculum. That certainly was how it was taught to me. The course I was teaching, however, had a definite pedagogical objective of bringing out the applicability and usefulness of linear algebra in Computer Science, which is essentially applied mathematics. In the era of machine learning and artificial intelligence, linear algebra is the branch of mathematics that holds the most relevance to computing.

One question I got from one of my brightest students the first time I taught the course was why they were forced to learn this particular branch of mathematics. It was not a defiant or rebellious question, but one of pure curiosity. I did not have a ready answer then, but I think I have one now. When we embark on any profession, we have to start with the tools of the trade. For instance, if we want to be a musician, we have to learn the notes before we can perform. If we want to be a writer, we need to know the vocabulary and grammar of our chosen language first. Similarly, in order to be a computer scientist, we have to have the necessary mathematical skills. And linear algebra is arguably the most critical expertise needed, especially when it comes to dealing with large quantities of data efficiently.

Linear algebra is a well-established field, and we can find several resources freely available on the Internet. In writing this book, I have made use of many of them. In fact, the whole process of writing can be thought of as an exercise in curating the right pieces of information at the right level, and standardizing them with consistent notations to form a coherent and fluid narrative.

Mathematics books often tend to be dry, unreadable collections of facts, theorems, proofs and problems. This type of discourse is understandable, given the nature of the subject that demands accuracy and completeness, often at the expense of readability. My objective was to write a book that would be read. For this reason, I set practicality as my goal, and even used, dare I say this, humor to keep my reader engaged.

I employed two more tricks to improve the readability. The first one is to restrict the field (over which our vectors and matrices are defined) to real numbers ($\mathbb{R}$) because of its relevance to computer science. The second trick is to pepper the text with "boxes," which are curious applications, background information, or other tidbits that are topic-adjacent to the matter under discussion.

From experience, I know that a book of this length takes about a year to write and another year to polish and publish. The first version of this book was done and dusted in about three months, almost ten times faster than normal. For this reason, it is continuously updated, corrected and improved upon over the last couple of years. How my students respond to the course on which the book is based will also inspire further revisions. So will the advent of new technologies (Large Language Models, for instance) that directly use Linear Algebra. These new versions and/or editions are made available periodically.

MANOJ THULASIDAS

Singapore

October 21, 2026

Introduction

I cannot teach anybody anything, I can only make them think.

—Socrates



This book is for an introductory course in linear algebra. It teaches the mathematical foundations of the subject to illustrate their relevance to computer science and its applications. It also prepares students for advanced numerical methods in computing, especially in machine learning and data analytics. Designed for a first course, this book covers basic concepts and techniques, providing an appreciation of the wide application of this discipline within the field of computer science.

The book requires the development of some theoretical results, with proofs and consequences employing some level of mathematical rigor, algebraic manipulation, geometry, and numerical algorithms. However, the main focus will be on the computational aspects and the applications of linear algebra.

Regardless of how application-oriented we would like to make it, linear algebra is a branch of pure mathematics. In a textbook written for a first course introducing it, we will have a hard time staying away from the purely theoretical. However, linear algebra touches upon so many aspects of the technologies that enhance our modern life that we may discover interesting connections and applications even from the most basic of its concepts. This book will bring out such connections wherever possible, thereby attempting to be an intellectual treat to its readers.

I.1 Why Learn Linear Algebra?

From a purely mathematical perspective, linear algebra is a branch that leads us to *mathematical maturity*. It allows us to appreciate the elegant interconnections among various areas of mathematics, such as algebra (solving equations), geometry (vector spaces), and the mathematical intuition behind advanced algorithms.

A few major branches of mathematics are highly relevant to scientific computing, numerical methods, and technology in general. Crucial among them are calculus (particularly multivariable calculus) and linear algebra. Roughly speaking, calculus corresponds to the analog world—concerned with continuous variables, their evolution, and their effects on one another. It plays an enormously important role in all branches of engineering and the physical sciences. However, the modern world is fundamentally digital. And the mathematics of digital technology, at a high level, is linear algebra.

For computer science specifically, discrete mathematics is another highly relevant branch. It is a highly engaging field, full of puzzles and brain-teasers, encompassing number theory, game theory, and other fascinating concepts.

Equally relevant to all fields involving data and number crunching (ranging from the social sciences and medicine to data science) are statistics and probability. Later in this book, we will see some statistical quantities (covariance and its principal components, for instance) and even certain operations quintessentially associated with calculus (like error minimization in linear regression, viewed as a projection operation) elegantly emerging from linear algebra operations.

While we can list a number of impressive applications of linear algebra in computer science (such as the PageRank algorithm that made Google what it is, along with various image and data compression techniques), the true role of this branch of mathematics is akin to the alphabet, vocabulary, and grammar in learning a language. If we want to be a writer, we must master all these structural aspects of our chosen language. Having these background skills alone will not automatically make us a writer, but *without* them, we will *never* be a good one.

Linear algebra, in much the same way, provides the essential foundation and backdrop for several pivotal numerical algorithms in computer science.

I.2 Learning Objectives and Competencies

The learning objectives of the book include imparting the knowledge and foundational concepts of linear algebra, as well as developing a set of skills and an appreciation for its application in computing. Upon the successful completion of a course based on this book, students should have a sound understanding of the following:

- The concept of linearity as it applies to expressions, functions, and transformations.
- The notion of vectors and matrices, and their operations.
- Important characteristics of a matrix, such as its four fundamental subspaces, rank, determinant, eigenvalues and eigenvectors, and various factorizations.
- How to use the characteristics of a matrix to solve a linear system of equations using algorithms such as Gaussian and Gauss-Jordan elimination.
- Important concepts of vector spaces such as linear independence, bases, dimensions, orthogonality, and the Gram-Schmidt process for orthonormalization.
- Properties of special categories of matrices, such as symmetric and positive definite matrices.
- Eigenvalue, singular value, and other decompositions, alongside their applications.

- Several algorithms rooted in linear algebra, such as linear regression, the QR algorithm and power iteration for eigenanalysis, and Cholesky decomposition.

The learning objectives listed above translate to the following learning outcomes or competencies for students. Upon the successful completion of a course based on this book, students should be able to perform the following:

1. Determine the existence and uniqueness of the solution to a linear system, and find its complete solution by choosing an effective method such as Gaussian elimination, factorization, or diagonalization.
2. Test for the linear independence of vectors, and the orthogonality of vectors and vector spaces.
3. Compute the rank, determinant, inverse, Gram-Schmidt orthogonalization, and various factorizations of a matrix.
4. Visualize the four fundamental subspaces of a matrix, identify their relation to systems of linear equations, and find their dimensions and bases.
5. Identify special properties of a matrix, such as symmetry and positive definiteness, and use this information to facilitate the calculation of matrix characteristics.
6. Describe the use of mathematical techniques from linear algebra as applied to computational problems.
7. Compute the eigenvalues and eigenvectors of a matrix, and use them for diagonalizing matrices, computing matrix powers, and solving advanced problems. *[Optional Topic]*
8. Perform diagonalization and the singular value decomposition of a matrix, and identify its principal components. *[Optional Topic]*

I.3 Organization

The book is organized into four parts, as listed below. For courses based on this book, we recommend following this topical flow, although instructors may shuffle certain sub-topics for clarity and ease of understanding.

Part I: Numerical Computations

In the first part, we cover the basics of linear algebra as it is traditionally taught in an undergraduate curriculum. Here, we work with vectors and matrices represented as numeric arrays, exploring their fundamental operations.

1. Functions, Equations, and Linearity
2. Vectors, Matrices, and Operations
3. Transposes and Determinants

Part II: Algebraic View

Building on basic operations, we transition to viewing matrices and vectors as encodings of linear equations, exploring algorithmic ways to solve them.

4. Gaussian Elimination
5. Ranks and Inverses of Matrices

Part III: Geometric View

In the third part, we explore the elegant geometry that arises from vectors and matrices, focusing on the vector spaces they define, alongside their properties and operations.

6. Vector Spaces, Bases, and Dimensions
7. Change of Basis, Orthogonality, and Gram-Schmidt
8. Review and Recap
9. The Four Fundamental Spaces
10. Projection, Least Squares, and Linear Regression

Part IV: Advanced Topics

In the final part, we discuss advanced operations on matrices, specifically eigenvalue and singular value decompositions, highlighting their significance and practical applications.

11. Eigenvalue Decomposition and Diagonalization
12. Special Matrices, Similarity, and Algorithms

13. Singular Value Decomposition

Part IV of the book, **Advanced Topics**, although significant as the basis of many computational algorithms, may be too difficult for an introductory undergraduate course in linear algebra. It may be revisited later on in the curriculum.

Although each part emphasizes a specific perspective as declared in its title, linear algebra resists strict compartmentalization into purely numeric, algebraic, or geometric domains. We will naturally encounter overlapping concepts throughout.

Finally, the text integrates computational resources to illustrate practical applications. We will see that most of the numeric calculations that used to be performed by hand are now carried out using mathematical programs such as `Matlab` or **`SageMath`**.

I.3.1 Our Focus

The value of learning linear algebra does not lie in developing arithmetic dexterity for numeric computations, as these are already neatly implemented in tools like **`SageMath`**. Even complex symbolic manipulations are readily handled by **`SageMath`** and other software such as Mathematica. What we focus on, therefore, are the aspects of linear algebra that improve our intuitive insights and conceptual understanding. Our hope is that these insights will help us become better computer and data scientists. For this reason, the exercises following each chapter summary differ significantly from those typically found in standard linear algebra textbooks; they are primarily objective-type questions designed to test our grasp of the theoretical, conceptual, and intuitive foundations.

Part I

Numerical Computations

1

Expressions, Functions, Equations and Linearity

> It is a monstrous thing to force a child to learn Latin or Greek or mathematics on the ground that they are an indispensable gymnastic for the mental powers. It would be monstrous even if it were true.
>
> —George Bernard Shaw



Formally, linear algebra deals with objects (mathematical entities) that transform in a *linear* fashion. After all, it has the word "Linear" in its name. Let's define linearity, along with what "transform" means. In order to do that, however, we have to take a step back and look at some of the foundations.

1.1 Variables and Expressions

When we combine one or more mathematical *variables* in a valid way (using operations like addition, multiplication, exponentiation *etc.*, or using *functions* like sine, logarithm etc.), we get a mathematical *expression*. Here are some examples of expressions: $3x, \sin x, x+y, \boldsymbol{A}\boldsymbol{x}, \ln x$. Fundamentally, it is expressions (which are a combination of variables) that can be linear or non-linear.

A *variable* is a symbol (like a, x, $\boldsymbol{A}$ etc.) that is a placeholder or a container for a *value*. Note that the value can be a single number, or a complicated one like a vector or a matrix, although we have not defined them yet. For now, however, we are thinking of the variables as placeholders of single, real values. For a variable x, we will state that it is real using this mathematical jargon: $x \in \mathbb{R}$ where $\mathbb{R}$ represents the set of all real numbers and $\in$ says, "is a member of."

When we have an *expression*, we can also think of it as a relationship between its value and the set of variables it contains, which is what we mean by a *function*. For example, $4x + y$ is an expression; $f(x, y) = 4x + y$ is a function. We usually write a function of a single variable as $f(x) =$ an expression in x. Some examples of single-variable functions are: $f(x) = 3x, f(x) = e^{-x^2}$. We require the functions to be *single-valued*.

In other words, we can think of an expression as *defining* or *specifying* a *function*, with the additional constraint that one set of inputs (corresponding to the variables in the expression) gives one unique output (which is the value of the expression). For example, the relationship $y^2 = x$ (or $f(x) = \pm\sqrt{x}$) is *not* a function by our definition because a single positive input for x yields two possible values for $f(x)$. However, explicitly choosing the principal root, $f(x) = \sqrt{x}$, satisfies our single-valued requirement.

We can think of a function as a *transformation*: A mathematical object that transforms the inputs to the outputs. Although we are thinking of the inputs (x, y etc.) and the output ($f(x, y)$) as numeric values, they can be other mathematical entities, like vectors and matrices (to be defined). Again, for now, let's limit ourselves to real values of the inputs and outputs for our discussion.

Summarizing:

- We have a variable that contains a value. *e.g.*, x, y, a, b.
- When we combine variables using mathematical operations, we get an expression. *e.g.*, $3x, x + y, ax + by, x^y$.
- Label an expression and we get a function, as long as the expression is single valued. *e.g.*, $f(x) = 3x, f(x, y) = ax+by$.
- A function *transforms* its inputs to the output.

1.1.1 Linearity of Expressions

Definition: An expression of a single, real value ($x \in \mathbb{R}$) is *linear* if it satisfies the following two conditions:

- **Homogeneity**: When the value of x is multiplied by a real number, the value of the expression also gets multiplied by the same number.
- **Additivity**: The value of the expression for the sum of two variables is the sum of the corresponding values of the expression.

It is easier to state the conditions of linearity using the definitions of functions or transformations.

1.1.2 Linearity of Functions

Definition: We call a transformation (or a function) of a single, real value ($x \in \mathbb{R}$) *linear* if it satisfies the following two conditions:

- **Homogeneity**: When the value of x is multiplied by a real number, the value of the function also gets multiplied by the same number.

$$f(sx) = sf(x) \qquad \forall\, s, x \in \mathbb{R}$$

- **Additivity**: The output of the function for the sum of two inputs is the sum of the corresponding outputs.

$$f(x + x') = f(x) + f(x') \qquad \forall\, x, x' \in \mathbb{R}$$

As we can see, linearity is a property of the expression itself. When a function is defined by such an expression, the function's linearity is directly determined by the expression. In other words, when we write $f(x) =$ **Expression**, if the **Expression** is linear, then the function $f(x)$ is linear.

Here are some examples of linear functions: $f(x) = 3x, f(x) = 0$. Some functions that are not linear would be: $f(x) = x^2, f(x) = 5$. Note that $f(x) = mx + c$ is *not* linear if $c \neq 0$ because it does not satisfy the linearity conditions listed above. This fact may look weird to us because we know that $y = mx + c$ is the equation to a line. We will take a closer look at it very soon.

1.2 Equations and Equality

In order to understand why $f(x) = mx + c$ is *not* linear, we have to understand what an *equation* is, and, even more fundamentally, what equality (or the equal sign, $=$) means. When we write $f(x) = mx+c$, we are *defining* a function. The equal sign ($=$) in $f(x) = mx + c$ is one of assignment: We are *assigning* the symbol f to the expression of one variable $mx + c$, which will feel highly intuitive to anyone with exposure to programming languages. This function, indeed, is not a linear function according to our definition of linearity because the expression $mx + c$ is not linear.

When we write $y = mx + c$, on the other hand, we do not mean an assignment or a definition. It is a statement of truth, or a recipe for computing any y given a value for x. It is an *equation*. In fact, it is an equation in *two* variables, $x, y \in \mathbb{R}$. In other words, the equal sign ($=$) in $y = mx + c$ is an assertion of truth or a condition. We call such assertions of truth equations. In the context of linear algebra, our preferred form of equations is: expression $=$ constant, or function (which is a placeholder for an expression) $=$ constant. Therefore, we will write $y = mx + c$ as $-mx + y = c$.

CS Connection: Overloading the Equals Sign


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

To summarize, equations in one variable have the form $f(x) = b$ (where $x, b \in \mathbb{R}$ in our context). If the function $f(x)$ is linear (meaning its underlying expression is linear), we consider the equation $f(x) = b$ to be linear as well. In fact, in the case of expressions (or functions) of one variable, we know a bit more than that because we have defined what we mean by linearity: The only possible form $f(x)$ can take so that it is linear is ax. So the most general linear equation of one variable is simply $ax = b$.

How do we define linearity for two (or more) variables? In particular, is $y = mx+c$ a linear *equation*? To align this with our preferred form, we can rearrange $y = mx + c$ to $-mx + y = c$. We know that $mx + c$ is *not* a linear *expression*, and $f(x) = mx + c$ is *not* a linear

function. However, $-mx + y$ is a function of *two* variables, $f(x, y)$. In order to determine whether it is linear, the linearity conditions listed earlier (homogeneity and additivity) need to be generalized for multiple variables.

1.3 Multivariable Functions

Let's generalize the homogeneity property to two variables. For the one variable case, it was $f(sx) = sf(x) \,\forall\, s, x \in \mathbb{R}$. For a two-variable function $f(x, y)$, let's say that homogeneity is satisfied when *both* variables are scaled simultaneously by the *same* scaling factor (or scalar).

$$f(sx, sy) = sf(x, y) \quad \forall\, s, x, y \in \mathbb{R}$$

For one variable, the additivity property was $f(x + x') = f(x) + f(x') \,\forall\, x, x' \in \mathbb{R}$. For a two-variable function, we generalize this by adding the inputs component-wise:

$$f(x + x', y + y') = f(x, y) + f(x', y') \quad \forall\, x, y, x', y' \in \mathbb{R}$$

If we define a new mathematical entity, consisting of two numbers (elements) in a specified order, for which scaling is defined as scaling each element, and addition is defined as the addition of the corresponding elements, we could reuse our original (one-variable) linearity conditions for the two-variable case as well, or indeed for the n-variable case. This mathematical entity (a group of numbers in a specified order, akin to a 1D array or an ordered tuple) is a **vector**. Let's define it as a column of numbers, with scaling and addition operations.

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \quad s\boldsymbol{x} = \begin{bmatrix} sx \\ sy \end{bmatrix}, \quad \boldsymbol{x'} = \begin{bmatrix} x' \\ y' \end{bmatrix}, \quad \boldsymbol{x} + \boldsymbol{x'} = \begin{bmatrix} x + x' \\ y + y' \end{bmatrix}$$

Our vector $\boldsymbol{x}$ has two real numbers in it, and we write $\boldsymbol{x} \in \mathbb{R}^2$. We can easily generalize the vectors such as $\boldsymbol{x}$ to n dimensions. All we have to do is write its n elements (say, $x_1, x_2, \cdots, x_n$) in a column. We would then write $\boldsymbol{x} \in \mathbb{R}^n$. With these definitions and generalization, we can restate the two linearity conditions as follows:

- **Homogeneity**: $f(s\boldsymbol{x}) = sf(\boldsymbol{x}) \quad \forall\, s \in \mathbb{R}, \boldsymbol{x} \in \mathbb{R}^n$
- **Additivity**: $f(\boldsymbol{x} + \boldsymbol{x'}) = f(\boldsymbol{x}) + f(\boldsymbol{x'}) \quad \forall\, \boldsymbol{x}, \boldsymbol{x'} \in \mathbb{R}^n$

For the sake of completeness, let's verify whether $f(x, y) = -mx + y$ is indeed linear. If we scale x and y by s, the value of the function gets scaled by the same factor:

$$f(sx, sy) = -msx + sy = s(-mx + y) = sf(x, y)$$

Therefore, the homogeneity condition is satisfied. For the additivity condition:

$$\begin{aligned} f(x + x', y + y') &= -m(x + x') + y + y' \\ &= (-mx + y) + (-mx' + y') \\ &= f(x, y) + f(x', y') \end{aligned}$$

Therefore, the second, additivity, condition also is satisfied, and $f(x, y) = -mx + y$ is indeed linear. And the *equation* $-mx + y = c$ is linear as well. To be clear, earlier we said $mx + c$, as an expression in (or $f(x) = mx + c$ as a function of) one variable is *not* linear, which is not in contradiction with the statement that $f(x, y) = -mx + y$ *is* linear as a function of (or $-mx + y$ as an expression in) *two variables*. And $-mx + y = c$ is a linear equation in two variables.

Note that linearity, at its core, is a property of mathematical *expressions*. We extend this concept to other mathematical constructs as follows:

- **Linear Functions:** A function is linear if the expression defining its output is linear.
- **Linear Equations:** An equation is linear if it can be arranged (using only addition, subtraction, and scalar multiplication) into the form of a linear expression equated to a constant. If every algebraic term it contains is of the first degree (or a constant), through basic addition and subtraction, these can always be gathered into a linear expression equated to a constant.

1.4 Vectors and Matrices

We stated earlier that the most general linear equation of one variable was $ax = b$. We start with the linear equation $-mx + y = c$. What is the most general linear equation (or system of linear equations) in two or n dimensions?

Let's start by defining a multiplication operation between two vectors: The product of two vectors is the sum of the products of their corresponding elements (conceptually similar to a "zip and sum" or a multiply-accumulate operation in programming). With this definition, and a notational trick of writing the first of the two vectors horizontally, we write:

$$\begin{bmatrix} a_1 & a_2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = a_1 x + a_2 y \implies \begin{bmatrix} -m & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = -mx + y$$

We use this horizontal trick to lay the groundwork for a generalized definition of matrix multiplication, of which this vector product will soon become a special case. We will explore this machinery in much more detail in the next chapter.

1.4.1 Systems of Linear Equations

Our linear equation $-mx+y = c$ and the more general $a_1x+a_2y = b$ then become:

$$\begin{bmatrix} -m & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = c \qquad \text{and} \qquad \begin{bmatrix} a_1 & a_2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = b$$

If we had one more equation, $a_{21}x + a_{22}y = b_2$, we could simply add another row in the compact notation above. In fact, this notation is capable of handling as many equations as we like. For instance, if we had a system of three equations:

1. $a_{11}x + a_{12}y = b_1$
2. $a_{21}x + a_{22}y = b_2$
3. $a_{31}x + a_{32}y = b_3$

We could write this system of three equations much more compactly as:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \quad \text{or} \quad \boldsymbol{Ax} = \boldsymbol{b} \tag{1.1}$$

Here, the 2D table of numbers we called $\boldsymbol{A}$ is a **matrix**. What we have written down as $\boldsymbol{Ax} = \boldsymbol{b}$ is a system of three linear equations (in 2 dimensions, but readily extended to n dimensions).

1.4.2 Linear Transformations

Now that we have gotten a glimpse of vectors and matrices, and defined what linearity means in the context of linear algebra, let's quickly summarize the discussion using the concept of linear transformations.

A function $f(x)$ can be thought of as a transformation or a mapping. For our purposes, there is very little difference between the terms *function*, *transformation*, and *mapping*. When we write $y = f(x)$, we are saying that we can take any input $x \in \mathbb{R}$ and pass it through f to obtain a unique output $y \in \mathbb{R}$. Here, f is a mapping from $\mathbb{R}$ to $\mathbb{R}$. Of all possible functions $f(x)$, the simplest one is a linear transformation, $f(x) = ax$. In fact, it is the *only* form of linear transformation in one dimension—the only one that strictly respects linearity as we previously defined it:

$$f(x) = ax, \quad x \in \mathbb{R}, \quad f : \mathbb{R} \to \mathbb{R}$$

Consequently, the most general form of a linear equation (following our *expression* = *constant* convention) in one dimension is:

$$ax = b, \quad a, b \in \mathbb{R}$$

What is the simplest possible transformation mapping a multi-dimensional space $\mathbb{R}^n$ to a *different* multi-dimensional space $\mathbb{R}^m$? In computer science terms, what is the simplest mathematical function that takes an array of size n and returns a newly computed array of size m? This turns out to be the linear transformation encoded entirely within a matrix:

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}, \quad \boldsymbol{x} \in \mathbb{R}^n, \boldsymbol{y} \in \mathbb{R}^m, \quad \boldsymbol{A} : \mathbb{R}^n \to \mathbb{R}^m$$

We can prove (as we indeed will, later on) that every linear transformation mapping $\mathbb{R}^n \to \mathbb{R}^m$ has a *unique* matrix $\boldsymbol{A}$ associated with it. The matrix $\boldsymbol{A}$, therefore, represents the active mapping logic from $\mathbb{R}^n$ to $\mathbb{R}^m$, taking the input vector $\boldsymbol{x}$ and computing the output vector $\boldsymbol{y}$. When we set this transformation equal to a constant target vector, we get the deceptively simple equation:

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$$

As we saw earlier, the equation $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ acts as a highly compact representation for a full system of linear equations. The properties

of such systems as encoded in the matrix $\boldsymbol{A}$—the conditions under which the system can be solved, the underlying transformations they represent, and the geometry and intuitions behind them—will form the core exploration of the rest of this book.

CS Connection: Matrices as Callable Functions

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

1.5 The Big Picture

For our purposes in computer science, the most appropriate set of numbers to build vectors and matrices is the set of real numbers, arranged in 1D or 2D arrays (columns, rows, or tables). A scalar is simply a single real number, $s \in \mathbb{R}$. It can also be thought of as a 1×1 matrix. A vector is an $n \times 1$ matrix, $\boldsymbol{x} \in \mathbb{R}^n$. Vectors with fewer than four components live in spaces that we can easily understand and visualize: $\mathbb{R}^2$ is a 2D plane and $\mathbb{R}^3$ is the 3D geometric space we inhabit. We can mathematically extrapolate to understand, if not visually render, higher dimensions ($\mathbb{R}^n$ for $n > 3$) just as easily. As we get more sophisticated later on, we will qualify this statement and draw a strict distinction between the coordinate spaces where we live and the formal vector spaces where vectors live.

Matrices (which often represent datasets or transformations), on the other hand, live in spaces that are much harder to visualize: $\boldsymbol{A} \in \mathbb{R}^{m \times n}$. However, as we will see, the mathematical properties of these matrix spaces are remarkably similar to those of standard vector spaces.

Although we will primarily deal with vectors and matrices as concrete arrays of numbers, it is crucial to keep in mind that they are, in fact, abstract entities. They are defined entirely by the operations specified on them (such as scaling, addition, and the inner product).

In computer science, representing vectors explicitly as arrays of numbers makes practical sense because our raw datasets consist of numerical features. However, any mathematical entity that satisfies these strict operational rules can be treated as a vector. This

abstraction plays a massive role in physics and advanced applied mathematics, where entities like polynomials, continuous functions, or even audio signals are processed using the exact same rules of linear algebra.

The overarching power of this subject is that it allows us to decouple ourselves from the underlying representation. We can stop worrying about array indexing and start working purely with the structural properties of the entities. As the book progresses, we will slowly abstract away the individual elements of matrices and vectors. Towards the end, in Part IV, our intuition will become completely representation-independent. We will then be in a position to leverage this high-level intuition to design highly efficient, sophisticated algorithms and computational techniques.

CS Connection: Mathematical Polymorphism


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

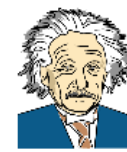

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

2

Vectors and Matrices

So far as the theories of mathematics are about reality, they are not certain; so far as they are certain, they are not about reality.

—Albert Einstein



Vectors and matrices, alongside their associated operations, form the foundational building blocks of linear algebra. While the previous chapter provided a brief, intuitive introduction, we will now examine them formally. For a computer science student, this transition is akin to moving from an introductory programming class to a formal Data Structures course.

2.1 Vectors

2.1.1 What Are Vectors?

For our purposes in computer science, a vector is an *ordered* list of numbers. Just like elements in a typed array, these numbers must all share the same underlying type. In our case, they are typically real numbers (analogous to floating-point variables), because real-world

Groups, Rings, and Fields


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

data—like weights, pixel intensities, or spatial coordinates—tends to be continuous. Mathematically, we formalize this "data type" by saying our vectors are defined over the *field* of real numbers.

This brings up an important mathematical concept: what exactly is a *field*? We will briefly digress in the box titled **Groups, Rings, and Fields** to define these structures. If we have an m-dimensional vector $\boldsymbol{x}$, we indicate its size and its underlying field by writing $\boldsymbol{x} \in \mathbb{R}^m$.

We see this mathematical type system in action in computational tools such as **SageMath**, when we define vectors and matrices explicitly over the *ring* of integers ($\mathbb{Z}$, called `ZZ` in **SageMath**), or over the *fields* of rationals ($\mathbb{Q}$, `QQ`), real numbers ($\mathbb{R}$, `RR`), or complex numbers ($\mathbb{C}$, `CC`)[1].

In order to build a realistic example of a vector that we might come across in computer science, let's think of a data set where we have multiple observations with three quantities:

1. w: Weight in kg
2. h: Height in cm
3. a: Age in years

A specific data point could be represented as the tuple $(w, h, a) = (76, 175, 51)$. We can treat this record as a mathematical vector over the field of real numbers. By standard convention, our vectors are always written as *column vectors*—a vertical arrangement of numbers. Therefore, we write:

$$\boldsymbol{x} = \begin{bmatrix} 76 \\ 175 \\ 51 \end{bmatrix} \in \mathbb{R}^3 \tag{2.1}$$

[1] See `https://www.mathsisfun.com/sets/number-types.html` for a quick refresher on common sets of numbers in mathematics.

A column vector of this dimension is technically just an $m \times 1$ matrix (in this case, 3×1). Note an important pedagogical distinction: although the numbers within the vector (also known as its *elements*, *entries*, or *components*) belong to the same mathematical field, they do not have to represent the same physical units. Weight, height, and age measure completely different things.

However, once encoded into a vector, the machinery of linear algebra acts strictly as a syntactical engine: it processes the numerical structure uniformly, completely ignoring their semantic meaning or physical units. Because of this abstraction, the *order* (or index) of the elements in a vector is strictly significant—swapping height and weight changes the meaning of the data entirely.

CS Connection: Mathematical Fields as Data Types

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

2.1.2 Basic Operations

Scalar Multiplication

We define the *scalar multiplication* of a vector by a number as follows: The result is a new vector where each of its elements is multiplied by that number. In other words, the number (called a *scalar*) *scales* the vector.

Definition: For any vector $\boldsymbol{x}$ and any scalar s, the result of scalar multiplication ($s\boldsymbol{x}$) is defined as follows:

$$\forall\ \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n \text{ and } s \in \mathbb{R}, \quad s\boldsymbol{x} \stackrel{\text{def}}{=} \begin{bmatrix} sx_1 \\ sx_2 \\ \vdots \\ sx_n \end{bmatrix} \in \mathbb{R}^n \tag{2.2}$$

We say that the set of vectors is *closed* under scalar multiplication. In programming terms, we can think of this as a strict type signature: if we pass a scalar and a vector into a multiplication function, it is guaranteed to return another valid vector, never a different data structure. Note that for a vector defined over a certain field (like $\mathbb{R}$),

the scalar must also belong to that same field. It can be any number in that field, including zero.

The fact that the scaling factor can be zero, combined with the closure property, implies the strict existence of the *zero vector* (denoted by $\mathbf{0}$, which is a vector whose elements are all zeros). Because scaling any vector by zero mathematically yields an entity filled with zeros, and because the operation must strictly return a vector, this entity must logically be a vector itself. The zero vector is a fundamental identity element in linear algebra, just like the number zero is in standard arithmetic.

Scalar multiplication is commutative: $s\boldsymbol{x} = \boldsymbol{x}s$. This is not a pre-condition or an arbitrary rule, but a direct consequence of the definition: because it operates element-wise, it simply inherits the commutative property of multiplication from the underlying field of real numbers.

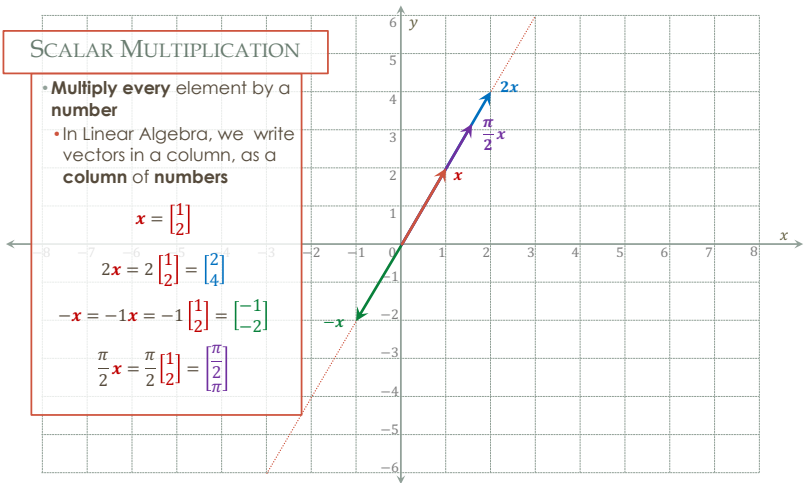


Fig. 2.1 Example of scalar multiplication of a vector $\boldsymbol{x} \in \mathbb{R}^2$. Note that all the scaled versions lie on the same line defined by the original vector. Interactive Visualization

Figure 2.1 shows an example of scalar multiplication. The original vector $\boldsymbol{x} \in \mathbb{R}^2$ has elements 1 and 2, which we plot as one unit along the x-axis and two units along the y-axis. The vector is graphically represented as an arrow from the origin to the point $(1, 2)$, shown in red. Note how the scaled versions (in blue, green, and purple) of the vector are all collinear with the original one. Note also that the line

defined by the original vector and its scaled versions always passes through the origin.

Vector Addition

We define the addition of two vectors element-wise: the result is a new vector whose elements are the sum of the corresponding elements in the original two vectors. In programming, this is akin to iterating through two arrays of the same length and adding the values at each index. Because the sum of two vectors is always another vector, we say the set of vectors is *closed* under addition.
Definition: For any two vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, the result of their addition $(\boldsymbol{x_1} + \boldsymbol{x_2})$ is defined as follows:

$$\begin{gathered} \forall\, \boldsymbol{x_1} = \begin{bmatrix} x_{11} \\ x_{21} \\ \vdots \\ x_{n1} \end{bmatrix} \text{ and } \boldsymbol{x_2} = \begin{bmatrix} x_{12} \\ x_{22} \\ \vdots \\ x_{n2} \end{bmatrix} \in \mathbb{R}^n, \\ \boldsymbol{x_1} + \boldsymbol{x_2} \stackrel{\text{def}}{=} \begin{bmatrix} x_{11} + x_{12} \\ x_{21} + x_{22} \\ \vdots \\ x_{n1} + x_{n2} \end{bmatrix} \in \mathbb{R}^n \end{gathered} \tag{2.3}$$

Vector addition is commutative ($\boldsymbol{x_1} + \boldsymbol{x_2} = \boldsymbol{x_2} + \boldsymbol{x_1}$) because the underlying scalar addition of the elements is commutative. Crucially, we can only add vectors that have the exact same number of elements (their *dimension*). In computer science, attempting to add two arrays of different lengths without a explicitly defined fallback raises a dimension or shape mismatch error.

Strictly speaking, vectors defined over different mathematical fields should not be added. However, because our vectors are almost entirely members of $\mathbb{R}^n$, this will rarely impact us. Even if we do mix types, we rely on the subset hierarchy: integers ($\mathbb{Z}$) $\subseteq$ rationals ($\mathbb{Q}$) $\subseteq$ reals ($\mathbb{R}$) $\subseteq$ complex numbers ($\mathbb{C}$). If we add a vector defined over the field of integers to one over complex numbers, the result simply "upcasts" to a vector over the field of complex numbers. As programmers, we are so used to this implicit type casting (e.g., adding

an integer to a float yields a float) that we might not even realize we are technically committing a linear algebra felony!

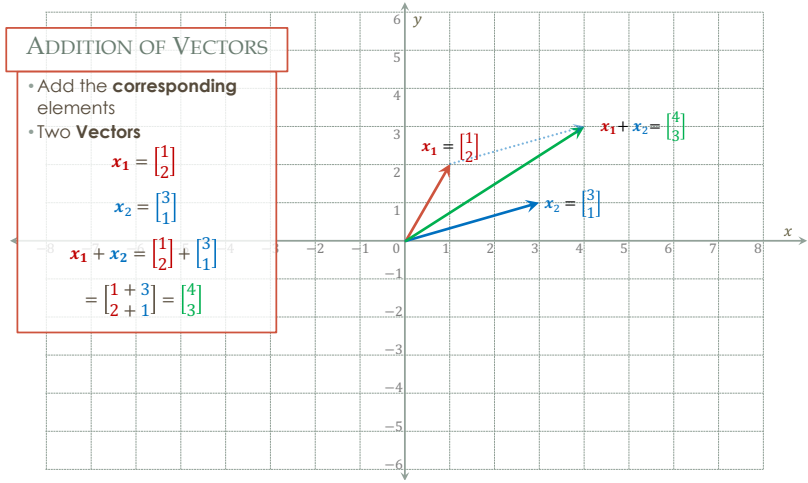


Fig. 2.2 Adding two vectors $\boldsymbol{x_1}, \boldsymbol{x_2} \in \mathbb{R}^2$. We add the elements of $\boldsymbol{x_2}$ to the corresponding elements of $\boldsymbol{x_1}$, which we can visualize as moving $\boldsymbol{x_2}$ (in blue) to the tip of $\boldsymbol{x_1}$ (in red), resulting in the dashed blue arrow. The sum of the two vectors is the green arrow, drawn from the origin to the tip of the dashed arrow. Interactive Visualization

Figure 2.2 illustrates the addition of two vectors $\boldsymbol{x_1}, \boldsymbol{x_2} \in \mathbb{R}^2$. Geometrically, we can think of the addition operation as transporting $\boldsymbol{x_2}$ (in blue) to the tip of $\boldsymbol{x_1}$ (in red). This forms the dashed blue arrow, which is parallel to and the same size as the original $\boldsymbol{x_2}$. The sum of the two vectors is shown in green, drawn from the origin to the tip of the dashed arrow. We could, of course, transport $\boldsymbol{x_1}$ (the red vector) to the tip of $\boldsymbol{x_2}$ (the blue vector) and perform the addition that way as well. This completes a parallelogram with $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ as adjacent sides, where the diagonal from the origin represents the sum.

CS Connection: Points vs. Vectors in Graphics

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Now that we have defined addition and scaling, we can ask: does every vector have an *additive inverse*? In other words, for each vector

$\boldsymbol{x} \in \mathbb{R}^n$, can we find another vector $\boldsymbol{x'}$ such that $\boldsymbol{x} + \boldsymbol{x'} = \mathbf{0}$? Note that $\mathbf{0}$ is not the scalar number 0, but the zero vector $\mathbf{0} \in \mathbb{R}^n$.

If we scale $\boldsymbol{x}$ by -1 to create $\boldsymbol{x'} = -1\boldsymbol{x}$, our definition of vector addition guarantees that $\boldsymbol{x} + \boldsymbol{x'} = \mathbf{0}$. Therefore, every vector in $\mathbb{R}^n$ strictly possesses an additive inverse.

2.1.3 Linear Combinations

Because the set of vectors is closed under both scalar multiplication and vector addition, we can safely compose these two operations. If we have two vectors in $\mathbb{R}^n$, $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, and any two scalars, $s_1, s_2 \in \mathbb{R}$, we can construct a new vector $\boldsymbol{y} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$. Because of closure, we are guaranteed that $\boldsymbol{y} \in \mathbb{R}^n$ is a valid vector.

In computer science and machine learning, this operation is commonly referred to as a *weighted sum*—where the scalars act as the "weights" applied to the data vectors. This notion of linear combinations is the absolute foundation of linear algebra, and we will rely on it extensively.

For example, using the two vectors from Figure 2.2, if we set our scaling factors to $s_1 = 2$ and $s_2 = -1$, we get:

$$\boldsymbol{y} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2} = 2\begin{bmatrix}1\\2\end{bmatrix} + (-1)\begin{bmatrix}3\\1\end{bmatrix} = \begin{bmatrix}2\\4\end{bmatrix} + \begin{bmatrix}-3\\-1\end{bmatrix} = \begin{bmatrix}-1\\3\end{bmatrix}$$

By choosing different values for s_1 and s_2, we can generate an infinite number of different $\boldsymbol{y}$ vectors.

This raises an important structural question: Is it possible to find more than one set of weights (s_1 and s_2) that produce the exact same target vector $\boldsymbol{y}$? In other words, is this representation *unique*?

The answer is no, and we can intuitively see why through geometry. Suppose we want to find the scaling factors for the green vector $\boldsymbol{y}$ in Figure 2.3. Draw a line through the tip of the green vector, running parallel to the blue vector $\boldsymbol{x_2}$. Let it intersect the infinite line defined by the red vector. This intersection gives us the scaled vector $\boldsymbol{x_1'}$. The ratio of the signed length of $\boldsymbol{x_1'}$ to the length of original $\boldsymbol{x_1}$ gives us exactly s_1. (The sign is positive if they point in the same direction, and negative otherwise).

By making a similar parallel construction for the other axis, we extract s_2. Because two non-parallel lines in $\mathbb{R}^2$ can intersect at exactly one unique point, there is only one valid geometric parallelogram

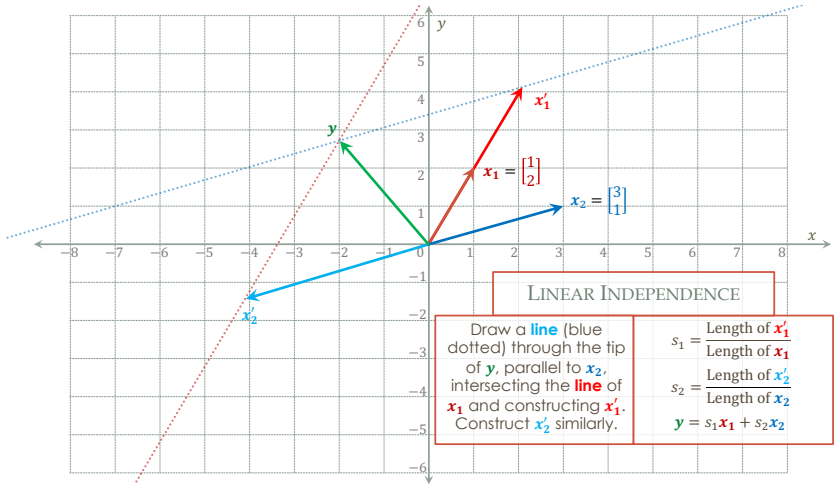


Fig. 2.3 Given *any* green vector $\boldsymbol{y}$ and the two basis vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, how do we find the scaling factors s_1 and s_2 such that $\boldsymbol{y} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$? This geometric construction shows that any vector in this plane can be written as a linear combination of $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$.
Interactive Visualization

that can be formed. Therefore, the scaling factors s_1 and s_2 must be strictly unique.

A related question is whether we can combine these vectors to yield the zero vector without trivially setting our weights to zero. That is, can we solve:

$$\boldsymbol{y} = \mathbf{0} = \begin{bmatrix}0\\0\end{bmatrix} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$$

where $s_1 \neq 0$ and $s_2 \neq 0$?

As a direct corollary to our geometric "proof" of uniqueness, the answer is again no—at least, not for the specific non-parallel vectors shown in our figure. Vectors that satisfy this strict condition are called *linearly independent*, a core concept we will formally define in just a moment. We will explore the deeper algebraic machinery behind this in a later chapter dedicated to vector spaces.

2.2 Advanced Vector Concepts

2.2.1 Linear Independence

In Figure 2.3, we saw that we can generate any arbitrary vector $\boldsymbol{y}$ as a linear combination of $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$. In other words, we can always find scalars s_1 and s_2 such that $\boldsymbol{y} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$. Later in this book, we will formalize this by saying that $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ *span* $\mathbb{R}^2$. This is a concise way of stating that all vectors in $\mathbb{R}^2$ can be written as a unique linear combination of these two vectors.

The fact that $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ can uniquely span $\mathbb{R}^2$ brings us to another pivotal concept in linear algebra: *Linear Independence*. We will dedicate Chapter 6 to this, but it is worth introducing now. The vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ in our first figure are linearly independent.

A set of vectors is linearly independent if none of them can be expressed as a linear combination of the rest. In a computer science or machine learning context, we can think of linear dependence as *data redundancy*. If a vector can be constructed entirely from the others, it provides no new "information" or mathematical dimension to the system. For two vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ in $\mathbb{R}^2$, being linearly independent simply means that $\boldsymbol{x_1}$ is not a scalar multiple of $\boldsymbol{x_2}$.

Another equivalent, highly rigorous definition is that $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ are linearly independent if the *only* solution to the equation $\boldsymbol{0} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$ is the trivial one: $s_1 = 0$ and $s_2 = 0$.

While Figure 2.3 shows a linearly independent pair, Figure 2.4 illustrates a pair, $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, that are linearly *dependent*. Because they point along the exact same line, they are redundant. If we try to reach an arbitrary 2D target vector $\boldsymbol{y}$ using our formula $\boldsymbol{y} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$, we will fail—unless $\boldsymbol{y}$ coincidentally lies on that exact same line. And even if it does, we lose our uniqueness guarantee; there would be an infinite number of valid s_1 and s_2 combinations that could reach it.

Notice that in Figure 2.4, $\boldsymbol{x_2} = 2\boldsymbol{x_1}$. We can rearrange this algebraically to $2\boldsymbol{x_1} - \boldsymbol{x_2} = \boldsymbol{0}$. Here, the zero vector $\boldsymbol{0}$ has been written as a linear combination of $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ using non-zero weights ($s_1 = 2$ and $s_2 = -1$). Because we found a non-trivial way to reach $\boldsymbol{0}$, our formal algebraic test fails, proving that these vectors are indeed linearly dependent.

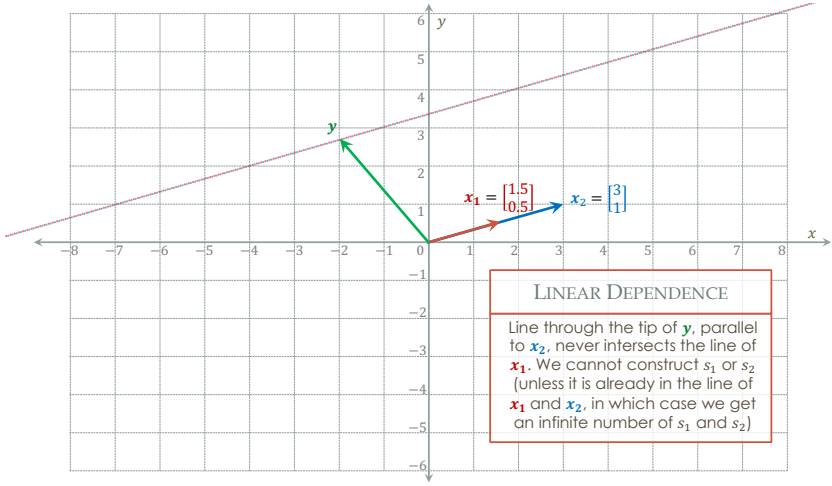


Fig. 2.4 Two linearly dependent vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, which cannot form a linear combination to reach the arbitrary green vector $\boldsymbol{y} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$ (unless $\boldsymbol{y}$ happens to fall perfectly on their shared line).

2.2.2 The Dot Product

We have now defined how to multiply a vector by a scalar and how to add two vectors together. In computer science terms, we know that trying to add a scalar directly to a vector results in a strict type mismatch. But what happens when we want to multiply a vector by another vector? Let's define the most fundamental form of vector-vector multiplication: the *dot product*.

Dot Product

Definition: For any two vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, their dot product $(\boldsymbol{x_1} \cdot \boldsymbol{x_2})$

is defined as follows:

$$\begin{aligned} \forall\, \boldsymbol{x_1} = \begin{bmatrix} x_{11} \\ x_{21} \\ \vdots \\ x_{n1} \end{bmatrix} \text{ and } \boldsymbol{x_2} = \begin{bmatrix} x_{12} \\ x_{22} \\ \vdots \\ x_{n2} \end{bmatrix} \in \mathbb{R}^n, \\ \boldsymbol{x_1} \cdot \boldsymbol{x_2} \overset{\text{def}}{=} x_{11}x_{12} + x_{21}x_{22} + \cdots + x_{n1}x_{n2} \\ = \sum_{i=1}^{n} x_{i1}x_{i2} \in \mathbb{R} \end{aligned} \tag{2.4}$$

In algorithmic terms, computing the dot product involves iterating through both vectors simultaneously, multiplying the elements at each corresponding index, and keeping a running sum of those products. In hardware and systems design, this is known as the classic Multiply-Accumulate (MAC) operation, which forms the absolute backbone of neural network computations. Naturally, just like with vector addition, the dot product is only defined for vectors of the exact same dimension; otherwise, our iteration would throw an index out-of-bounds error.

CS Connection: The Hardware of the Dot Product


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

The dot product of two vectors is also commonly known as their *scalar product* because the final return type of the operation is a single scalar number ($\in \mathbb{R}$). This terminology is critical: it should not be confused with *scalar multiplication*, which scales a vector and returns a new vector ($\in \mathbb{R}^n$). Another widely used mathematical name for the dot product is the *inner product*.

Cosine Similarity

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
Available in the Full Version of the book.

2.2.3 Norm of a Vector

Definition: For a vector $\boldsymbol{x}$, its norm, $\|\boldsymbol{x}\|$, is defined as follows:

$$\forall\, \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n, \quad \|\boldsymbol{x}\| \stackrel{\text{def}}{=} \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} \\ = \sqrt{\sum_{i=1}^{n} x_i^2} \quad \in \mathbb{R} \tag{2.5}$$

From the definition of the dot product, we can see that $\|\boldsymbol{x}\| = \sqrt{\boldsymbol{x} \cdot \boldsymbol{x}}$. Related to the dot product, the norm of a vector is a rigorous measure of its size, length, or magnitude. Note an important mathematical property: if we scale a vector by a factor s, the norm scales by the *absolute value* of that factor: $\|s\boldsymbol{x}\| = |s|\|\boldsymbol{x}\|$. (Because a norm represents a magnitude or distance, it can never be negative).

If we divide a vector by its norm (which means we perform scalar multiplication by the reciprocal of the norm), the resulting vector has a unit length of exactly 1. This process is called *normalizing* a vector, a procedure highly common in computer graphics and machine learning feature scaling. We typically use the "hat" notation $\hat{\boldsymbol{x}}$ to indicate unit vectors:

$$\hat{\boldsymbol{x}} = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|}$$

The norm we defined above is the *Euclidean norm*. However, other norms are possible and are frequently used in computer science. They are generalized as the p-norm, defined below for various values of $p \geq 1$:

$$\|\boldsymbol{x}\|_p \stackrel{\text{def}}{=} \sqrt[p]{\sum_{i=1}^{n} |x_i|^p} \quad \in \mathbb{R} \tag{2.6}$$

This p-norm becomes the standard Euclidean norm (often referred to in machine learning as the L_2 norm) when $p = 2$. When $p = 1$, it becomes what is known as the Manhattan (or Taxicab) norm (the L_1 norm), which measures distance strictly along grid lines. As $p \to \infty$, the p-norm converges to the maximum absolute value among the vector's elements, known as the infinity norm (L_∞) or simply the maximum norm. In machine learning, we will see L_1 and L_2 norms used constantly for model regularization algorithms like Lasso and Ridge regression.

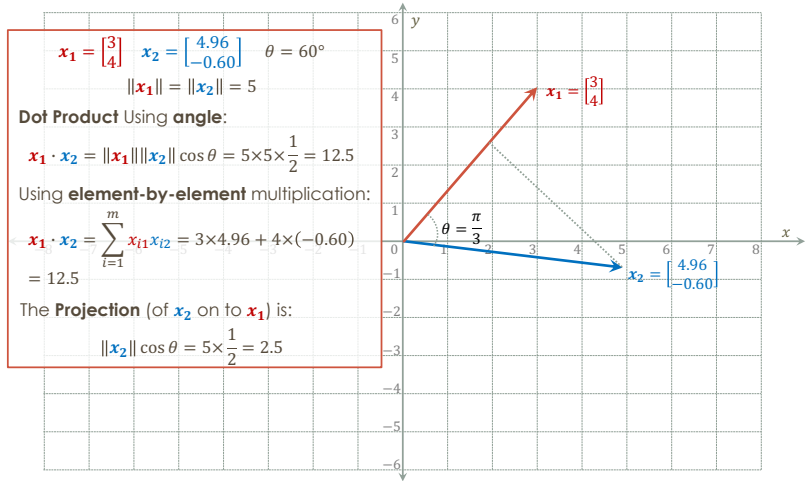


Fig. 2.5 Dot product between two vectors $\boldsymbol{x_1}, \boldsymbol{x_2} \in \mathbb{R}^2$. We compute the dot product using the elements of the vectors as well as the angle between them to demonstrate that both the algebraic and geometric approaches yield the exact same result. Interactive Visualization

Dot Product as Projection

Geometrically, the dot product can also be defined using the angle between the two vectors. Let's consider two vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ separated by an angle θ. Their dot product is:

$$\boldsymbol{x_1} \cdot \boldsymbol{x_2} \overset{\text{def}}{=} \|\boldsymbol{x_1}\|\|\boldsymbol{x_2}\| \cos\theta$$

If we algebraically rearrange this definition, we reveal that it elegantly describes the *projection* of one vector onto the direction of the

Quantum Computing & Dirac Notation

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
Available in the Full Version of the book.

other:

$$\|\boldsymbol{x_2}\| \cos\theta = \frac{\boldsymbol{x_1} \cdot \boldsymbol{x_2}}{\|\boldsymbol{x_1}\|} = \left(\frac{\boldsymbol{x_1}}{\|\boldsymbol{x_1}\|}\right) \cdot \boldsymbol{x_2} = \hat{\boldsymbol{x}}_1 \cdot \boldsymbol{x_2}$$

Here, $\hat{\boldsymbol{x}}_1$ is the normalized unit vector pointing in the exact direction of the first vector. The term $\|\boldsymbol{x_2}\| \cos\theta$ is the scalar projection (the "shadow") of the second vector onto this direction. Consequently, if the two vectors are perfectly perpendicular ($\theta = \frac{\pi}{2}$), their projection is zero, meaning their dot product is zero. If the angle is zero, the projected length is exactly equal to the full length of the second vector.

In machine learning and natural language processing, this geometric interpretation is incredibly powerful. By dividing the dot product by the magnitudes of the vectors, we isolate $\cos\theta$. This metric is known as *Cosine Similarity*, and it is the standard algorithm used to calculate how semantically similar two documents or word embeddings are, regardless of their size.

CS Connection: Cosine Similarity in NLP

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
CS Connections are available in the Full Version of the book.

It is a mathematically remarkable fact that our two completely different definitions of the dot product—one algebraic (row-column multiplication) and one geometric (angles and lengths)—are perfectly equivalent:

$$\sum_{i=1}^{n} x_{i1} x_{i2} = \|\boldsymbol{x_1}\| \|\boldsymbol{x_2}\| \cos\theta$$

This equivalence can be rigorously proven using the Law of Cosines. Figure 2.5 visually demonstrates this relationship, using two vectors in $\mathbb{R}^2$ to show how one perfectly projects onto the other.

2.2.4 Generalized Vectors

We should keep in mind that, in its highest generality, vectors are defined strictly by their *operations*—namely scalar multiplication, vector addition, and the inner product.

In software engineering terms, we can think of a "Vector" as an Interface or an Abstract Base Class. Any set of mathematical entities—literally any at all—that successfully "implements" these operations with the correct commutative, associative, and distributive properties can be formally treated as vectors. They do not have to be physical arrows in space or 1D arrays of numbers; they could be polynomials, continuous functions, or sound waves.

Once an object satisfies these baseline operational rules, the vast, powerful machinery of linear algebra stands ready to process it. This abstraction helps us ensure logical consistency, derive new insights, and deepen our understanding of highly complex systems. This "polymorphic" view of vectors is exactly how linear algebra is utilized in modern advanced physics, most notably in special relativity and the quantum mechanical states of subatomic particles.

2.3 Matrices

We introduced the notion of a matrix when we explored the conditions of linearity for functions and expressions with two variables in §1.4, page 16. Now it is time to describe it more completely and formally. Exactly like vectors, matrices are tables of numbers. In computer science, we typically implement them as 2D arrays or data tensors.

The numbers we are dealing with are typically real (represented as `float` or `double` data types in various programming languages). We will, therefore, talk about matrices over the field of real numbers, and write $\boldsymbol{A} \in \mathbb{R}^{m \times n}$. This notation formally states that $\boldsymbol{A}$ is a matrix over the field of reals, structured with m rows and n columns. Note the order: rows first, always. (Computer science students might recognize this convention as it directly mirrors the row-major memory layout used in languages like C, C++, and Python).

The first two operations on matrices that we will define are identical to the ones for vectors.

2.3.1 Scalar Multiplication

Definition: For any matrix $\boldsymbol{A}$ and any scalar s, the result of scalar multiplication ($s\boldsymbol{A}$) is defined as follows:

$$\begin{gathered}\forall\ \boldsymbol{A} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & a_{ij} & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m\times n} \text{ and } s \in \mathbb{R}, \\ s\boldsymbol{A} \stackrel{\text{def}}{=} \begin{bmatrix} sa_{11} & \cdots & sa_{1n} \\ \vdots & sa_{ij} & \vdots \\ sa_{m1} & \cdots & sa_{mn} \end{bmatrix} \in \mathbb{R}^{m\times n}\end{gathered} \tag{2.7}$$

In other words, when we multiply a matrix by a scalar, we multiply each of its individual elements by that scalar. In data science libraries, this is known as *broadcasting* a scalar across an array. The resulting matrix is strictly of the same dimension: If $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, then $s\boldsymbol{A} \in \mathbb{R}^{m\times n}$.

2.3.2 Matrix Addition

Definition: For any two matrices $\boldsymbol{A}$ and $\boldsymbol{B}$, their sum (the result of their addition, $\boldsymbol{A} + \boldsymbol{B}$) is defined as follows:

$$\begin{gathered}\forall\ \boldsymbol{A} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & a_{ij} & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \text{ and } \boldsymbol{B} = \begin{bmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & b_{ij} & \vdots \\ b_{m1} & \cdots & b_{mn} \end{bmatrix} \in \mathbb{R}^{m\times n}, \\ \boldsymbol{A} + \boldsymbol{B} \stackrel{\text{def}}{=} \begin{bmatrix} a_{11} + b_{11} & \cdots & a_{1n} + b_{1n} \\ \vdots & a_{ij} + b_{ij} & \vdots \\ a_{m1} + b_{m1} & \cdots & a_{mn} + b_{mn} \end{bmatrix} \in \mathbb{R}^{m\times n}\end{gathered} \tag{2.8}$$

To add two matrices, we simply iterate through and add their corresponding elements. Naturally, just like arrays in code, the two matrices we are trying to add *must* have the exact same dimensions to avoid a shape mismatch.

As we can see, although the notations are slightly different, vectors and matrices share the exact same logic when it comes to scalar multiplication and addition. Soon, we will see that the dot product—a quintessentially vector-type operation (associated with the cosine of the angle between them)—is also, in fact, a matrix operation. The

reason for this connection is simple: A vector in linear algebra is just a matrix with exactly one column; it is a *column matrix*. Everything we specify for matrices applies flawlessly to vectors as well.

It may be worth our time to compare these first two matrix/vector operations (scalar multiplication and addition) back to the conditions of linearity (homogeneity and additivity, described in §1.3, page 16). The fact that they run perfectly parallel is not an accident. It points directly to the deep, underlying structure of linear algebra that we will uncover.

2.3.3 Properties of Scaling and Addition

We have to keep in mind that in matrix (and therefore vector) addition, the dimensions must strictly match: $\boldsymbol{A} + \boldsymbol{B}$ is defined if and only if they both have the exact same number of rows and columns. In particular, a 1D vector cannot be added directly to a 2D matrix (of more than one column).

Furthermore, vectors of different lengths cannot interact with each other; they live in entirely different mathematical spaces. A zero vector in $\mathbb{R}^3$ (our 3-D space) is fundamentally not the same object as the zero vector in $\mathbb{R}^2$ (a plane). In object-oriented programming, these would be entirely different classes, and attempting to add them together would cause the compiler to throw a fatal type error.

When dealing with matrices (or vectors) of compatible sizes, we can list the mathematical properties of scalar multiplication and addition. Both of these operations exhibit:

Commutativity

The order in which the matrices (or vectors) appear in the operations does not matter.

$$s\boldsymbol{x} = \boldsymbol{x}s; \quad s\boldsymbol{A} = \boldsymbol{A}s$$
$$\boldsymbol{x_1} + \boldsymbol{x_2} = \boldsymbol{x_2} + \boldsymbol{x_1}; \quad \boldsymbol{A} + \boldsymbol{B} = \boldsymbol{B} + \boldsymbol{A}$$

Associativity

We can group and perform the operations in any order we want.

$$s_1 s_2 \boldsymbol{x} = s_1(s_2 \boldsymbol{x}) = (s_1 s_2)\boldsymbol{x}; \quad s_1 s_2 \boldsymbol{A} = (s_1 s_2)\boldsymbol{A} = s_1(s_2 \boldsymbol{A})$$
$$\boldsymbol{x_1} + \boldsymbol{x_2} + \boldsymbol{x_3} = (\boldsymbol{x_1} + \boldsymbol{x_2}) + \boldsymbol{x_3} = \boldsymbol{x_1} + (\boldsymbol{x_2} + \boldsymbol{x_3})$$
$$\boldsymbol{A} + \boldsymbol{B} + \boldsymbol{C} = (\boldsymbol{A} + \boldsymbol{B}) + \boldsymbol{C} = \boldsymbol{A} + (\boldsymbol{B} + \boldsymbol{C})$$

Distributivity

Scalar multiplication distributes perfectly over matrix addition.

$$s(\boldsymbol{x_1} + \boldsymbol{x_2}) = s\boldsymbol{x_1} + s\boldsymbol{x_2}; \quad s(\boldsymbol{A} + \boldsymbol{B}) = s\boldsymbol{A} + s\boldsymbol{B}$$
$$(s_1 + s_2)\boldsymbol{x} = s_1\boldsymbol{x} + s_2\boldsymbol{x}; \quad (s_1 + s_2)\boldsymbol{A} = s_1\boldsymbol{A} + s_2\boldsymbol{A}$$

These properties are not arbitrarily imposed rules. They are direct, logical consequences of the element-wise definitions of scalar multiplication and matrix addition. In other words, they can be rigorously proven to be true starting from the underlying definitions.

2.4 Matrix Multiplication

We will now define and describe how matrices multiply. Two matrices can be multiplied to yield a new matrix, but only under highly specific conditions. Matrix multiplication forms the absolute backbone of much of the subject matter to follow—especially in algorithms like neural networks and 3D graphics rendering. For that reason, we will look at it carefully from multiple perspectives.

2.4.1 Conformance Requirement

In order to multiply one matrix by another, the number of *columns* of the *first* matrix must exactly equal the number of *rows* of the *second* matrix. The resulting product matrix will inherit the number of rows from the first matrix, and the number of columns from the second.

In other words, if $\boldsymbol{A} \in \mathbb{R}^{m \times k}$ and $\boldsymbol{B} \in \mathbb{R}^{k \times n}$, their "inner dimensions" (k) match, so they can be multiplied to produce a new matrix $\boldsymbol{C} = \boldsymbol{AB} \in \mathbb{R}^{m \times n}$. Think of this like connecting software interfaces: the output data shape of process $\boldsymbol{A}$ must perfectly match the expected input data shape of process $\boldsymbol{B}$.

Note that this conformance requirement is entirely different from the one for matrix addition, where the individual matrices and the sum must all be the exact *same* size. In contrast, for matrix multiplication, if two rectangular matrices are of the exact same size ($m \times n$), they generally *cannot* be multiplied together (unless they are square matrices where $m = n$).

Row-by-Column Multiplication

Definition: For conformant matrices, we define standard matrix multiplication as follows: The element in the ith row and the jth column of the product is the sum of the products of the elements in the ith row of the first matrix and the jth column of the second matrix. (Computer science students beware: this is *not* element-wise Hadamard multiplication, which simply multiplies corresponding slots together).

More formally, for any two conformant matrices $\boldsymbol{A} \in \mathbb{R}^{m \times k}$ and $\boldsymbol{B} \in \mathbb{R}^{k \times n}$, their product ($\boldsymbol{C} = \boldsymbol{AB}$) is defined as follows:

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & \cdots & a_{1k} \\ \vdots & a_{il} & \vdots \\ a_{m1} & \cdots & a_{mk} \end{bmatrix} \text{ and } \boldsymbol{B} = \begin{bmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & b_{lj} & \vdots \\ b_{k1} & \cdots & b_{kn} \end{bmatrix},$$

$$\boldsymbol{AB} = \boldsymbol{C} = \begin{bmatrix} c_{11} & \cdots & c_{1n} \\ \vdots & c_{ij} & \vdots \\ c_{m1} & \cdots & c_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n} \quad \text{where} \tag{2.9}$$

$$c_{ij} \overset{\text{def}}{=} a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ik}b_{kj} = \sum_{l=1}^{k} a_{il}b_{lj}$$

Since this purely algebraic definition is a bit cumbersome to parse, let us visualize exactly what it is saying using Figure 2.6.

Crucially, note that matrix multiplication is *not* commutative: $\boldsymbol{AB} \neq \boldsymbol{BA}$ in general. In fact, for most non-square matrices, if they are conformant for the multiplication $\boldsymbol{AB}$, they simply are not conformant for $\boldsymbol{BA}$, meaning the reverse product will throw a dimension error and is mathematically undefined.

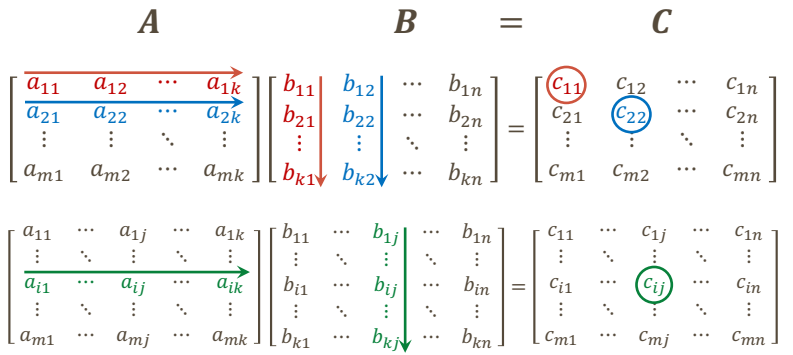


Fig. 2.6 Illustration of matrix multiplication $\boldsymbol{AB} = \boldsymbol{C}$. In the top panel, the element c_{11} (in red) of the product $\boldsymbol{C}$ is obtained by taking the elements in the first row of $\boldsymbol{A}$, multiplying them with the elements in the first column of $\boldsymbol{B}$ (shown with red letters and arrows), and summing them up. The element c_{22} (blue) is the sum-product of the second row of $\boldsymbol{A}$ and the second column of $\boldsymbol{B}$. In the bottom panel, we see a general element c_{ij} (green) as the sum-product of the ith row of $\boldsymbol{A}$ and the jth column of $\boldsymbol{B}$.

2.4.2 Dot Product as Matrix Product

Now that we have defined general matrix multiplication $\boldsymbol{AB} = \boldsymbol{C}$ (where $\boldsymbol{A} \in \mathbb{R}^{m\times k}$, $\boldsymbol{B} \in \mathbb{R}^{k\times n}$, and $\boldsymbol{C} \in \mathbb{R}^{m\times n}$), let's consider a special case where $m = 1$ and $n = 1$. In this scenario, both $\boldsymbol{A}$ and $\boldsymbol{B}$ contain exactly k numbers, but arranged horizontally and vertically, respectively:

$$\boldsymbol{A} = \begin{bmatrix} a_1 & a_2 & \cdots & a_k \end{bmatrix} = \boldsymbol{a}^{\mathsf{T}}, \quad \boldsymbol{B} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_k \end{bmatrix} = \boldsymbol{b} \tag{2.10}$$

$$\boldsymbol{C} = [c_{11}] = a_1 b_1 + a_2 b_2 + \cdots + a_k b_k = \sum_{i=1}^{k} a_i b_i = s$$

In this case, $\boldsymbol{C}$ has become a matrix of one row and one column, which is mathematically indistinguishable from a scalar. We can, therefore, safely use the scalar symbol s to represent it. $\boldsymbol{B}$ is a column matrix with k rows, which is exactly what we earlier defined

as a vector (remember, by default, all our vectors are column vectors). Let's use the lowercase symbol $\boldsymbol{b}$ instead of $\boldsymbol{B}$ to stay consistent with our vector notation.

Now, $\boldsymbol{A}$ is a special matrix with k elements arranged in a single row. While some texts call this a "row vector," we will formally call it the *transpose* of a column vector $\boldsymbol{a}$, denoted by $\boldsymbol{a}^\mathsf{T}$. We obtain the transpose of a matrix by swapping its rows and columns. In the case of a column vector (a vertical sequence of numbers), its transpose is formed by simply laying it down horizontally.

What we have written above is mathematically identical to our definition of the vector dot product in Eqn (2.4), letting $\boldsymbol{a} = \boldsymbol{x_1}$ and $\boldsymbol{b} = \boldsymbol{x_2}$. In other words, the vector dot product $\boldsymbol{a} \cdot \boldsymbol{b}$ is strictly equivalent to the matrix product $\boldsymbol{a}^\mathsf{T}\boldsymbol{b}$.

Note that $\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{b} \cdot \boldsymbol{a}$ and $\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \boldsymbol{b}^\mathsf{T}\boldsymbol{a}$, as is evident from the summation in Equations (2.4) and (2.10). Because this equivalence is incredibly useful when manipulating algebraic equations in machine learning, let's restate it with its own reference number:

$$\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \boldsymbol{b}^\mathsf{T}\boldsymbol{a} = \boldsymbol{b} \cdot \boldsymbol{a} \tag{2.11}$$

As we can clearly see, the vector dot product is merely a special, constrained case of matrix multiplication. In fact, *all* operations we define for vectors are special cases of the corresponding operations for matrices. This is unsurprising because vectors are simply matrices with one column. We actually used this matrix multiplication trick way back in Eqn (1.1) as a notational convenience.

Multiplication as Dot Products

Definition: Consider the product $\boldsymbol{AB} = \boldsymbol{D}$ (where $\boldsymbol{A} \in \mathbb{R}^{m \times k}$, $\boldsymbol{B} \in \mathbb{R}^{k \times n}$, and $\boldsymbol{D} \in \mathbb{R}^{m \times n}$). We use $\boldsymbol{D}$ instead of $\boldsymbol{C}$ here to avoid possible visual confusion between the column vectors $\boldsymbol{c_j}$ and the elements of the product matrix. We can think of $\boldsymbol{A}$ as consisting of m row vectors $\boldsymbol{r_i}^\mathsf{T}$ (each with k elements), and $\boldsymbol{B}$ as consisting of n column vectors $\boldsymbol{c_j}$ (also with k elements). The scalar element d_{ij} in the product matrix is simply the dot product of the ith row of $\boldsymbol{A}$ and the jth column of $\boldsymbol{B}$:

$$d_{ij} = \boldsymbol{r_i} \cdot \boldsymbol{c_j} = \boldsymbol{r_i}^\mathsf{T}\boldsymbol{c_j}$$

Here, we are restating matrix multiplication rather cyclically, using the definition of the vector dot product. Because of the matrix dimensions, all $\boldsymbol{r}_i^\mathsf{T}$ rows and $\boldsymbol{c}_j$ columns strictly share k elements, allowing the dot product to execute safely. In computer science, this localized property is vital: because each d_{ij} relies *only* on one specific row and column, every element in the output matrix can be computed entirely independently at the same time. This is exactly why GPUs (which have thousands of parallel threads) are so phenomenally fast at matrix math!

2.4.3 Block-wise Multiplication

We can perform matrix multiplication in a *block-wise* fashion, where we segment the initial matrices into smaller, conformant sub-matrices (blocks), and multiply them block-by-block. While this might sound like a needlessly complex exercise, we actually just utilized it above. When we defined matrix multiplication as the dot product of individual rows and columns, we were simply dividing the matrices into the smallest possible 1D blocks and processing them.

In algorithms, block-wise multiplication acts like a divide-and-conquer strategy. In the product $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$, if $\boldsymbol{A}$ is segmented into $m \times k$ blocks named $\boldsymbol{A}_{il}$ and $\boldsymbol{B}$ is segmented into $k \times n$ blocks named $\boldsymbol{B}_{lj}$, we can compute an entire output block $\boldsymbol{C}_{ij}$ as:

$$\boldsymbol{C}_{ij} = \sum_{l=1}^{k} \boldsymbol{A}_{il}\boldsymbol{B}_{lj}$$

provided that the sub-block $\boldsymbol{A}_{il}$ is dimensionally conformant to multiply with $\boldsymbol{B}_{lj}$. If we compare this equation to Eqn (2.9), we will immediately see that the standard formula is just a special case where we partitioned $\boldsymbol{A}$ and $\boldsymbol{B}$ into blocks of single scalar elements.

Segmenting matrices into conformant blocks is a foundational optimization in software engineering (often called "loop tiling") used to keep data chunks small enough to fit perfectly inside a CPU's ultra-fast cache memory.

CS Connection: Block Matrices and Cache Optimization

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Special Cases of Block-wise Multiplication

Here is a practical situation where block-wise multiplication is invaluable: If we have a square matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ and a wide matrix $\boldsymbol{B} \in \mathbb{R}^{n \times 2n}$, we can certainly multiply them to get $\boldsymbol{AB} \in \mathbb{R}^{n \times 2n}$. However, we can also conceptualize $\boldsymbol{B}$ as being composed of two separate $n \times n$ square matrices, $\boldsymbol{B_1}$ and $\boldsymbol{B_2}$, sitting side-by-side.

We can write this partitioned matrix as $\boldsymbol{B} = [\boldsymbol{B_1} \mid \boldsymbol{B_2}]$. The notation $[\boldsymbol{C} \mid \boldsymbol{D}]$ indicates a new matrix where the columns of $\boldsymbol{C}$ and $\boldsymbol{D}$ are horizontally concatenated. Later in the book, we will formally call these "augmented matrices." Using block multiplication:

$$\boldsymbol{AB} = \boldsymbol{A}[\boldsymbol{B_1} \mid \boldsymbol{B_2}] = [\boldsymbol{AB_1} \mid \boldsymbol{AB_2}]$$

This behaves exactly as if we distributed the matrix multiplier $\boldsymbol{A}$ across the partition boundaries. We will use this specific trick to understand several advanced techniques later. However, we must keep in mind that the order of the matrices remains completely rigid. $\boldsymbol{BA} \neq [\boldsymbol{B_1 A} \mid \boldsymbol{B_2 A}]$. In fact, given their dimensions, $\boldsymbol{B}$ and $\boldsymbol{A}$ are not even conformant to be multiplied in that reverse order.

Another highly common case occurs when we treat the second matrix simply as a horizontal sequence of column vectors. If $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{B} \in \mathbb{R}^{n \times k}$, we can think of $\boldsymbol{B}$ as k distinct column vectors $\boldsymbol{b_i}$ standing side-by-side, written as $\boldsymbol{B} = [\boldsymbol{b_1} \mid \boldsymbol{b_2} \mid \cdots \mid \boldsymbol{b_k}]$. We can then apply our block-wise rule:

$$\boldsymbol{AB} = \boldsymbol{A}[\boldsymbol{b_1} \mid \boldsymbol{b_2} \mid \cdots \mid \boldsymbol{b_k}] = [\boldsymbol{Ab_1} \mid \boldsymbol{Ab_2} \mid \cdots \mid \boldsymbol{Ab_k}]$$

Earlier, we described $\boldsymbol{Ax} = \boldsymbol{b}$ as a single set of m linear equations with n unknowns. We can think of the block-wise expansion above as solving k distinct sets of equations simultaneously! In machine learning, this is known as *batch processing*—applying our model $\boldsymbol{A}$ to an entire batch of data vectors ($\boldsymbol{b_i}$) at the exact same time. We will circle back to this powerful concept in Chapter 5.

2.4.4 Column and Row Pictures

We can think of our first matrix as composed of individual column vectors, and the final product as a linear combination of these specific vectors. This gives us the *column picture* of matrix multiplication. This column-centric view is a foundational perspective that underpins several critical concepts in linear algebra, and we will return to it time and again in this book.

Recall that we defined the linear combinations of vectors in §2.1.3 (page 27). Let's now restate the definition of matrix multiplication using this notion: the product is a linear combination of the columns of the first matrix, scaled by the individual elements of the second matrix.

To keep it simple, let's first consider the multiplication of a matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ by a single column vector $\boldsymbol{x} \in \mathbb{R}^n$. The dimensions are conformant, and the multiplication is allowed. We will get a vector as the product, which we will call $\boldsymbol{b} \in \mathbb{R}^m$. We can conceptualize $\boldsymbol{A}$ as being composed of n distinct column vectors ($\boldsymbol{c_i} \in \mathbb{R}^m$) standing side-by-side. The product $\boldsymbol{Ax}$ is strictly the linear combination of these columns of $\boldsymbol{A}$, weighted by the scalar components of $\boldsymbol{x}$:

$$\boldsymbol{A} = \begin{bmatrix} | & | & & | \\ \boldsymbol{c_1} & \boldsymbol{c_2} & \cdots & \boldsymbol{c_n} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{m \times n}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n \tag{2.12}$$

$$\boldsymbol{Ax} = \boldsymbol{b} = x_1\boldsymbol{c_1} + x_2\boldsymbol{c_2} + \cdots + x_n\boldsymbol{c_n} \in \mathbb{R}^m$$

In machine learning, this view is highly intuitive: if $\boldsymbol{A}$ represents a layer of learned weights and $\boldsymbol{x}$ is our input data, the output $\boldsymbol{b}$ is simply a blended combination of the model's feature columns, weighted by the specific input values.

Similarly, performing the multiplication on the *left* gives us the *row picture*. Take $\boldsymbol{x}^\mathsf{T}$ to be a matrix of a single row ($\boldsymbol{x}^\mathsf{T} \in \mathbb{R}^{1 \times m}$), which simply means $\boldsymbol{x}$ is a standard column vector $\boldsymbol{x} \in \mathbb{R}^m$. If $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, the product $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}$ is a new row matrix $\boldsymbol{b}^\mathsf{T} \in \mathbb{R}^{1 \times n}$. The row picture dictates that $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}$ is a linear combination of the *rows* of the matrix $\boldsymbol{A}$.

Because the column and row pictures can initially be abstract and hard to grasp, we illustrate them in Figure 2.7 using example matrices

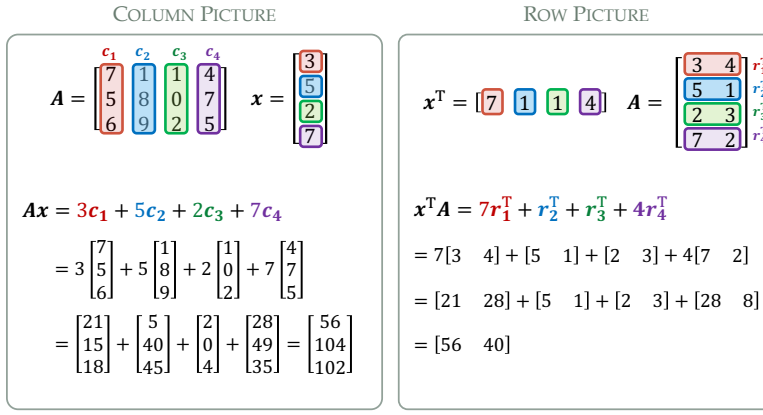


Fig. 2.7 Illustration of matrix multiplication using the column and row pictures. On the left, we have $\boldsymbol{Ax}$, where the product (a column vector $\boldsymbol{b}$) is the linear combination of the columns of $\boldsymbol{A}$. On the right, we have $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}$, where the product (a row vector $\boldsymbol{b}^{\mathsf{T}}$) is a linear combination of the rows of $\boldsymbol{A}$.

with color-coded rows and columns. We can fall back on our standard row-by-column multiplication definition (Eqn (2.9)) and satisfy ourselves that the column and row pictures mathematically guarantee the exact same numeric answers. To put it as a mnemonic: matrix multiplication computes a linear combination of the *columns* of the matrix on the *left*, or conversely, a linear combination of the *rows* of the matrix on the *right*.

These column and row pictures are, in fact, just special cases of the block-wise multiplication we discussed earlier. In the column picture, we are segmenting the first matrix into vertical blocks (columns), which perfectly conform for multiplication by the single-element blocks (scalars) of the vector. The sum of these individual products yields the linear combination of the columns. We can visualize the row picture as an identical, but horizontal, block-wise segmentation.

If, instead of a single vector $\boldsymbol{x}$, we multiplied by a multi-column matrix $\boldsymbol{B}$, the product would naturally have multiple columns. Every single column of the product would simply be a linear combination of the columns of $\boldsymbol{A}$, scaled by the elements of the corresponding column in $\boldsymbol{B}$. In other words, in $\boldsymbol{AB}$ (where $\boldsymbol{A} \in \mathbb{R}^{m\times k}$ and $\boldsymbol{B} \in \mathbb{R}^{k\times n}$), the

Other Operations



Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions** *Only $7.95. Scan, Click or Tap to buy.*

Available in the Full Version of the book.

Convolution



Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions** *Only $7.95. Scan, Click or Tap to buy.*

Available in the Full Version of the book.

Tensors and Tensor Products in Quantum Computing



Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions** *Only $7.95. Scan, Click or Tap to buy.*

Available in the Full Version of the book.

product has n columns, each of which is a unique linear combination of the columns of $\boldsymbol{A}$.

Considering this a valuable teachable moment, let's look at it from the rigorous perspective of block-wise multiplication once more. Let's think of $\boldsymbol{B}$ as being composed of n column matrices stacked side-by-side: $\boldsymbol{B} = [\boldsymbol{b_1} \mid \boldsymbol{b_2} \mid \cdots \mid \boldsymbol{b_n}]$. Then, by applying block-wise multiplication, we get:

$$\boldsymbol{AB} = \boldsymbol{A}[\boldsymbol{b_1} \mid \boldsymbol{b_2} \mid \cdots \mid \boldsymbol{b_n}] = [\boldsymbol{Ab_1} \mid \boldsymbol{Ab_2} \mid \cdots \mid \boldsymbol{Ab_n}]$$

This equation explicitly states that each ith column of the product is computed as $\boldsymbol{Ab_i}$. And as we just proved with the column picture, $\boldsymbol{Ab_i}$ is simply a linear combination of the columns of $\boldsymbol{A}$, strictly using the scalar coefficients provided by $\boldsymbol{b_i}$.

2.4.5 Inner and Outer Products

The vector dot product is strictly commutative: $\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{b} \cdot \boldsymbol{a}$. This property can be easily verified by examining its element-wise definition in Eqn (2.10). When written using our matrix multiplication notation, commutativity means that $\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \boldsymbol{b}^\mathsf{T}\boldsymbol{a}$.

However, note that general matrix multiplication by itself is *not* commutative. In particular, $\boldsymbol{a}^\mathsf{T}\boldsymbol{b} \neq \boldsymbol{a}\boldsymbol{b}^\mathsf{T}$ (although both multiplications are between dimensionally conformant matrices, and are therefore legally allowed). The latter product, $\boldsymbol{a}\boldsymbol{b}^\mathsf{T}$, is in fact not a scalar at all. If $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^k$, the result is a full $k \times k$ matrix.

This resulting matrix, $\boldsymbol{a}\boldsymbol{b}^\mathsf{T}$, is called the *outer product* (as opposed to the dot product, which is the *inner product*) of the two vectors. In computer science and statistics, the outer product is highly valuable because it computes a matrix of all possible pairwise interactions between the elements of the two vectors. This is the underlying engine for computing covariance matrices and deep learning attention mechanisms. The outer product possesses several unique properties that will become crucial to us later in the book.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

3

Transposes and Determinants

I would rather have questions that can't be answered than answers that can't be questioned.

—Richard Feynman



Now that we have defined matrices and mastered their basic operations, we turn to another fundamental manipulation: taking the transpose. We will also introduce the determinant–a single scalar value that encodes critical information about a matrix, including a beautiful geometric interpretation. Finally, we will explore the nomenclature of special matrices. Familiarizing ourselves with this vocabulary will be essential for the concepts we tackle in later chapters.

3.1 Transpose of a Matrix

We get the transpose of a matrix by flipping it over its *main diagonal*. Before defining it more formally, let's first look at an example in Eqn (3.1), where the main diagonal is highlighted in bold. The

transpose of a matrix is obtained, essentially, by switching its rows and columns.

$$\boldsymbol{A} = \begin{bmatrix} \mathbf{7} & 1 & 1 & 4 \\ 8 & \mathbf{8} & 0 & 7 \\ 6 & 9 & \mathbf{2} & 8 \end{bmatrix} \in \mathbb{R}^{3\times 4} \quad \boldsymbol{A}^{\mathsf{T}} = \begin{bmatrix} \mathbf{7} & 8 & 6 \\ 1 & \mathbf{8} & 9 \\ 1 & 0 & \mathbf{2} \\ 4 & 7 & 8 \end{bmatrix} \in \mathbb{R}^{4\times 3} \tag{3.1}$$

The *main diagonal* consists of the elements whose row number equals their column number: a_{ii}. Flipping a matrix over this diagonal means we take the element a_{ij} and move it to the position of a_{ji}.

To make future definitions easier to write, let's introduce a shorthand notation for a matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ as $\left[a_{ij}\right]$. In this context, a_{ij} represents a generic element in $\boldsymbol{A}$, with row index i ($1 \leq i \leq m$) and column index j ($1 \leq j \leq n$). With this notation established, we can state the formal definition of the transpose as follows:

Matrix Transpose

Definition: For any matrix $\boldsymbol{A} = \left[a_{ij}\right] \in \mathbb{R}^{m\times n}$, its transpose is defined as $\boldsymbol{A}^{\mathsf{T}} \overset{\text{def}}{=} \left[a_{ji}\right] \in \mathbb{R}^{n\times m}$.

From a computational perspective, taking a transpose does not necessarily mean we have to physically move all these elements around in computer memory. In highly optimized numerical libraries, transposing a matrix is often an $\mathcal{O}(1)$ operation; the software simply changes the underlying "strides" to read the data in column-major order rather than row-major order.

3.1.1 Properties of Transposes

From the definitions of transposes and the basic operations of matrices, we can prove the following properties.

1. **Transpose of a Transpose:** If we take the transpose twice, we get the original matrix back. Mathematically, this operation is an *involution*. In modern numerical libraries, calling a double transpose (like `A.T.T`) incurs zero computational cost, as the software simply toggles the array's memory view flags back to

their original state.

$$(\boldsymbol{A}^{\mathsf{T}})^{\mathsf{T}} = \boldsymbol{A}$$

2. **Transpose of a Sum:** The transpose of the sum of two matrices is the sum of the transposes of the individual matrices.

$$(\boldsymbol{A} + \boldsymbol{B})^{\mathsf{T}} = \boldsymbol{A}^{\mathsf{T}} + \boldsymbol{B}^{\mathsf{T}}$$

3. **Scalar Multiplication:** The operation of taking the transpose of a matrix commutes with scalar multiplication.

$$(s\boldsymbol{A})^{\mathsf{T}} = s\boldsymbol{A}^{\mathsf{T}}$$

4. **Transpose of a Scalar:** A scalar can be considered a matrix of one row and one column. Therefore, it is its own transpose. This is a crucial edge case to remember when writing algorithms involving vector dot products, which often output a 1×1 matrix.

$$s \equiv \begin{bmatrix} s \end{bmatrix} \implies s^{\mathsf{T}} = \begin{bmatrix} s \end{bmatrix}^{\mathsf{T}} = \begin{bmatrix} s \end{bmatrix} = s$$

3.1.2 Product Rule

The transpose of the product of two matrices is the product of the transposes, taken in the reverse order.

$$(\boldsymbol{A}\boldsymbol{B})^{\mathsf{T}} = \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}$$

We can prove this product rule by looking at the (i, j) element of the product matrices on the left and right-hand sides, although it is slightly tedious. Before diving into the formal proof, let's look at the dimensions of the matrices involved to see why this reverse-order rule is strictly necessary.

Let's consider $\boldsymbol{A} \in \mathbb{R}^{m \times k}$ and $\boldsymbol{B} \in \mathbb{R}^{k \times n}$ so that $\boldsymbol{A}\boldsymbol{B} \in \mathbb{R}^{m \times n}$. The dimensions must align this way due to the conformance requirement of matrix multiplication, which states that the number of columns of the first matrix must equal the number of rows of the second. Otherwise, the product is undefined. In particular, for $m \neq n$, $\boldsymbol{B}\boldsymbol{A}$ is not defined.

By the definition of the transpose, we have:

$$\boldsymbol{A} \in \mathbb{R}^{m \times k} \implies \boldsymbol{A}^{\mathsf{T}} \in \mathbb{R}^{k \times m} \quad \text{and} \quad \boldsymbol{B} \in \mathbb{R}^{k \times n} \implies \boldsymbol{B}^{\mathsf{T}} \in \mathbb{R}^{n \times k}$$

Notice that the number of *columns* of $\boldsymbol{B}^{\mathsf{T}}$ is exactly the same as the number of *rows* of $\boldsymbol{A}^{\mathsf{T}}$. Therefore, the product $\boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}$ is perfectly well-defined, while $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{B}^{\mathsf{T}}$ would trigger a dimension mismatch error (unless $m = n$). Furthermore, the resulting dimensions yield $\boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}} \in \mathbb{R}^{n\times m}$.

From the definition of the transpose, we also know that:

$$\boldsymbol{AB} \in \mathbb{R}^{m\times n} \implies (\boldsymbol{AB})^{\mathsf{T}} \in \mathbb{R}^{n\times m}$$

This perfectly matches the dimensions of the product of the transposes in reverse order: $\boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}} \in \mathbb{R}^{n\times m}$. Therefore, purely from a programmatic type-checking perspective, the product rule of transposes is the only way the operations remain strictly defined.

Let's illustrate this product rule using an example with two simple matrices, $\boldsymbol{A}$ and $\boldsymbol{B}$:

$$\boldsymbol{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \text{and} \quad \boldsymbol{B} = \begin{bmatrix} 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\boldsymbol{AB} = \begin{bmatrix} 34 & 40 \\ 79 & 94 \end{bmatrix} \implies (\boldsymbol{AB})^{\mathsf{T}} = \begin{bmatrix} 34 & 79 \\ 40 & 94 \end{bmatrix}$$

$$\boldsymbol{B}^{\mathsf{T}} = \begin{bmatrix} 3 & 5 & 7 \\ 4 & 6 & 8 \end{bmatrix} \quad \text{and} \quad \boldsymbol{A}^{\mathsf{T}} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

$$\boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}} = \begin{bmatrix} 34 & 79 \\ 40 & 94 \end{bmatrix} = (\boldsymbol{AB})^{\mathsf{T}}$$

CS Connection: The Reverse Order Property

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

We have postponed the proof for the product rule of transposes for as long as possible. Now, we will present two proofs.

Proof 1: Here, we will use the row-column matrix multiplication of matrices to prove the product rule. In $\boldsymbol{C} = \boldsymbol{AB}$, where $\boldsymbol{A} \in \mathbb{R}^{m\times k}$ and $\boldsymbol{B} \in \mathbb{R}^{k\times n}$, let's denote $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}' = \left[a'_{ij}\right]$, $\boldsymbol{B}^{\mathsf{T}} = \boldsymbol{B}' = \left[b'_{ij}\right]$, and $\boldsymbol{C}^{\mathsf{T}} = \boldsymbol{C}' = \left[c'_{ij}\right]$.

(1) Row-column matrix multiplication:	$c_{ij} = \sum_{p=1}^{k} a_{ip} b_{pj}$
(2) Definition of transpose:	$c'_{ij} = c_{ji}$
(3) Using (1), and switching $i \leftrightarrow j$:	$c_{ji} = \sum_{p=1}^{k} a_{jp} b_{pi}$
(4) Definition of transpose:	$= \sum_{p=1}^{k} a'_{pj} b'_{ip}$
(5) Rearranging scalar terms:	$c'_{ij} = \sum_{p=1}^{k} b'_{ip} a'_{pj}$
(6) Recognizing (5) as a matrix product:	$\boldsymbol{C'} = \boldsymbol{B}^{\mathsf{T}} \boldsymbol{A}^{\mathsf{T}}$

CS Connection: Procedural vs. Declarative Matrix Operations

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Proof 2: Here, we will use the dot-product view of matrix multiplication to prove the product rule. In $\boldsymbol{C} = \boldsymbol{AB}$, let's denote $\boldsymbol{C}^{\mathsf{T}} = \boldsymbol{C'}$. Noting that the element c_{ij} of $\boldsymbol{C}$ is the dot product of the ith row of $\boldsymbol{A}$ and the jth column of $\boldsymbol{B}$, we can write:

(1) The element c_{ij} as a dot product:	$c_{ij} = \boldsymbol{a}_i \cdot \boldsymbol{b}_j$
(2) Definition of transpose:	$c'_{ij} = c_{ji}$
(3) Using (1), and switching $i \leftrightarrow j$:	$c_{ji} = \boldsymbol{a}_j \cdot \boldsymbol{b}_i$
(4) Since the dot product is commutative:	$c_{ji} = \boldsymbol{b}_i \cdot \boldsymbol{a}_j$
(5) Using (2) and definition of transposes:	$c'_{ij} = \boldsymbol{b}_i \cdot \boldsymbol{a}_j = \boldsymbol{b}'_i \cdot \boldsymbol{a}'_j$
(6) Writing (5) in matrix form:	$\boldsymbol{C'} = \boldsymbol{C}^{\mathsf{T}} = \boldsymbol{B}^{\mathsf{T}} \boldsymbol{A}^{\mathsf{T}}$

In step (5), we used the fact that the ith row of a matrix is identical to the ith column of its transpose.

CS Connection: Vectorization as a Paradigm Shift


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

In both proofs, we have successfully shown that $\boldsymbol{C}^{\mathsf{T}} = (\boldsymbol{AB})^{\mathsf{T}} = \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}$.

3.2 Definitions and Matrices with Special Properties

Main Diagonal: The line of elements in a matrix with the same row and column indexes is known as the main diagonal. It may also be referred to as the leading diagonal. In other words, it is the line formed by the elements a_{ii} in $\boldsymbol{A} = \begin{bmatrix} a_{ij} \end{bmatrix} \in \mathbb{R}^{m \times n}$. Note that the number of rows and columns in the matrix $\boldsymbol{A}$ do not have to be equal.

Square Matrix: When the number of rows in a matrix is the same as the number of columns, we call it a square matrix. If $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, then $\boldsymbol{A}$ is a square matrix.

Here is an interesting fact: For any matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, $\boldsymbol{AA}^{\mathsf{T}} \in \mathbb{R}^{m \times m}$ and $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} \in \mathbb{R}^{n \times n}$. Therefore, for any matrix (of any size), its product with its transpose (multiplying on the right or left) is always a square matrix.

Symmetric Matrix: A matrix is symmetric when its transpose is identical to itself. $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A} \implies \boldsymbol{A}$ is symmetric. This condition cannot be satisfied by a non-square matrix, and therefore all symmetric matrices are square matrices. For a symmetric matrix, $a_{ij} = a_{ji}$.

For a square matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, $\boldsymbol{A} + \boldsymbol{A}^{\mathsf{T}}$ is always a symmetric matrix, which is easily proved by the fact that the transpose of the sum of two matrices is the sum of their transposes.

CS Connection: Memory Optimization via Symmetry


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Skew-Symmetric Matrix: A matrix is skew-symmetric (also known as antisymmetric) when its transpose is the negative of itself. $\boldsymbol{A}^{\mathsf{T}} = -\boldsymbol{A} \implies \boldsymbol{A}$ is skew-symmetric. Like the previous case, this condition cannot be met by a non-square matrix, and therefore all skew-symmetric matrices are square matrices. For a skew-symmetric matrix, $a_{ij} = -a_{ji}$.

For a square matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $\boldsymbol{A} - \boldsymbol{A}^{\mathsf{T}}$ is always a skew-symmetric matrix, which can be proven using the same technique as for $\boldsymbol{A} + \boldsymbol{A}^{\mathsf{T}}$ being symmetric.

Gram Matrix: Another interesting fact: For any matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, what is $(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{\mathsf{T}}$? By the product rule of matrix transposes,

$$(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{\mathsf{T}} = \boldsymbol{A}^{\mathsf{T}}(\boldsymbol{A}^{\mathsf{T}})^{\mathsf{T}} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$$

In other words, $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ is symmetric. So is $\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}$, by the same argument. The matrix $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ is called the Gram Matrix.

CS Connection: The Ubiquity of Gram Matrices


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Diagonal Matrix: A matrix that has zero elements everywhere other than the main diagonal elements is called a diagonal matrix. It is usually a square matrix, but we can call a non-square matrix a diagonal matrix without ambiguity because we have a clear definition for the main diagonal. Using symbols, $\boldsymbol{A} = \left[a_{ij}\right] \in \mathbb{R}^{m\times n}$ is a diagonal matrix if $a_{ij} = 0$ for all $i \neq j$. Note that we do not specify the values of the diagonal elements in any way: They may or may not be zero. In particular, a zero matrix (with all zero elements) is also a diagonal matrix.

Identity Matrix: The identity matrix is a square, diagonal matrix with ones along the diagonal and zeros everywhere else. $\boldsymbol{A} = \boldsymbol{I}$

if and only if $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $a_{ii} = 1$, and $a_{ij} = 0$ for $i \neq j$. We use the symbol $\boldsymbol{I}$ or $\boldsymbol{I}_n$ to refer to the identity matrix.

Unit Vectors: We can think of the columns of the identity matrix as unit vectors defining directions in space. For $\boldsymbol{I}_3 \in \mathbb{R}^{3\times 3}$, the unit vectors ($\hat{i}, \hat{j}$, and $\hat{k}$, as some physicists denote them) would be:

$$\boldsymbol{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \hat{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \hat{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \hat{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Now, we can write any general vector, say,

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad \text{as} \quad \boldsymbol{x} = x\hat{i} + y\hat{j} + z\hat{k}$$

Note that the unit vectors, as defined here, all have unit length. They are also perpendicular to each other because the dot product of any distinct pair of them is zero.

Upper and Lower Triangular Matrices: An upper triangular matrix ($\boldsymbol{U} = \begin{bmatrix} u_{ij} \end{bmatrix} \in \mathbb{R}^{m\times n}$) is one with all the elements *below* the main diagonal equal to zero.

$$u_{ij} = 0 \text{ for } i > j$$

Similarly, a lower triangular matrix ($\boldsymbol{L} = \begin{bmatrix} l_{ij} \end{bmatrix} \in \mathbb{R}^{m\times n}$) is one with all the elements *above* the main diagonal equal to zero.

$$l_{ij} = 0 \text{ for } i < j$$

Here are some examples:

$$\boldsymbol{L} = \begin{bmatrix} a & 0 \\ c & d \end{bmatrix} \in \mathbb{R}^{2\times 2} \quad \boldsymbol{U} = \begin{bmatrix} a & b \\ 0 & d \end{bmatrix} \in \mathbb{R}^{2\times 2}$$

Again, note that we do not specify anything about nonzero elements in the definitions. They can be zeros as well, and a diagonal matrix, in principle, is both an upper and a lower triangular matrix at the same time.

Morphisms


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

CS Connection: The Efficiency of Triangular Matrices


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Inverse of a Matrix: A (square) matrix ($\boldsymbol{A}$), when multiplied by its inverse ($\boldsymbol{A}^{-1}$) will result in the identity matrix ($\boldsymbol{I}$): $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$. For non-square matrices, we can have *left* and *right* inverses, but they will be different from each other.

Singular Matrix: Not all square matrices have inverses. If a matrix is *noninvertible*, it is called singular.

3.3 Determinant of a Matrix

The determinant of a square matrix is a scalar value derived from its elements. It contains a massive amount of information about the matrix and the linear transformation (see §1.4.2, page 18) that it represents. We use the symbol $|\boldsymbol{A}|$, $\Delta\boldsymbol{A}$, or $|\boldsymbol{A}|$ to denote the determinant of the square matrix $\boldsymbol{A}$.

Before actually defining the determinant, let's formally state what we said in the previous paragraph:

$$|\boldsymbol{A}| = f(a_{ij}) \quad \text{where} \quad \boldsymbol{A} \in \mathbb{R}^{n\times n}$$

This states that the determinant is a function (a hitherto unknown one) of the elements of the square matrix. We will soon specify exactly what this function is. Before defining it, however, let us list some interesting facts about the determinant.

Thinking of $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ as a linear transformation, we can say that $|\boldsymbol{A}| = 0$ means that it is *not* a one-to-one mapping or an isomorphism. In other words, $\boldsymbol{A}$ takes multiple vectors in $\mathbb{R}^n$ to the *same*

transformed vector (again in $\mathbb{R}^n$). See the box titled "**Morphisms**" for the formal nomenclature and some more details.

If the determinant is not zero, then $\boldsymbol{A}$ represents an isomorphism, and we can invert its transformation. We can find another matrix to do the reversal, and we will call it the inverse of $\boldsymbol{A}$, denoting it using the symbol $\boldsymbol{A}^{-1}$, which is indeed the same inverse we defined on page 56. Transformations that are not isomorphisms cannot be reversed, and the corresponding matrices are noninvertible; they are singular matrices. To use the right mathematical lingo, the determinant $|\boldsymbol{A}|$ being nonzero is a necessary and sufficient condition for the matrix $\boldsymbol{A}$ to be invertible.

CS Connection: Determinants, Condition Numbers, and Reality


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

As we shall see later, when the matrix $\boldsymbol{A}$ is invertible, the system of linear equations $\boldsymbol{Ax} = \boldsymbol{b}$ has a unique solution. We can think of this characteristic of the determinant as an algebraic property because it says something about the solutions of linear equations. In addition, we will find a geometric property as well, stating that the determinant behaves like the volume of a parallelepiped that the matrix represents, which we will explore once we define what the determinant actually is. A parallelepiped, by the way, is the higher-dimensional generalization of a parallelogram, which is essentially a sheared rectangle in 2-D.

3.3.1 2×2 Matrices

If we have a matrix $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$ as in the equation below, its determinant is defined as $|\boldsymbol{A}| = ad - bc$. To restate it formally,

$$\boldsymbol{A} = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \in \mathbb{R}^{2\times 2} \quad |\boldsymbol{A}| = \begin{vmatrix} a & c \\ b & d \end{vmatrix} \stackrel{\text{def}}{=} ad - bc \in \mathbb{R} \tag{3.2}$$

We can think of $\boldsymbol{A}$ as having two column vectors ($\boldsymbol{c}_1$ and $\boldsymbol{c}_2$) standing side-by-side.

$$\boldsymbol{A} = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \in \mathbb{R}^{2\times 2} \quad \boldsymbol{c}_1 = \begin{bmatrix} a \\ b \end{bmatrix}, \boldsymbol{c}_2 = \begin{bmatrix} c \\ d \end{bmatrix} \in \mathbb{R}^2$$

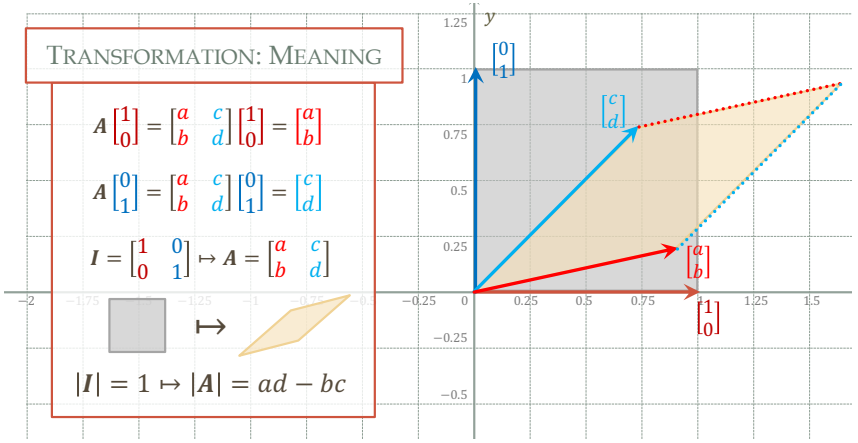


Fig. 3.1 The matrix $\boldsymbol{A}$ transforms the unit vectors to its columns, $\boldsymbol{a}_1$ and $\boldsymbol{a}_2$, thus transforming the unit square to a parallelogram. Interactive Visualization

How does $\boldsymbol{A}$ transform the unit vectors? (The ith unit vector is the ith column of the identity matrix $\boldsymbol{I}$). The transformed versions are just $\boldsymbol{c}_1$ and $\boldsymbol{c}_2$, the columns of $\boldsymbol{A}$.

$$\boldsymbol{A}\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}a & c\\b & d\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}a\\b\end{bmatrix} = \boldsymbol{c}_1 \quad \boldsymbol{A}\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}a & c\\b & d\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}c\\d\end{bmatrix} = \boldsymbol{c}_2$$

It is easiest to use the column picture of matrix multiplication (§2.4.4, page 44) to understand this fact: The first product above is a trivial linear combination of the columns of $\boldsymbol{A}$, taking one of the first column and zero of the second column ($1 \times \boldsymbol{c}_1 + 0 \times \boldsymbol{c}_2$), giving us the first column $\boldsymbol{c}_1$ of $\boldsymbol{A}$ back. Similarly, the second unit vector transforms to the second column $\boldsymbol{c}_2$ of $\boldsymbol{A}$.

As we can see in Figures 3.1 and 3.2, the transformation that $\boldsymbol{A}$ performs on the unit square with vertices at $(0,0)$, $(1,0)$, $(1,1)$, and $(0,1)$ takes it to a parallelogram with vertices at $(0,0)$, (a,b), $(a+c, b+d)$, and (c,d). And the area of this parallelogram, as Figure 3.2 proves without words[1], is indeed the determinant as defined in Eqn (3.2). In $\mathbb{R}^n$, instead of the area, we get the (signed) volume of a hyper-parallelepiped. Note that we have proved only the absolute

[1] The proof is recreated from Mathematics StackExchange, attributed to Solomon W. Golomb. If the geometrical version of the proof is hard to digest, there is an algebraic version as well.

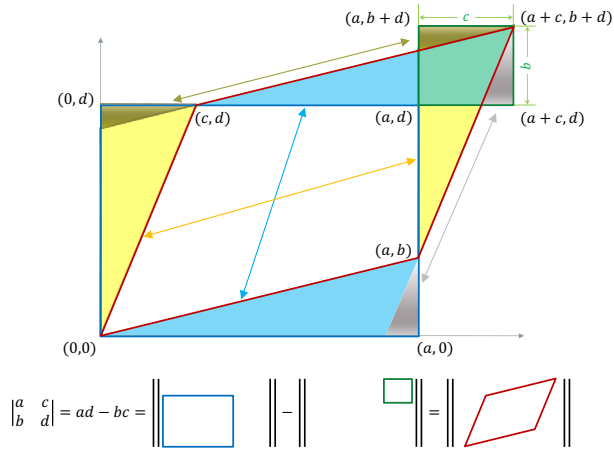


Fig. 3.2 The parallelogram that results from the transformation of the unit square by the action of $\boldsymbol{A}$. The picture proves, without words, that its area is the same as $|\boldsymbol{A}|$.

value of the area in Figure 3.2; we will learn more about the sign properties in the following examples.

In Figure 3.3, we have three different examples of $\boldsymbol{A}$ in $\mathbb{R}^2$ and their determinants. In the left panel, we see how $\boldsymbol{A}$ transforms the red and blue unit vectors (which have unit elements in the first and second dimension respectively). The red unit vector gets transformed to the first column of $\boldsymbol{A}$ (shown in a lighter shade of red), and the blue one to the second column (light blue vector). If we complete the parallelogram, its area is 2, which is the determinant, $|\boldsymbol{A}|$.

In the middle panel of Figure 3.3, we have a different $\boldsymbol{A}$, which does something strange to the red and blue unit vectors: They get transformed to a single line, which is a collapsed parallelogram with zero area. And by the definition of $|\boldsymbol{A}|$, it is indeed zero. Looking at the column vectors in $\boldsymbol{A}$, we can see that they are scalar multiples of each other; they are not linearly independent.

In the right panel of the same figure, we have shuffled the columns of $\boldsymbol{A}$, so that the parallelogram is identical to the one in the left panel, but the determinant now is negative. We, therefore, call the determinant the *signed* area of the parallelogram formed with the column vectors as adjacent sides. Why is the area negative in this case? It is because $\boldsymbol{A}$ has flipped the order of the transformed vectors: Our blue unit vector is to the left of the red one. In the left panel,

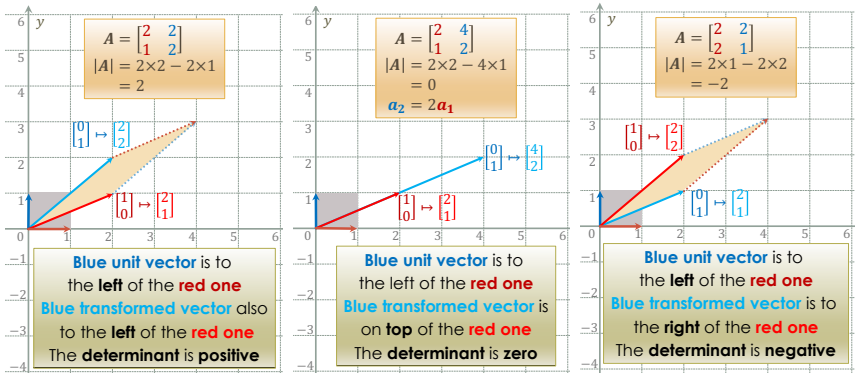


Fig. 3.3 Determinants as areas: The matrix $\boldsymbol{A}$ transforms the unit square (shaded grey) into the amber parallelogram. The determinant $|\boldsymbol{A}|$ is the signed area of this parallelogram. The sign is negative when the transformed unit vectors "flip." Interactive Visualization

the transformed blue vector is still to the left. But in the right panel, the blue one has gone to the *right* of the red one after transformation, thereby attributing a negative sign to the determinant.

To use a more formal language, to go from the first (red) unit vector to the second (blue) one, we go in the counterclockwise direction. The direction is the same for the transformed versions in the left panel of Figure 3.3, in which case the determinant is positive. So is the area. In the right panel, the direction for the transformed vectors is clockwise, opposite of the unit vectors. In this case, the determinant and the signed area are negative.

3.3.2 3×3 Matrices

We have defined the determinant of a 2×2 matrix in Eqn (3.2). We now extend it to higher dimensions recursively. For $\boldsymbol{A} \in \mathbb{R}^{3\times3}$, we have:

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \in \mathbb{R}^{3\times3} \tag{3.3}$$

$$|\boldsymbol{A}| \overset{\text{def}}{=} a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix} \in \mathbb{R}$$

Since each of the 2×2 determinants in Eqn (3.3) is defined in Eqn (3.2), we have a recursive formula for the determinant of $\boldsymbol{A} \in$

$\mathbb{R}^{3\times3}$. Notice that the first term has a positive sign, the second one a negative sign, and the third a positive sign again. This pattern of alternating signs extends to higher dimensions as well.

CS Connection: Recursive Determinants and Base Cases



Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions** *Only $7.95. Scan, Click or Tap to buy.*

CS Connections are available in the Full Version of the book.

Minors and Cofactors

The first submatrix whose determinant appears in Eqn (3.3) (multiplying a_{11}) is obtained by removing the first row and the first column from $\boldsymbol{A}$. More generally, the term multiplying the element a_{ij} is the determinant of the submatrix obtained by removing the ith row and the jth column. The determinants of these smaller submatrices are called the *minors* of $\boldsymbol{A}$, denoted by M_{ij}.

A minor paired with its associated alternating sign is called a *cofactor*, defined mathematically as $C_{ij} = (-1)^{i+j}M_{ij}$.

What we did in Eqn (3.3) was expand the determinant along the first row. However, we could have done it along the first column as well. In fact, we can compute the determinant by expanding along *any* row or column and summing up the resulting element-cofactor products. With these definitions of minors and cofactors, we can rewrite Eqn (3.3) much more compactly. For example, expanding $|\boldsymbol{A}|$ along the ith row yields:

$$|\boldsymbol{A}| = \sum_{j=1}^{3}(-1)^{i+j}a_{ij}M_{ij} = \sum_{j=1}^{3}a_{ij}C_{ij} \tag{3.4}$$

CS Connection: Optimizing Determinants for Sparse Data



Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions** *Only $7.95. Scan, Click or Tap to buy.*

CS Connections are available in the Full Version of the book.

Determinant as Volume

For 2×2 matrices, we saw that the determinant was the (signed) area of the parallelogram to which the unit square transformed. In the 3×3 case, it becomes the volume of the parallelepiped that is the transformed version of the unit cube. It is also signed: If two unit vectors flip orientation when transformed, the determinant gets multiplied by -1. If one more unit vector flips, the sign reverts back to the original.

CS Connection: The Jacobian and Probabilistic Volume

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

3.3.3 $n \times n$ Matrices

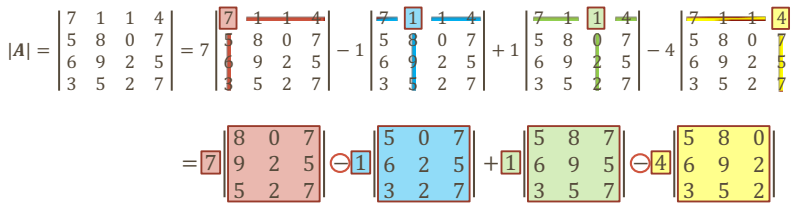


Fig. 3.4 Illustration of minors and cofactors using an example 4×4 matrix. The minor is the determinant of the submatrix obtained by removing the row and column corresponding to each element, as shown. Notice the sign in the summation. The minor with the associated sign is the cofactor.

We can extend the notion of volume to $\mathbb{R}^n$. If we think of $\boldsymbol{A}$ as being composed of n column vectors,

$$\boldsymbol{A} = \begin{bmatrix} | & | & & | \\ \boldsymbol{c}_1 & \boldsymbol{c}_2 & \cdots & \boldsymbol{c}_n \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n \times n} \quad \boldsymbol{c}_i \in \mathbb{R}^n$$

the determinant, $|\boldsymbol{A}|$, is the *signed volume* of the n-dimensional parallelepiped formed with edges $\boldsymbol{c}_i$.

Laplace Formula

What we wrote down in Eqn (3.4) is, in fact, the general version of the recursive formula for computing the determinant, expanding over the ith row:

$$|\boldsymbol{A}| = \sum_{j=1}^{n} (-1)^{i+j} a_{ij} M_{ij} = \sum_{j=1}^{n} a_{ij} C_{ij} \tag{3.5}$$

This is the Laplace formula (or Laplace expansion). We could do the expansion over the jth column as well:

$$|\boldsymbol{A}| = \sum_{i=1}^{n} (-1)^{i+j} a_{ij} M_{ij} = \sum_{i=1}^{n} a_{ij} C_{ij}$$

In fact, one of the fundamental properties of determinants is that they are invariant under transposition. In other words, expanding by rows yields the exact same result as expanding by columns, meaning $|\boldsymbol{A}| = \left|\boldsymbol{A}^{\mathsf{T}}\right|$.

CS Connection: Why We Avoid Laplace Expansion

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

3.4 Properties of Determinants

We have touched upon some of the characteristics and properties of determinants in the previous sections. Let's list them all here formally for completeness.

1. Identity matrices (of any dimension, $\boldsymbol{I} \in \mathbb{R}^{n \times n}$) have a determinant of one.

   $$|\boldsymbol{I}| = 1$$

   This property is consistent with the fact that a hypercube of unit sides has a unit (hyper)volume.

2. If we exchange two rows (or two columns), the determinant changes sign. As a consequence, it should be impossible to get

Deriving $|\boldsymbol{A}|$

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
Available in the Full Version of the book.

back to the exact same matrix by performing an odd number of row or column exchanges.

3. If we multiply a single row (or a single column) by a scalar s, the determinant gets multiplied by the same factor:

$$\boldsymbol{A} = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \quad \boldsymbol{A}_1 = \begin{bmatrix} sa & sc \\ b & d \end{bmatrix} \quad \boldsymbol{A}_2 = \begin{bmatrix} sa & c \\ sb & d \end{bmatrix}$$

$$\implies |\boldsymbol{A}_1| = |\boldsymbol{A}_2| = s\,|\boldsymbol{A}|$$

Since determinants behave like volumes, we can see that if we double one side of a hypercube or a parallelepiped, its volume gets doubled. If we double *all* sides (multiplying the entire matrix by s), then its volume gets multiplied by s^n.

CS Connection: Stability via the Log-Determinants

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
CS Connections are available in the Full Version of the book.

4. If we can express all elements in one row (or a column) as sums of two numbers each, we can split the whole determinant as the sum of two determinants:

$$\begin{vmatrix} a+a' & c+c' \\ b & d \end{vmatrix} = \begin{vmatrix} a & c \\ b & d \end{vmatrix} + \begin{vmatrix} a' & c' \\ b & d \end{vmatrix}$$

5. If one row (or column) is all zeros, then the determinant is zero. This statement should be self-evident from the formal definition of determinants (Eqn (3.5)): We simply expand the Laplace formula over the row or column containing all zeros to instantly get $|\boldsymbol{A}| = 0$.

6. The determinant does not change if we add or subtract any multiple of one row (or column) from any other row (or column). While this property may sound a bit mysterious, it corresponds to the fact that matrix equations (such as $\boldsymbol{Ax} = \boldsymbol{b}$) actually encode systems of linear equations, which we can add and subtract without affecting their solvability.

7. If two rows (or columns) of the matrix are identical, the determinant is zero, which is a direct corollary of the previous two properties.

8. For a triangular matrix (e.g., an upper triangular matrix, $\boldsymbol{U}$: $u_{ij} = 0$ for $i > j$), the determinant is simply the product of its main diagonal elements. We can appreciate the veracity of this property by expanding the determinant over the first column for an upper triangular matrix, or the first row of a lower triangular matrix.

CS Connection: High-Performance Determinants


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9. If the determinant is zero, the matrix cannot be inverted. We have not fully defined or discussed matrix inversion yet, but the inverse of a matrix is something that mathematically reverses its transformation. If $\boldsymbol{Ax} = \boldsymbol{b}$, and $\boldsymbol{x} = \boldsymbol{A^{-1}b}$, we call $\boldsymbol{A^{-1}}$ the inverse of $\boldsymbol{A}$. What this property guarantees is that if $|\boldsymbol{A}| = 0$, we cannot find $\boldsymbol{A^{-1}}$. As we saw earlier, such noninvertible matrices are called singular matrices, and they have deep implications for much of what we will learn from now on.

10. The determinant of the product of two matrices (of the same size) is the product of their determinants: $|\boldsymbol{AB}| = |\boldsymbol{A}|\,|\boldsymbol{B}|$.

11. The determinant of the transpose of a matrix is the same as that of the original matrix ($\left|\boldsymbol{A^T}\right| = |\boldsymbol{A}|$). We could have used this final property to avoid repeatedly specifying "or column" in most of the rules above.

3.5 Summary: Numerical Computations

This first part of the book (comprising Chapters 1, 2, and 3) was originally framed around numerical computations involving vectors and matrices. However, the moment we start speaking of vectors, we are already thinking in geometrical terms. In this chapter, we saw how even a purely numerical value like the determinant has a profound geometric meaning.

In the first chapter, to provide motivation for the idea of matrices, we introduced linear equations. We will expand upon this in the next part of the book, which focuses extensively on the algebraic view of linear algebra. As we delve deeper into systems of linear equations, we will fully realize the connections between determinants and system solvability that we have only hinted at so far.

As we can see, although we might initially try to keep algebra and geometry separated, it is neither possible nor advisable to do so. They are simply different, complementary views of the exact same underlying subject of linear algebra.

CS Connection: The Duality of Geometric and Algebraic Thinking


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Part II

Algebraic View

4

Systems of Linear Equations

Confidence is what you have before you understand the problem.

—Woody Allen



4.1 When Can We Solve Linear Equations?

If we have a system of linear equations, we may or may not have solutions—we may have a unique solution or an infinite number of solutions. As we will see very soon, it is not easy to determine the solvability of a system even for a relatively small number of equations. We can, of course, perform a series of manipulations on the equations (like adding them, eliminating variables, substituting the solved ones back, *etc.*) to arrive at their solutions. How do we get a computer to solve the system of equations, though? We need an algorithm, both to determine the solvability conditions and to actually find the solutions. Such an algorithm is called *Gaussian elimination* or *row reduction*.

Table 4.1 Various permutations of simultaneous equations in two variables, showing solvability

	Equations	Solution	Comment
1	$x + y = 5$ $x - y = 1$	$x = 3$ $y = 2$	Unique solution
2	$x + y = 5$	$x = t$ $y = 5 - t$	Infinity of solutions Only one equation
3	$x + y = 5$ $2x + 2y = 10$	$x = t$ $y = 5 - t$	Infinity of solutions Really, only one equation
4	$x + y = 5$ $x + y = 1$	–	No solutions Inconsistent equations
5	$x + y = 5$ $x - y = 1$ $3x - y = 7$	$x = 3$ $y = 2$	Unique solution Really only two equations
6	$x + y = 5$ $x - y = 1$ $3x - y = 9$	–	No solutions Inconsistent equations

Let's start with a simple system of two linear equations, and see what the problem really is. Here, we will have only two variables, x and y. We are told that if we have two equations and two unknowns, we can solve them, as though the equality of the number of variables to the number of equations is the necessary and sufficient condition for solvability. However, as Table 4.1 shows, it is only part of the story, and not always true.

In the first row of the table, we see the ideal case: We have two good equations for our two unknowns, and we can easily find their solutions.

In the second row, we have too few equations. What happens is not that we do not have a solution. Any x and y satisfying the one equation we have is a solution, and we have infinitely many of them.

In the third row of Table 4.1, we do have two equations, but the second equation is *derived* from the first and is therefore not

independent. In effect, we have only one equation and an infinite number of solutions, as in the second row.

In the fourth row, we have two equations, but they are not *consistent* with each other. They cannot both be true at the same time for any pair of values of x and y.

Things get complicated in the fifth row, where we seem to have three equations. But the third one can be derived from the first two. It is, in fact, Eq. $1 + 2 \times$ Eq. 2, which means, in reality, we only have two good equations for two unknowns. We therefore get a good solution, much like the first row of Table 4.1.

The sixth row looks similar to the fifth, but the third equation there is different on the right-hand side. It cannot be derived from the other two, and is inconsistent with them. Therefore, we have no solutions.

In light of these results, we can state the solvability condition, in a general case as follows: If we have a system of n *independent* and *consistent* linear equations in n unknowns, we can find a *unique* solution for them. We are yet to define the concepts of independence and consistency, though.

Independence

Definition: An equation in a system of linear equations is considered independent if it cannot be derived from the rest using algebraic manipulations.

If we multiply an equation by a scalar, or add two equations, the new equation we get is *not* independent. Again, notice the similarity of the dependence of equations to our requirements for linearity (§1.3, page 16).

Consistency

Definition: An inconsistent system of linear equations is one with no solutions.

The concept of consistency is harder to pin down. For now, we are defining it rather cyclically, as in the statement above. It is possible, however, to visualize why some equations are inconsistent

with others. In the fourth row of Table 4.1, for instance, the lines described by the two equations are parallel to each other.

4.1.1 Visualizing Equations

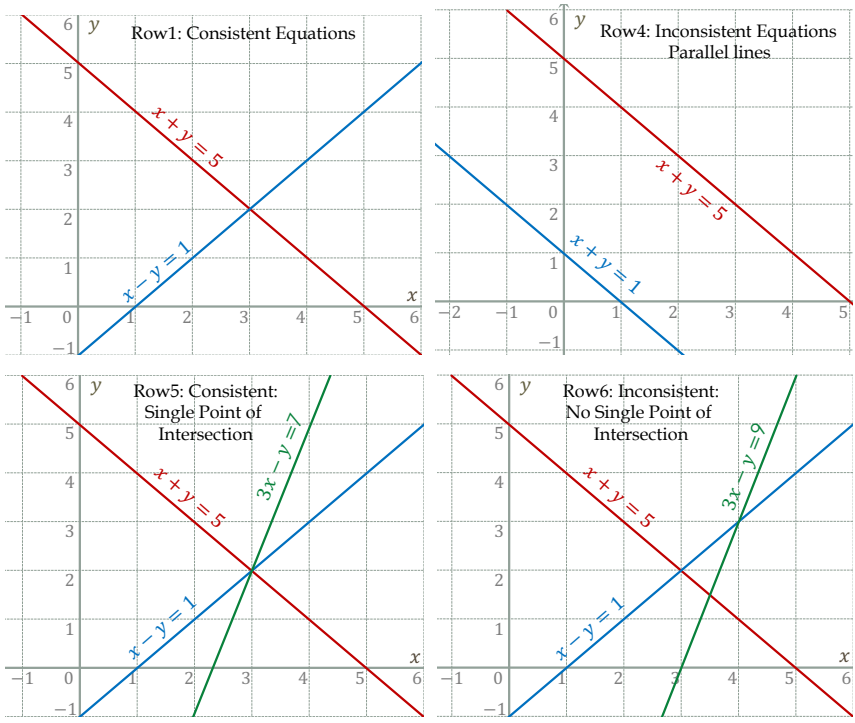


Fig. 4.1 Visualizing equations listed in Table 4.1. Clockwise from top-left: Row 1, 4, 5 and 6 in the table.

In Table 4.1, our equations represent lines because we have only two variables (x and y) and we are dealing with $\mathbb{R}^2$, which is a plane. In Figure 4.1, we can see the lines corresponding to the equations in the various rows in the table. For the first row, we have two good (independent and consistent) equations, corresponding to the red and blue lines in the top-left panel. This system has a nice solution, where the two lines intersect, at $x = 3, y = 2$. When people say "as many equations as unknowns," what they have in mind is this type of independent and consistent system.

In the second row of Table 4.1, we have only one equation, which corresponds to the red one in all panels of Figure 4.1. Every point on the line is a solution to the equation, which goes for the third row too.

The fourth row of the table is shown in the top-right panel of Figure 4.1 as two parallel lines, which will never meet. No point on the red line will ever be a point on the blue line, and the equations do not have a solution, which is why they are inconsistent.

In the fifth row of Table 4.1, we have an extra third equation, shown as a green line in the bottom-left panel of Figure 4.1. It is consistent with the other two equations because it goes through the same solution point $x = 3, y = 2$. The equations are, in fact, not independent; any one of them can be derived from the other two. The three equations are consistent with each other, and any two of them would be enough to fully solve the system.

The sixth row shows the case where the third equation is not consistent. Note that in this case, we do not have parallel lines, but three lines not having a common point (which would have been the solution). For future use, it may be interesting to wonder what an approximate, or *best possible*, solution would be. Is the centroid of the triangle formed by the three lines, perhaps, a good candidate as the best possible solution?

4.1.2 Generalizing to $\mathbb{R}^n$

The representation of the one-dimensional space $\mathbb{R}$, as we learn in school, is the number line. An equation, such as $x = 1$, or, in general, $ax = b$, defines a point on this line.

When we move to $\mathbb{R}^2$, the same equation, $x = 1$, defines a vertical line. In general, however, we have an equation $a_1x + a_2y = b$, which defines a line with a slope. If we have two such equations, then we get two lines, with the possibility of them intersecting and giving us a solution to the system of the two equations.

Similarly, in $\mathbb{R}^3$, a single linear equation, such as $x = 1$, defines a plane. It is a plane parallel to the yz plane, at unit distance from it. $x = 0$ would be the yz plane. A general equation of the form $a_1x + a_2y + a_3z = b$ gives us a plane with some orientation. Another linear equation defines another plane. If the equations are consistent and independent, the planes will intersect, giving us the solution, which is a line. Since the solution is a line, we have an infinity of

Table 4.2 Properties and behavior of linear equations and solutions

Equations	$\mathbb{R}^2$	$\mathbb{R}^3$	$\mathbb{R}^n$ $(n > 3)$
One equation	A line Infinite solutions	A plane Infinite solutions	An $(n-1)$-subspace Infinite solutions
Two independent and consistent equations	A point of intersection Unique solution	A line of intersection Infinite solutions	An $(n-2)$-subspace as intersection. ∞ solutions
Three independent and consistent equations	Cannot happen	A point of intersection Unique solution	An $(n-3)$-subspace as intersection. ∞ solutions
n independent and consistent equations	Cannot happen	Cannot happen	A point of intersection. Unique solution
Two independent, but inconsistent equations	Parallel lines No intersection No solutions	Parallel planes No intersection No solutions	Parallel subspaces No intersection No solutions
Three independent, but inconsistent equations	Lines that make $\|\|\|, \nparallel, \triangle$[1] No solutions	Planes making $\|\|\|, \nparallel, \triangle$ in cross section[2] No solutions	Hard to visualize No common intersection No solutions
n independent, but inconsistent equations	Lines that make $\|\|\|, \nparallel, \triangle$[3] No solutions	Planes making $\|\|\|, \nparallel, \triangle$ in cross section[4] No solutions	Hard to visualize No common intersection No solutions

[1,3] These symbols represent three parallel lines, two parallel lines plus an intersecting line, or three lines making a triangle.
[2,4] These symbols represent three planes, which, when sliced perpendicularly by a plane, show up as three parallel lines, two parallel lines plus an intersecting line, or three lines making a triangle. In other words, the three planes are parallel, two parallel with one intersecting, or the three making a triangular tube respectively.

solutions with two linear equations in $\mathbb{R}^3$. If we have one more linear equation, we have another plane, potentially intersecting this solution line at a point, giving us a unique solution.

CS Connection: Hyperplanes in Machine Learning

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

With this picture in mind, let's list the behavior of systems of linear equations and their solutions in $\mathbb{R}^2$, $\mathbb{R}^3$, and extrapolate to $\mathbb{R}^n$, as in Table 4.2.

Summarizing the insights from Table 4.2:

- The number of *independent* equations in a system of *consistent* linear equations can never be greater than the number of unknowns.

- If a system of linear equations has more *independent* equations than unknowns, it is necessarily *inconsistent* and has no solutions.

As we can see, the behavior of systems of equations is much more complicated than what we learned in high school. Furthermore, it looks hard to generalize to higher dimensions, and harder still to understand how a given system is going to behave. Imagine if we have 50 unknowns and 100 equations—a relatively small system that we may come across in our computer/data science career. Do they have solutions? Are the equations independent? Consistent? Are there too many? Or too few? We clearly need a systematic way, an algorithm, to tell us these things.

CS Connection: Overdetermined Systems in Data Science

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

4.1.3 Matrix as System of Linear Equations

The first step in developing an algorithm is to cast our linear equations in a compact matrix form. We, in fact, did this in Chapter 1 (§1.4, page 16), when we introduced vectors and matrices. Let's do it again, looking at the deceptively simple equation $\boldsymbol{Ax} = \boldsymbol{b}$ that will become the mainstay of our discussion for the rest of the book. Writing it out explicitly:

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & a_{ij} & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n} \quad \text{and} \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n, \tag{4.1}$$

$$\boldsymbol{Ax} = \boldsymbol{b}, \quad \text{where} \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \in \mathbb{R}^m$$

Now that we know matrix multiplication, we can see that $\boldsymbol{Ax} = \boldsymbol{b}$ represents a system of m linear equations of the form

$$a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n = b_i \quad \text{where } 1 \leq i \leq m$$

in n unknowns x_j, for $1 \leq j \leq n$.

CS Connection: Vectorization and High-Performance Computing


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

We can perform algebraic operations on these equations, such as adding or subtracting them, or multiplying them by a scalar. In fact, these manipulations are exactly the linear combinations we described in §2.1.3 (page 27). By formalizing these equation manipulations and casting them as operations we can perform on the coefficient matrix $\boldsymbol{A}$ in $\boldsymbol{Ax} = \boldsymbol{b}$, we arrive at the most foundational algorithm in linear algebra. It is called *Gaussian elimination* (also known as *row reduction*), and it is critical for determining the solvability of, and computing the exact solutions for, systems of linear equations.

4.2 Gaussian Elimination

Gaussian elimination is the algorithm to transform a matrix by applying a series of *elementary row operations* to arrive at its *row-echelon form* (REF). When working with systems of linear equations and their solutions, we apply Gaussian elimination to the *augmented matrix* of the system. (We will soon define the terms in *italics*.) Gaussian elimination, however, has applications other than solving equations, such as computing determinants, finding inverses, determining ranks of matrices, and so on.

4.2.1 Elementary Row Operations

Analogous to the algebraic manipulation of equations, let's first define a set of *elementary row operations* as listed below:

- Swap any two rows.
- Multiply any row by a scalar.
- Add a multiple of any row to another.

We will use these row operations to build an algorithm to solve the system of linear equations $\boldsymbol{Ax} = \boldsymbol{b}$, much the same way we would use the corresponding algebraic operations to solve the equations

symbolically. The advantage of the matrix formulation is that we are dealing with numbers, and we can program a computer to perform the operations once the algorithm is ready.

Do you notice the similarity between these elementary operations encoding algebraic manipulations of equations and some of the properties of determinants (§3.4, page 63)? We will make use of this similarity in using Gaussian elimination for determinant calculation.

CS Connection: Pivoting for Numerical Stability

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

4.2.2 Augmented Matrix

We have to keep in mind that what we are doing when performing row operations on $\boldsymbol{A}$ is basically the same as the algebraic manipulation of equations. Therefore, we have to apply the same operations to the right-hand side, $\boldsymbol{b}$, as well. For this reason, it may be best to add an extra column to $\boldsymbol{A}$ with the elements of $\boldsymbol{b}$. Such a matrix is called the *augmented matrix*. It is merely a convenient bookkeeping technique, which turns out to be useful when implementing row operations in a computer program.

$$\left[\boldsymbol{A} \mid \boldsymbol{b}\right] = \left[\begin{array}{ccc|c} a_{11} & \cdots & a_{1n} & b_1 \\ a_{21} & \cdots & a_{2n} & b_2 \\ \vdots & a_{ij} & \vdots & b_i \\ a_{m1} & \cdots & a_{mn} & b_m \end{array}\right] \tag{4.2}$$

Now that we are naming matrices, let's also call $\boldsymbol{A}$ (either as part of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ or by itself) the *coefficient matrix*, and the $\boldsymbol{b}$ part the *constant matrix* or *constant vector*.

Before describing the algorithm for Gaussian elimination, let's look at the endpoint of the algorithm, which is the form in which we would like to have our matrix $\boldsymbol{A}$ or $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$.

Row-Echelon Form

Definition: A matrix is considered to be in its row-echelon form (REF) if it satisfies the following two conditions:

1. All rows with zero elements are at the bottom of the matrix.
2. The *first nonzero element* in any row is strictly to the right of the first nonzero element in the row above it.

Pivots

Definition: The leading nonzero element in a row of a matrix in its row-echelon form is called a pivot. The corresponding column is called the pivot column. Pivots are also called the *leading coefficients*.

The largest number of pivots a matrix can have is the smaller of its dimensions (number of rows and columns). In other words, for $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, the largest number of pivots would be $\min(m, n)$.

Rank

Definition: The number of pivots of a matrix (in its REF) is its rank. We will have a better definition of rank later on. If a matrix has its largest possible rank (which is the largest possible number of pivots, $\min(m, n)$), it is called a *full-rank* matrix. If a matrix is not full-rank, we call it rank-deficient, and its rank deficiency is $\min(m, n) - \text{rank}(\boldsymbol{A})$.

CS Connection: Rank Deficiency in Machine Learning


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

We have a few examples of matrices in their REF in Eqn (4.3) below, where the pivots are shown in **bold**. The first matrix shows a square matrix of size 4×4, and it has four pivots, and is therefore full-rank. The second matrix is 2×3, and has two pivots—the largest possible number. It is also full-rank. The third one is a 4×4 matrix,

Gaussian Elimination: Algorithm



Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

but has only three pivots. It is rank-deficient by one. The fourth matrix also has a rank deficiency of one.

$$\begin{bmatrix} \mathbf{5} & 2 & 11 & 3 \\ 0 & \mathbf{3} & 7 & 13 \\ 0 & 0 & \mathbf{17} & 2 \\ 0 & 0 & 0 & \mathbf{13} \end{bmatrix} \begin{bmatrix} \mathbf{5} & 11 & 3 \\ 0 & 0 & \mathbf{13} \end{bmatrix} \begin{bmatrix} \mathbf{5} & 2 & 11 & 3 \\ 0 & 0 & \mathbf{17} & 2 \\ 0 & 0 & 0 & \mathbf{13} \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{5} & 2 & 11 & 3 \\ 0 & 0 & \mathbf{17} & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{4.3}$$

4.2.3 The Algorithm

With the definitions of REF, pivots, and ranks in place, we can state the Gaussian elimination algorithm, running on a matrix $\boldsymbol{A} = \begin{bmatrix} a_{ij} \end{bmatrix} \in \mathbb{R}^{m \times n}$ as follows:

1. If $a_{11} = 0$, loop over the rows of $\boldsymbol{A}$ to find the row that has a nonzero element in the first column.
2. If found, swap it with the first row. If not, ignore the first row and column and move on to the second row (calling it the first).
3. Multiply the first row by $-\frac{a_{i1}}{a_{11}}$ and add it to the ith row to get zeros in the first column of all rows other than the first one.
4. Now consider the submatrix from the second row, second column to the last row, last element (*i.e.*, from a_{22} to a_{mn}) as the new matrix:
$$\boldsymbol{A} \leftarrow \boldsymbol{A}[a_{22} : a_{mn}], m \leftarrow m - 1, n \leftarrow n - 1$$
5. Loop back to step 1 and perform all the steps until all rows or columns are exhausted.

Note that in step 3, we are subtracting a number after dividing it by a_{11}, which may cause numerical instability in the algorithm when $a_{11} \to 0$. For this reason, we may want to modify the first step to find the row with the largest absolute value in the first column. (In other words, modify the first step to read, "Loop over the rows of $\boldsymbol{A}$ to find the row that has the *largest absolute value* ($\neq 0$) in the first

column," which is the way Gaussian elimination is implemented in most programs such as **SageMath**.)

CS Connection: The Power of Partial Pivoting


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

4.3 Applications of Gaussian Elimination

Although the primary objective of Gaussian elimination is to solve systems of linear equations, we also use it for a couple of other purposes.

We saw that the rank of a matrix is the same as the number of pivots, which we can get directly from Gaussian elimination (also known as row reduction, as a reminder).

At the end of Gaussian elimination, we have an upper triangular matrix. If we start with a square matrix whose determinant we need to compute, we can get it (or its absolute value) by simply taking the product of the diagonal elements (which are the pivots) because the elementary row operations we perform in Gaussian elimination do not change the absolute value of the determinant.

CS Connection: The Algorithmic Cost of Determinants


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

For these applications, we can also perform Gaussian elimination by column (as column reduction) instead of by row. However, for the main application of solving a system of linear equations, we can do it only row-wise. Besides, column-wise Gaussian elimination is the same as row reduction on the transpose of the matrix anyway.

4.3.1 Solution to System of Linear Equations

To solve a system of linear equations, we apply the algorithm of Gaussian elimination to its augmented matrix. Let's first look at

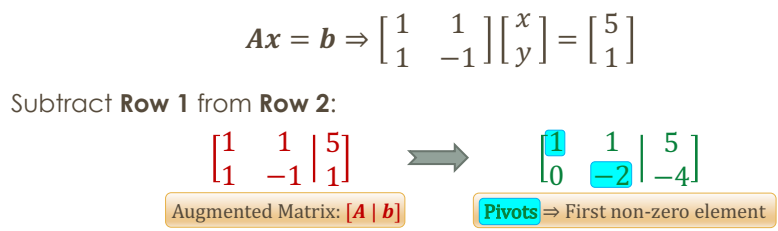


Fig. 4.2 Gaussian elimination on a simple augmented matrix, showing the pivots.

a simple example from the first row in Table 4.1. As shown in Figure 4.2, the system of linear equations $x + y = 5, x - y = 1$ translates to an augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ with two rows and single-step Gaussian elimination, after which, we get two pivots, 1 and -2. Although it is a simple system, we can make a few observations about it:

- The rank of the 2×2 matrix $\boldsymbol{A}$ is two, and it is a full-rank matrix.
- Since the coefficient matrix ($\boldsymbol{A}$) is square and full-rank, we can infer that the system has a unique solution.
- The determinant $|\boldsymbol{A}|$ is the product of the pivots $= -2$. It can also be calculated directly: $|\boldsymbol{A}| = a_{11}a_{22} - a_{21}a_{12} = (1 \times -1) - (1 \times 1) = -2$.
- The last row of the REF form of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ stands for the equation $-2y = -4$, which gives us $y = 2$.
- We can *back-substitute* this value of y in the second-to-last row (which is the first row) to solve for the other variable, giving us $x + 2 = 5 \implies x = 3$.

Back Substitution

Definition: The process of solving a system of linear equations from the REF of the augmented matrix of the system is known as back substitution. The equation corresponding to the last nonzero row is solved first, and the solution is substituted in the one for the row above, and so on, until all the variables are solved.

Table 4.3 Illustration of solvability conditions based on the characteristics of REF

	Equations	$[\boldsymbol{A} \mid \boldsymbol{b}]$	REF($[\boldsymbol{A} \mid \boldsymbol{b}]$)	Observations[1]
1	$x + y = 5$ $x - y = 1$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 1 & -1 & 1 \end{array}\right]$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 0 & -2 & -4 \end{array}\right]$	REF has no $0 = b_i \implies$ Solvable Rank = # Vars $\implies$ Unique Solution
2	$x + y = 5$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \end{array}\right]$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \end{array}\right]$	REF has no $0 = b_i \implies$ Solvable Rank < # Vars $\implies$ Infinity of Solns
3	$x + y = 5$ $2x + 2y = 10$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 2 & 2 & 10 \end{array}\right]$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 0 & 0 & 0 \end{array}\right]$	REF has no $0 = b_i \implies$ Solvable Rank < # Vars $\implies$ Infinity of Solns
4	$x + y = 5$ $x + y = 6$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 1 & 1 & 6 \end{array}\right]$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 0 & 0 & 1 \end{array}\right]$	REF has $0 = b_i \implies$ Inconsistency Not Solvable
5	$x + y = 5$ $x - y = 1$ $3x - y = 7$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 1 & -1 & 1 \\ 3 & -1 & 7 \end{array}\right]$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 0 & -2 & -4 \\ 0 & 0 & 0 \end{array}\right]$	REF has no $0 = b_i \implies$ Solvable Rank = # Vars $\implies$ Unique Solution
6	$x + y = 5$ $x - y = 1$ $3x - y = 9$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 1 & -1 & 1 \\ 3 & -1 & 9 \end{array}\right]$	$\left[\begin{array}{cc\|c} 1 & 1 & 5 \\ 0 & -2 & -4 \\ 0 & 0 & 2 \end{array}\right]$	REF has $0 = b_i \implies$ Inconsistency Not Solvable

[1] Note: When we say $0 = b_i$ in the observations, we mean with $b_i \neq 0$.

With Gaussian elimination and back substitution, we can solve a system of linear equations as fully as possible. Moreover, we can say a lot about the solvability of the system by looking at the pivots, as we shall illustrate using the examples in Table 4.1. We have the augmented matrices for these equations, their REF, and our observations on the solvability of the system based on the properties of the REF in Table 4.3. These observations are, in fact, general statements about the solvability of systems of linear equations, as listed below.

Solvability Conditions of a system of linear equations based on the properties of the REF of its augmented matrix:

1. If we have a row in the REF (of the augmented matrix $[\boldsymbol{A} \mid \boldsymbol{b}]$) with all zeros in the coefficient ($\boldsymbol{A}$) part and a nonzero element in the constant ($\boldsymbol{b}$) part, the system is not solvable.

2. If the number of pivots in the REF is the same as the number of unknowns, we have a unique solution, provided the system is solvable by the first condition.

3. If the number of pivots is smaller than the number of variables, we have an infinity of solutions, provided the system is solvable (by the first condition).

The first solvability condition above can be stated in a variety of ways:

- If a certain linear combination of the rows of the coefficient matrix gives all zeros, the same combination of the elements of the constant vector also should give zero. Else, the system is inconsistent and not solvable.
- If the rank of the augmented matrix is greater than the rank of the coefficient matrix, the system is inconsistent and not solvable.
- If the pivot in any row of the REF of the augmented matrix is in the augmented column (meaning, in the constants column, coming from $\boldsymbol{b}$), the system is not solvable.

Knowing that the number of pivots in the REF is the same as the rank of the matrix, we can restate the solvability conditions more formally (albeit absolutely equivalently and therefore superfluously) as follows: For a system of linear equations $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ with m equations and n unknowns (in other words, $\boldsymbol{A} \in \mathbb{R}^{m\times n}$), with $\text{rank}(\boldsymbol{A}) = r$ and $\text{rank}\big(\big[\boldsymbol{A} \mid \boldsymbol{b}\big]\big) = r'$, we have:

1. If $r' > r$, the system is inconsistent and unsolvable.
2. If $n = r = r'$, the system has a unique solution.
3. If $n > r$, the system has an infinity of solutions.

Notes:

- The largest value the rank of any matrix can have is the smaller of its dimensions.
- The rank r of the coefficient matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ can never be larger than the rank r' of $\big[\boldsymbol{A} \mid \boldsymbol{b}\big] \in \mathbb{R}^{m\times(n+1)}$.
- As a corollary, if $n > m$, we have too few equations. The system, if solvable, will always have infinitely many solutions.

We will look at some more examples to illustrate the solvability conditions when we discuss the elementary matrices (the matrices associated with the elementary operations) after the next important topic on finding the complete solution of a system of linear equations when we have an infinite number of solutions.

Complete Solution

Looking at the solvability of the systems of equations in Table 4.3, we see that in several of them, we have either a unique solution or no solutions at all. In the case of no solution, we give up right away. When we have a unique solution, we can find it using back substitution. What is more complicated is when we have infinitely many solutions, as in rows 2 and 3 in Table 4.1, but the solution was still trivial to find because it was just a line representing one of the equations in the system.

Here is a system of equations with a nontrivial infinity of solutions, along with its augmented matrix and its REF.

$$\begin{matrix} x+y+z=6 \\ 2x+2y+z=9 \\ x+y=3 \end{matrix} \implies \left[\boldsymbol{A} \mid \boldsymbol{b}\right] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 3 \end{array}\right] \xrightarrow{\text{REF}} \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & 0 & 0 \end{array}\right]$$

From the REF above, we can start back-substituting: Row 3 does not say anything. Row 2 says:

$$-z = -3 \implies z = 3$$

Substituting it into the row above, we get:

$$x + y + 3 = 6 \quad \text{or} \quad x + y = 3$$

By convention, we take the variable corresponding to the non-pivot column (which is column 2 in this case, corresponding to the variable y) as a *free variable*. It can take any value $y = t$. Once y takes a value, x is fixed: $x = 3 - t$. So, the complete solution to this system of equations is:

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 3-t \\ t \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 3 \end{bmatrix} + t \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$$

Note that a linear equation (such as $z = 3$) in $\mathbb{R}^3$ defines a plane. $x + y = 3$ also defines a plane. (The pair $x = 3 - t$ and $y = t$ is a parametric equation for the same plane.) The intersection of these two planes is a line. And any point on this line is a solution to our set of linear equations.

Let's take one more example, this time with four variables x_1, x_2, x_3, and x_4:

$$x_1 + x_2 + x_3 + 2x_4 = 6$$
$$2x_1 + 2x_2 + x_3 + 7x_4 = 9$$

$$\left[\boldsymbol{A} \mid \boldsymbol{b}\right] = \left[\begin{array}{cccc|c} 1 & 1 & 1 & 2 & 6 \\ 2 & 2 & 1 & 7 & 9 \end{array}\right] \xrightarrow{\text{REF}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 2 & 6 \\ 0 & 0 & -1 & 3 & -3 \end{array}\right]$$

Columns 1 and 3 have pivots, which means x_2 and x_4 are free variables. Assigning values t_1 and t_2 to them, the last row gives us the equation $-x_3 + 3t_2 = -3$ or $x_3 = 3 + 3t_2$. The first row stands for the equation:

$$x_1 + x_2 + x_3 + 2x_4 = 6$$

Now we know $x_2 = t_1$, $x_3 = 3 + 3t_2$, and $x_4 = t_2$. Substituting, we get:

$$x_1 + t_1 + 3 + 3t_2 + 2t_2 = 6$$

This equation gives us the parametrized value of $x_1 = 3 - t_1 - 5t_2$ and the complete solution:

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 3 - t_1 - 5t_2 \\ t_1 \\ 3 + 3t_2 \\ t_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 3 \\ 0 \end{bmatrix} + t_1 \begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \end{bmatrix} + t_2 \begin{bmatrix} -5 \\ 0 \\ 3 \\ 1 \end{bmatrix} \tag{4.4}$$

CS Connection: Degrees of Freedom in Robotics


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

We have chosen to decompose the complete solution in this way (as the sum of one vector and a linear combination of two others) because it is the right form for a later topic. Just as a preview, we will call the first vector a *particular solution* and the linear combination the *null space* later on.

General Solution

The complete solution in Eqn (4.4) is also called the general solution. Let's take a closer look at it. It has the form $\boldsymbol{x}_p + t_1\boldsymbol{x}_{s_1} + t_2\boldsymbol{x}_{s_2}$. The

first vector, $\boldsymbol{x}_p$, is the so-called *particular solution*. We can get it by setting all the free variables (the ones corresponding to the non-pivot columns, which are the second and fourth variables in our example above) to the value zero. The second and third terms in the solution form a linear combination of two vectors $\boldsymbol{x}_{s_1}$ and $\boldsymbol{x}_{s_2}$. These vectors are called the special solutions of the system. We can see that they are, in fact, the solutions to the equations when the right-hand side, the constants part, is zero, which is to say $\boldsymbol{b} = \boldsymbol{0}$.

When a system of linear equations is of the form $\boldsymbol{Ax} = \boldsymbol{0}$, it is called a homogeneous system because all the terms in the system are of the same order one in the variables (as opposed to some with order zero if we had a nonzero $\boldsymbol{b}$). Therefore, the special solutions are also called homogeneous solutions.

Lastly, the linear combinations of the special solutions in our example, $t_1\boldsymbol{x}_{s_1} + t_2\boldsymbol{x}_{s_2}$, define a plane in $\mathbb{R}^4$, as two linearly independent vectors always form a plane going through the origin $\boldsymbol{0}$. What the addition of $\boldsymbol{x}_p$ does is to shift the plane to its tip, namely the coordinate point $(3, 0, 3, 0)$, if we allow ourselves to visualize it in a coordinate space. In other words, the complete solution is any vector whose tip is on this plane defined by the special solutions $\boldsymbol{x}_{s_1}$ and $\boldsymbol{x}_{s_2}$, and shifted by the particular solution $\boldsymbol{x}_p$.

4.3.2 Matrix Factorization

Elementary Matrices

Each of the row operations that we perform on a matrix $\boldsymbol{A}$ in Gaussian elimination can be thought of as a matrix multiplying $\boldsymbol{A}$ on the left. This insight comes from the row picture of matrix multiplication (§2.4.4, page 44). For easy reference, here are the elementary row operations once more:

1. Swap any two rows.

   e.g., Swap the second and third rows ($r_3 \leftrightarrow r_2$).

2. Multiply any row by a scalar.

   e.g., Multiply the second row by 3 ($r_2 \leftarrow 3r_2$).

3. Add a multiple of any row to another.

e.g., Subtract three times the first row from the third ($r_3 \leftarrow -3r_1 + r_3$).

For any $\boldsymbol{A} \in \mathbb{R}^{3\times n}$, here are the so-called elementary matrices (or operators) that would implement the examples listed above:

$$\boldsymbol{E_1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \qquad \boldsymbol{E_2} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad \boldsymbol{E_3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -3 & 0 & 1 \end{bmatrix}$$

By the row picture of matrix multiplication, $\boldsymbol{U} = \boldsymbol{E_1 A}$ can be stated as follows, denoting the rows of $\boldsymbol{A}$ by r_i and the rows of $\boldsymbol{U}$ by r_i':

- r_1' is a linear combination of the rows of $\boldsymbol{A}$: $r_1' = 1r_1 + 0r_2 + 0r_3 = r_1$
- r_2' is another linear combination: $r_2' = 0r_1 + 0r_2 + 1r_3 = r_3$
- r_3' is another linear combination: $r_3' = 0r_1 + 1r_2 + 0r_3 = r_2$

As we can see, $\boldsymbol{E_1}$ implements the swap of the second and third rows of any $\boldsymbol{A} \in \mathbb{R}^{3\times n}$. In particular, it does it for the identity matrix $\boldsymbol{I}$ in $\mathbb{R}^{3\times 3}$. Therefore, we can see that the elementary matrices differ from the identity matrix of the same size by one elementary row operation. In other words, every elementary row operation can be represented by an **elementary matrix** $\boldsymbol{E}$ multiplying $\boldsymbol{A}$ on the left. We can easily compute $\boldsymbol{E}$ by taking the identity matrix $\boldsymbol{I}$ and applying that exact row operation to it.

Summarizing, we can capture each elementary row operation that we apply to a matrix in Gaussian elimination as a matrix multiplying it on the left. This matrix is called the elementary matrix or elementary operator, denoted by $\boldsymbol{E}$. Since we are reducing a matrix $\boldsymbol{A}$ to its REF, it is customary to think of its rows being replaced by linear combinations of other rows: In other words, we usually write $r_i \leftarrow$ (rather than $r_i' =$) a specified linear combination, as we see in our examples.

Figure 4.3 shows the elementary matrix (in blue) that reduced our augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ for the system of linear equations $x + y = 5, x - y = 1$ to REF. We needed only one elementary operation in this example. We have more examples of elementary matrices in Tables 4.6 to 4.10.

$$\boldsymbol{Ax} = \boldsymbol{b} \Rightarrow \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

Subtract **Row 1** from **Row 2**:

$$\left[\begin{array}{cc|c} 1 & 1 & 5 \\ 1 & -1 & 1 \end{array}\right] \Rightarrow \left[\begin{array}{cc|c} 1 & 1 & 5 \\ 0 & -2 & -4 \end{array}\right]$$

Augmented Matrix: $[\boldsymbol{A} \mid \boldsymbol{b}]$ — Pivots ⇒ First non-zero element

The ***Elementary Row Operation*** as a Matrix:

$$\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \left[\begin{array}{cc|c} 1 & 1 & 5 \\ 1 & -1 & 1 \end{array}\right] = \left[\begin{array}{cc|c} 1 & 1 & 5 \\ 0 & -2 & -4 \end{array}\right] = \boldsymbol{U}$$

Using the **row picture** for the first matrix multiplication above:

- First Row of the product $\boldsymbol{U}$ = 1× **Row 1** + 0× **Row 2** of $[\boldsymbol{A} \mid \boldsymbol{b}]$
- Second Row of the product $\boldsymbol{U}$ = −1× **Row 1** + 1× **Row 2** of $[\boldsymbol{A} \mid \boldsymbol{b}]$

Fig. 4.3 Gaussian elimination on a simple augmented matrix, showing the elementary operator that implements the row reduction.

LU Decomposition

As we have seen, the Row-Echelon Form (REF), the end result of Gaussian elimination, is always an upper triangular matrix. We hinted at this by calling it $\boldsymbol{U}$ in one of our equations. Since Gaussian elimination can be applied to any matrix, we can always get an upper triangular matrix from any matrix by applying the elementary row operations listed earlier (in §4.2.1, page 75). Table 4.4 shows an

Table 4.4 $\boldsymbol{A = LU}$ Decomposition

$[\boldsymbol{A}] \rightarrow$ REF	Row Op.	$\boldsymbol{E_i}$	Inv. Op.	$\boldsymbol{E_i^{-1}}$
$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 1 \\ 1 & 3 & 0 \end{bmatrix}$	$r_2 \leftarrow -2r_1 + r_2$ $r_3 \leftarrow -r_1 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	$r_2 \leftarrow 2r_1 + r_2$ $r_3 \leftarrow r_1 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$
$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & -1 \\ 0 & 2 & -1 \end{bmatrix}$	$r_3 \leftarrow 2r_2 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix}$	$r_3 \leftarrow -2r_2 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix}$
$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & -1 \\ 0 & 0 & -3 \end{bmatrix}$	$\boldsymbol{L} = \boldsymbol{E_1^{-1}E_2^{-1}} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & -2 & 1 \end{bmatrix}$		$\boldsymbol{U}$ = REF = $\begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & -1 \\ 0 & 0 & -3 \end{bmatrix}$	

Let's focus on one elementary row operation, $\boldsymbol{E_2}$ (second row, under the column $\boldsymbol{E_i}$). This operation replaces row 3 with the sum of twice row 2 and row 3. The inverse of this operation would be to replace row 3 with row 3 − twice row 2, as shown in the column Inv. Op. The elementary operation for this inverse operation is, in fact, the inverse of $\boldsymbol{E_2}$, which we can find in the last column, under $\boldsymbol{E_i^{-1}}$.

example of this process of getting a $\boldsymbol{U}$ out of a matrix by finding its REF.

We have more examples coming up (in Tables 4.4 to 4.10), where we will write down the elementary matrix that implemented each row operation. We will call these elementary matrices $\boldsymbol{E_i}$. Referring to Table 4.4, we can therefore write the REF form as:

$$\boldsymbol{U = E_2 E_1 A = EA}$$

Note the order in which the elementary matrices appear in the product. We perform the first row operation $\boldsymbol{E_1}$ on $\boldsymbol{A}$, get the product $\boldsymbol{E_1 A}$, and apply the second operation $\boldsymbol{E_2}$ on this product, and so on.

We have not yet fully discussed the inverse of matrices, but the idea of the inverse undoing what the original matrix does is probably clear enough at this point. Now, starting from the upper triangular matrix $\boldsymbol{U}$ that is the REF of $\boldsymbol{A}$, we can write the following:

$$\boldsymbol{U = EA = E_2 E_1 A \implies A = E_1^{-1} E_2^{-1} U = LU}$$

where we have called the matrix $\boldsymbol{E_1^{-1} E_2^{-1}}$, which is a lower triangular matrix (because each of the $\boldsymbol{E_i^{-1}}$ is a lower triangular matrix), $\boldsymbol{L}$. Again notice the order in which the inverses $\boldsymbol{E_i}$ appear in the equation: We undo the last operation first before moving on to the previous one.

This statement is the famous $\boldsymbol{LU}$ decomposition, which states that any matrix can be written as the product of a lower triangular matrix $\boldsymbol{L}$ and an upper triangular matrix $\boldsymbol{U}$. The algorithm we use to perform the decomposition is Gaussian elimination. Notice that all the elementary matrices and their inverses have unit determinants, and the row operations we carry out do not change the determinant of $\boldsymbol{A}$, such that $|\boldsymbol{A}| = |\boldsymbol{U}|$.

Permutation Matrices

In discussing the example in Table 4.4, we cleverly glossed over one fact: We did not use the first elementary row operation (namely, swapping rows). Row exchanges introduce symmetric matrices as their elementary matrices. They are not lower triangular. The matrix implementing row exchanges is called a *permutation matrix*, and is denoted by $\boldsymbol{P}$.

Table 4.5 $\boldsymbol{A = PLU}$ Decomposition

$[\boldsymbol{A}] \rightarrow$ REF	Row Op.	$\boldsymbol{E_i}$	Inv. Op.	$\boldsymbol{E_i^{-1}}$
$\begin{bmatrix}1&1&1\\2&2&1\\1&3&0\end{bmatrix}$	$r_2 \leftrightarrow r_3$	$\begin{bmatrix}1&0&0\\0&0&1\\0&1&0\end{bmatrix}$	$r_2 \leftrightarrow r_3$	$\begin{bmatrix}1&0&0\\0&0&1\\0&1&0\end{bmatrix}$
$\begin{bmatrix}1&1&1\\1&3&0\\2&2&1\end{bmatrix}$	$r_2 \leftarrow -r_1 + r_2$ $r_3 \leftarrow -2r_1 + r_3$	$\begin{bmatrix}1&0&0\\-1&1&0\\-2&0&1\end{bmatrix}$	$r_2 \leftarrow r_1 + r_2$ $r_3 \leftarrow 2r_1 + r_3$	$\begin{bmatrix}1&0&0\\1&1&0\\2&0&1\end{bmatrix}$
$\begin{bmatrix}1&1&1\\0&2&-1\\0&0&-1\end{bmatrix}$	$\boldsymbol{P} = \begin{bmatrix}1&0&0\\0&0&1\\0&1&0\end{bmatrix}$	$\boldsymbol{L} = \boldsymbol{E_2^{-1}} = \begin{bmatrix}1&0&0\\1&1&0\\2&0&1\end{bmatrix}$	$\boldsymbol{U}$ = REF = $\begin{bmatrix}1&1&1\\0&2&-1\\0&0&-1\end{bmatrix}$	

Every permutation matrix (of single row exchange) differs from the identity matrix of the same size by one or more row exchanges[5]. And it is its own inverse (because exchanging a row with another is undone by the same exchange): $\boldsymbol{P}^2 = \boldsymbol{I}$. To introduce another name, the matrices that are their own inverses are called involutory. And the permutation matrices implementing a single row exchange are involutory.

Here are all possible permutation matrices that swap one or more pairs of rows in $\mathbb{R}^2$ and $\mathbb{R}^3$:

$$\underset{r_1 \leftrightarrow r_2}{\begin{bmatrix}0&1\\1&0\end{bmatrix}} \quad \underset{r_1 \leftrightarrow r_2}{\begin{bmatrix}0&1&0\\1&0&0\\0&0&1\end{bmatrix}} \quad \underset{r_2 \leftrightarrow r_3}{\begin{bmatrix}1&0&0\\0&0&1\\0&1&0\end{bmatrix}} \quad \underset{r_3 \leftrightarrow r_1}{\begin{bmatrix}0&0&1\\0&1&0\\1&0&0\end{bmatrix}} \quad \underset{\substack{r_1 \leftrightarrow r_2\\ r_2 \leftrightarrow r_3}}{\begin{bmatrix}0&1&0\\0&0&1\\1&0&0\end{bmatrix}}$$

Note that the last one does multiple row exchanges and consequently $\boldsymbol{P}^2 \neq \boldsymbol{I}$: It is not involutory.

$\boldsymbol{PLU}$ Decomposition

Since the elementary operation of row exchanges breaks our decomposition of $\boldsymbol{A = LU}$, we keep the permutation part separate, and come up with the general, unbreakable, universally applicable decomposition $\boldsymbol{A = PLU}$. We can see an example of this decomposition in Table 4.5.

[5] In principle, we can have permutation matrices that perform multiple row exchanges, but for the purposes of this book, we focus on one row exchange at a time.

Untangling Permutations: $PA = LU$ Trick

In our discussion of the $\boldsymbol{PLU}$ decomposition, we neatly separated the permutation matrices ($\boldsymbol{P}$) from the lower triangular matrix ($\boldsymbol{L}$). However, if we look closely at the Gaussian elimination algorithm with partial pivoting (§4.2.3, page 78), a subtle paradox emerges: a computer program does not magically know all the necessary row swaps in advance.

The algorithm evaluates the need for swaps on the fly. As the code runs, the row operations and swaps are physically tangled up in a sequence like this:

$$\boldsymbol{E}_3\boldsymbol{P}_2\boldsymbol{E}_2\boldsymbol{E}_1\boldsymbol{P}_1\boldsymbol{A} = \boldsymbol{U}$$

If the operations are interleaved, how can we mathematically pull all the permutation matrices to the front to achieve the clean $\boldsymbol{PA} = \boldsymbol{LU}$ form?

To answer this, we must first establish that permutation matrices and elementary elimination matrices do *not* commute. That is, $\boldsymbol{PE} \neq \boldsymbol{EP}$. We can prove this easily with a 2×2 example. Let $\boldsymbol{P}$ be the permutation matrix that swaps Row 1 and Row 2, and let $\boldsymbol{E}$ be the elementary matrix that adds k times Row 1 to Row 2:

$$\boldsymbol{P} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \boldsymbol{E} = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$$

Computing the product in both directions yields entirely different matrices:

$$\boldsymbol{PE} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix} = \begin{bmatrix} k & 1 \\ 1 & 0 \end{bmatrix}$$

$$\text{while} \quad \boldsymbol{EP} = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & k \end{bmatrix}$$

Since we cannot simply swap the order of $\boldsymbol{P}$ and $\boldsymbol{E}$, we rely on a brilliant mathematical sleight of hand. Because every permutation matrix of a single row exchange is its own inverse ($\boldsymbol{P}^{-1} = \boldsymbol{P}$), we can insert $\boldsymbol{PP}$ (which evaluates to the identity matrix $\boldsymbol{I}$) right after an elimination matrix without altering the equation:

$$\boldsymbol{PE}_1 = \boldsymbol{PE}_1(\boldsymbol{PP}) = (\boldsymbol{PE}_1\boldsymbol{P})\boldsymbol{P}$$

If we evaluate that grouped term, $\boldsymbol{PE}_1\boldsymbol{P}$, it yields a brand new elementary matrix, which we will call $\boldsymbol{E}_1'$. This new matrix performs the exact same elimination, but targets the newly swapped row. Therefore, we can safely pass a permutation matrix through an elimination matrix by slightly altering the elimination:

$$\boldsymbol{PE}_1 = \boldsymbol{E}_1'\boldsymbol{P}$$

Using this trick, we can push all the permutation matrices to the far right (right next to $\boldsymbol{A}$) by constantly modifying the $\boldsymbol{E}$ matrices they pass through. Let's untangle a short sequence:

$$\boldsymbol{E}_2\boldsymbol{PE}_1\boldsymbol{A} = \boldsymbol{U} \implies \boldsymbol{E}_2(\boldsymbol{E}_1'\boldsymbol{P})\boldsymbol{A} = \boldsymbol{U}$$

Suddenly, all the eliminations are grouped together on the left, and the permutation is applied directly to $\boldsymbol{A}$. Taking the inverse of those grouped eliminations forms our standard lower triangular matrix $\boldsymbol{L}$:

$$\boldsymbol{PA} = (\boldsymbol{E}_2\boldsymbol{E}_1')^{-1}\boldsymbol{U} \implies \boldsymbol{PA} = \boldsymbol{LU}$$

CS Connection: Efficient LU Decomposition

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

4.3.3 Computing Determinants

As briefly mentioned earlier, we can use Gaussian elimination to compute the determinant of a square matrix. Once we get the REF ($\boldsymbol{U}$), the determinant is simply the product of the diagonal elements. The reason is that the elementary row operations do not change the absolute value of the determinant. Or rather, they change the determinant in a way we can keep track of.

- Swapping two rows will result in the determinant flipping sign (Property 1 in §3.4, page 63). We therefore have to keep track of how many times we swapped rows when applying Gaussian elimination to compute determinants.
- Although we defined it as an elementary row operation (for future use), we do not scale rows in Gaussian elimination. If

we did, we would have to keep track of it as well because the determinant then would be multiplied by the same scaling factor (Property 3 in §3.4, page 63).

- The third elementary operation (adding/subtracting a multiple of a row from another one) does not change the determinant (by Property 6 in §3.4, page 63).

Why would we want to compute determinants this way rather than using the Laplace expansion (Eqn (3.4))? The computational complexity of the Laplace expansion is $\mathcal{O}(n!)$ while that of the Gaussian elimination method is only $\mathcal{O}(n^3)$, which is a huge gain for large values of n. As a special case, if the matrix does not have a full set of pivots, the determinant is zero, which means we do not even have to complete Gaussian elimination; we can stop the moment we get a row without a pivot.

4.4 More Examples

In Table 4.6, we have the first example of a system of three equations: We have as many equations as unknowns. After Gaussian elimination, we can see that we have a full set of pivots, and $\boldsymbol{A}$ is indeed full-rank. Therefore, we get a unique solution through back substitution, as shown after the table.

Table 4.6 Example 1: Gaussian elimination and solvability: Unique solution

Equations	$\left[\boldsymbol{A} \mid \boldsymbol{b}\right] \rightarrow$ REF	Row Operation	Elementary Matrix $\boldsymbol{E_i}$
$x + y + z = 6$ $2x + 2y + z = 9$ $x + 3y = 7$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 3 & 0 & 7 \end{array}\right]$	$r_2 \leftarrow -2r_1 + r_2$ $r_3 \leftarrow -r_1 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 2 & -1 & 1 \end{array}\right]$	$r_2 \leftrightarrow r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \end{array}\right]$	REF	

Back Substitution:
The last row of the REF($\boldsymbol{A}$) says $-z = -3 \implies z = 3$.
Substituting it into the row above, $2y - 3 = 1 \implies y = 2$.
Substituting z and y into the first row, $x + 2 + 3 = 6 \implies x = 1$.
The complete and unique solution is: $(x, y, z) = (1, 2, 3)$

Table 4.7 Example 2: Gaussian elimination and solvability: No solutions

Equations	$\left[\boldsymbol{A} \mid \boldsymbol{b}\right] \rightarrow$ REF	Row Operation	Elementary Matrix $\boldsymbol{E_i}$
$x + y + z = 6$ $2x + 2y + z = 9$ $x + y = 6$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 6 \end{array}\right]$	$r_2 \leftarrow -2r_1 + r_2$ $r_3 \leftarrow -r_1 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & -1 & 0 \end{array}\right]$	$r_3 \leftarrow -r_2 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & 0 & 3 \end{array}\right]$	REF	

The last row of the REF($\boldsymbol{A}$) is $\left[\begin{array}{ccc\|c} 0 & 0 & 0 & 3 \end{array}\right]$, saying $0 = 3 \implies$ The system is inconsistent.

We can restate the determinant computation more formally in terms of the $\boldsymbol{PLU}$ decomposition we discussed above. If we start with a square matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, we can make the following statements:

1. By the product rule of determinants (Property 10 in §3.4, page 63, $|\boldsymbol{AB}| = |\boldsymbol{A}|\,|\boldsymbol{B}|$), we can see that $|\boldsymbol{A}| = |\boldsymbol{P}|\,|\boldsymbol{L}|\,|\boldsymbol{U}|$.
2. $|\boldsymbol{P}| = \pm 1$. (Exercise: Prove this statement.)
3. $|\boldsymbol{L}| = 1$. (Exercise: Prove this statement.)
4. Therefore, $|\boldsymbol{A}| = |\boldsymbol{P}|\,|\boldsymbol{U}| = \pm\,|\boldsymbol{U}|$, with the sign given by $|\boldsymbol{P}|$.
5. Since $\boldsymbol{U}$ is triangular, its determinant is the product of its diagonal elements:
$$|\boldsymbol{U}| = \prod_{i=1}^{n} u_{ii}$$
6. In particular, if $\boldsymbol{U}$ does not have a full set of pivots (and is rank-deficient), at least one of $u_{ii} = 0 \implies |\boldsymbol{A}| = 0$.

In the second example (Table 4.7), we have modified the third equation such that it is no longer consistent with the first two. We again have as many equations as unknowns. However, after Gaussian elimination, we can see that the last row reads $\left[\begin{array}{ccc\|c} 0 & 0 & 0 & 3 \end{array}\right]$. It translates to an equation $0 = 3$, which indicates that the equations are inconsistent. Note that $\text{rank}\left(\left[\boldsymbol{A} \mid \boldsymbol{b}\right]\right) = 3 > \text{rank}(\boldsymbol{A}) = 2$.

In the third example (Table 4.8), we have added one more equation to the system in Table 4.6. Thus, we have four equations for three unknowns. But the augmented matrix reduces with the last row

reading $0 = 0$, which means the fourth equation is consistent with the rest. And $\text{rank}\big(\big[\boldsymbol{A} \mid \boldsymbol{b}\big]\big) = \text{rank}(\boldsymbol{A}) = 3$, which is the same as the number of unknowns. Hence, we have a unique solution.

The fourth example is similar to the third one; we have added a fourth equation in Table 4.9 as well. However, the new equation is not consistent with the rest.

In the last example (Table 4.10), which we used to illustrate the complete solution with free variables, we have as many equations as unknowns. After Gaussian elimination, we get the last row reading $\left[\begin{array}{ccc|c} 0 & 0 & 0 & 0 \end{array}\right]$; no zero = nonzero row, indicating that the equations are consistent. However, $\text{rank}\big(\big[\boldsymbol{A} \mid \boldsymbol{b}\big]\big) = \text{rank}(\boldsymbol{A}) = 2$ with three unknowns, which means we have infinitely many combinations of (x, y, z) that can satisfy these equations.

4.5 Gaussian Elimination: A Foundational Algorithm

In this chapter, we looked at the applications of Gaussian elimination in computing determinants and solving equations. We briefly introduced matrix inverses.

Table 4.8 Example 3: More equations than unknowns, but solvable

Equations	$\big[\boldsymbol{A} \mid \boldsymbol{b}\big] \rightarrow$ REF	Row Operation	Elementary Matrix $\boldsymbol{E_i}$
$x + y + z = 6$ $2x + 2y + z = 9$ $x + 3y = 7$ $3x + y + z = 8$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 3 & 0 & 7 \\ 3 & 1 & 1 & 8 \end{array}\right]$	$r_2 \leftarrow -2r_1 + r_2$ $r_3 \leftarrow -r_1 + r_3$ $r_4 \leftarrow -3r_1 + r_4$	$\left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ -3 & 0 & 0 & 1 \end{array}\right]$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 2 & -1 & 1 \\ 0 & -2 & -2 & -10 \end{array}\right]$	$r_2 \leftrightarrow r_3$	$\left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{array}\right]$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \\ 0 & -2 & -2 & -10 \end{array}\right]$	$r_4 \leftarrow r_2 + r_4$	$\left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{array}\right]$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & -3 & -9 \end{array}\right]$	$r_4 \leftarrow -3r_3 + r_4$	$\left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -3 & 1 \end{array}\right]$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & 0 & 0 \end{array}\right]$	REF	

The last row of the REF($\boldsymbol{A}$), $\left[\begin{array}{ccc|c} 0 & 0 & 0 & 0 \end{array}\right]$, says $0 = 0 \implies$ The system is consistent.

Table 4.9 Example 4: More equations than unknowns, with no solutions

Equations	$[\boldsymbol{A} \mid \boldsymbol{b}] \to$ REF	Row Operation	Elementary Matrix $\boldsymbol{E_i}$
$x+y+z=6$ $2x+2y+z=9$ $x+3y=7$ $3x+y+z=11$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 3 & 0 & 7 \\ 3 & 1 & 1 & 11 \end{array}\right]$	$r_2 \leftarrow -2r_1 + r_2$ $r_3 \leftarrow -r_1 + r_3$ $r_4 \leftarrow -3r_1 + r_4$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ -3 & 0 & 0 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 2 & -1 & 1 \\ 0 & -2 & -2 & -7 \end{array}\right]$	$r_2 \leftrightarrow r_3$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \\ 0 & -2 & -2 & -7 \end{array}\right]$	$r_4 \leftarrow r_2 + r_4$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & -3 & -6 \end{array}\right]$	$r_4 \leftarrow -3r_3 + r_4$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -3 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & 0 & 3 \end{array}\right]$	REF	

The last row of the REF($\boldsymbol{A}$) is $\left[\begin{array}{ccc\|c} 0 & 0 & 0 & 3 \end{array}\right]$, saying $0 = 3 \implies$ The system is inconsistent.

Beyond its role as a practical algorithm, Gaussian elimination serves as a foundational diagnostic tool. When we are stumped by a matrix problem on an exam and have no idea how to get started, we perform Gaussian elimination. Reducing the matrix to its row-echelon form systematically exposes its underlying structure. The resulting arrangement of pivots, free variables, and zero rows almost always provides the necessary insight to determine how to proceed.

The interplay between ranks, determinants, and inverses will be further explored in the next chapter.

Table 4.10 Example 5: Gaussian elimination and solvability: Infinity of solutions

Equations	$[\boldsymbol{A} \mid \boldsymbol{b}] \to$ REF	Row Operation	Elementary Matrix $\boldsymbol{E_i}$
$x+y+z=6$ $2x+2y+z=9$ $x+y=3$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 3 \end{array}\right]$	$r_2 \leftarrow -2r_1 + r_2$ $r_3 \leftarrow -r_1 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & -1 & -3 \end{array}\right]$	$r_3 \leftarrow -r_2 + r_3$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}$
	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & 0 & 0 \end{array}\right]$	REF	

The last row of the REF($\boldsymbol{A}$) is $\left[\begin{array}{ccc\|c} 0 & 0 & 0 & 0 \end{array}\right]$, indicating consistency. With two independent equations for three unknowns, we get an infinity of solutions.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

5

Ranks and Inverses of Matrices

Everything is simpler than you think and at the same time more complex than you imagine.

—Johann Wolfgang von Goethe



Along with the determinant, the rank of a matrix is a fundamental number that reveals a great deal about its properties. Unlike the determinant, however, rank is defined for all matrices, not merely square ones. Furthermore, rank is a non-negative integer, whereas the determinant evaluates to a scalar belonging to the same field over which the matrix is defined. For square matrices, rank and determinant are closely related to each other. In fact, most things in linear algebra tend to be interconnected. In this chapter, we will summarize what we have already learned about rank and expand upon it. The full discussion, however, will wait until we look at the underlying geometry of matrices and vectors.

5.1 The Concept of Rank

Earlier, we stated that the rank of a matrix is the number of pivots in its row echelon form (REF), which we get through Gaussian elimination. This algorithm is merely a series of elementary row operations, each of which is about taking a linear combination of the rows of the matrix. And, right from the opening chapters where we introduced vectors and matrices (in §2.1.3, page 27), we talked about linear combinations.

If we can combine a bunch of rows in a matrix so as to get a different row, then the rows are not linearly independent. To state it without ambiguity using mathematical lingo, for a matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, we say that a nonzero i^{th} row ($\boldsymbol{r}_i^{\mathsf{T}}$) is a linear combination of the other rows if we can write:

$$\boldsymbol{r}_i^{\mathsf{T}} = \sum_{j=1, j \neq i}^{m} s_j \boldsymbol{r}_j^{\mathsf{T}}, \quad \boldsymbol{r}_j \in \mathbb{R}^n, \quad s_j \in \mathbb{R}$$

When we can write one row as the linear combination of the rest, we say that that row is *not* linearly independent. Note that any row that is not linearly independent can be reduced to zero using the elementary row operations. Once a row is reduced to zero, it does not have a pivot. Conversely, all nonzero rows do have pivots, and they are linearly independent. Therefore, we can state that the rank of a matrix is the number of linearly independent rows.

5.1.1 Row and Column Ranks

Strictly speaking, the number of linearly independent rows would be the row rank of the matrix. Similarly, we can define a column rank, which is the number of linearly independent columns of the matrix. Or, the number of pivots when Gaussian elimination is performed column-wise, using column-reduction operations.

We have a theorem that states that the row rank is the same as the column rank. It is fairly easy to prove this fact. We will provide some hints and leave it as an exercise after looking at various possible shapes of matrices, the Gauss-Jordan algorithm and the canonical form it produces (in §5.2.2, page 104). We also have the full proof in one of the advanced topics, in §12.2.2 (page 256).

5.1.2 Matrix Shapes and Rank Limits

A general matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ can be tall if the number of rows is greater than the number of columns: $m > n$. Or it can be wide (or, less charitably, fat) if it has more columns than rows: $m < n$. When $m = n$, what we have is a square matrix.

For a square matrix, the best possible situation is that we may have a pivot in every row/column. We cannot have more pivots than that. Therefore, the rank r of the matrix has a maximum value of $m = n$.

$$\boldsymbol{A} \in \mathbb{R}^{n \times n} \implies \text{rank}(\boldsymbol{A}) \leq n$$

For a "tall" matrix, the best scenario is when we have a pivot in each of the columns. We cannot have more than one pivot in the same column. So we have $r \leq n$ when $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, $n < m$. Similarly, for a "wide" matrix, we have, $r \leq m$ when $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, $n > m$. Combining these two limits, we can write:

$$\boldsymbol{A} \in \mathbb{R}^{m \times n} \implies \text{rank}(\boldsymbol{A}) \leq \min(m, n)$$

If the rank $r = \min(m, n)$, we call $\boldsymbol{A}$ a full-rank matrix; else, it is rank deficient. If $r = m$, it is often called a full-row-rank (or if $r = n$, full-column-rank) matrix.

5.1.3 Properties of Ranks

Here are some of the properties of ranks of matrices. Although listed without any formal proof, the properties are either obvious, or derived from the fact that the rank is the number of linearly independent columns or rows, and that matrix multiplication is about taking their linear combinations (as in the row and column pictures). The proofs will have to wait till we learn more about the geometry associated with matrices because ranks are closely linked to it.

For $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, we have:

1. The only matrix that has a rank of zero is the zero matrix, which is a matrix that has all entries as zeros.

2. If $\boldsymbol{B} \in \mathbb{R}^{n \times k}$, conformant for multiplication with $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ on the right,

$$\text{rank}(\boldsymbol{AB}) \leq \min(\text{rank}(\boldsymbol{A}), \text{rank}(\boldsymbol{B}))$$

3. If $\boldsymbol{B} \in \mathbb{R}^{n\times k}$, conformant for multiplication with $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ on the right, with $\text{rank}(\boldsymbol{A}) = r$ and $\text{rank}(\boldsymbol{B}) = n$ (full-row rank $\implies k \geq n$),

$$\text{rank}(\boldsymbol{AB}) = \text{rank}(\boldsymbol{A}) = r$$

This property follows from the column picture of matrix multiplication and the definition of rank as the number of linearly independent columns: The product $\boldsymbol{AB}$ has columns that are linear combinations of the r independent columns of $\boldsymbol{A}$. Note that $m \geq r$ and $n \geq r$ (since $\text{rank}(\boldsymbol{A}) = r$), $k \geq n$ (since $\text{rank}(\boldsymbol{B}) = n$) $\implies k \geq r$. In other words, the product $\boldsymbol{AB} \in \mathbb{R}^{m\times k}$ has enough rows and columns to accommodate the rank of r. Or, stating it using still other words, while multiplying on the right, $\boldsymbol{B}$ has enough rows and columns to preserve the rank of $\boldsymbol{A}$, subject to the condition $\text{rank}(\boldsymbol{AB}) \leq \min(\text{rank}(\boldsymbol{A}), \text{rank}(\boldsymbol{B}))$.

4. Similarly, if $\boldsymbol{B} \in \mathbb{R}^{k\times m}$, conformant for multiplication with $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ on the left, with $\text{rank}(\boldsymbol{B}) = m$ (full-column rank),

$$\text{rank}(\boldsymbol{BA}) = \text{rank}(\boldsymbol{A})$$

This property follows from the row picture of matrix multiplication and the definition of rank as the number of linearly independent rows. The explanation is the transpose case of the previous case. Keep in mind that these two properties (3 and 4) do not violate the condition in property (2).

5. If $\boldsymbol{B} \in \mathbb{R}^{m\times n}$, same dimensions as $\boldsymbol{A}$ so that we can add them,

$$\text{rank}(\boldsymbol{A} + \boldsymbol{B}) \leq \text{rank}(\boldsymbol{A}) + \text{rank}(\boldsymbol{B})$$

6. As a corollary of the previous property, we can say that if $\text{rank}(\boldsymbol{A}) = r$, we can write it as a sum of r matrices of rank one, and we do not need more matrices.

7. A very important property that we will prove later on (much later, in §12.2.2, page 256):

$$\text{rank}\left(\boldsymbol{AA}^{\mathsf{T}}\right) = \text{rank}\left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\right) = \text{rank}(\boldsymbol{A}) = \text{rank}\left(\boldsymbol{A}^{\mathsf{T}}\right)$$

$A^T A$ is called the Gram matrix, and has properties that make it important in machine learning.

CS Connection: Low-Rank Matrices and Data Compression


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

5.2 Gauss-Jordan Elimination and Canonical Forms

While discussing the Gaussian elimination algorithm in the previous chapter, we ended up with the $A = PLU$ decomposition. The U matrix had the same determinant as A (and P contained the information about the sign of the determinant).

If we are not really concerned about computing the determinant, but only want to solve the system of linear equations $Ax = b$, we can keep applying elementary row operations instead of the back substitution we discussed earlier. This algorithm to solve linear equations is known as *Gauss-Jordan Elimination*.

As a reminder to ourselves, we have the *elementary row operations*, as listed below:

- Swap any two rows.
- Multiply any row by a scalar.
- Add a multiple of any row to another.

In the previous chapter, although we listed the three elementary operations, we did not use the second one, namely scaling a row. In the Gauss-Jordan algorithm, we will use it as well.

Before stating the steps involved in Gauss-Jordan elimination, let's look at the example in Figure 5.1. Here, we start from the REF of the augmented matrix of the equations $x + y = 5$, $x - y = 1$ (see Figure 4.2), and keep applying the elementary row operations, which do not affect the solution of the system of equations.

In this case, the REF of the augmented matrix, $\left[A \mid b\right]$, becomes the identity matrix in the coefficient (A) side, at which point the constants column contains the solution. Remember, each row of the augmented matrix (whether row reduced or not) represents an

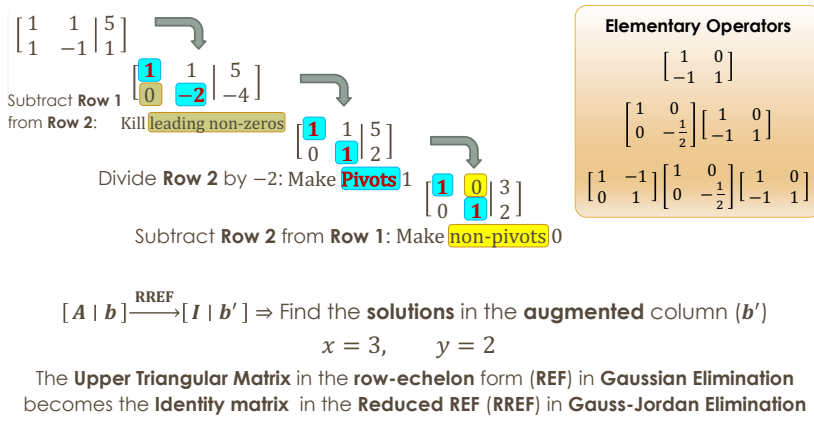


Fig. 5.1 Example of Gauss-Jordan elimination to solve a simple system of linear equations

equation. What equation does the top row of the final matrix from Gauss-Jordan elimination represent? It reads: $1x + 0y = 3$ or $x = 3$. Similarly, the second row gives us the equation $y = 2$.

The aim of the Gauss-Jordan algorithm is to get to an identity matrix in the coefficient part of the augmented matrix. In other words, try to make $\boldsymbol{A}$ in $\boldsymbol{Ax} = \boldsymbol{b}$ an $\boldsymbol{I}$, failing which, get as close to the identity matrix as possible. This form of the coefficient matrix is called the *Reduced Row Echelon Form*, or RREF, also known as the *Canonical Form*. For a full-rank, square matrix, its canonical form is the identity matrix of the same size, as we saw in the example in Figure 5.1. We will soon discuss other combinations of shapes and ranks and their canonical forms.

Reduced Row Echelon Form

Definition: A matrix is in its Reduced Row Echelon or Canonical Form if it has the following characteristics:

1. It is in REF. (See §4.2.2, page 76)
2. Every nonzero row is scaled such that the pivot value is one.
3. In every pivot column, the only nonzero element is the pivot.

Gauss-Jordan Elimination: Algorithm


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

As we can see, the *Reduced* Row Echelon Form (RREF) is the Row Echelon Form (REF) with some special properties, as its name indicates. The actual definition of REF and RREF are a bit fluid: Different textbooks may define them differently. For instance, some may suggest that REF should have all pivots equaling unity, through scaling. Such differences, however, do not affect the conceptual significance of the forms and perhaps not even their applications.

Note that while the REF (which is the result of Gaussian elimination) can have different shapes depending on the order in which the row operations are performed, RREF (the result of Gauss-Jordan) is immutable: A matrix has a unique RREF. In this sense RREF is more fundamental than REF, and points to the underlying characteristics of the matrix.

5.2.1 The Gauss-Jordan Algorithm

With the insight from the example in Figure 5.1, we can state the steps in the Gauss-Jordan algorithm (for a matrix $\boldsymbol{A} = \begin{bmatrix} a_{ij} \end{bmatrix} \in \mathbb{R}^{m \times n}$) as follows:

1. Run the Gaussian elimination algorithm (§4.2.3, page 78) to get REF($\boldsymbol{A}$).
2. Loop (with index i) over the rows of REF($\boldsymbol{A}$) and scale the ith row by $\frac{1}{a_{ik}}$ (where k is the column where Pivot$_i$ appears) so that Pivot$_i = 1$.
3. Loop (with index j) over all the elements of the pivot column from 1 to $i - 1$, multiply the ith row with $-a_{jk}$ and add it to the jth row to get zeros in the kth position for all rows above the ith row.
4. Loop back to step 2 (incrementing i) and iterate until all rows or columns are exhausted.

Let's look at two examples to illustrate the outputs of the Gauss-Jordan algorithm. Starting from the REF of the augmented matrices in Tables 4.7 and 4.10, we find their RREF. The examples are in Tables 5.1 and 5.2.

Table 5.1 Gauss-Jordan on a full-rank, square coefficient matrix, giving us the unique solution

Equations	$\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$	REF	RREF
$x + y + z = 6$ $2x + 2y + z = 9$ $x + 3y = 7$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 3 & 0 & 7 \end{array}\right]$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & -1 & -3 \end{array}\right]$	$\left[\begin{array}{ccc\|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array}\right]$

Here, we start from the REF of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ (as in Table 4.7), and do further row reduction to get $\boldsymbol{I}$ in the coefficient part. In other words, in total, we apply the Gauss-Jordan algorithm. We can then read out the solution $(x, y, z) = (1, 2, 3)$ from the augmented part of the RREF.

If we started from the REF of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ in Table 4.9, where we have inconsistent equations, the RREF would be an uninteresting 4×4 identity matrix. What it says is that we have a rank of four in $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$, with only three unknowns, indicating that we have no solutions.

Notice that we did not write down the elementary matrices for the Gauss-Jordan algorithm, as we diligently did for Gaussian elimination? The reason for doing that in Gaussian elimination was to arrive at the $\boldsymbol{A} = \boldsymbol{PLU}$ decomposition so as to compute its determinant. We do not use Gauss-Jordan to compute determinants, although we could, if we wanted to.

5.2.2 Shapes of Canonical Forms

We looked at what we called "tall" and "wide" matrices in §5.1.2. What are the canonical forms of matrices of these various shapes? Remember, canonical form is an alias for RREF.

Square Matrices

Let's start with a square matrix of full rank: $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, $\text{rank}(\boldsymbol{A}) = n$. Since all the rows have pivots, and since we can scale the pivots to get ones, we will end up with the identity matrix $\boldsymbol{I}$ as the canonical form. In fact, this is what we did in Table 5.1, for the coefficient matrix.

$$\boldsymbol{A} \in \mathbb{R}^{n \times n}, \quad \text{rank}(\boldsymbol{A}) = n \implies \boldsymbol{A} \xrightarrow{\text{RREF}} \boldsymbol{I}_n$$

where the symbol $\boldsymbol{I}_n$ stands for the identity matrix in $\mathbb{R}^{n \times n}$.

Table 5.2 Gauss-Jordan on a rank-deficient coefficient matrix, resulting in an infinity of solutions

Equations	$\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$	REF	RREF
$x + y + z = 6$ $2x + 2y + z = 9$ $x + y = 3$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 3 \end{array}\right]$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & 0 & 0 \end{array}\right]$	$\left[\begin{array}{ccc\|c} 1 & 1 & 0 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 0 \end{array}\right]$

This second example starts from the REF of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ from Table 4.10. The Gauss-Jordan algorithm gets us as close to the identity matrix as possible in the coefficient part of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$. The solution is: $z = 3$ and $x + y = 3$, which is usually written as $(x, y, z) = (3 - t, t, 3)$.

What happens if the matrix is rank deficient? If $\text{rank}(\boldsymbol{A}) = r < n$, we have only r pivots, which the Gauss-Jordan algorithm will place in the first r rows. The last $n - r$ rows of the RREF, therefore, will be zeros. The top r rows will contain the identity matrix $\boldsymbol{I}$ because the pivots will be scaled to one, and they will be used to eradicate other elements in the pivot columns.

The pivot columns, however, may not be ideally placed to give us an $\boldsymbol{I}_r$ matrix in the top-left corner, but may appear interspersed among other pivot-less columns. As we remember from solving linear equations, columns with no pivots result in free variables, and we may call these columns part of a matrix $\boldsymbol{F}$. Thus the canonical form of a rank-deficient square matrix looks like:

$$\boldsymbol{A} \in \mathbb{R}^{n \times n}, \quad \text{rank}(\boldsymbol{A}) = r < n \implies \boldsymbol{A} \xrightarrow{\text{RREF}} \left[\begin{array}{c} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r \times (n-r)} \\ \hline \boldsymbol{0}_{(n-r) \times n} \end{array}\right]$$

We use the expression $\boldsymbol{I} \bowtie \boldsymbol{F}$ to indicate that $\boldsymbol{I}$ and $\boldsymbol{F}$ are shuffled in together. Note that $\bowtie$ is *not* a standard notation for this purpose.

In Table 5.2, we have an example of such a system: The coefficient matrix is a square matrix with rank deficiency. Looking at the RREF column of the table, we can see that pivot columns do make an identity matrix, but the pivot-less, free-variable columns are interspersed among it. The pivot columns are 1 and 3, and the free variable is in column 2. The bottom $m - r = 1$ row is a zero row.

Tall Matrices

For a "tall" matrix of full (column) rank, we will get pivots for the first n rows, and the remaining $m - n$ rows will be zeros. The n

Table 5.3 Gauss-Jordan on a rank-deficient "wide" coefficient matrix

Equations	$\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$	REF	RREF
$x + y + z = 6$ $2x + 2y + z = 9$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \end{array}\right]$	$\left[\begin{array}{ccc\|c} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \end{array}\right]$	$\left[\begin{array}{ccc\|c} 1 & 1 & 0 & 3 \\ 0 & 0 & 1 & 3 \end{array}\right]$

This third example is similar to the one in Table 5.2, but we removed the third equation. The Gauss-Jordan algorithm gets us as close to the identity matrix as possible in the coefficient part of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$. Notice how the columns of the identity matrix and the pivot-less columns are interspersed.

pivots can be normalized so that the top part of the canonical form becomes $\boldsymbol{I}_n$, and we will have:

$$\boldsymbol{A} \in \mathbb{R}^{m \times n}, \quad \text{rank}(\boldsymbol{A}) = n < m \implies \boldsymbol{A} \xrightarrow{\text{RREF}} \left[\begin{array}{c} \boldsymbol{I}_n \\ \hline \boldsymbol{0}_{(m-n)\times n} \end{array}\right]$$

If the tall matrix is rank deficient, we again have an $\boldsymbol{I}$ and $\boldsymbol{F}$ in the top r rows, followed by zeros. The pivot and pivot-less columns may get interspersed among each other, as in the case of the square matrix, giving us a canonical form that looks identical to the one for the rank-deficient square matrix, but the indices are different: It has one instance of n replaced by m.

$$\boldsymbol{A} \in \mathbb{R}^{m \times n}, \quad \text{rank}(\boldsymbol{A}) = r < n < m \implies \boldsymbol{A} \xrightarrow{\text{RREF}} \left[\begin{array}{c} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r\times(n-r)} \\ \hline \boldsymbol{0}_{(m-r)\times n} \end{array}\right]$$

Wide Matrices

Moving on to "wide" matrices (with more columns than rows), if they have full (row) rank, they will get a canonical form that may not cleanly separate into $\boldsymbol{I}$ and $\boldsymbol{F}$. We can see an example in Table 5.3, which has two independent and consistent equations, and therefore a full-row-rank $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$. The first and third columns have pivots, and make an identity matrix, but the free-variable column appears in between. The general shape of a full-row-rank, wide matrix, therefore, looks like:

$$\boldsymbol{A} \in \mathbb{R}^{m \times n}, \quad \text{rank}(\boldsymbol{A}) = m < n \implies \boldsymbol{A} \xrightarrow{\text{RREF}} \left[\boldsymbol{I}_m \bowtie \boldsymbol{F}_{m\times(n-m)}\right]$$

For a rank-deficient wide matrix also, the canonical form is similar to the other rank-deficient cases. What we had for the coefficient matrix (or for the whole augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$) in Table 5.2

was a wide matrix (not enough independent equations) with rank deficiency. Looking at the RREF column of the table, we can see that pivot elements do make an identity matrix, but the pivot-less, free variable columns are interspersed among it. The pivot columns are 1 and 3, and the free variable is in column 2. The remaining $m - r$ rows are zeros. We represent this as:

$$\boldsymbol{A} \in \mathbb{R}^{m \times n}, \quad \text{rank}(\boldsymbol{A}) = r < m < n \implies \boldsymbol{A} \xrightarrow{\text{RREF}} \left[\begin{array}{c} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r \times (n-r)} \\ \hline \boldsymbol{0}_{(m-r) \times n} \end{array}\right]$$

Summary

We see that the Gauss-Jordan algorithm tries very hard to get to an identity matrix as the RREF. If the matrix has full rank, it succeeds fully in the case of square and tall matrices, with a clean identity matrix visible in the RREF. In the case of a wide matrix, it may not be able to cleanly separate the identity matrix from the pivot-less columns. If the matrix is rank deficient, in general, the RREF will have the columns of the identity matrix (which are pivot columns) interspersed with the free-variable columns. In any case, the RREF is unique for a matrix, unlike REF, the output of Gaussian elimination, which depends on the order in which the elementary row operations are performed.

Earlier, we stated that the row and column ranks were the same for any matrix, and that we would provide a hint to prove it. This whole section is the hint, plus the fact that elementary row operations do not change row or column rank. Neither do elementary column operations. There is nothing that prevents us from doing both, one after the other, if we want, and then asserting the fact that the row and column ranks do not change.

5.3 Matrix Inverses and Invertibility

We came across the inverse of matrices on a few previous occasions, starting with the nomenclature, where we defined it as that matrix which, when multiplied with the original, gives us the identity matrix. We again saw it in the properties of determinants: If a matrix has a zero determinant, it cannot be inverted. While talking about elementary

matrices, we spoke of their inverses as those matrices that would reverse the effect of multiplying with the original matrices. Let's recap and expand on what we have learned about inverses.

5.3.1 Conditions for Invertibility

A square matrix is invertible if we can compute its inverse. Invertibility can be expressed in a variety of ways, as listed below. For a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, if $\boldsymbol{A^{-1}}$ exists, the following statements are equivalent:

- $\boldsymbol{A}$ is invertible.
- $\boldsymbol{A}$ is *not* singular.
- $\boldsymbol{A}$ is *not* degenerate.
- $\boldsymbol{A}$ has n pivots in its REF.
- $\boldsymbol{A}$ has full rank: $\text{rank}(\boldsymbol{A}) = n$.
- The system of linear equations $\boldsymbol{Ax} = \boldsymbol{b}$ has a unique solution.
- $\boldsymbol{A}$ represents a linear automorphism. (See §**??**, page **??**.)

These are the statements about the invertibility of $\boldsymbol{A}$ that we have learned so far. We have quite a few more of such statements, related to the linear independence of the rows and columns of $\boldsymbol{A}$, the geometry of its so-called fundamental spaces, its eigenvalues *etc.* We will list them later, when appropriate.

5.3.2 Properties of Inverses

In addition to the various statements about invertibility, we also have a list of some interesting properties of inverses.

1. The inverse of an inverse is the original matrix: $\left(\boldsymbol{A^{-1}}\right)^{-1} = \boldsymbol{A}$.
2. **Scaling**: When we multiply a matrix by a scalar, its inverse gets multiplied by the reciprocal of the scalar:

$$\boldsymbol{(sA)^{-1}} = s^{-1}\boldsymbol{A^{-1}} \text{ because}$$
$$(s\boldsymbol{A})\, s^{-1}\boldsymbol{A^{-1}} = (ss^{-1})\boldsymbol{AA^{-1}} = 1\boldsymbol{I} = \boldsymbol{I}$$

3. **Product Rule:** Similar to the product rule of transposes, the inverse of a product is the product of the inverses of the factors in the reverse order:

$$(\boldsymbol{AB})^{-1} = \boldsymbol{B}^{-1}\boldsymbol{A}^{-1} \text{ because } (\boldsymbol{AB})\boldsymbol{B}^{-1}\boldsymbol{A}^{-1}$$
$$= \boldsymbol{A}\left(\boldsymbol{BB}^{-1}\right)\boldsymbol{A}^{-1} = \boldsymbol{AIA}^{-1} = \boldsymbol{AA}^{-1} = \boldsymbol{I}$$

   We used this property in constructing the lower triangular matrix $\boldsymbol{L}$ (on $\boldsymbol{A} = \boldsymbol{LU}$) from the inverses of elementary matrices. (See §4.3.2, page 87.)

4. **Involutory Matrix:** A matrix whose inverse is the same as itself is called an involutory matrix. If $\boldsymbol{P}$ is involutory, $\boldsymbol{P}^2 = \boldsymbol{I}$, such as the single-row-exchange permutation matrices.

5. The determinant of the inverse of a matrix is the reciprocal of its determinant:

$$\left|\boldsymbol{A}^{-1}\right| = \frac{1}{|\boldsymbol{A}|}$$

5.3.3 Left and Right (Generalized) Inverses

The left inverse of a matrix $\boldsymbol{A}$ is the matrix that produces $\boldsymbol{I}$ when multiplied on the left of $\boldsymbol{A}$. Similarly, the right inverse is the matrix that yields $\boldsymbol{I}$ when multiplied on the right. In other words, if $\boldsymbol{BA} = \boldsymbol{I}$, then $\boldsymbol{B} = \boldsymbol{A}^{-1}_{\textbf{Left}}$. And if $\boldsymbol{AC} = \boldsymbol{I}$, then $\boldsymbol{C} = \boldsymbol{A}^{-1}_{\textbf{Right}}$.

Although we did not specify it, the inverse $\boldsymbol{A}^{-1}$ we have been talking about is a double-sided inverse: $\boldsymbol{AA}^{-1} = \boldsymbol{I} = \boldsymbol{A}^{-1}\boldsymbol{A}$. For a full-rank, square matrix, we can prove that if $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C} \in \mathbb{R}^{n\times n}$, $\text{rank}(\boldsymbol{A}) = n$, and $\boldsymbol{AB} = \boldsymbol{CA} = \boldsymbol{I}$, then $\boldsymbol{B} = \boldsymbol{C}$.

For "tall" or "wide" matrices (as described in §5.1.2, page 99), the situation is a bit more complicated, even if they are of full rank. Remember, for tall matrices, full rank means full column rank, and for wide matrices, it means full row rank. To state this with mathematical precision:

$$\text{Tall, full rank:} \implies \boldsymbol{A} \in \mathbb{R}^{m\times n}, \quad m > n, \quad \text{rank}(\boldsymbol{A}) = r = n$$
$$\text{Wide, full rank:} \implies \boldsymbol{A} \in \mathbb{R}^{m\times n}, \quad m < n, \quad \text{rank}(\boldsymbol{A}) = r = m$$

For tall matrices, what is the shape of $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$? It is a square matrix of size $n \times n = r \times r$. It has a rank of r, as we stated in Property 7

while listing the properties of ranks in §5.1.3, page 99. When $\boldsymbol{A}$ has full column rank, $r = n$, and $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ is full rank as well. This so-called Gram matrix is therefore an invertible matrix, and $(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}$ exists. It is also symmetric because $(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^\mathsf{T} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A}$. With a bit of matrix algebra wizardry, we can write:

$$(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}) = \boldsymbol{I} \implies \left((\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\right)\boldsymbol{A} = \boldsymbol{I} \tag{5.1}$$

This is of the form $\boldsymbol{B}\boldsymbol{A} = \boldsymbol{I}$, and therefore $\left((\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\right)$ is something like an inverse of $\boldsymbol{A}$ when multiplied on the left. We will call this the left inverse of $\boldsymbol{A}$, $\boldsymbol{A}^{-1}_{\textbf{Left}}$. Remember, $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ $(m > n)$ is not square. Although we can multiply $\boldsymbol{A}$ on the right with $\boldsymbol{A}^{-1}_{\textbf{Left}}$ (because dimensional conformance requirements are met), what we will get is a matrix in $\mathbb{R}^{m\times m}$ with a rank of $n < m$. And the product cannot be an identity matrix because it is rank-deficient.

In data science, our matrices tend to be "tall", with $m \gg n$, and $\boldsymbol{A}^\mathsf{T}\boldsymbol{A} \in \mathbb{R}^{n\times n}$ is a nice, small matrix. Moreover, it tends to be a full-rank matrix because data points are usually measurements, and one measurement is very unlikely to be a linear combination of others. $\boldsymbol{A}\boldsymbol{A}^\mathsf{T} \in \mathbb{R}^{m\times m}$, on the other hand, is a horrible, huge matrix with a rank $r = n \ll m$, which means it is hopelessly rank-deficient.

When we have a "wide" matrix, however, it is the other Gram matrix, $\boldsymbol{A}\boldsymbol{A}^\mathsf{T} \in \mathbb{R}^{m\times m}$, that is the nice matrix. All we wrote down above for the left inverse will work for the right inverse with minor changes. Altogether, we get the following results:

$$\begin{aligned}
\text{Tall, full rank:} \quad & m > n,\ \text{rank}(\boldsymbol{A}) = n \quad \implies \quad \boldsymbol{A}^{-1}_{\textbf{Left}} = (\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} \\
\text{Wide, full rank:} \quad & m < n,\ \text{rank}(\boldsymbol{A}) = m \quad \implies \quad \boldsymbol{A}^{-1}_{\textbf{Right}} = \boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{A}^\mathsf{T})^{-1}
\end{aligned}$$

5.4 Computational Methods for Inverses and Systems

5.4.1 Gauss-Jordan Method to Compute Inverse

The inverse of $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, if it exists, is easily computed as:

$$\left[\boldsymbol{A} \mid \boldsymbol{I}\right] \xrightarrow{\text{RREF}} \left[\boldsymbol{I} \mid \boldsymbol{A}^{-1}\right]$$

We have two ways of seeing why this works. The first explanation below is more elegant and less complicated than the second one that follows.

A Using Block Multiplication

The first and most elegant explanation of why Gauss-Jordan elimination gives $\boldsymbol{A}^{-1}$ uses the **Special Case of Block-wise Multiplication** we discussed on page 42. If we have a block matrix $\left[\boldsymbol{A} \mid \boldsymbol{I}\right] \in \mathbb{R}^{n\times 2n}$, we can multiply it by another matrix $\boldsymbol{E} \in \mathbb{R}^{n\times n}$ as:

$$\boldsymbol{E}\left[\boldsymbol{A} \mid \boldsymbol{I}\right] = \left[\boldsymbol{EA} \mid \boldsymbol{EI}\right] = \left[\boldsymbol{EA} \mid \boldsymbol{E}\right]$$

If $\boldsymbol{E}$ happens to be the elementary matrix that takes $\boldsymbol{A}$ to its RREF, then the process of multiplying by $\boldsymbol{E}$ on the left is the same as performing Gauss-Jordan elimination: $\boldsymbol{E}\left[\boldsymbol{A} \mid \boldsymbol{I}\right] = \text{RREF}\left(\left[\boldsymbol{A} \mid \boldsymbol{I}\right]\right)$. And since we start with a full-rank, invertible $\boldsymbol{A}$, its RREF is nothing but $\boldsymbol{I}$. Putting it all together, we write:

$$\text{RREF}\left(\left[\boldsymbol{A} \mid \boldsymbol{I}\right]\right) = \left[\boldsymbol{EA} \mid \boldsymbol{E}\right] = \left[\boldsymbol{I} \mid \boldsymbol{E}\right]$$

Comparing the left blocks in the equation above, we get $\boldsymbol{EA} = \boldsymbol{I} \implies \boldsymbol{E} = \boldsymbol{A}^{-1}$, and we can see $\boldsymbol{E}$ sitting as the right block in the final term. In other words,

$$\left[\boldsymbol{A} \mid \boldsymbol{I}\right] \xrightarrow{\text{RREF}} \left[\boldsymbol{I} \mid \boldsymbol{A}^{-1}\right]$$

B Using Super-Augmented Matrices

The previous explanation was elegant because we worked at the level of matrices (using matrix algebra, as it were), rather than descending to the column/row/element level. If we need another explanation, we can get it by looking at $\boldsymbol{A}$ as encoding one system of equations $\boldsymbol{Ax} = \boldsymbol{b}$, and Gauss-Jordan giving us the solution. If we have multiple systems of equations with the same coefficient matrix, we can perform Gauss-Jordan elimination on all of them in one go.

Limiting ourselves to the simple case of a full-rank, square coefficient matrix $\boldsymbol{A}$ so that $\boldsymbol{A}^{-1}$ exists, if we have $\boldsymbol{Ax} = \boldsymbol{b}_1$ and $\boldsymbol{Ax} = \boldsymbol{b}_2$, we could write a "super" augmented matrix like $\left[\boldsymbol{A} \mid [\boldsymbol{b}_1\, \boldsymbol{b}_2]\right]$. If the RREF of this augmented matrix becomes $\left[\boldsymbol{I} \mid [\boldsymbol{b}'_1\, \boldsymbol{b}'_2]\right]$, we know that $\boldsymbol{b}'_1$ is the solution to the first system, and $\boldsymbol{b}'_2$ that of the second one. In other words:

$$\left[\boldsymbol{A} \mid [\boldsymbol{b}_1\, \boldsymbol{b}_2]\right] \xrightarrow{\text{RREF}} \left[\boldsymbol{I} \mid [\boldsymbol{b}'_1\, \boldsymbol{b}'_2]\right] \implies \boldsymbol{Ab}'_1 = \boldsymbol{b}_1 \quad \text{and} \quad \boldsymbol{Ab}'_2 = \boldsymbol{b}_2$$

More generally, we could write this complicated super system as (again, limiting ourselves to $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{B'} \in \mathbb{R}^{n \times n}$):

$$\boldsymbol{AX} = \boldsymbol{B} \implies \left[\boldsymbol{A} \mid \boldsymbol{B}\right] \xrightarrow{\text{RREF}} \left[\boldsymbol{I} \mid \boldsymbol{B'}\right]$$

Noting that $\boldsymbol{X}$ is a symbolic matrix (x_{ij} are symbols, not numbers) and $\boldsymbol{B'}$ is the solution that satisfies the equations, we can confidently write:

$$\boldsymbol{AB'} = \boldsymbol{B}$$

We know that there is nothing special about this $\boldsymbol{B}$. In other words, if we have an $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, we can build any number of systems of linear equations by an arbitrary choice of $\boldsymbol{B}$. Let's therefore choose $\boldsymbol{B} = \boldsymbol{I}$ and investigate what it says. With this choice, we get:

$$\boldsymbol{AB'} = \boldsymbol{I}$$

What does this equation mean? It says that $\boldsymbol{B'}$ is the matrix that, when multiplied with $\boldsymbol{A}$, results in the identity matrix $\boldsymbol{I}$, which is the definition of the inverse of $\boldsymbol{A}$. In other words, $\boldsymbol{B'} = \boldsymbol{A^{-1}}$.

Remembering again that $\boldsymbol{B'}$ is merely part of the result of Gauss-Jordan running on $\left[\boldsymbol{A} \mid \boldsymbol{B}\right]$, and that $\boldsymbol{B}$ now is $\boldsymbol{I}$, we get the same cryptic recipe for $\boldsymbol{A^{-1}}$:

$$\left[\boldsymbol{A} \mid \boldsymbol{I}\right] \xrightarrow{\text{RREF}} \left[\boldsymbol{I} \mid \boldsymbol{A^{-1}}\right]$$

Elaborating on it: To find the $\boldsymbol{A^{-1}}$,

1. Construct the augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{I}\right]$.
2. Run the Gauss-Jordan algorithm on it.
3. If we get the identity matrix $\boldsymbol{I}$ in the place of the original matrix $\boldsymbol{A}$ in $\left[\boldsymbol{A} \mid \boldsymbol{I}\right]$, the augmented part will now contain the inverse $\boldsymbol{A^{-1}}$.
4. If we cannot get $\boldsymbol{I}$, it means $\boldsymbol{A}$ is singular.

We see that Gauss-Jordan elimination (which is an extension of Gaussian elimination) gives us a recipe for computing the inverse of any invertible matrix $\boldsymbol{A}$. It so happens that the Gauss-Jordan method of finding the inverse is the one that is computationally most efficient.

5.4.2 Analytic Computation using Cofactors

In addition to the Gauss-Jordan algorithm, we also have an analytic formula, similar to the Laplace expansion for determinant computation, Eqn (3.5), to directly compute the inverse using the same cofactors we defined in §3.3.2, page 61.

Refreshing our memory, every element in a matrix $\boldsymbol{A} = \begin{bmatrix} a_{ij} \end{bmatrix}$ has a *cofactor* C_{ij}, which is the determinant of the submatrix obtained by deleting the ith row and j^{th} column of $\boldsymbol{A}$, with an associated sign. We can put these cofactors (being just numbers) in a matrix of the same size to get $\boldsymbol{C} = [C_{ij}]$. The *transpose* of this matrix of cofactors goes by many names, such as the *adjugate*, the *classical adjoint*, and the *adjoint* matrix.

Once we have the matrix of cofactors, we can compute its transpose (which is the adjugate matrix) and then the inverse as:

$$\boldsymbol{A^{-1}} = \frac{1}{|\boldsymbol{A}|}\boldsymbol{C^{\mathsf{T}}} \tag{5.2}$$

Although this equation looks compact, it is horribly expensive computationally. As we can see from the formula, when $|\boldsymbol{A}| = 0$, we cannot compute $\boldsymbol{A^{-1}}$ and the matrix $\boldsymbol{A}$ is singular.

CS Connection: The Cardinal Sin of Matrix Inversion


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

5.4.3 Inverse of a 2×2 Matrix

For a 2×2 matrix, we can apply Eqn (5.2) and write down its inverse.

$$\boldsymbol{A} = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \implies |\boldsymbol{A}| = ad - bc \text{ and } \boldsymbol{A^{-1}} = \frac{1}{ad - bc}\begin{bmatrix} d & -c \\ -b & a \end{bmatrix}$$

Remember this formula as: Swap the diagonal elements and switch the sign of the non-diagonal elements. And then divide by the determinant.

5.4.4 Cramer's Rule

Related to the matrix of cofactors and the analytic formulas for the determinant and inverse of matrices (Equations (3.5) and (5.2)) is a beautifully compact and elegant (albeit computationally useless) formula for solving a system of linear equations. This formula is known as Cramer's Rule. It states that for a system of linear equations $\boldsymbol{Ax} = \boldsymbol{b}$ with a square, invertible $\boldsymbol{A}$ and $\boldsymbol{x} = [x_j]$, the solution is given by:

$$x_j = \frac{|\boldsymbol{A}_j|}{|\boldsymbol{A}|} \tag{5.3}$$

where $\boldsymbol{A}_j$ is the matrix formed by replacing the jth column of $\boldsymbol{A}$ with $\boldsymbol{b}$.

To recap, we have learned four ways to solve the system of linear equations $\boldsymbol{Ax} = \boldsymbol{b}$:

1. Run Gaussian elimination on the augmented matrix $\big[\boldsymbol{A} \mid \boldsymbol{b}\big]$, followed by back substitution.

2. Perform Gauss-Jordan elimination on the augmented matrix $\big[\boldsymbol{A} \mid \boldsymbol{b}\big]$ (which is not really different from the previous method).

3. If $\boldsymbol{A}$ is a full-rank, square matrix, $\boldsymbol{Ax} = \boldsymbol{b} \implies \boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$. This is often written in some programs as $\boldsymbol{x} = \boldsymbol{A}\backslash\boldsymbol{b}$, indicating something like dividing by $\boldsymbol{A}$ on the left.

4. Use Cramer's Rule, again if $\boldsymbol{A}$ is a full-rank, square matrix.

As we can see, although methods (3) and (4) are elegant, they work only for the ideal situation of unique solutions. Besides, they are prohibitively expensive to compute for nontrivial matrix sizes (especially Cramer's Rule). To top it off, their numeric stability is also questionable.

CS Connection: The Computational Cost of Cramer's Rule


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

5.5 Summary: Algebraic View

We have completed the algebraic view of linear algebra, by which we mean determining the solvability conditions of systems of linear equations and characterizing their solutions. Along the way, we touched upon the concepts of ranks, determinants, inverses, and a couple of matrix decompositions.

While looking at linear equations, we spoke of their shapes in coordinate spaces: A linear equation is a line in $\mathbb{R}^2$ and a plane in $\mathbb{R}^3$, and their solutions are the intersections of these lines or planes. While this perspective is undoubtedly geometric, the geometric view of Linear Algebra that we will explore in the next part of the book reveals the deeper and richer structure of vector spaces, subspaces, and their properties. The insights from this geometry, as we shall see, have direct relevance to certain algorithms and a broader impact on how we discuss and present our ideas in computer science.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Part III

Geometric View

6

Vector Spaces, Bases and Dimensions

To those who do not know mathematics, it is difficult to get across a real feeling as to the beauty, the deepest beauty, of nature. . . If you want to learn about nature, to appreciate nature, it is necessary to understand the language that she speaks in.

—Richard Feynman



The notion of linear combinations is central to linear algebra, which is why we introduced it in the very first chapter of this book. Now that we are taking a geometric view of the subject, we will explore this concept—and its associated mechanics—in much greater detail and with formal rigor.

6.1 Linear Combinations

Let's start by reminding ourselves of the basic operations on vectors (and indeed matrices):

- Vectors can be scaled. A vector multiplied by a scalar is another vector, which means the collection of vectors is closed under scalar multiplication.

- Vectors can be added. The set of vectors is closed under vector addition as well: When we add vectors, we get other vectors.

We can do both these operations together: Take a bunch of vectors, scale them, and add the scaled versions, which is exactly what we mean by a linear combination. As we can see from the basic operations, the set of vectors is closed under linear combinations as well; when we take linear combinations, what we get are other vectors. Stating this rather obvious fact formally:

$$\begin{aligned} &\forall \boldsymbol{x_i} \in \mathbb{R}^n,\ s_i \in \mathbb{R} \text{ and any nonnegative integer } k, \\ &\qquad \boldsymbol{z} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2} + \cdots + s_k\boldsymbol{x_k} \\ &\qquad = \sum_{i=1}^{k} s_i\boldsymbol{x_i} \in \mathbb{R}^n \text{ is a linear combination.} \end{aligned} \tag{6.1}$$

6.1.1 Visualizing Linear Combinations

Let's look at a few examples of linear combinations:

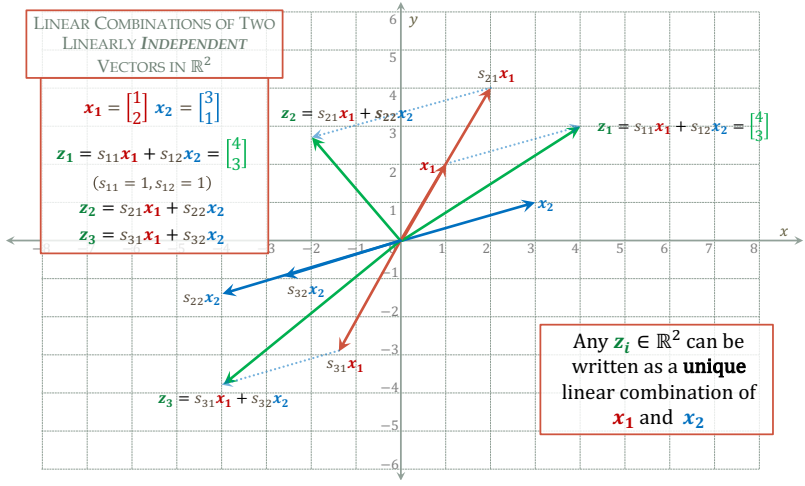


Fig. 6.1 Linear combinations of two vectors in $\mathbb{R}^2$. We can create all possible vectors in $\mathbb{R}^2$ starting from our $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$. Interactive Visualization

1. Two vectors in $\mathbb{R}^2$: In Figure 6.1, we start with two vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$. The first linear combination we make is the sum $\boldsymbol{z_1} = \boldsymbol{x_1} + \boldsymbol{x_2}$, which is the same as $\boldsymbol{z_1} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$, with the scalar multipliers s_1 and s_2 set to 1. Then we change the

values of the scalars s_1 and s_2 so as to get other vectors. As we can see, we can get to any vector in $\mathbb{R}^2$ by the appropriate choice of s_1 and s_2.

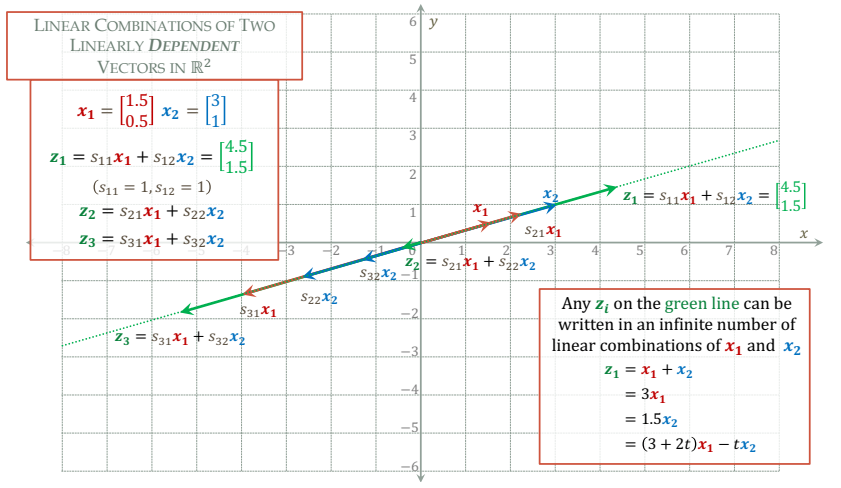


Fig. 6.2 Linear combinations of two vectors in $\mathbb{R}^2$. Starting from our $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, we cannot get out of the line defined by the vectors, no matter what scaling factors we try. The vectors are not *linearly independent*. Interactive Visualization

2. In Figure 6.2, we again have two vectors. In this case, however, when we take all possible linear combinations, we do not seem to be able to get to all vectors in $\mathbb{R}^2$; we are confined to one line. In order to see why, we have to first notice that the two vectors we defined were not independent of each other. Our second vector, $\boldsymbol{x_2}$, is a scalar multiple of $\boldsymbol{x_1}$: $\boldsymbol{x_2} = 2\boldsymbol{x_1}$. Therefore, when we take a linear combination $\boldsymbol{z} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$, what we are actually doing, in effect, is only scaling one vector:

$$\boldsymbol{z} = s_1\boldsymbol{x_1} + 2s_2\boldsymbol{x_1} = (s_1 + 2s_2)\boldsymbol{x_1} = s\boldsymbol{x_1}$$

All the scaled versions of $\boldsymbol{x_1}$ fall in the same direction as $\boldsymbol{x_1}$.

3. In the third example, Figure 6.3, we move on to $\mathbb{R}^3$, but with vectors very similar to the first case, but with a third component (because vectors in $\mathbb{R}^3$ have three components) equal to zero. When we take all possible linear combinations $\boldsymbol{z} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$, we can see that the third component of $\boldsymbol{z}$ can never be anything other than zero, and therefore all such linear combinations live on the xy plane.

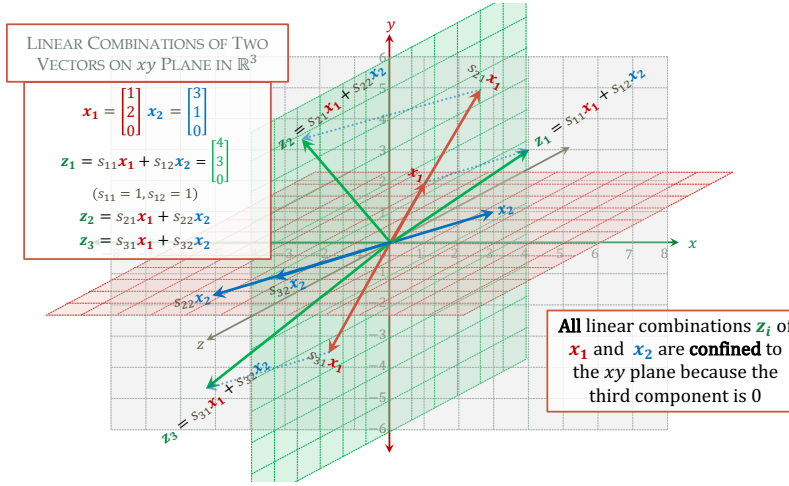


Fig. 6.3 Linear combinations of two vectors in $\mathbb{R}^3$. All the vectors we create starting from our $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ live on the xy plane. We do not have enough vectors to *span* the whole of $\mathbb{R}^3$.

Interactive Visualization

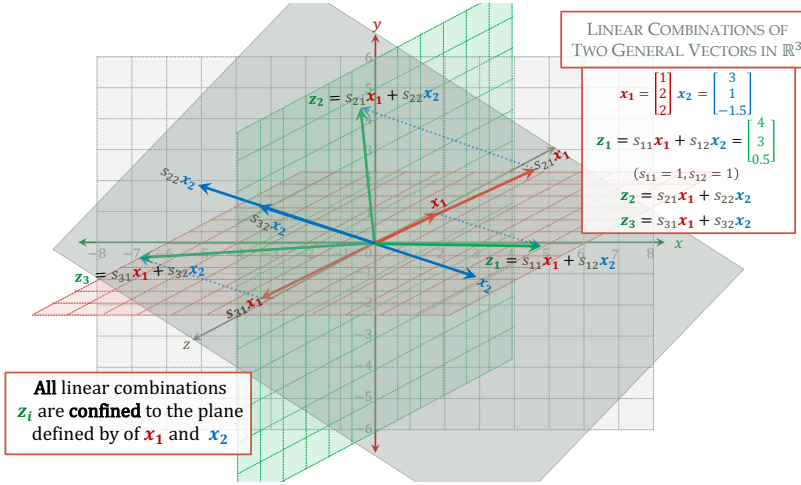


Fig. 6.4 Linear combinations of two vectors in $\mathbb{R}^3$. Here again, we do not have enough vectors to span $\mathbb{R}^3$. The linear combinations of $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ live in a plane defined by the two vectors.

Interactive Visualization

4. In Figure 6.4, we have added some nonzero values as the third component of $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$. Still, the linear combinations of the two vectors, $\boldsymbol{z} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$, all fall on a plane defined by the two vectors, or, equivalently, by the three points $(0, 0, 0)$,

$(1, 2, 2)$, and $(3, 1, -1.5)$. Two vectors are not enough to span $\mathbb{R}^3$.

5. As we can also see from Figure 6.4, we have a hard time depicting general vectors in $\mathbb{R}^3$. In this fifth example, we are going to move to three vectors, but will not attempt to draw them. If we have the following three vectors, we still cannot get all vectors in $\mathbb{R}^3$:

$$\boldsymbol{x_1} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \quad \boldsymbol{x_2} = \begin{bmatrix} 3 \\ 1 \\ 0 \end{bmatrix} \quad \boldsymbol{x_3} = \begin{bmatrix} 11 \\ 17 \\ 0 \end{bmatrix}$$

   In addition to the two vectors in Figure 6.3, we have added a third vector, but with its third component equal to 0. Now we take all possible linear combinations $\boldsymbol{z} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2} + s_3\boldsymbol{x_3}$. The third component of $\boldsymbol{z}$ will have to be zero no matter what scaling factors we try, and we are confined to the xy plane. Although it may not be obvious, $\boldsymbol{x_3}$ is a linear combination of $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$, which we can see by solving for s_1 and s_2 in the equation $\boldsymbol{x_3} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$.

6. In the next example, we start with the two vectors in Figure 6.4 and add a third vector, which we know is a linear combination of the two. We have these three vectors in $\mathbb{R}^3$:

$$\boldsymbol{x_1} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} \quad \boldsymbol{x_2} = \begin{bmatrix} 3 \\ 1 \\ -1.5 \end{bmatrix} \quad \boldsymbol{x_3} = \begin{bmatrix} 9 \\ 8 \\ 3 \end{bmatrix} = 3\boldsymbol{x_1} + 2\boldsymbol{x_2}$$

   The three vectors, being linear combinations of each other, are on the same plane in $\mathbb{R}^3$. All other linear combinations we can make with them will fall on the same plane.

7. In the last example, we will start from the simpler vectors in Figure 6.3, which live on the xy plane in $\mathbb{R}^3$. We add a simple third vector with a component in the third direction:

$$\boldsymbol{x_1} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \quad \boldsymbol{x_2} = \begin{bmatrix} 3 \\ 1 \\ 0 \end{bmatrix} \quad \boldsymbol{x_3} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

   Note that none of these three vectors can be written as the linear combination of the other two. Now we are no longer confined

to any plane. We can indeed create any vector in $\mathbb{R}^3$ as a linear combination of these three.

We have listed seven examples with many permutations of linear combinations of vectors above. In order to make sense of them, and to introduce some terms that we will define very soon, let's take stock of what we have done. In the first example, we had two *linearly independent* vectors in $\mathbb{R}^2$. Two linearly independent vectors are enough to *span* $\mathbb{R}^2$, which is the collection of all possible vectors with two components. We will soon call $\mathbb{R}^2$ a *vector space*. In the second example, we had two vectors, but they were not linearly independent. What they spanned was a subset of $\mathbb{R}^2$, which we will call a *subspace*. In the third example, we went on to $\mathbb{R}^3$, but with two linearly independent vectors, which were not enough to span the whole vector space. Instead, they spanned a subspace that was the xy plane. In the fourth example, we again had only two vectors in $\mathbb{R}^3$, spanning a subspace. In the fifth and sixth examples, we added a third vector, but it was not linearly independent of the other two, and therefore, the spans of the three vectors were still only subspaces. Finally, in the seventh example, we had three good, linearly independent vectors that could span the entirety of $\mathbb{R}^3$.

CS Connection: RGB Colors as Linear Combinations

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Our job now is to define the unfamiliar terms in the previous paragraph, express them in mathematical language, and generalize them to higher dimensions, $\mathbb{R}^n$. Clearly, we will not be able to visualize them, but that inability will not prevent us from developing insights and intuitions that will apply to all dimensions and datasets.

6.1.2 Spans of Vectors

Definition: The set of all possible linear combinations of a given set of vectors is their span. Written mathematically, given k vectors

$\boldsymbol{x_i} \in \mathbb{R}^n$, their span is defined as:

$$\text{span}(\{\boldsymbol{x_i}\}) = \left\{ \boldsymbol{z} \mid \boldsymbol{z} = \sum_{i=1}^{k} s_i \boldsymbol{x_i} \right\} \text{ for any } k \text{ scalars } s_i \in \mathbb{R} \tag{6.2}$$

In Figure 6.1, the span of the red and blue vectors is all possible vectors in $\mathbb{R}^2$. In Figure 6.2, the span of the two vectors is a much smaller subset: Only those vectors that are in the same direction (the dotted line in the figure) as the original two, collinear vectors.

6.1.3 Cardinality

Definition: The number of vectors in the span is called its cardinality. Cardinality is just a fancy name for the size or the number of elements in a set. It applies to our notion of span of vectors as well as the set of vectors that define a span, as in Eqn (6.2) for instance. The cardinality of the former is indeed infinity (as there are an infinity of linear combinations we can form), while the cardinality of the latter (the vectors of which we are forming the linear combinations) is k.

6.1.4 Linear Independence

Definition: The vectors in a set are linearly independent if we cannot write any one of them as a linear combination of the others. An equivalent definition, again in mathematical lingo, states that a given set of k vectors $\boldsymbol{x_i} \in \mathbb{R}^n$ is linearly independent if and only if the only solution to the equation

$$\sum_{i=1}^{k} s_i \boldsymbol{x_i} = \boldsymbol{0} \tag{6.3}$$

is the trivial solution where all scalars $s_i = 0$. Clearly, with all $s_i = 0$, the sum is always the zero vector. What we are looking for is a set of scaling factors with at least some nonzero numbers such that the sum is $\boldsymbol{0}$. If we can find such a set, the vectors are not linearly independent. They are linearly dependent.

Looking at the example in Figure 6.2, we have $\boldsymbol{x_2} = 2\boldsymbol{x_1} \implies \boldsymbol{x_2} - 2\boldsymbol{x_1} = \boldsymbol{0}$, which gives us the zero vector as a linear combination with nonzero scalars, meaning $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ are not linearly independent. Similarly, for the sixth example in the list above, $\boldsymbol{x_3} = 3\boldsymbol{x_1} + 2\boldsymbol{x_2}$, which rearranges to $3\boldsymbol{x_1} + 2\boldsymbol{x_2} - \boldsymbol{x_3} = \boldsymbol{0}$. Once again, we have the

zero vector as a linear combination with nonzero scalars, implying linear dependence.

6.2 Vector Spaces and Subspaces

One of the critical notions in linear algebra is that of a vector space, also called a linear vector space. Conceptually, it is more intimately connected to sets rather than space. Let us take a closer look.

6.2.1 Vector Spaces

For our purposes in this book, we will define a vector space as a set of all possible vectors. In fact, while defining vectors, their operations, and their properties, we were indirectly defining vector spaces as well. For instance, we said when we scale a vector or add two vectors, we get another vector, which is the same as saying that the *set* of vectors is closed under scalar multiplication and vector addition. We also listed the associativity, commutativity, and distributivity properties of vector operations, which are, in fact, properties of the underlying vector space. Although mathematical puritans may look askance at such sloppiness, we will consider it a pardonable foible of computer scientists. We will, however, present the full and precise definition in all its mathematical glory in §8.1, page 158.

Vector Space

Definition: The set $\mathcal{V}$ of all vectors $\boldsymbol{x} \in \mathbb{R}^n$ is a vector space (AKA linear space) if the operations of scalar multiplication and vector addition (with the right properties as specified earlier) are defined for $\boldsymbol{x}$, and $\mathcal{V}$ is closed under the operations.

As we mentioned earlier, any mathematical object that can fulfill the requirements of vectors is a vector, as far as linear algebra is concerned. And a closed set of any such vectors would be a vector space. Here are some examples of vector spaces:

Coordinate Space: The usual Euclidean space of points is a vector space, treating the numbers and coordinates in them as vectors. For instance, the good old number line (same as $\mathbb{R}$) is a vector

space. If we treat the (x, y) coordinates in $\mathbb{R}^2$ as vectors, the two-dimensional plane, being a collection of such coordinates, is a vector space. Such spaces ($\mathbb{R}^n$) are called coordinate spaces, naturally.

Space of Matrices: Matrices also have operations identical to vectors, as we defined them. Vectors are indeed single-column matrices. Therefore, we can treat a set of matrices (of the same size) as a vector space. For instance, if we create a set $\mathcal{M} = \{\boldsymbol{A} \mid \boldsymbol{A} \in \mathbb{R}^{m \times n}\}$, it would be a vector space.

Complex Numbers: The set of complex numbers ($\mathbb{C}$) is a vector space as well, very similar to the coordinate space of $\mathbb{R}^2$.

6.2.2 Subspace

Definition: Given a vector space $\mathcal{V}$, a subset of it is a vector (or linear) subspace $\mathcal{S}$ if it is closed under the same operations of scalar multiplication and vector addition defined in $\mathcal{V}$. In other words, for any vectors $\boldsymbol{x}_i \in \mathcal{S}$, the same operations (as defined in $\mathcal{V}$) applied on them result in other vectors $\boldsymbol{y}_i \in \mathcal{S}$.

In practice, we can say that all linear combinations of the vectors in $\mathcal{S}$ have to be in $\mathcal{S}$ for it to qualify as a subspace. Therefore, a practical definition of a subspace $\mathcal{S} \subseteq \mathcal{V}$ is that it is the span of a given set of vectors in $\mathcal{V}$.

In Figure 6.2, we have two vectors, $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, that are scaled versions of each other. All their linear combinations (which are, in fact, just scaled versions of $\boldsymbol{x}_1$) form a line in $\mathbb{R}^2$. This line represents a subspace because, when we add any two vectors in this line, we get another vector that is collinear with it. Note that the zero vector has to be a part of any subspace because the trivial linear combination of the vectors (that define the span, which is the subspace) with zero scalars is still a linear combination and has to be in the subspace.

6.2.3 Unions and Intersections

If we have two subspaces, their intersection is a subspace, but their union, in general, is not. Let's formally prove the intersection part of this statement.

1. Let $\mathcal{S}_1$ and $\mathcal{S}_2$ be the subspaces and $\mathcal{I} = \mathcal{S}_1 \cap \mathcal{S}_2$, with $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{I}$.

2. By the definition of the intersection, $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{S}_1$ and $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{S}_2$.
3. Since $\mathcal{S}_1$ is a subspace, for any two scalars, s_1 and s_2, $s_1\boldsymbol{x}_1 + s_2\boldsymbol{x}_2 \in \mathcal{S}_1$.
4. Since $\mathcal{S}_2$ is a subspace, for the same scalars, $s_1\boldsymbol{x}_1 + s_2\boldsymbol{x}_2 \in \mathcal{S}_2$ as well.
5. Therefore, $s_1\boldsymbol{x}_1 + s_2\boldsymbol{x}_2 \in \mathcal{I}$, since it is in both $\mathcal{S}_1$ and $\mathcal{S}_2 \implies \mathcal{I} = \mathcal{S}_1 \cap \mathcal{S}_2$ is a subspace.

There is a strict condition under which the union of two subspaces is also a subspace: it occurs if and only if one subspace is entirely contained within the other. Mathematically, this means one must be a subset of the other:

$$\mathcal{S}_1 \subseteq \mathcal{S}_2 \implies \mathcal{S}_1 \cup \mathcal{S}_2 = \mathcal{S}_2 \quad \text{(which is a subspace)}$$

$$\mathcal{S}_2 \subseteq \mathcal{S}_1 \implies \mathcal{S}_1 \cup \mathcal{S}_2 = \mathcal{S}_1 \quad \text{(which is a subspace)}$$

CS Connection: Subspaces and Linear Constraints

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

6.2.4 Orthogonal Subspaces

Orthogonality in linear algebra is a generalization of perpendicularity in geometry; the latter applies to lines or other shapes where the concept of angles makes sense. Two lines are perpendicular when the angle between them is $90°$. On the other hand, two vectors are orthogonal when their dot product is zero. In our definition of vectors for this purpose, there is little difference between orthogonal vectors and perpendicular vectors, but there can be with other definitions of vectors and their dot products. We shall, therefore, never again talk of perpendicular vectors.

We can extend the concept of orthogonality to vector subspaces as well. Two subspaces are orthogonal to each other when every vector in one subspace is perpendicular to every vector in the other one.

Here are two subspaces: The xy plane and the yz plane in $\mathbb{R}^3$. Are they orthogonal subspaces? From the geometric shapes, they look like two planes at right angles to each other. But is every vector in the xy subspace orthogonal to those in the yz subspace? Clearly not, because we have a whole line of vectors along the y axis that are in both subspaces. They are clearly not orthogonal to themselves. Any

Notational Abuse

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
Available in the Full Version of the book.

two subspaces with a nontrivial (meaning more than the zero vector, $\mathbf{0}$) intersection, therefore, cannot be orthogonal to each other. Note that all subspaces have to contain the zero vector.

To look at a positive example, the xy plane and the z axis can be considered two subspaces. They are indeed orthogonal to each other. Every vector on the xy plane is orthogonal to every vector along the z axis. Here is a generalized proof using symbolic vectors.

1. A general vector on the xy plane and a general vector along the z axis have the form:
$$\boldsymbol{x} = \begin{bmatrix} x \\ y \\ 0 \end{bmatrix} \quad \boldsymbol{z} = \begin{bmatrix} 0 \\ 0 \\ z \end{bmatrix}$$
2. The dot product between any vector on the xy plane and any vector along the z axis is therefore:
$$\boldsymbol{x} \cdot \boldsymbol{z} = \boldsymbol{x}^{\mathsf{T}} \boldsymbol{z} = 0 \implies \boldsymbol{x} \perp \boldsymbol{z}$$
3. Therefore, by the definition of orthogonal subspaces, the xy plane $\perp$ z axis.

Both perpendicularity and orthogonality are indicated by the same symbol $\perp$.

6.3 Bases and Dimensions

6.3.1 Bases

Definition: A basis of a vector space or a vector subspace is a set of vectors that meets two criteria:

1. They span the vector space or the vector subspace.
2. They are linearly independent.

The vectors in the basis are called basis vectors, obviously. As we can see, the concept of a basis builds on the related concepts of linear combinations and spans of vectors. We can define a basis in a variety of ways. Here is another definition: A basis of a vector space or vector subspace is the minimum set of vectors that span it. Yet another one: A set of vectors in a vector space (or subspace) is called a basis if every element in it can be written as a unique and finite linear combination of them. All these definitions are indeed equivalent.

Illustrating it with a couple of examples and counterexamples:

- In Figure 6.1 illustrating linear combinations, we have two vectors and linear combinations like $\boldsymbol{z}$:

$$\boldsymbol{x_1} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \text{ and } \boldsymbol{x_2} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \quad \text{Test vector: } \boldsymbol{z} = \begin{bmatrix} 5 \\ 1 \end{bmatrix} \in \mathbb{R}^2$$

  $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ span all of $\mathbb{R}^2$ and are linearly independent. Therefore, they form a basis for the vector space $\mathbb{R}^2$. Given any vector $\boldsymbol{z} \in \mathbb{R}^2$, we can find unique scalars s_1 and s_2 such that $\boldsymbol{z} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$.

- In Figure 6.2, we have two candidate basis vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$:

$$\boldsymbol{x_1} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} \text{ and } \boldsymbol{x_2} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \quad \text{Test vector: } \boldsymbol{z} = \begin{bmatrix} 5 \\ 1 \end{bmatrix} \in \mathbb{R}^2$$

  $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ do not span all of $\mathbb{R}^2$ and are not linearly independent. Therefore, they are not a basis for the vector space $\mathbb{R}^2$. Given our test vector $\boldsymbol{z}$, we are not able to find scalars s_1 and s_2 such that $\boldsymbol{z} = s_1\boldsymbol{x_1} + s_2\boldsymbol{x_2}$.

- However, considering the subspace indicated by the line in the same Figure 6.2, are the two vectors in the previous example a basis? The vectors $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ do span this subspace, but they are still not a basis because they are not linearly independent. For this reason, a new vector $\boldsymbol{z_2}$, which is in this subspace, can be expressed as an infinite number of linear combinations of $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$.

- The reason why the two vectors did not form a basis for the subspace in the previous example was that we had too many

vectors: for a subspace formed as a span of one vector (which is a line of vectors), we need only one vector in the basis. Similarly, for a subspace that looks like a plane (span of two vectors) as in Figure 6.4, we need exactly two vectors in the basis. If we had a third vector, it is necessarily a linear combination of the other two that are already in the basis, and is therefore not linearly independent.

CS Connection: Alternate Bases and Data Compression

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

6.3.2 Dimensions

Definition: The dimension of a vector space or subspace is the number of vectors in its basis. Or, in fancier language, the dimension is the cardinality of any basis of the space or subspace.

Note that a vector space or vector subspace (which we may as well start calling spaces and subspaces, dropping the vector part, now that we know them well enough) may have many different bases, but all of them will have the same cardinality. The dimension of a space or a subspace is an immutable property.

Earlier, we spoke of orthogonal subspaces. We can restate orthogonality in terms of their bases: Two subspaces are orthogonal to each other if all the basis vectors of one are orthogonal to all the basis vectors of the other.

In $\mathbb{R}^3$, we spoke of the xy plane as a subspace being orthogonal to the z axis (taking the liberty to mix vector and coordinate spaces). The dimension of the first space is 2 and the second one is 1, adding up to the dimension of the containing space $\mathbb{R}^3$. We saw that the xy plane and the yz plane do not form orthogonal subspaces, and one reason is that their dimensions add up to 4, which is more than the dimension of $\mathbb{R}^3$.

In $\mathbb{R}^4$, however, we will be able to find two-dimensional subspaces that are orthogonal to each other. Remember that subspaces cannot have nontrivial (nonzero) intersections. Therefore, we can have two planes intersecting at a point in four dimensions.

6.3.3 Orthogonal Complements

Definition: If we have a subspace $\mathcal{S} \subseteq \mathbb{R}^n$ of dimension r, then its orthogonal complement $\mathcal{S}^\perp$ is the collection of all vectors $\boldsymbol{x} \in \mathbb{R}^n$ that are orthogonal to *all* vectors in $\mathcal{S}$:

$$\mathcal{S}^\perp = \left\{ \boldsymbol{x} \mid \boldsymbol{x}^\mathsf{T} \boldsymbol{y} = 0 \ \forall \ \boldsymbol{y} \in \mathcal{S} \right\}$$

As a consequence, the dimension of $\mathcal{S}^\perp$ is $n - r$, so that the dimensions of the subspace and its orthogonal complement add up to the dimension of the containing space.

In the earlier example of orthogonal subspaces, the xy plane and the z axis (in the coordinate space $\mathbb{R}^3$) are, in fact, more than orthogonal to each other. They are orthogonal complements, which means there are no other dimensions left in the space beyond what is accounted for by these two. To look at a counterexample, the x and z axes define orthogonal subspaces, but not orthogonal complements.

Keep in mind that although all the dimensions are accounted for when we think of orthogonal subspaces (such as the xy plane and the z axis in $\mathbb{R}^3$), there are still plenty of vectors not in either. In fact, most vectors in $\mathbb{R}^3$ are not on the xy plane or the z axis; they are linear combinations of the ones in these two subspaces.

6.4 Geometry of Linear Equations

Earlier, we listed a set of equations in Table 4.1 and visualized some of them in Figure 4.1, as part of the algebraic view of solving equations. Our linear equations formed lines in two-dimensional xy planes (which are *coordinate* spaces, not *vector* spaces, underscoring the need for care in distinguishing between them as described in the box on "**Notational Abuse**"). Now we want to look at the equations again as vectors in some vector space, where we present the geometry in a completely different way.

6.4.1 The Column Picture

Looking at the system of linear equations $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ one row at a time, as an equation making a shape in the coordinate space, is the row picture. Our equations in Figure 4.1 have two variables each, and the row picture produces visualizations in the coordinate space

of $\mathbb{R}^2$. In general, for $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, the row picture operates in $\mathbb{R}^n$ (where n is the number of variables). Our new way of looking at the equations, which we will call the column picture, will mean that we are working in $\mathbb{R}^m$, with a dimension equal to the number of rows in $\boldsymbol{A}$, or, equivalently, the number of equations in the system. In our examples in Figures 4.1 and 6.5 to 6.7, if the row picture looks simpler than the column picture, it is only because we have two variables, and potentially more than two equations. As the coefficient matrix $\boldsymbol{A}$ becomes large, both pictures become equally challenging to visualize, but the column view gives us deeper insights.

CS Connection: GPUs and the Column Picture

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

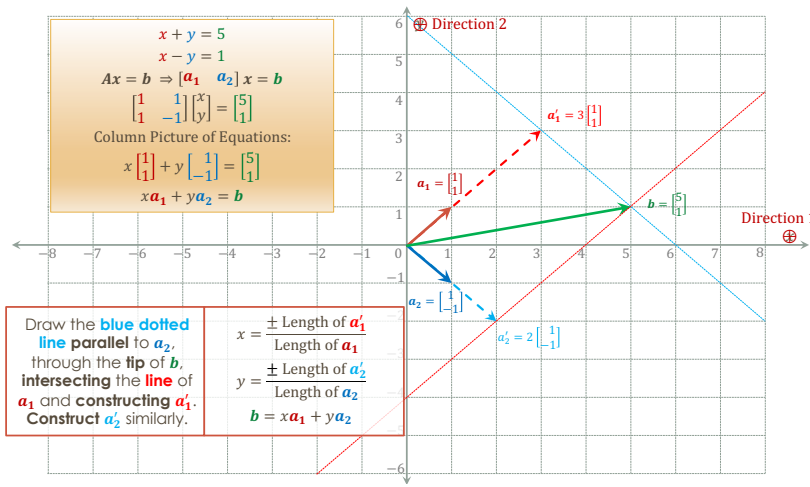


Fig. 6.5 Two linear equations on two unknowns with a unique solution. The scalars for the column vectors of $\boldsymbol{A}$ to produce $\boldsymbol{b}$ as linear combinations are the solution to the system of equations. Note that the length of $\boldsymbol{a_i'}$ is signed (as indicated by $\pm$): It is positive if $\boldsymbol{a_i'}$ is in the same direction $\boldsymbol{a_i}$, negative otherwise.

In Figure 6.5, we have our favorite system of two equations, this time shown as a linear combination of vectors that form the columns of the coefficient matrix $\boldsymbol{A}$. Here is the system:

$$\begin{aligned} x + y &= 5 \\ x - y &= 1 \end{aligned} \quad \boldsymbol{Ax} = \boldsymbol{b} \quad \boldsymbol{A} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \ \boldsymbol{b} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

By the column picture of matrix multiplication, we know that the product $\boldsymbol{Ax}$ is a linear combination of the columns of $\boldsymbol{A}$, taken with the scaling factors in $\boldsymbol{x}$, which are x and y. The system of equations $\boldsymbol{Ax} = \boldsymbol{b}$ says that the solution is the correct values for the scalars in the linear combination of columns of $\boldsymbol{A}$ that will yield $\boldsymbol{b}$. How we find these right values is depicted in Figure 6.5. Note that we had to relabel the axes (in Figures 6.5 to 6.7) as Directions 1 and 2 because they are indicators in a vector space now, not in the coordinate space. In particular, the x and y in our equations have nothing to do with these directions, once again highlighting the need for care described in the box above on notational abuse.

To convince ourselves that this system does have a solution in this case, let's outline, as a series of steps, or an algorithm of sorts, how we can get to the right values for the scalars[1] referring to Figure 6.5:

1. Draw a line parallel to the second blue vector (the second column of $\boldsymbol{A}$, $\boldsymbol{a_2}$), going through the tip of the green $\boldsymbol{b}$ vector. It is shown in blue as a thin dashed line.

2. Scale the first red vector (the first column of $\boldsymbol{A}$, $\boldsymbol{a_1}$) to reach this line. The scaling required tells us what the scalar should be. For our equations, the scalar for $\boldsymbol{a_1}$ is $x = 3$ in $\boldsymbol{b} = x\boldsymbol{a_1} + y\boldsymbol{a_2}$.

3. Similarly, draw the red dashed line parallel to $\boldsymbol{a_1}$, going through the tip of the green $\boldsymbol{b}$ vector.

4. Scale $\boldsymbol{a_2}$ to reach this line, which gives us the scalar $y = 2$ in $\boldsymbol{b} = x\boldsymbol{a_1} + y\boldsymbol{a_2}$.

Following the steps listed above, we can see that the solution is the scalars needed for $\boldsymbol{a_1}$ and $\boldsymbol{a_2}$, giving us $(x, y) = (3, 2)$. Although we presented our steps above as an "algorithm," we should point out that it is never used as a method for solving the equations. It is

[1] In listing these steps, we break our own rule about notational abuse: We talk about drawing lines in the vector space, which we cannot do. A vector space contains only vectors; it does not contain lines. It is coordinate spaces that contain lines. This predicament of ours shows the difficulty in staying absolutely rigorous about concepts. Perhaps pure mathematical rigor for its own sake is not essential, especially for an applied field like computer science.

an algorithm only for mentally constructing the requisite scalars for the right linear combination, illustating that it is possible to do so, and that the scalars are unique in this case of linear equations with a unique solution.

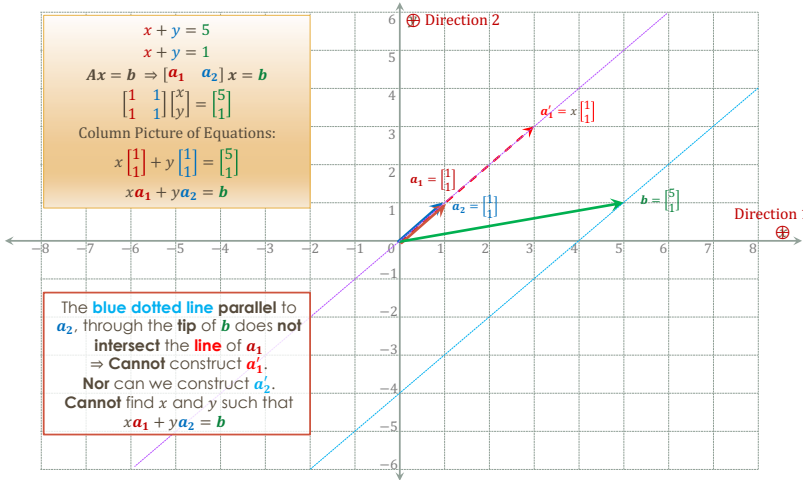


Fig. 6.6 Two inconsistent linear equations on two unknowns with no solution. All linear combinations of the column vectors of $\boldsymbol{A}$ fall on the purple line, and the right hand side does not. Therefore, $\boldsymbol{b}$ is not in the span of $\boldsymbol{a_1}$ and $\boldsymbol{a_2}$. Hence no solution. Note that we have drawn the red and blue vectors slightly offset from each other for visibility; they are supposed to be on top of each other.

6.4.2 Handling Inconsistent Systems

Let's now move on to the case where we have two linear equations that are not consistent with each other: $x + y = 5$ and $x + y = 1$. In Figure 4.1, we saw that they were parallel lines, which would never meet. What do they look like in our advanced geometric view in the vector space? The column vectors of the coefficient matrix are now identical. As we now know, all possible linear combinations of the two vectors $\boldsymbol{a_1}$ and $\boldsymbol{a_2}$ fall in a subspace that is a line defined by the direction of either of them. Their span, to use the technical term, is only a subspace. The green vector $\boldsymbol{b}$ that we would like to create is not along this line, which means no matter what scalars we try, we will never be able to get $\boldsymbol{b}$ out of $\boldsymbol{a_1}$ and $\boldsymbol{a_2}$. The system has no solutions.

If we try the steps of our little construction algorithm above, we see that while we can draw a line parallel to $\boldsymbol{a_2}$ going through the tip of $\boldsymbol{b}$, there is no scaling factor (other than ∞, to be absolutely rigorous) that will take $\boldsymbol{a_1}$ to this line.

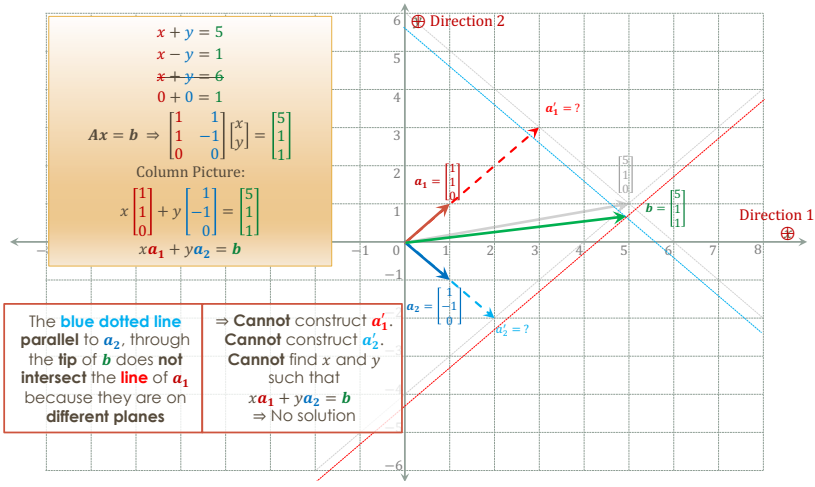


Fig. 6.7 Three linear equations on two unknowns with no solution. Here, $\boldsymbol{a_1}$ and $\boldsymbol{a_2}$ are not collinear, but span the plane of the page. The right hand side, $\boldsymbol{b}$, has a component in the third direction, perpendicular to the page coming toward us, indicated only by the shadow that $\boldsymbol{b}$ casts.

In Figure 6.7, we have three equations, with the third one inconsistent with the first two. The geometric view is in three dimensions, as opposed to the algebraic visualization of the equations, which still stays in two dimensions because of the number of variables. In other words, the geometric view is based on the column vectors of $\boldsymbol{A}$, which have as many elements as equations, or the number of rows of $\boldsymbol{A}$. The algebraic view is based on the number of unknowns, which is the same as the number of columns of $\boldsymbol{A}$.

CS Connection: Inconsistent Systems in Data Science

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

The fact that the vector space now has three directions makes it harder for us to visualize it. We have simplified it: First, we reduced

the third equation to a simpler form. Secondly, we indicate the third direction (assumed to be roughly perpendicular to the page, coming toward us) only by the shadows that the vector and the dashed lines would cast if we were to shine light on them from above the page.

Here are the equations, the columns of $\boldsymbol{A}$ and the RHS:

$$\begin{aligned} &x + y = 5 \\ &x - y = 1 \\ &x + y = 6 \text{ (reduced to } 0 = 1) \end{aligned} \qquad \boldsymbol{a_1} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \boldsymbol{a_2} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \qquad \boldsymbol{b} = \begin{bmatrix} 5 \\ 1 \\ 1 \end{bmatrix}$$

Running our construction algorithm on this system:

1. Drawing a line parallel to $\boldsymbol{a_2}$ (blue vector), going through the tip of the green $\boldsymbol{b}$, we get the thin dashed line in blue. This line is one unit above the plane of the page (because the third component of $\boldsymbol{b}$ is one).

2. Trying to scale the first red vector ($\boldsymbol{a_1}$) to reach this blue dashed line, we fail because the scaled versions go under the blue line.

3. Similarly, we fail trying to scale the blue $\boldsymbol{a_2}$ as well, for the same reason. We cannot find x and y such that $\boldsymbol{b} = x\boldsymbol{a_1} + y\boldsymbol{a_2}$, because of the pesky, nonzero third component in $\boldsymbol{b}$.

4. However, we can see that the shadows of these red and blue dashed lines (shown in grey) on the plane of the page do meet at the tip of the shadow of $\boldsymbol{b}$. Think of these shadows as projections and we have a teaser for a future topic.

In Figure 6.7, we took a coefficient matrix such that the third components of its column vectors were zero so that we could visualize the system relatively easily: Most of the action was taking place on the xy-plane. In a general case of three equations on two unknowns, even when we have nonzero third components, the two column vectors still make a plane as their span. If the RHS vector is in the span of the two vectors, we get a unique solution. If not, the equations are inconsistent and we get no solution.

The geometric view of linear algebra, much like the algebraic view, concerns itself with the solvability conditions, the characteristics of the solutions *etc.*, but from the geometry of the vector spaces associated with the coefficient matrix (as opposed to row-reduction type operations in the algebraic view). As we saw in this chapter, and as

we will appreciate even more in later chapters, the backdrop of the geometric view is the column picture of matrix multiplication.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

7

Bases, Orthogonality, and the Gram-Schmidt Process

Mathematics takes us still further from what is human, into the region of absolute necessity, to which not only the world, but every possible world, must conform.

—Bertrand Russell



Throughout this book, we have represented vectors as column matrices, with their numerical components stacked vertically. But where do these specific numbers actually come from? How are they determined? They are, in fact, dictated by an underlying basis that we haven't formally discussed until now. In this chapter, we will expand our understanding of bases, learn how vector components shift when we change from one basis to another, and explore the desirable properties of basis vectors.

7.1 Bases and Components

A basis of a space[1] (such as $\mathbb{R}^n$) is the minimal set of vectors that span it. *Minimal* implies that the basis vectors are linearly independent. In $\mathbb{R}^n$, we can have a maximum of n linearly independent vectors $\boldsymbol{a_i} \in \mathbb{R}^n$. Any set of such n linearly independent vectors forms a basis for $\mathbb{R}^n$. Once we have a basis, we can express any vector $\boldsymbol{x} \in \mathbb{R}^n$ as a *unique* linear combination of those basis vectors. The components (or coordinates) of $\boldsymbol{x}$ are simply the coefficients (the scaling factors) of this linear combination. Because the linear combination is unique, the components are unique as well.

7.1.1 Components of a Vector

Definition: For any $\boldsymbol{x} \in \mathbb{R}^n$, if the set of n vectors $\boldsymbol{a_i}$ (which are columns of a matrix $\boldsymbol{A}$) forms a basis for $\mathbb{R}^n$ and

$$\boldsymbol{x} = \sum_{i=1}^{n} x_{i|\boldsymbol{A}} \boldsymbol{a_i} \tag{7.1}$$

then the n scalars $x_{i|\boldsymbol{A}} \in \mathbb{R}$ are the components (or coordinates) of $\boldsymbol{x}$ in the basis $\boldsymbol{a_i}$, or in the basis matrix $\boldsymbol{A}$.

If we place these basis vectors as the columns of a matrix, we get a square matrix $\boldsymbol{A} = \begin{bmatrix} \boldsymbol{a_i} \end{bmatrix} \in \mathbb{R}^{n \times n}$. Because we can represent a set of basis vectors this way, we will start referring to the matrices themselves as the basis of spaces and subspaces. This is why we denote the components of $\boldsymbol{x}$ in the $\boldsymbol{A}$ basis as $x_{i|\boldsymbol{A}}$ in Eqn (7.1).

7.1.2 The Identity Matrix as a Basis

To answer the question that opened this chapter: We already had components for our vectors, but where did they come from? These default components are, in fact, derived using the identity matrix as the underlying basis. Let's illustrate this with an example. Consider a vector $\boldsymbol{x} \in \mathbb{R}^2$:

$$\boldsymbol{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \boldsymbol{I}\boldsymbol{x} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 3 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 2\boldsymbol{q_1} + 3\boldsymbol{q_2}$$

[1] From this chapter onward, when we say space or subspace, we mean a vector space or a vector subspace.

Here, we've called the basis vectors $\boldsymbol{q_1}$ and $\boldsymbol{q_2}$. (Some textbooks, especially in physics, refer to these unit vectors as $\hat{i}$ and $\hat{j}$, writing $\boldsymbol{x} = 2\hat{i} + 3\hat{j}$). In either notation, the vector has a component of 2 in the first direction and 3 in the second. This demonstrates how a two-dimensional vector gets its components from $\boldsymbol{I_2}$, a concept that naturally extends to higher dimensions: for any $\boldsymbol{x} \in \mathbb{R}^n$, it is $\boldsymbol{I_n}$ that provides its default components.

Identity matrices are arguably the perfect basis. First, their column vectors all have a norm equal to one (which is why they are called *unit* vectors). Second, each column is orthogonal to the others. Finally, the matrix is diagonal, meaning the dimension of the space it spans is simply the sum of its diagonal elements. This sum is known as the trace (usually written as Tr($\boldsymbol{A}$), trace($\boldsymbol{A}$), or Tr. $\boldsymbol{A}$). Because this is a critical concept in theoretical linear algebra, let's formally define it:

Trace

Definition: For any $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, its trace is defined as the sum of its diagonal elements:

$$\text{trace}(\boldsymbol{A}) = \sum_{i=1}^{n} a_{ii}$$

When we use the identity matrix as our basis (as we almost always do by default), the components of the vectors directly correspond to the coordinates of the points where the tips of the vectors lie. For this reason, the identity basis is often called the *coordinate basis*, and the components may simply be called *coordinates*. Furthermore, a vector originating from the origin is often called a *position vector* to distinguish it from other types of vectors (like electric or magnetic fields, which have specific values and directions at various points in coordinate space).

7.1.3 Bases of Subspaces

Everything we have discussed regarding bases and components for full vector spaces also applies to subspaces, but with one important distinction. As we know, a subspace lives inside a larger parent space. For example, we can have a subspace of dimension r inside $\mathbb{R}^n$, where

$r < n$. To span this subspace, we need exactly r basis vectors, but each of these vectors is still n-dimensional: $\boldsymbol{a_i} \in \mathbb{R}^n$. If we place these r vectors into a matrix as columns, we get $\boldsymbol{A} \in \mathbb{R}^{n \times r}$. This is no longer a square matrix, but rather a "tall" matrix.

Remember, this subspace of dimension r is *not* $\mathbb{R}^r$. In particular, a two-dimensional subspace (a plane going through the origin) inside $\mathbb{R}^3$ is *not* the same as $\mathbb{R}^2$. Let's build on the third row of Table 7.1 to illustrate this. We will take the two basis vectors from that example and make them three-dimensional by adding a third component. The subspace we are now considering is the span of these two vectors, which forms a plane in the coordinate space $\mathbb{R}^3$: all linear combinations of the two vectors lie on this plane. We will use these same two vectors to form the basis matrix $\boldsymbol{A}$ and write our original vector $\boldsymbol{x}$ (in the coordinate basis) as $[\boldsymbol{x}]_{\boldsymbol{A}}$ (in the new basis for the subspace). We have:

$$\boldsymbol{A} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad \boldsymbol{x} = 2\boldsymbol{a_1} + 3\boldsymbol{a_2} = \begin{bmatrix} 7 \\ 5 \\ 2 \end{bmatrix} \implies [\boldsymbol{x}]_{\boldsymbol{A}} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Notice that $[\boldsymbol{x}]_{\boldsymbol{A}}$ has only two components, even though it represents a vector in $\mathbb{R}^3$. Why is that? While the vectors physically reside in $\mathbb{R}^3$, they are entirely defined by linear combinations of the two column vectors of $\boldsymbol{A}$. The two scaling factors used in this linear combination directly correspond to the two components of the vector in this new basis.

For full spaces ($\mathbb{R}^n$), we used Eqn (7.3) to compute $[\boldsymbol{x}]_{\boldsymbol{A}}$ relying on the standard matrix inverse, $\boldsymbol{A^{-1}}$. However, for a subspace, our basis matrix $\boldsymbol{A}$ is "tall," having more rows (m) than columns ($r < m$). What is the equivalent of Eqn (7.3) in this scenario? This is where the left inverse (introduced in Eqn (5.1)) comes to the rescue. Because its r columns are linearly independent basis vectors, $\boldsymbol{A}$ is a tall matrix with full column rank. Consequently, the product $\boldsymbol{A^\mathsf{T}A}$ is a full-rank, square matrix of size $r \times r$, which means it is invertible. As a reminder, the left inverse is defined as:

$$(\boldsymbol{A^\mathsf{T}A})^{-1}(\boldsymbol{A^\mathsf{T}A}) = \boldsymbol{I} \implies \boldsymbol{A^{-1}_{\text{Left}}} = (\boldsymbol{A^\mathsf{T}A})^{-1}\boldsymbol{A^\mathsf{T}}$$

Returning to our change-of-basis problem, we can find the components in the new basis $\boldsymbol{A}$ by following the exact same logic used to

derive Eqn (7.3):

$$\boldsymbol{A}[\boldsymbol{x}]_{\boldsymbol{A}} = \boldsymbol{x} \implies [\boldsymbol{x}]_{\boldsymbol{A}} = \boldsymbol{A}_{\textbf{Left}}^{-1}\boldsymbol{x} = (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{x} \tag{7.2}$$

Let's verify this formula by walking through our 3D example.

$$\boldsymbol{x} = \begin{bmatrix} 7 \\ 5 \\ 2 \end{bmatrix}, \quad \boldsymbol{A} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 0 \end{bmatrix} \implies \boldsymbol{A}^{\mathsf{T}} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix} \text{ and } \boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} = \begin{bmatrix} 6 & 3 \\ 3 & 2 \end{bmatrix}$$

$$\left|\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\right| = 3 \text{ and } (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1} = \frac{1}{3}\begin{bmatrix} 2 & -3 \\ -3 & 6 \end{bmatrix} = \begin{bmatrix} \frac{2}{3} & -1 \\ -1 & 2 \end{bmatrix}$$

$$(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}_{\textbf{Left}}^{-1} = \begin{bmatrix} \frac{2}{3} & -1 \\ -1 & 2 \end{bmatrix}\begin{bmatrix} 2 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \\ 0 & 1 & -1 \end{bmatrix}$$

$$[\boldsymbol{x}]_{\boldsymbol{A}} = \boldsymbol{A}_{\textbf{Left}}^{-1}\boldsymbol{x} = \begin{bmatrix} \frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \\ 0 & 1 & -1 \end{bmatrix}\begin{bmatrix} 7 \\ 5 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

It is a tedious calculation, but it correctly yields exactly the scalars from the linear combination we started with: $\boldsymbol{x} = 2\boldsymbol{a_1} + 3\boldsymbol{a_2}$. Happily, we rarely have to do these calculations by hand in computer science; we have tools like `SageMath` to handle the heavy lifting for us. We will gain a much deeper geometric understanding of this left inverse when we explore projection operations two chapters from now.

7.1.4 Change of Basis

One of the most important applications of linear algebra involves the change of basis: moving from one basis to another. We see this computational application in multiplayer video games, where the underlying 3D world is rendered from the unique perspective of each player. We also see it in the perspective correction features on our smartphones.

Let's start with the examples detailed in Table 7.1 and illustrated in Figure 7.1. In the first row, the vector $\boldsymbol{x}$ has components 7 and 5 in the standard basis we are used to, which we call the identity or coordinate basis. In the table, this representation is written as $[\boldsymbol{x}]_{\boldsymbol{I}}$.

In the second row of Table 7.1, we use the matrix $\boldsymbol{A}'$ as the new basis. Notice that the basis vectors $\boldsymbol{a_1'}$ and $\boldsymbol{a_2'}$ still point in the same direction as the corresponding unit vectors from the identity basis ($\boldsymbol{q_1}$ and $\boldsymbol{q_2}$), but they have been scaled up. Looking at the components of

Table 7.1 Examples of change of basis.

Basis Matrix	Basis Vectors	Linear Combination	Vector Components
$\boldsymbol{A} = \boldsymbol{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\boldsymbol{q_1} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \boldsymbol{q_2} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$	$\boldsymbol{x} = 7\boldsymbol{q_1} + 5\boldsymbol{q_2}$	$[\boldsymbol{x}]_{\boldsymbol{I}} = \begin{bmatrix} 7 \\ 5 \end{bmatrix}$
$\boldsymbol{A'} = \begin{bmatrix} 3.5 & 0 \\ 0 & 2.5 \end{bmatrix}$	$\boldsymbol{a'_1} = \begin{bmatrix} 3.5 \\ 0 \end{bmatrix}, \boldsymbol{a'_2} = \begin{bmatrix} 0 \\ 2.5 \end{bmatrix}$	$\boldsymbol{x} = 2\boldsymbol{a'_1} + 2\boldsymbol{a'_2}$	$[\boldsymbol{x}]_{\boldsymbol{A'}} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
$\boldsymbol{A''} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$	$\boldsymbol{a''_1} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \boldsymbol{a''_2} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\boldsymbol{x} = 2\boldsymbol{a''_1} + 3\boldsymbol{a''_2}$	$[\boldsymbol{x}]_{\boldsymbol{A''}} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$

How the same vector $\boldsymbol{x}$ is represented in various bases. The notation of square brackets around a vector with a matrix as a subscript stands for the vector represented in the basis of that matrix: $[\boldsymbol{x}]_{\boldsymbol{A}}$ is the representation of $\boldsymbol{x}$ using the columns of $\boldsymbol{A}$ as the basis vectors.

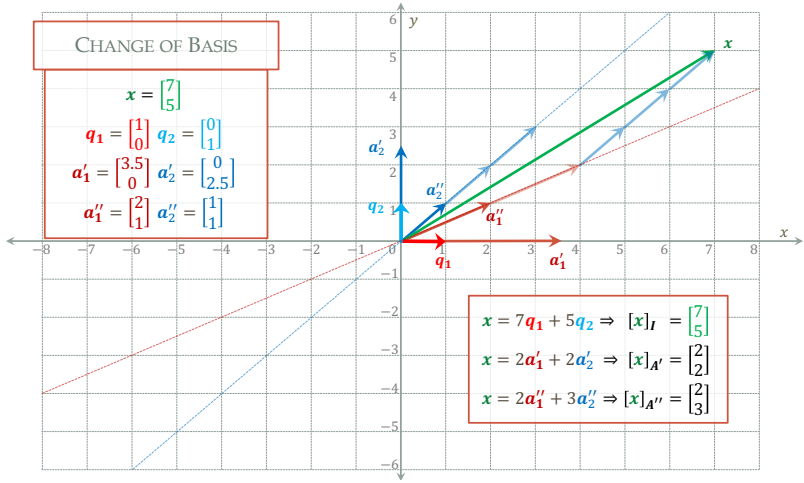


Fig. 7.1 Visualizing the change-of-basis examples listed in Table 7.1. The green vector is represented in three different bases, giving vastly different values for its components.
Interactive Visualization

the same vector in this new basis $\boldsymbol{A'}$ (denoted as $[\boldsymbol{x}]_{\boldsymbol{A'}}$), we see that they are smaller than the original coordinates: $7 \to 2$ and $5 \to 2$. This tells us that because the basis vectors are larger, we only need to take two steps in each direction to reach the tip of our vector $\boldsymbol{x}$. When the basis vectors get bigger, the vector components get smaller. This inverse relationship strongly hints that a change of basis fundamentally involves the inverse of the basis matrix.

In the third row of Table 7.1, things get significantly more complex. Now, the basis vectors point in arbitrary directions (they are not orthogonal to each other) and have arbitrary sizes (far from unit length). The first component is now 2 and the second is 3, as described in Table 7.1 and illustrated in Figure 7.1.

How do we compute these new components given the original vector and the new basis vectors? In other words, how do we perform a change of basis in a generalized way? Let's assume our original basis is the coordinate basis $\boldsymbol{I_n} \in \mathbb{R}^{n\times n}$, and the new basis is $\boldsymbol{A} \in \mathbb{R}^{n\times n}$. The column vectors of $\boldsymbol{A} = \begin{bmatrix}\boldsymbol{a_i}\end{bmatrix} \in \mathbb{R}^{n\times n}$ are also specified in the identity basis. Eqn (7.1) tells us how to arrive at the components of $\boldsymbol{x}$ (which are x_i) in this basis. Remember, we denote the representation of $\boldsymbol{x}$ in $\boldsymbol{A}$ as $[\boldsymbol{x}]_{\boldsymbol{A}}$.

To find the components of $\boldsymbol{x}$ in the $\boldsymbol{A}$ basis, we must find the vector $[\boldsymbol{x}]_{\boldsymbol{A}}$ such that $\boldsymbol{x} = \boldsymbol{A}[\boldsymbol{x}]_{\boldsymbol{A}}$. This is mathematically identical to solving a system of linear equations, as shown in Eqn (7.3) below. Because we know that $\boldsymbol{A}$ is a valid basis matrix, it must be square, full rank, and therefore invertible. Thus, we can isolate the components in the new basis $\boldsymbol{A}$ through matrix inversion:

$$\boldsymbol{x} = \sum_{i=1}^{n} x_{i|\boldsymbol{A}}\boldsymbol{a_i} \implies \boldsymbol{x} = \boldsymbol{A}[\boldsymbol{x}]_{\boldsymbol{A}} \implies [\boldsymbol{x}]_{\boldsymbol{A}} = \boldsymbol{A^{-1}x} \tag{7.3}$$

Let's verify Eqn (7.3) using the most complicated example from our table (the third row). Recalling the standard prescription for the inverse of a 2×2 matrix (swap the diagonal elements, flip the signs of the off-diagonal elements, and divide by the determinant), we have:

The original vector in the coordinate basis: $\boldsymbol{x} = \begin{bmatrix}7\\5\end{bmatrix}$

The new basis: $\boldsymbol{A} = \begin{bmatrix}2 & 1\\1 & 1\end{bmatrix} \implies |\boldsymbol{A}| = 1$ and $\boldsymbol{A^{-1}} = \begin{bmatrix}1 & -1\\-1 & 2\end{bmatrix}$

The vector in the new basis:

$$[\boldsymbol{x}]_{\boldsymbol{A}} = \boldsymbol{A^{-1}x} = \begin{bmatrix}1 & -1\\-1 & 2\end{bmatrix}\begin{bmatrix}7\\5\end{bmatrix} = \begin{bmatrix}2\\3\end{bmatrix}$$

Note that we used the generic symbol $\boldsymbol{A}$ for the new basis rather than $\boldsymbol{A''}$ as written in Table 7.1. Comparing our calculated $[\boldsymbol{x}]_{\boldsymbol{A}}$ with the $[\boldsymbol{x}]_{\boldsymbol{A''}}$ from the table, we can satisfy ourselves that the matrix equation in Eqn (7.3) works perfectly.

When we change the basis to $\boldsymbol{A}$, what happens to vector dot products, and consequently, to the norm of the vector? Using the definition of the dot product from way back in Eqn (2.11) and our new change of basis rule in Eqn (7.3), we can write:

$$\boldsymbol{x} \cdot \boldsymbol{y} = \boldsymbol{x}^{\mathsf{T}}\boldsymbol{y} = (\boldsymbol{A}[\boldsymbol{x}]_{\boldsymbol{A}})^{\mathsf{T}} \boldsymbol{A}[\boldsymbol{y}]_{\boldsymbol{A}} = [\boldsymbol{x}]_{\boldsymbol{A}}^{\mathsf{T}} \left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\right) [\boldsymbol{y}]_{\boldsymbol{A}}$$

If the dot product is to remain unchanged, we require $\boldsymbol{x} \cdot \boldsymbol{y} = \boldsymbol{x}^{\mathsf{T}}\boldsymbol{y} = [\boldsymbol{x}]_{\boldsymbol{A}}^{\mathsf{T}}[\boldsymbol{y}]_{\boldsymbol{A}}$, which means we need $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} = \boldsymbol{I}$ for the new basis $\boldsymbol{A}$. What type of matrices satisfy this exact condition? As we will see in Eqn (7.5), orthogonal matrices do. Therefore, preserving the norm of a vector under a change of basis requires the basis matrix to be orthogonal, which means that its columns are orthonormal.

7.2 Orthogonality and Projections

7.2.1 Orthogonal Vectors

We have encountered orthogonal vectors in a few places earlier in this book. If we imagine vectors as line segments in *coordinate* spaces with arrows at their tips, we can say that orthogonal vectors are simply perpendicular vectors. We are clearly more sophisticated than that by now, and our abstract *vector* spaces do not inherently have lines and arrows. Orthogonal vectors are, therefore, those vectors that have their inner (or dot) product equal to zero.

Orthogonal Vectors

Definition: Two vectors, $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, are orthogonal to each other if and only if

$$\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y} = \boldsymbol{y}^{\mathsf{T}}\boldsymbol{x} = 0$$

Note that the vectors in the inner product commute. Note also that the less sophisticated definition of the inner product (namely, $\boldsymbol{x} \cdot \boldsymbol{y} = \|\boldsymbol{x}\|\|\boldsymbol{y}\| \cos\theta$) shows that the inner (or dot) product is zero when $\boldsymbol{x}$ and $\boldsymbol{y}$ are perpendicular, because the angle θ between them is $\frac{\pi}{2}$ and $\cos(\frac{\pi}{2}) = 0$.

We still stay away from defining the inner product strictly via angles because, by now, we know that the machinery of linear algebra may

be applied to vector-like objects where we cannot easily talk about directions and angles. For example, in Fourier transforms or the wave functions of quantum mechanics, vectors are functions, and the inner product is defined with no reference to any kind of angles. We can still have *orthogonal* "vectors" in such spaces when the inner product is zero. Clearly, we cannot have *perpendicular* vectors without abusing the notion and notation a bit too much for our (or at least, the author's) liking.

Earlier, in Eqn (2.5), we defined the Euclidean norm of a vector $\|\boldsymbol{a}\|$:

$$\|\boldsymbol{a}\|^2 = \boldsymbol{a}^\mathsf{T}\boldsymbol{a}$$

Using this definition, we can prove that the inner product of orthogonal vectors has to be zero, albeit not completely devoid of the lack of sophistication associated with perpendicularity.

If we have $\boldsymbol{a} \perp \boldsymbol{b}$, then we know that $\|\boldsymbol{a}\|$ and $\|\boldsymbol{b}\|$ form the sides of a right-angled triangle (where the geometric intuition of perpendicularity re-emerges) whose hypotenuse is of length $\|\boldsymbol{a}+\boldsymbol{b}\|$. Then, by the Pythagorean theorem, we can write:

$$\begin{aligned}
\|\boldsymbol{a}\|^2 + \|\boldsymbol{b}\|^2 &= \|\boldsymbol{a}+\boldsymbol{b}\|^2 \\
\boldsymbol{a}^\mathsf{T}\boldsymbol{a} + \boldsymbol{b}^\mathsf{T}\boldsymbol{b} &= (\boldsymbol{a}+\boldsymbol{b})^\mathsf{T}(\boldsymbol{a}+\boldsymbol{b}) \\
&= (\boldsymbol{a}^\mathsf{T}+\boldsymbol{b}^\mathsf{T})(\boldsymbol{a}+\boldsymbol{b}) \\
&= \boldsymbol{a}^\mathsf{T}\boldsymbol{a} + \boldsymbol{b}^\mathsf{T}\boldsymbol{b} + \boldsymbol{b}^\mathsf{T}\boldsymbol{a} + \boldsymbol{a}^\mathsf{T}\boldsymbol{b} \\
\implies 0 &= \boldsymbol{b}^\mathsf{T}\boldsymbol{a} + \boldsymbol{a}^\mathsf{T}\boldsymbol{b} \\
\implies \boldsymbol{b}^\mathsf{T}\boldsymbol{a} &= \boldsymbol{a}^\mathsf{T}\boldsymbol{b} = 0 \quad \text{since } \boldsymbol{b}^\mathsf{T}\boldsymbol{a} = \boldsymbol{a}^\mathsf{T}\boldsymbol{b}
\end{aligned}$$

7.2.2 Orthogonal and Orthonormal Bases

We snuck the concept of orthogonality into the previous chapter, but it deserves a fuller treatment here. Orthogonal vectors are vectors whose dot product is zero: $\boldsymbol{x}$ and $\boldsymbol{y}$ are orthogonal if $\boldsymbol{x}^\mathsf{T}\boldsymbol{y} = 0$. If all the vectors in our basis are mutually orthogonal, we call it an orthogonal basis.

If our basis vectors are orthogonal to each other *and* are all of unit length, we have an orthonormal basis. The identity matrix is a prime example of a nice orthonormal basis, but it is certainly not the only

one. Before looking at specific examples, let's study orthonormal bases in general.

Let's define our orthonormal basis matrix as $\boldsymbol{Q} = [\boldsymbol{q_i}] \in \mathbb{R}^{n\times n}$, with column vectors $\boldsymbol{q_i} \in \mathbb{R}^n$. The fact that the columns $\boldsymbol{q_i}$ form an orthonormal basis implies two things: the dot product of any two distinct columns is zero, and the norm of every column is one. Both characteristics can be stated compactly as:

$$\boldsymbol{q_i^\mathsf{T} q_j} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \tag{7.4}$$

As a result, the matrix $\boldsymbol{Q}$ inherits some incredibly useful properties:

1. The inverse of an orthogonal matrix is its transpose:

$$\boldsymbol{Q^{-1} = Q^\mathsf{T}} \implies \boldsymbol{Q^\mathsf{T} Q = Q Q^\mathsf{T} = I} \tag{7.5}$$

   Proof: We can easily prove this by viewing the product $\boldsymbol{Q^\mathsf{T} Q}$ as the dot products of the columns of $\boldsymbol{Q}$. Due to the piecewise definition in Eqn (7.4), only the elements where the row number matches the column number survive, yielding the identity matrix.

2. An orthogonal matrix does not change the norm of a vector:

$$\|\boldsymbol{Qx}\| = \|\boldsymbol{x}\| \tag{7.6}$$

   Proof: Consider the squared norm, $\|\boldsymbol{Qx}\|^2$:

$$\begin{aligned} \|\boldsymbol{Qx}\|^2 &= (\boldsymbol{Qx})^\mathsf{T}(\boldsymbol{Qx}) = \boldsymbol{x^\mathsf{T} Q^\mathsf{T} Q x} \\ &= \boldsymbol{x^\mathsf{T} Q^{-1} Q x} = \boldsymbol{x^\mathsf{T} I x} \\ &= \boldsymbol{x^\mathsf{T} x} = \|\boldsymbol{x}\|^2 \\ \implies \|\boldsymbol{Qx}\| &= \|\boldsymbol{x}\| \end{aligned}$$

3. Orthogonal matrices have unit determinants ($|\boldsymbol{Q}| = \pm 1$): Conceptually, this follows from the fact that orthogonal matrices do not scale the vectors they multiply. We can formally prove it using determinant properties:

$$\begin{aligned} \boldsymbol{Q^{-1} = Q^\mathsf{T}} &\implies \left|\boldsymbol{Q^{-1}}\right| = \left|\boldsymbol{Q^\mathsf{T}}\right| = |\boldsymbol{Q}| \implies \frac{1}{|\boldsymbol{Q}|} = |\boldsymbol{Q}| \\ &\implies |\boldsymbol{Q}|^2 = 1 \implies |\boldsymbol{Q}| = \pm 1 \end{aligned}$$

A quick heads-up on a frustrating historical naming convention: while we make a strict mathematical distinction between orthogonal and orthonormal vectors, standard linear algebra terminology refers to a matrix with orthonormal columns simply as an "orthogonal matrix." The term "orthonormal matrix" is rarely used in advanced literature. We will also adopt this standard–if slightly confusing–convention.

CS Connection: The Orthonormal Advantage

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

7.2.3 Example: Rotation Matrices

If we think of matrix multiplication ($\boldsymbol{Ax} = \boldsymbol{b}, \boldsymbol{A} \in \mathbb{R}^{n \times n}$) as a transformation $\boldsymbol{A} : \boldsymbol{x} \mapsto \boldsymbol{b}$, we can appreciate that an orthogonal matrix $\boldsymbol{Q}$ represents a transformation that either rotates a vector, reflects it, or shuffles its elements (a permutation).

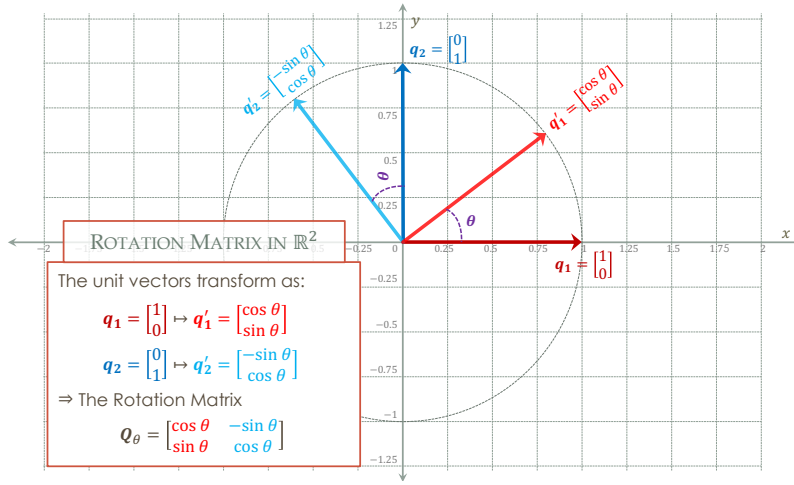


Fig. 7.2 The rotation matrix in $\mathbb{R}^2$ can be written down by looking at where the unit vectors go under a rotation through a specified angle.

An important class of orthogonal matrices is rotation matrices. They are orthogonal because they do not change the norm (the length) of the vector being rotated. As illustrated in Figure 7.2, we can easily derive the rotation matrix $\boldsymbol{Q_\theta}$ in $\mathbb{R}^2$. Conceptually, the inverse of

Linearity in Relativity

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

a rotation matrix must be a matrix that rotates the vector back in the opposite direction: $\boldsymbol{Q}_{\boldsymbol{\theta}}^{-1}$ should simply be $\boldsymbol{Q}_{-\boldsymbol{\theta}}$. Furthermore, because it is an orthogonal matrix, its inverse must also be equal to its transpose ($\boldsymbol{Q}_{\boldsymbol{\theta}}{}^{\mathsf{T}}$).

$$\boldsymbol{Q}_{\boldsymbol{\theta}} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \quad |\boldsymbol{Q}_{\boldsymbol{\theta}}| = \cos^2\theta + \sin^2\theta = 1$$

$$\boldsymbol{Q}_{\boldsymbol{\theta}}^{-1} = \boldsymbol{Q}_{-\boldsymbol{\theta}} = \begin{bmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} = \boldsymbol{Q}_{\boldsymbol{\theta}}{}^{\mathsf{T}}$$

Rotation in $\mathbb{R}^3$ is defined by three angles—pitch, roll, and yaw—as they are commonly known in aviation. A full 3D rotation matrix can actually be constructed as the product of three independent 2D-style rotations along the x, y, and z axes.

CS Connection: The Ubiquity of 3D Rotations

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

7.3 The Gram-Schmidt Process

The Gram-Schmidt process is an algorithm that transforms a matrix with linearly independent columns (such as a square, invertible matrix) into an orthogonal one through a sequence of steps. These steps are essentially elementary column operations, akin to the row operations we performed in Gaussian and Gauss-Jordan elimination.

Upon completing the Gram-Schmidt process on a matrix $\boldsymbol{A}$, we obtain a matrix $\boldsymbol{Q}$ that represents a rotated version of the identity matrix $\boldsymbol{I}$, potentially with permuted rows. As we will see, the algorithm preserves the direction of the first column vector and maintains the

relative order of the subsequent vectors. In other words, if the second column vector of $\boldsymbol{A}$ is to the "right" of the first one, the second column of $\boldsymbol{Q}$ will also be to the right of the first one. (To be mathematically precise, by "right," we mean that we must rotate clockwise to go from the first column to the second. More formally, the span of the first k columns of $\boldsymbol{A}$ is identical to the span of the first k columns of $\boldsymbol{Q}$.)

The Gram-Schmidt process proceeds by projecting each column of the matrix it is currently processing onto the orthonormal vectors found so far. Before getting into the algorithm, let's formalize our understanding of projections.

7.3.1 Vector Projections

Earlier, in Chapter 2, we looked at the vector dot product as a projection using the cosine of the angle between them, in Figure 2.5. To remind ourselves, if we have two vectors $\boldsymbol{a_1}$ and $\boldsymbol{a_2}$ with an angle θ between them, the projection of $\boldsymbol{a_2}$ onto $\boldsymbol{a_1}$ has length $\|\boldsymbol{a_2}\| \cos\theta$. From the definition of the dot product using the angle, we can write:

$$\|\boldsymbol{a_2}\| \cos\theta = \frac{\boldsymbol{a_1} \cdot \boldsymbol{a_2}}{\|\boldsymbol{a_1}\|} = \frac{\boldsymbol{a_1}^\mathsf{T}}{\|\boldsymbol{a_1}\|}\boldsymbol{a_2}$$

To move to a purely matrix-based notation, we can use the matrix definition of dot products, $\boldsymbol{a_1} \cdot \boldsymbol{a_2} = \boldsymbol{a_1}^\mathsf{T}\boldsymbol{a_2} = \boldsymbol{a_2}^\mathsf{T}\boldsymbol{a_1}$, which changes the formula above to:

$$\|\boldsymbol{a_2}\| \cos\theta = \frac{\boldsymbol{a_1}^\mathsf{T}}{\|\boldsymbol{a_1}\|}\boldsymbol{a_2} = \boldsymbol{a_2}^\mathsf{T}\frac{\boldsymbol{a_1}}{\|\boldsymbol{a_1}\|}$$

This formula gives us the *length* of the projection, which we called x in Figure 7.3. For our purposes, we want a *vector* as the projection, which would be a vector pointing in the direction of $\boldsymbol{a_1}$, with its norm equal to the length of the projection x.

Let's call this projection vector $\boldsymbol{a_{2\|}}$. The direction of $\boldsymbol{a_1}$ is specified by the unit vector $\boldsymbol{q_1}$:

$$\boldsymbol{q_1} = \frac{\boldsymbol{a_1}}{\|\boldsymbol{a_1}\|}$$

Putting it all together, we write:

$$\boldsymbol{a_{2\|}} = (\boldsymbol{a_2}^\mathsf{T}\boldsymbol{q_1})\boldsymbol{q_1} \tag{7.7}$$

This form of the projection is exactly what we will use in the Gram-Schmidt process coming up in the next section.

However, we want to go a bit further with Eqn (7.7) and develop what we will call a projection matrix. Knowing the definition of the norm of a vector ($\|\boldsymbol{a}_1\|^2 = \boldsymbol{a}_1^\mathsf{T}\boldsymbol{a}_1$), we can write the projection in a form that we will use later when we start projecting vectors onto subspaces rather than other single vectors. The derivation of this form of projection is shown in Figure 7.3, which we will repeat as a recasting of Eqn (7.7) merely to get used to the vector/matrix manipulations, if nothing else.

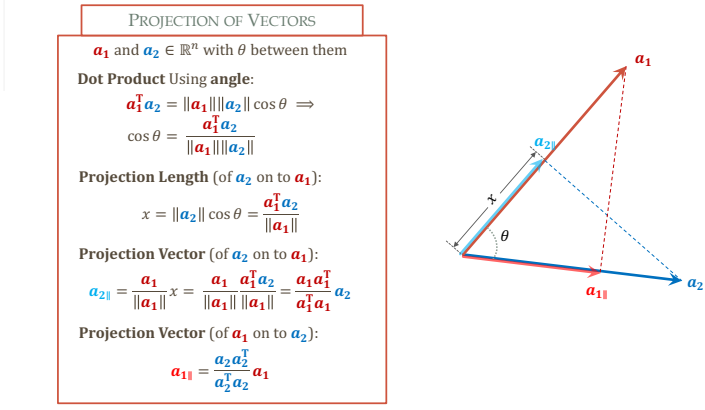


Fig. 7.3 Dot product between two vectors $\boldsymbol{a}_1, \boldsymbol{a}_2 \in \mathbb{R}^n$, shown on the plane defined by the two vectors.

Rewriting Eqn (7.7) using the definitions of the norm ($\|\boldsymbol{a}_1\|$) and the direction ($\boldsymbol{q}_1$):

$$\begin{aligned}\boldsymbol{a}_{2\|} &= \boldsymbol{q}_1(\boldsymbol{q}_1^\mathsf{T}\boldsymbol{a}_2) = \frac{\boldsymbol{a}_1}{\|\boldsymbol{a}_1\|}\left(\frac{\boldsymbol{a}_1^\mathsf{T}}{\|\boldsymbol{a}_1\|}\boldsymbol{a}_2\right) \\ &= \boldsymbol{a}_1\left(\frac{\boldsymbol{a}_1^\mathsf{T}\boldsymbol{a}_2}{\|\boldsymbol{a}_1\|^2}\right) = \left(\frac{\boldsymbol{a}_1\boldsymbol{a}_1^\mathsf{T}}{\boldsymbol{a}_1^\mathsf{T}\boldsymbol{a}_1}\right)\boldsymbol{a}_2\end{aligned} \tag{7.8}$$

In the relatively complicated manipulations in Eqn (7.8), we have a couple of observations to write down:

- The entities in the parentheses in all but the last RHS are scalars. If we were to call it, say s, it commutes with matrix (and vector) multiplication:

$$s\boldsymbol{a}_1\boldsymbol{a}_2 = \boldsymbol{a}_1 s\boldsymbol{a}_2 = \boldsymbol{a}_1\boldsymbol{a}_2 s$$

- In the first RHS, we have the projection of $\boldsymbol{a_2}$ equal to a scalar times the direction of $\boldsymbol{a_1}$, which makes perfect intuitive sense.
- This pattern repeats itself in all the RHS until the very last one: the projection vector is just $\boldsymbol{a_1}$ scaled.
- In the last and final RHS (where we used the associativity of matrix multiplication to put the parentheses where we wanted), the entity multiplying $\boldsymbol{a_2}$ is a matrix: it is $\boldsymbol{a_1 a_1^\mathsf{T}} \in \mathbb{R}^{n\times n}$ multiplied by a scalar $(\boldsymbol{a_1^\mathsf{T} a_1})^{-1}$.
- We can think of this scaled matrix as an operator, which, when applied to any vector $\boldsymbol{a_2}$, gives its projection onto $\boldsymbol{a_1}$.

We will call this scaled matrix $\boldsymbol{P}$, the projection matrix. Let's write it down once more as the definition of the projection matrix, which we will revisit in a couple of chapters.

$$\boldsymbol{P} = \boldsymbol{a_1}(\boldsymbol{a_1^\mathsf{T} a_1})^{-1}\boldsymbol{a_1^\mathsf{T}} \tag{7.9}$$

7.3.2 The Algorithm

Given that an orthonormal basis is the best possible basis we can ever hope to have, we need an algorithm to make any matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ orthogonal. An orthogonal matrix is one in which the columns are normalized and orthogonal to one another. In other words, it is a matrix that could serve as a basis matrix as described in §7.2.2 with the associated properties. Orthogonalization is the process or algorithm that can make a square matrix orthogonal.

Since our objective is to construct an orthogonal basis matrix, we will normalize each column vector and ensure it is orthogonal to the others through an iterative process, beginning with the first column. Let's set the stage by defining our symbols. We will start with a square matrix $\boldsymbol{A}$ and end up with another square matrix $\boldsymbol{Q}$:

$$\boldsymbol{A} = \begin{bmatrix}\boldsymbol{a_i}\end{bmatrix}, \quad \boldsymbol{Q} = \begin{bmatrix}\boldsymbol{q_i}\end{bmatrix} \in \mathbb{R}^{n\times n}, \quad \boldsymbol{A} \xrightarrow{\text{G-S}} \boldsymbol{Q}$$
$$\implies \boldsymbol{q_i^\mathsf{T} q_j} = \begin{cases} 0 & i \neq j \\ 1 & i = j \end{cases}$$

Here are the standard steps in the Gram-Schmidt process, as illustrated in Figure 7.4:

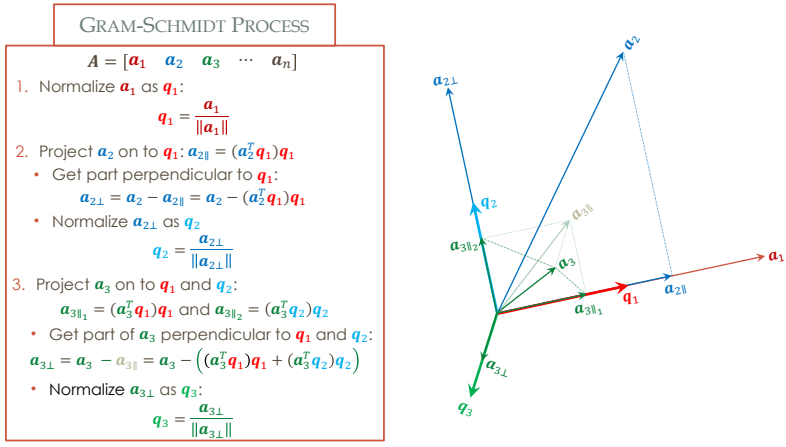


Fig. 7.4 Illustration of the Gram-Schmidt process running on a matrix $\boldsymbol{A}$. The first column (red) is normalized to get the unit vector $\boldsymbol{q_1}$, which is then used to create $\boldsymbol{q_2}$ from the second (blue) column. Both $\boldsymbol{q_1}$ and $\boldsymbol{q_2}$ are used in projecting the third (green) column and computing $\boldsymbol{q_3}$. Interactive Visualization

1. Take the first column of $\boldsymbol{A}$ and normalize it to get the first column of $\boldsymbol{Q}$:
$$\boldsymbol{q_1} = \frac{\boldsymbol{a_1}}{\|\boldsymbol{a_1}\|}$$
2. Use $\boldsymbol{q_1}$ to derive $\boldsymbol{q_2}$ (or generally, $\boldsymbol{q_j}$ for $j = 2$) using the following steps:
   (a) Take the second column of $\boldsymbol{A}$ and project it onto $\boldsymbol{q_1}$ to find the component of $\boldsymbol{a_2}$ parallel to it, utilizing Eqn (7.7):
   $$\boldsymbol{a_{2\|}} = (\boldsymbol{a_2^T q_1})\boldsymbol{q_1}$$
   (b) Subtract $\boldsymbol{a_{2\|}}$ from $\boldsymbol{a_2}$ to isolate the perpendicular component:
   $$\boldsymbol{a_{2\perp}} = \boldsymbol{a_2} - \boldsymbol{a_{2\|}} = \boldsymbol{a_2} - (\boldsymbol{a_2^T q_1})\boldsymbol{q_1}$$
   (c) Normalize $\boldsymbol{a_{2\perp}}$ to obtain the second column of $\boldsymbol{Q}$:
   $$\boldsymbol{q_2} = \frac{\boldsymbol{a_{2\perp}}}{\|\boldsymbol{a_{2\perp}}\|}$$
3. Now compute $\boldsymbol{q_j}$ using the previously normalized vectors $\boldsymbol{q_i}$ ($1 \leq i < j$), following these steps:

(a) Take $\boldsymbol{a_j}$, the next column of $\boldsymbol{A}$, and project it successively onto each $\boldsymbol{q_i}$ to find the parallel components:

$$\boldsymbol{a_{j\|i}} = (\boldsymbol{a_j^\mathsf{T} q_i})\boldsymbol{q_i}$$

(b) Subtract all parallel parts $\boldsymbol{a_{j\|i}}$ from $\boldsymbol{a_j}$ to isolate the component perpendicular to all preceding $\boldsymbol{q_i}$:

$$\boldsymbol{a_{j\perp}} = \boldsymbol{a_j} - \sum_{i=1}^{j-1} \boldsymbol{a_{j\|i}} = \boldsymbol{a_j} - \sum_{i=1}^{j-1} (\boldsymbol{a_j^\mathsf{T} q_i})\boldsymbol{q_i}$$

(c) Normalize $\boldsymbol{a_{j\perp}}$ to yield the next column of $\boldsymbol{Q}$, which is $\boldsymbol{q_j}$:

$$\boldsymbol{q_j} = \frac{\boldsymbol{a_{j\perp}}}{\|\boldsymbol{a_{j\perp}}\|}$$

4. Iterate until we run out of columns (i.e., when $j = n$).

Note that Step 2 in the algorithm above is technically identical to Step 3 with $j = 2$, but we chose to spell it out explicitly to make the intuition behind the process clearer. In fact, even the first step can be thought of as a special case of the third step, making it very straightforward to program as a generalized algorithm.

7.3.3 Numerical Considerations

When we examine the steps of the Gram-Schmidt process, we see a cycle: normalize a vector, use it to compute the next one, normalize that result, and work our way through the entire matrix $\boldsymbol{A}$. As we know, floating-point operations on a computer have inherent precision limits, and the very first step necessarily incurs a small rounding error. Because this result is then used in subsequent steps, the errors compound. For this reason, the standard Gram-Schmidt process can be numerically unstable.

The *modified* Gram-Schmidt algorithm mitigates this problem by breaking down the summation in Step 3(b) iteratively, like so:

$$\begin{aligned} \boldsymbol{a_{j\perp}^{(0)}} &= \boldsymbol{a_j} \\ \boldsymbol{a_{j\perp}^{(i)}} &= \boldsymbol{a_{j\perp}^{(i-1)}} - (\boldsymbol{a_{j\perp}^{(i-1)^\mathsf{T}} q_i})\boldsymbol{q_i} \quad 1 \le i \le j-1 \\ \boldsymbol{a_{j\perp}} &= \boldsymbol{a_{j\perp}^{(j-1)}} \end{aligned} \tag{7.10}$$

In other words, instead of projecting the original $\boldsymbol{a_j}$ onto all available orthonormal vectors $\boldsymbol{q_i}$ and subtracting the grand sum at the end, we project it onto $\boldsymbol{q_1}$ first, and immediately subtract it from $\boldsymbol{a_j}$ to get a partially perpendicular vector $\boldsymbol{a_{j\perp}^{(1)}}$. Then, instead of projecting the original $\boldsymbol{a_j}$ onto $\boldsymbol{q_2}$, we project the newly updated $\boldsymbol{a_{j\perp}^{(1)}}$ instead. This continuous updating significantly disrupts the accumulation of rounding errors.

7.3.4 QR Decomposition

Because the Gram-Schmidt algorithm relies entirely on taking linear combinations of the columns of the original matrix, it can be expressed as a matrix multiplication on the right:

$$\boldsymbol{A}, \boldsymbol{Q} \in \mathbb{R}^{n \times n}, \quad \boldsymbol{A} \xrightarrow{\text{G-S}} \boldsymbol{Q} \implies \boldsymbol{AX} = \boldsymbol{Q}, \quad \boldsymbol{X} \in \mathbb{R}^{n \times n}$$

Furthermore, since $\boldsymbol{q_1}$ is just a scaled version of $\boldsymbol{a_1}$, the first column of $\boldsymbol{X}$ has only one nonzero element (x_{11}). In general, each $\boldsymbol{q_i}$ is a linear combination of all preceding $\boldsymbol{a_j}$ ($1 \leq j \leq i$). Because of this strict ordering, $\boldsymbol{X}$ is an upper triangular matrix.

We are often more interested in the inverse of this matrix, which we will call $\boldsymbol{R}$. The inverse of an upper triangular matrix is also an upper triangular matrix. This fact can be intuitively appreciated by noticing that reversing the process (expressing $\boldsymbol{a_i}$ as a linear combination of $\boldsymbol{q_j}$) also relies only on preceding vectors ($1 \leq j \leq i$). A formal general proof of this property is left as an exercise.

Using these variable names, we can rewrite the equation:

$$\boldsymbol{AX} = \boldsymbol{Q} \implies \boldsymbol{A} = \boldsymbol{QX^{-1}} \implies \boldsymbol{A} = \boldsymbol{QR}$$

By leveraging the fact that $\boldsymbol{Q}$ is an orthogonal matrix (meaning its inverse is simply its transpose, $\boldsymbol{Q^{-1}} = \boldsymbol{Q^{\mathsf{T}}}$), we can easily solve for $\boldsymbol{R}$:

$$\boldsymbol{R} = \boldsymbol{Q^{-1}A} = \boldsymbol{Q^{\mathsf{T}}A}$$

As we can see, the Gram-Schmidt algorithm naturally leads to the decomposition of a square, full-rank matrix into an orthogonal matrix $\boldsymbol{Q}$ and an upper triangular matrix $\boldsymbol{R}$. Fittingly, this is known as the $\boldsymbol{QR}$ decomposition.

Why Do QR Decomposition?

The $\boldsymbol{QR}$ decomposition makes it easy for us to solve systems like $\boldsymbol{Ax} = \boldsymbol{b}$. Instead of directly dealing with a computationally messy matrix $\boldsymbol{A}$, we substitute $\boldsymbol{A} = \boldsymbol{QR}$:

$$\boldsymbol{Ax} = \boldsymbol{QRx} = \boldsymbol{b} \implies \boldsymbol{Rx} = \boldsymbol{Q}^{\mathsf{T}}\boldsymbol{b}$$

We are, essentially, multiplying both sides of $\boldsymbol{Ax} = \boldsymbol{b}$ on the left by $\boldsymbol{Q}^{\mathsf{T}}$. Does it alter the underlying solution set? Remember that multiplication on the left is the same as taking linear combinations of the rows of the matrix, much the same way we performed Gaussian elimination earlier. It does not change the solutions. Because $\boldsymbol{Q}$ is a square orthogonal matrix, its transpose is its exact inverse. Multiplying a linear system by an invertible matrix is algebraically equivalent to applying a sequence of perfectly reversible row operations. The surface-level numbers change, but the actual solution set remains identical.

This substitution ($\boldsymbol{A} = \boldsymbol{QR}$) reduces a complex solving problem into a computationally cheap matrix-vector multiplication ($\boldsymbol{Q}^{\mathsf{T}}\boldsymbol{b}$) followed by a fast back-substitution on $\boldsymbol{R}$. Remember that $\boldsymbol{R}$ is an upper triangular matrix, which is the perfect shape for back-substitution.

Just as Gaussian elimination utilizes row operations to preserve the solution set of a system, the Gram-Schmidt process utilizes elementary column operations to preserve the span of its columns. We will formally define this concept as the column space, $\mathcal{C}(\boldsymbol{A})$, in later chapters. For now, just know that this specific property is exactly what makes the $\boldsymbol{QR}$ decomposition the mathematical engine under the hood of linear regression.

Handling Non-Square Matrices

Up until now, we have exclusively applied the Gram-Schmidt process to square matrices. We did this because square, full-rank matrices are conceptually the easiest starting point: they form a complete basis for the entire space $\mathbb{R}^n$, and their resulting $\boldsymbol{Q}$ matrix is a perfectly square, easily invertible orthogonal matrix.

However, in computer science and data analysis, we rarely deal with perfectly square data. What happens if we apply the Gram-

Schmidt process to a "tall" matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ (where $m > n$) with linearly independent columns?

The beauty of the Gram-Schmidt algorithm is that the mathematical steps remain exactly the same. We take the n columns of $\boldsymbol{A}$ and orthogonalize them one by one. Because there are only n columns, the algorithm naturally terminates after n steps. The result is a set of n orthonormal vectors that still live in the larger m-dimensional space.

If we place these output vectors into a matrix, we get $\boldsymbol{Q}_1 \in \mathbb{R}^{m \times n}$. Because $\boldsymbol{Q}_1$ is tall, it is not a strictly orthogonal matrix (it has no standard inverse, and $\boldsymbol{Q}_1\boldsymbol{Q}_1^{\mathsf{T}} \neq \boldsymbol{I}_m$). However, its columns *are* orthonormal, meaning its left inverse is its transpose: $\boldsymbol{Q}_1^{\mathsf{T}}\boldsymbol{Q}_1 = \boldsymbol{I}_n$. The corresponding upper triangular matrix $\boldsymbol{R}_1$ tracks the linear combinations just as before, and it remains a square $n \times n$ matrix. This is known as the *Thin $\boldsymbol{QR}$ Decomposition*:

$$\boldsymbol{A} = \boldsymbol{Q}_1\boldsymbol{R}_1$$

Rank-Deficient Matrices

For the sake of completeness, let's also consider the case when the matrix $\boldsymbol{A}$ is not full rank. Whether the matrix is square or tall, a rank-deficient matrix means that its column vectors are linearly dependent. How does the Gram-Schmidt process work for them?

During the execution of the algorithm, whenever we reach a column $\boldsymbol{a}_j$ that is merely a linear combination of the previously processed columns, subtracting its projections onto the existing $\boldsymbol{q}_i$ vectors will cancel it out entirely. This leaves us with the zero vector: $\boldsymbol{a}_{j\perp} = \boldsymbol{0}$. Because we cannot normalize the zero vector (which would require a mathematically illegal division by zero), the algorithm simply skips it. Ultimately, the process successfully generates exactly r orthonormal vectors, where $r = \text{rank}(\boldsymbol{A})$.

Wide Matrices

This brings us naturally to "wide" matrices, where $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and $m < n$. Because the number of columns (n) exceeds the number of dimensions they live in (m), the columns of a wide matrix are guaranteed to be linearly dependent. In other words, a wide matrix can never have full column rank; its maximum possible rank is m.

If we apply the Gram-Schmidt process to a wide matrix, it will operate exactly as described above for rank-deficient matrices. It will orthogonalize the independent columns and seamlessly skip the dependent ones (which reduce to zero vectors), ultimately yielding exactly $r \leq m$ orthonormal basis vectors.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

8

Review and Recap

The story so far:
In the beginning the Universe was created.
This has made a lot of people very angry and been widely regarded as a bad move.

—Douglas Adams



8.1 Formalizing the Abstraction: Vector Spaces

We have come about halfway in our linear algebra journey. The way we discussed our topics in this book was explicitly geared toward computer science. Specifically, our vectors and matrices were over the field of real numbers, $\mathbb{R}$. We could generalize this to any field, $\mathbb{F}$ or $\mathbb{K}$, and almost all of our statements and discussions would still stand. However, for computer scientists, we believe the most appropriate field to work with is $\mathbb{R}$.

We also presented certain definitions, such as the pivotal one of a vector space, incrementally: we defined vectors first, their operations, and the properties thereof, and finally stated that a vector space is

merely a complete collection or closed set of all possible vectors of the same dimension.

We may find some value in taking a step back and defining the concept of a vector space in a general and formal way, which starts from scratch and lists all the properties and operations we are looking for. Here is such a definition, which also provides a good summary of our initial, introductory chapters.

Vector Space

Definition: A vector space over a field $\mathbb{K}$ is a set of elements equipped with two operations. We will call the elements "vectors" and use the symbol $\mathcal{S}$ for the set.

1. **Addition** (denoted by $+$): For any two vectors, $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{S}$, addition assigns a third (not necessarily distinct) vector (called the sum) $\boldsymbol{z} \in \mathcal{S}$. We will write $\boldsymbol{z} = \boldsymbol{x} + \boldsymbol{y}$.
2. **Scalar Multiplication**: For an element $s \in \mathbb{K}$ (which we will call a scalar) and a vector $\boldsymbol{x} \in \mathcal{S}$, scalar multiplication assigns a new (not necessarily distinct) vector $\boldsymbol{z} \in \mathcal{S}$ such that $\boldsymbol{z} = s\boldsymbol{x}$.

These two operations have to satisfy the properties listed below:

Commutativity: Addition is commutative, meaning the order in which the vectors are added does not matter. For any two vectors $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{S}$,

$$\boldsymbol{x}_1 + \boldsymbol{x}_2 = \boldsymbol{x}_2 + \boldsymbol{x}_1$$

We can make scalar multiplication also commutative by defining $s\boldsymbol{x} = \boldsymbol{x}s$.

Associativity: Both operations should respect associativity, which means we can group and perform the operations in any order we want. For any two scalars $s_1, s_2 \in \mathbb{K}$ and a vector $\boldsymbol{x} \in \mathcal{S}$,

$$s_1 s_2 \boldsymbol{x} = s_1(s_2\boldsymbol{x}) = (s_1 s_2)\boldsymbol{x}$$

and for any three vectors $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3 \in \mathcal{S}$,

$$\boldsymbol{x}_1 + \boldsymbol{x}_2 + \boldsymbol{x}_3 = (\boldsymbol{x}_1 + \boldsymbol{x}_2) + \boldsymbol{x}_3 = \boldsymbol{x}_1 + (\boldsymbol{x}_2 + \boldsymbol{x}_3)$$

Distributivity: Scalar multiplication distributes over vector addition. For any scalar $s \in \mathbb{K}$ and any two vectors $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{S}$,

$$s(\boldsymbol{x}_1 + \boldsymbol{x}_2) = s\boldsymbol{x}_1 + s\boldsymbol{x}_2$$

and for any two scalars $s_1, s_2 \in \mathbb{K}$ and a vector $\boldsymbol{x} \in \mathcal{S}$,

$$(s_1 + s_2)\boldsymbol{x} = s_1\boldsymbol{x} + s_2\boldsymbol{x}$$

Identity of Addition: The set $\mathcal{S}$ has an identity of addition (called the zero vector) $\mathbf{0} \in \mathcal{S}$ such that $\boldsymbol{x} + \mathbf{0} = \mathbf{0} + \boldsymbol{x} = \boldsymbol{x}$.

Additive Inverse: For every $\boldsymbol{x} \in \mathcal{S}$, the set $\mathcal{S}$ has an inverse of addition $-\boldsymbol{x}$ such that $\boldsymbol{x} + (-\boldsymbol{x}) = \mathbf{0}$, the identity of addition.

Identity of Scalar Multiplication: The field $\mathbb{K}$ has a unit element, 1 (called the multiplicative identity) such that for every $\boldsymbol{x} \in \mathcal{S}$, $1\boldsymbol{x} = \boldsymbol{x}1 = \boldsymbol{x}$.

If $\mathcal{S}$ is a set that satisfies all these conditions, then it is a vector space.

As we can see, we do not say anything about what the "vectors" are. In particular, we are free to call our matrices by the name vectors also, and consider a collection of matrices that satisfy these conditions to be a vector space. What are such matrices? They are matrices of the same dimensions, of which our vectors are examples.

CS Connection: Matrices as Generalized Vectors

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

8.2 The Two Geometries: Row and Column Pictures

When we started visualizing linear equations (Figure 4.1), we were doing it in the coordinate space, where we have points, lines, and planes. A linear equation in the *coordinate space* of $\mathbb{R}^2$, for instance, is a line. As we know, each row of the augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ represents an equation, which has a shape in the coordinate space. In other words, when we think of equations, we are thinking of rows, and we can consider this thinking process the row picture of $\boldsymbol{Ax} = \boldsymbol{b}$.

The dimension of this coordinate space is the same as the number of unknowns.

We have a wholly separate *vector* space $\mathbb{R}^2$ as well, where we have all possible vectors with two components and nothing else. When we think of $\boldsymbol{Ax} = \boldsymbol{b}$ as a system of equations demanding that $\boldsymbol{b}$ be the right linear combination of the *columns* of $\boldsymbol{A}$ as specified by $\boldsymbol{x}$, we are working with the latter. If we have two rows, our column vectors have two elements, and we are working with the vector space $\mathbb{R}^2$, regardless of how many unknowns we may have. If we have three equations, as in Figure 6.7, we are in the vector space $\mathbb{R}^3$, even though we have only two unknowns. This mode of thinking can be considered the column picture of systems of linear equations. This column picture is the more sophisticated, university-level thinking we are trying to foster in this course.

CS Connection: Row vs. Column Picture in Practice

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

8.2.1 The Column Picture of Matrix Multiplication

While describing matrix multiplication, we presented several different ways of understanding it. All these different pictures are indeed equivalent. First, we started with an element-wise definition. We then saw that vectors, being matrices of single columns, obeyed the same multiplication rules, and realized that the vector dot product is a form of matrix multiplication. We then redefined matrix multiplication as dot products of the rows of the first matrix with the columns of the second.

We also looked at the row and column pictures of matrix multiplication: multiplication on the left is the same as a linear combination of the rows of the second matrix, and multiplication on the right is the same as the linear combination of the columns of the first. Of all these different views of matrix multiplication, the column picture is the most useful one. Let's look at it once more, and apply it to our understanding of some of the crucial and basic concepts of linear algebra.

The Zero Vector


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

In the product $\boldsymbol{AX} = \boldsymbol{B}$ with $\boldsymbol{A} \in \mathbb{R}^{m\times k}$, $\boldsymbol{X} \in \mathbb{R}^{k\times n}$ and consequently $\boldsymbol{B} \in \mathbb{R}^{m\times n}$, the columns of the product $\boldsymbol{B}$ are linear combinations of the columns of $\boldsymbol{A}$, taken with the scalars in each column of $\boldsymbol{X}$.

More specifically, in our favorite equation $\boldsymbol{Ax} = \boldsymbol{b}$, $\boldsymbol{b}$ is a linear combination of the columns of $\boldsymbol{A}$ scaled by the components of $\boldsymbol{x}$. This view makes it possible for us to unearth a system of linear equations hiding behind the definitions of span and linear independence of vectors.

8.2.2 Span of Vectors

Earlier, we defined the span of vectors in Eqn (6.2). For our own nefarious ulterior motives, let's recast the definition using other notations and symbols. Given n vectors $\boldsymbol{a}_i \in \mathbb{R}^m$, their span is defined for any n scalars $x_i \in \mathbb{R}$ as:

$$\text{span}(\{\boldsymbol{a}_i\}) = \left\{ \boldsymbol{b} \mid \boldsymbol{b} = \sum_{i=1}^{n} x_i \boldsymbol{a}_i \right\} \tag{8.1}$$

We switched from our previous notation of vectors (from $\boldsymbol{x_i} \rightarrow \boldsymbol{a}_i$) and scaling factors (from $s_i \rightarrow x_i$). Can we unearth our favorite matrix equation $\boldsymbol{Ax} = \boldsymbol{b}$ from this new definition?

As we can see, we may redefine the span of a set of vectors as follows: if we arrange a set of vectors as the columns of a matrix $\boldsymbol{A}$, then their span is all possible vectors we can get as the product $\boldsymbol{Ax} = \boldsymbol{b}$ for all possible vectors $\boldsymbol{x}$. In the very next chapter, we will have a lot more to say about this way of looking at the span.

8.2.3 Linear Independence

The definition of linear independence in Eqn (6.3) also hides within it a system of linear equations. In order to rebrand linear dependency

also as a matrix equation that we will have the pleasure of meeting again later on, let's consider it for the columns $\boldsymbol{a}_i \in \mathbb{R}^m$ of $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, where the scalars are called $x_i \in \mathbb{R}$. The condition now becomes:

$$\sum_{i=1}^{n} x_i \boldsymbol{a}_i = \boldsymbol{0} \implies \boldsymbol{A}\boldsymbol{x} = \boldsymbol{0} \tag{8.2}$$

In other words, if we can find nontrivial solutions (meaning $\boldsymbol{x} \neq \boldsymbol{0}$) for the matrix equation $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0}$, then the columns of $\boldsymbol{A}$ are *not* linearly independent.

8.3 Algebraic Properties: Transposes and Inverses

The product rules for transposes and inverses are similar: when we transpose (or invert) a product, we apply the operations in the reverse order to the factor matrices.

$$(\boldsymbol{A}\boldsymbol{B})^{\mathsf{T}} = \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}} \quad \text{and} \quad (\boldsymbol{A}\boldsymbol{B})^{-1} = \boldsymbol{B}^{-1}\boldsymbol{A}^{-1}$$

The product rule for transposes gives us two interesting symmetric matrices from any arbitrary rectangular matrix $\boldsymbol{A}$:

$$\begin{aligned}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{\mathsf{T}} &= \boldsymbol{A}^{\mathsf{T}}(\boldsymbol{A}^{\mathsf{T}})^{\mathsf{T}} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} \quad \text{and} \\ (\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}})^{\mathsf{T}} &= (\boldsymbol{A}^{\mathsf{T}})^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}\end{aligned}$$

$\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ is the Gram matrix. The matrices $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$, $\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}$, and $\boldsymbol{A}^{\mathsf{T}}$ all have the same rank as $\boldsymbol{A}$. In particular, if $\boldsymbol{A}$ has full column rank, $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ is a full-rank, square matrix. And if $\boldsymbol{A}$ has full row rank, $\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}$ is a full-rank, square matrix.

Combining the invertibility of $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ and $\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}$ with the product rule for inverses, we can derive the left and right inverses.

$$(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} = \boldsymbol{I} \implies \boldsymbol{A}_{\textbf{Left}}^{-1} = (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}}$$

In writing the first part of the expression above, we assume that $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ (the Gram matrix) is invertible, which means $\boldsymbol{A}$ has full column rank. Similarly, a full-row-rank matrix $\boldsymbol{A}$ has a right inverse:

$$\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}(\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}})^{-1} = \boldsymbol{I} \implies \boldsymbol{A}_{\textbf{Right}}^{-1} = \boldsymbol{A}^{\mathsf{T}}(\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}})^{-1}$$

Keep in mind that $\boldsymbol{A}$ is not necessarily square, and therefore does not have a double-sided inverse $\boldsymbol{A}^{-1}$. However, the left inverse can take the place of an inverse, as it did in Eqn (7.2).

While describing the Gram-Schmidt process, we wrote down the projection operator in a special form in Eqn (7.8):

$$\boldsymbol{a}_{2\|} = \boldsymbol{a}_1(\boldsymbol{a}_1^\mathsf{T}\boldsymbol{a}_1)^{-1}\boldsymbol{a}_1^\mathsf{T}\boldsymbol{a}_2$$

Notice a similar pattern emerging? The projection of $\boldsymbol{a}_2$ onto $\boldsymbol{a}_1$ relies on the *left* inverse of $\boldsymbol{a}_1$, denoted as $\boldsymbol{a}_{1,\text{left}}^{-1} \in \mathbb{R}^{1\times n}$, multiplying $\boldsymbol{a}_2 \in \mathbb{R}^n$ on the *left*. This produces a scalar that then scales $\boldsymbol{a}_1$ to make the projection a vector.

CS Connection: Pseudoinverses and Machine Learning


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

8.4 Computational Mechanics: Algorithms and Matrix Shapes

8.4.1 Named Algorithms

As we have encountered a few algorithms (by which we mean step-by-step instructions to achieve an objective), it may be useful now to compare them in terms of their purposes and end products, which is what we have in Table 8.1.

When we started solving systems of linear equations, we encountered the Gaussian elimination algorithm. Its objective is to get to the row echelon form (REF) of the matrix (typically, the augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ of the system $\boldsymbol{Ax} = \boldsymbol{b}$). Once we have the REF, we can count the number of pivots as $\text{rank}(\boldsymbol{A})$, and fully solve the system by back substitution.

As a byproduct, Gaussian elimination produces a decomposition as well, $\boldsymbol{A} = \boldsymbol{PLU}$. $\boldsymbol{P}$, being a permutation matrix, has a determinant of ± 1, while $\boldsymbol{L}$, being the inverse of a bunch of elementary matrices, has a determinant of 1. Therefore, $|\boldsymbol{A}|$ can be computed from its REF, which is the upper triangular matrix $\boldsymbol{U}$, as the product of its diagonal elements, multiplied by $|\boldsymbol{P}| = \pm 1$.

Table 8.1 Named algorithms and their uses

Name	Type of Operation	Uses	Decomposition
Gaussian Elimination	Elementary Row Operations	Row Echelon Form (REF). Solve equations using back substitution. Determine rank. Compute determinants.	$\boldsymbol{PLU}$
Gauss-Jordan Elimination	Elementary Row Operations	Reduced Row Echelon Form (RREF). Solve equations. Determine rank. Compute inverses.	$\boldsymbol{CR}$
Gram-Schmidt Process	Elementary Column Operations	Orthonormal basis matrix. Works on full-column-rank $\boldsymbol{A}$.	$\boldsymbol{QR}$

Does Gauss-Jordan Yield a Decomposition?

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

Gauss-Jordan elimination is an extension of Gaussian elimination, where we scale the pivot rows so the pivots have a value of one. We also get rid of all the matrix elements above the pivot in each pivot column by subtracting appropriate multiples of the pivot row. Our objective is to get an identity matrix if we were to look only at the pivot rows and columns. The resulting form is called the reduced row echelon form or RREF. Once we have the RREF of the augmented matrix of a system of linear equations, we can read off the solutions from the pivot rows. It also leads to a column-row decomposition (also known as the rank factorization), as described in the box below.

CS Connection

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

The RREF that Gauss-Jordan elimination produces can also tell us about the rank, which is the number of pivots (also equal to the sum of pivots because they all have a value equal to one). But the truly interesting usage of this algorithm is in inverting a matrix, $\boldsymbol{A}$. For this purpose, we augment it with $\boldsymbol{I}$ and run the Gauss-Jordan algorithm on $[\boldsymbol{A} \mid \boldsymbol{I}]$. When the $\boldsymbol{A}$ part of the augmented matrix becomes $\boldsymbol{I}$, we have the inverse of $\boldsymbol{A}$ where $\boldsymbol{I}$ used to be.

$$[\boldsymbol{A} \mid \boldsymbol{I}] \xrightarrow{\text{RREF}} [\boldsymbol{I} \mid \boldsymbol{A}^{-1}]$$

If the algorithm fails to produce $\boldsymbol{I}$, it means that it could not find a pivot in at least one row, and the matrix is not invertible; it is rank-deficient.

The third algorithm, the Gram-Schmidt process, is different from the first two in the sense that it performs elementary column operations (not row operations like the other two). However, it is not such a big difference because a column operation is a row operation on the transpose. The purpose of Gram-Schmidt is to produce a matrix that is orthogonal. In the process, it creates the so-called $\boldsymbol{QR}$ decomposition, where $\boldsymbol{R}$, an upper triangular matrix, is the inverse of all the elementary operations performed on the matrix. But the process does not really have to keep track of the operations and take their inverses because of the basic property of orthogonal matrices, $\boldsymbol{Q}^{-1} = \boldsymbol{Q}^{\mathsf{T}}$. Therefore,

$$\boldsymbol{A} = \boldsymbol{QR} \implies \boldsymbol{R} = \boldsymbol{Q}^{-1}\boldsymbol{A} = \boldsymbol{Q}^{\mathsf{T}}\boldsymbol{A}$$

Once $\boldsymbol{Q}$ is obtained, $\boldsymbol{R}$ is only a matrix multiplication away.

We have created a neat little Table 8.1 to compare these three algorithms. It may be useful as a quick reference to their steps, purposes, and usages.

CS Connection: Algorithmic Stability in Practice


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

8.4.2 Pivots, Ranks, Inverses, and Determinants

Another topic worthy of a recap is the interconnected story of pivots, ranks, inverses, and determinants of matrices, and how it relates to the

solvability of the underlying system of linear equations. We looked at each of these topics at various points in our discussions so far. Because of the intimate interconnections among them, it is best to summarize it all as we do in Figure 8.1.

Matrices come in different shapes: square, "tall," and "wide," as we described way back in §5.1.2 (page 99). Each shape of matrix can be either full-rank or rank-deficient, thereby giving us six possible permutations. When we run Gauss-Jordan elimination on these matrices, depending on their rank, we will get different canonical forms, as we discussed in §5.2.2 (page 104). Figure 8.1 is an attempt to put all these things together, and comment on the solvability of the underlying system of linear equations.

Corresponding to each permutation[1] of the matrix shape and its rank status, we have a row in Figure 8.1. Referring to these rows, we can make the following statements.

The only matrix that can be inverted is in the first row. It has a nonzero determinant, and it leads to a unique solution for the system of equations that it represents. The matrix in the second row also has a determinant, but it is zero. The matrix is not invertible. Note that its RREF has at least one zero row at the bottom. If the entry in the augmented part of the matrix is also zero, then the equations are consistent, but we do not have enough of them. Therefore, we get an infinite number of solutions. We can see it because the RREF has $\boldsymbol{F}$ in it, which indicates columns with no pivots and therefore free variables. On the other hand, if the constant in this zero row is nonzero, we have inconsistency and hence no solution.

We can make a similar statement about the third row also: the system can be consistent (with $0 = 0$ in the bottom rows), in which case we get a unique solution, or it can be inconsistent, with no solutions. Note the absence of $\boldsymbol{F}$ in the RREF, indicating that we will never get an infinity of solutions.

The fourth row has a matrix with RREF with both free variables and zero rows. So we have the possibilities of no solution or an infinite number of solutions.

[1] To be very pedantic about it, it is not a permutation but an element of the Cartesian product of the sets of matrix shapes and rank statuses.

	Shape	Rank	RREF	Solvability
1	Square, Full-rank: $\boldsymbol{A} \in \mathbb{R}^{n \times n}$	n	$\boldsymbol{I}_n$	**Unique Solution**
2	Square, Rank-deficient: $\boldsymbol{A} \in \mathbb{R}^{n \times n}$	$r < n$	$\begin{bmatrix} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r \times (n-r)} \\ \hline \boldsymbol{0}_{(n-r) \times n} \end{bmatrix}$	**No Solution** or **Infinity of Solutions**
3	Tall, Full-rank: $\boldsymbol{A} \in \mathbb{R}^{m \times n}, m > n$	n	$\begin{bmatrix} \boldsymbol{I}_n \\ \hline \boldsymbol{0}_{(m-n) \times n} \end{bmatrix}$	**Unique Solution** or **No Solution**
4	Tall, Rank-deficient: $\boldsymbol{A} \in \mathbb{R}^{m \times n}, m > n$	$r < n$	$\begin{bmatrix} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r \times (n-r)} \\ \hline \boldsymbol{0}_{(m-r) \times n} \end{bmatrix}$	**No Solution** or **Infinity of Solutions**
5	Wide, Full-rank: $\boldsymbol{A} \in \mathbb{R}^{m \times n}, m < n$	m	$\begin{bmatrix} \boldsymbol{I}_m \bowtie \boldsymbol{F}_{m \times (n-m)} \end{bmatrix}$	**Infinity of Solutions**
6	Wide, Rank-deficient: $\boldsymbol{A} \in \mathbb{R}^{m \times n}, m < n$	$r < m$	$\begin{bmatrix} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r \times (n-r)} \\ \hline \boldsymbol{0}_{(m-r) \times n} \end{bmatrix}$	**No Solution** or **Infinity of Solutions**

Fig. 8.1 Properties and solvability of the linear equations $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ based on the RREF of the coefficient matrix $\boldsymbol{A}$.

When it comes to the full-rank "wide" matrix, we have no zero row, which means we will never have inconsistency. But we never have enough equations either (as indicated by the free variable part in the RREF), and we always have an infinity of solutions.

Rank-deficient "wide" matrices lead to situations similar to rows 2 and 4, with no solution or an infinite number of solutions. To reiterate, when a matrix is rank-deficient (as in rows 2, 4, or 6), regardless of its shape, we either have no solution (inconsistent equations) or an infinite number of solutions (not enough equations).

Figure 8.1 is color-coded for easy reference: whenever we see free variables (corresponding to columns with no pivots) in RREF, shown in purple, an infinity of solutions is a possibility, precluded only by the existence of inconsistent equations (shown in red) as indicated by zero rows in the $\boldsymbol{A}$ part of $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$, with the $\boldsymbol{b}$ part nonzero. The only instances where we get a unique solution is when we can have an $\boldsymbol{I}_n$ in the RREF, shown in green. We can also see that a tall matrix can never have an infinity of solutions and a wide one can never have a unique solution.

CS Connection: Matrix Shapes in Machine Learning


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

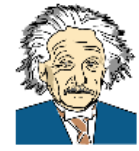

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

9

The Four Fundamental Spaces

Time and space are modes by which we think and not conditions in which we live.

—Albert Einstein



Our geometric view started with the notion that we can think of the solution to the system of linear equations $\boldsymbol{Ax} = \boldsymbol{b}$ as a quest for that special $\boldsymbol{x}$ whose components become the coefficients in taking the linear combination of the columns of $\boldsymbol{A}$ to give us $\boldsymbol{b}$. Since this opening statement was a bit too long and tortured, let's break it down. Here is what it means:

- The product of the matrix multiplication $\boldsymbol{Ax}$ is a linear combination of the columns $\boldsymbol{a}_i$ of $\boldsymbol{A}$.

- The linear combination is taken with coefficients equal to the components of $\boldsymbol{x} = x_i$. In other words, the product is:

$$\boldsymbol{Ax} = \sum_{i=1}^{n} x_i \boldsymbol{a}_i$$

- Our system of linear equations $\boldsymbol{Ax} = \boldsymbol{b}$, therefore, becomes the condition:

$$\sum_{i=1}^{n} x_i \boldsymbol{a}_i = \boldsymbol{b}$$

Solving this system means finding the right x_i so that the linear combination satisfies the condition.

We also learned that the collection of all possible linear combinations of a set of vectors is called the span of those vectors, and it is a vector subspace, which is a subset of a vector space. For example, $\mathbb{R}^m$ is a space[1] and n vectors $\boldsymbol{a}_i \in \mathbb{R}^m$ span a subspace contained within $\mathbb{R}^m$. Let's call this subspace $\mathcal{C} \subseteq \mathbb{R}^m$. If, among the n vectors that span $\mathcal{C}$, only $r \leq n$ of them are linearly independent, then those r vectors form a basis for $\mathcal{C}$ and the dimension of this subspace $\mathcal{C}$ is indeed r (which is the cardinality of the basis). In fact, even if $n > m$, the n vectors span only a subspace $\mathcal{C} \subset \mathbb{R}^m$ if the number of independent vectors $r < m$. Note that r can never be greater than m, the number of components of each of our vectors $\boldsymbol{a}_i \in \mathbb{R}^m$.

CS Connection: Subspaces and Data Compression

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9.1 Column Space

The choice of the symbols in the previous paragraph was not an accident. We are ready to introduce the concept of the column space of a matrix.

[1] Once again, we will be dropping the ubiquitous "vector" prefix from spaces and subspaces.

Column Space

Definition: The column space $\mathcal{C}$ of a matrix is the span of its columns.

$$\text{For a matrix } \boldsymbol{A} = \begin{bmatrix} \boldsymbol{a}_i \end{bmatrix} \in \mathbb{R}^{m \times n},\ \mathcal{C}(\boldsymbol{A}) \stackrel{\text{def}}{=} \left\{ \boldsymbol{z} \mid \boldsymbol{z} = \sum_{i=1}^{n} x_i \boldsymbol{a}_i \right\}$$

where $x_i \in \mathbb{R}$. Note that $\mathcal{C}(\boldsymbol{A}) \subseteq \mathbb{R}^m$, where m is the number of *rows* of $\boldsymbol{A}$ because each of its column vectors $\boldsymbol{a}_i \in \mathbb{R}^m$.

Are the columns of $\boldsymbol{A}$ ($\boldsymbol{a}_i$) a basis for $\mathcal{C}(\boldsymbol{A})$? Remembering that the basis of a space or subspace is the *minimal* set of vectors that span it, we can see that a decent basis for $\mathcal{C}(\boldsymbol{A})$ would be the linearly independent columns of $\boldsymbol{A}$. How many of them are there? We saw earlier that the number of linearly independent columns of $\boldsymbol{A}$ is its rank, rank$(\boldsymbol{A})$. To be more precise, we should call it the column-rank of $\boldsymbol{A}$, but we also saw that the column and row ranks were the same. Finally, the dimension of a space or subspace is the number of vectors in its basis. Therefore, the dimension of the column space is the rank. Conversely, we can define rank$(\boldsymbol{A})$ as the dimension of $\mathcal{C}(\boldsymbol{A})$.

9.1.1 Significance of Column Space

As we said in the introduction to this chapter, the solution to $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ is that $\boldsymbol{x}$ which produces the linear combination of the columns of $\boldsymbol{A}$ equalling $\boldsymbol{b}$. We then defined the column space $\mathcal{C}(\boldsymbol{A})$ as *all* possible linear combinations of the columns of $\boldsymbol{A}$. In other words, if $\boldsymbol{b} \notin \mathcal{C}(\boldsymbol{A})$, we will never be able to find a solution vector $\boldsymbol{x}$.

We can, in fact, turn the last statement about the solvability of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ and $\mathcal{C}(\boldsymbol{A})$ into an alternative definition of column space: $\mathcal{C}(\boldsymbol{A})$ is the set of all vectors $\boldsymbol{b}$ for which there exists an $\boldsymbol{x}$ such that $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.

Note that the elementary column operations, which we used in the Gram-Schmidt process, do not change the column space because they are all about taking linear combinations of the columns or scaling them. The span of the columns is unaffected by such linear operations. Therefore, in $\boldsymbol{A} \xrightarrow{\text{G-S}} \boldsymbol{Q}$, both $\boldsymbol{A}$ and its orthogonal cousin $\boldsymbol{Q}$ have the same column space: $\mathcal{C}(\boldsymbol{A}) = \mathcal{C}(\boldsymbol{Q})$.

On the other hand, the elementary row operations in Gaussian and Gauss-Jordan eliminations do affect the column space. What is

unaffected would be the linear combination of the rows, which we will call the row space.

CS Connection: Reachable States and Shadows


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9.1.2 Recipe to Compute Column Space

"Computing" a subspace means three things:

1. Identify the containing space, *e.g.*, $\mathcal{C}(\boldsymbol{A}) \subset \mathbb{R}^5$.
2. Compute the dimension of the subspace, *e.g.*, $\dim(\mathcal{C}(\boldsymbol{A})) = 3$.
3. Specify the basis vectors as a set, *e.g.*, Basis $= \{\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3\}$.

For a matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ of rank r, the dimension of $\mathcal{C}(\boldsymbol{A})$ is r, and the dimension of the containing space is m, the number of rows. Or, more compactly, $\mathcal{C}(\boldsymbol{A}) \subseteq \mathbb{R}^m$, $\dim(\mathcal{C}(\boldsymbol{A})) = r$.

How do we find the basis, or the linearly independent columns of $\boldsymbol{A}$? The pivot columns are the ones that are linearly independent in the reduced forms. It turns out that the corresponding columns in the *original matrix* $\boldsymbol{A}$ are the ones that are linearly independent, and therefore, they form a good basis for $\mathcal{C}(\boldsymbol{A})$. Let's see why.

In order to make the discussion easier, we are going to call the RREF of $\boldsymbol{A}$ by the name $\boldsymbol{R}$. As we know, the solution set of the system of equations $\boldsymbol{Ax} = \boldsymbol{b}$ can be obtained from its augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$ by finding its RREF, which is $\left[\boldsymbol{A} \mid \boldsymbol{b}\right] \xrightarrow{\text{RREF}} \left[\boldsymbol{R} \mid \boldsymbol{b}'\right]$, where $\boldsymbol{b} \neq \boldsymbol{b}'$ in general. But when $\boldsymbol{b} = \boldsymbol{0}$, the constants part does not get affected by row operations. In other words, $\boldsymbol{Ax} = \boldsymbol{0}$ and $\boldsymbol{Rx} = \boldsymbol{0}$ have the same solutions, since the solution of a system of linear equations is not affected by row operations (which are equivalent to equation manipulations).

Next, we can see that in $\boldsymbol{R}$, the pivot columns are linearly independent of each other by design. They are the only ones with a nonzero element (actually 1) in the pivot positions, and there is no way we can create a 1 by taking finite combinations of 0. Therefore, a linear combination of the pivot columns in $\boldsymbol{R}$ will be $\boldsymbol{0}$ if and only if the

coefficients multiplying them are all zero. Since the solution sets for $\boldsymbol{A}$ and $\boldsymbol{R}$ are the same, the same statement applies to $\boldsymbol{A}$ as well. Therefore, the columns in $\boldsymbol{A}$ that correspond to the pivot columns in $\boldsymbol{R}$ are linearly independent. And indeed, they form a basis for the column space of $\boldsymbol{A}$.

9.1.3 Worked Example

Let's compute the column space of the following matrix:

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 3 \end{bmatrix}$$

Performing elimination to find the RREF:

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 3 \end{bmatrix} \xrightarrow{\text{REF}} \begin{bmatrix} 1 & 1 & 1 & 6 \\ 0 & 0 & -1 & -3 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$
$$\xrightarrow{\text{RREF}} \begin{bmatrix} 1 & 1 & 0 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \boldsymbol{R}$$

Looking at $\boldsymbol{R}$, we can see that we have pivots in columns 1 and 3. The rank of the matrix is 2.

The column space lives in $\mathbb{R}^3$ because the columns have three components. The dimension of the column space = the rank = the number of pivots = 2. The basis (which is a set) consists of the column vectors in $\boldsymbol{A}$ (*not* in $\boldsymbol{R}$) corresponding to the pivot columns, namely 1 and 3. So here is our answer:

$$\mathcal{C}(\boldsymbol{A}) \subset \mathbb{R}^3; \quad \dim\left(\mathcal{C}(\boldsymbol{A})\right) = 2; \quad \text{Basis} = \left\{ \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \right\}$$

9.2 Null Space

So far, we have concerned ourselves mostly with the solution of $\boldsymbol{Ax} = \boldsymbol{b}$. Let's now take a look at the so-called homogeneous system of linear equations $\boldsymbol{Ax} = \boldsymbol{0}$. In fact, we did come across this system

when we were talking about linear independence in the last chapter, rebranding its condition as a matrix equation in Eqn (8.2). If $\boldsymbol{Ax} = \mathbf{0}$ holds for some nonzero vector $\boldsymbol{x}$ (in other words, if the homogeneous system has a nontrivial solution), then the columns of $\boldsymbol{A}$ are not linearly independent.

The collection of *all* vectors $\boldsymbol{x}$ that solve $\boldsymbol{Ax} = \mathbf{0}$ (called the solution set) is the null space of $\boldsymbol{A}$, denoted by $\mathcal{N}(\boldsymbol{A})$. Let's put down this statement as the definition of the null space.

Null Space

Definition: The null space $\mathcal{N}$ of a matrix $\boldsymbol{A}$ is the complete set of all vectors that form the solutions to the homogeneous system of linear equations $\boldsymbol{Ax} = \mathbf{0}$.

$$\mathcal{N}(\boldsymbol{A}) \stackrel{\text{def}}{=} \{\boldsymbol{x} \mid \boldsymbol{Ax} = \mathbf{0}\}$$

Note that if we have two vectors in the null space, then all their linear combinations are also in the null space. $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{N}(\boldsymbol{A}) \implies \boldsymbol{x} = s_1\boldsymbol{x}_1 + s_2\boldsymbol{x}_2 \in \mathcal{N}(\boldsymbol{A})$. If $\boldsymbol{Ax}_1 = \mathbf{0}$ and $\boldsymbol{Ax}_2 = \mathbf{0}$, then $\boldsymbol{A}(s_1\boldsymbol{x}_1 + s_2\boldsymbol{x}_2) = \mathbf{0}$ because of the basic linearity condition we learned way back in the first chapter. What all this means is that the null space is indeed a vector subspace. Furthermore, $\mathcal{N}(\boldsymbol{A})$ is complete by definition. In other words, we will not find a vector $\boldsymbol{x}$ that is not in $\mathcal{N}(\boldsymbol{A})$ such that $\boldsymbol{Ax} = \mathbf{0}$. In the cold hard language of mathematics, $\boldsymbol{x} \in \mathcal{N}(\boldsymbol{A}) \iff \boldsymbol{Ax} = \mathbf{0}$.

9.2.1 Significance of Null Space

For any and all vectors $\boldsymbol{x} \in \mathcal{N}(\boldsymbol{A})$ in the null space of $\boldsymbol{A}$, their dot products with the rows of $\boldsymbol{A}$ are zero. If $\boldsymbol{r}_i^{\mathsf{T}}$ is a row of $\boldsymbol{A}$, we have:

$$\boldsymbol{Ax} = \mathbf{0} \implies \begin{bmatrix} \vdots \\ \boldsymbol{r}_i^{\mathsf{T}} \\ \vdots \end{bmatrix} \boldsymbol{x} = \mathbf{0} \implies \boldsymbol{r}_i^{\mathsf{T}}\boldsymbol{x} = 0 \quad \forall\, i \tag{9.1}$$

The dot product being zero means that the vectors in $\mathcal{N}(\boldsymbol{A})$ and the rows of $\boldsymbol{A}$ are orthogonal. It then follows that any linear combination of the rows of $\boldsymbol{A}$ is also orthogonal to the vectors in $\mathcal{N}(\boldsymbol{A})$. And, the

collection of the linear combinations of the rows of $\boldsymbol{A}$ is indeed their span, which is a subspace we will soon call the row space of $\boldsymbol{A}$.

CS Connection: Null Space in Robotics

Get the **Full Edition** of LA4CS with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9.2.2 Recipe to Compute Null Space

The null space of a matrix would be the complete solution of the homogeneous system of equations $\boldsymbol{Ax} = \boldsymbol{0}$. When we write down the augmented matrix, it becomes $\left[\boldsymbol{A} \mid \boldsymbol{0}\right]$, and whatever row operations we perform on it will not change the constants (or right-hand side) part because it is $\boldsymbol{0}$. We can therefore solve $\boldsymbol{Ax} = \boldsymbol{0}$ by performing Gauss-Jordan elimination on the matrix $\boldsymbol{A}$ itself, without augmenting it. If we find columns with no pivots, we will have free variables and a null space.

Computing the null space $\mathcal{N}(\boldsymbol{A})$ for a matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ involves three explicit steps:

1. **Containing Space:** Identify the containing space, which is always the input space of the transformation, $\mathcal{N}(\boldsymbol{A}) \subseteq \mathbb{R}^n$ (where n is the number of columns of $\boldsymbol{A}$).

2. **Dimension (Nullity):** Perform Gauss-Jordan elimination to reach RREF$(\boldsymbol{A})$. Count the number of free variables (columns without pivots). The dimension of the null space is $\dim\left(\mathcal{N}(\boldsymbol{A})\right) = n - r$, where $r = \text{rank}(\boldsymbol{A})$.

3. **Basis Vectors:** Solve the homogeneous system $\boldsymbol{Ax} = \boldsymbol{0}$ by setting each free variable to 1 and all other free variables to 0, sequentially. Specify the resulting special solution vectors as a set, *e.g.*, Basis $= \{\boldsymbol{x}_{\perp,1}, \boldsymbol{x}_{\perp,2}, \ldots, \boldsymbol{x}_{\perp,n-r}\}$.

9.2.3 Worked Example

Let's compute the null space of the same matrix we used for $\mathcal{C}(\boldsymbol{A})$:

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 3 \end{bmatrix}$$

For the null space of our matrix $\boldsymbol{A}$, we will first solve the underlying $\boldsymbol{Ax} = \boldsymbol{0}$ equations completely, highlight a pattern, and present it as a possible shortcut, to be used with care.

We saw earlier that $\boldsymbol{Ax} = \boldsymbol{0}$ and $\boldsymbol{Rx} = \boldsymbol{0}$ (where $\boldsymbol{A} \xrightarrow{\text{REF}} \boldsymbol{R}$) have the same solution set, which is the null space. Writing down the equations explicitly, we have:

$$\begin{aligned} \boldsymbol{Rx} = \begin{bmatrix} 1 & 1 & 0 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} &= \boldsymbol{0} \\ \implies x_1 + x_2 + 3x_4 = 0 \quad \text{and} \quad x_3 + 3x_4 &= 0 \end{aligned} \tag{9.2}$$

Noting that x_2 and x_4 are free variables (because the corresponding columns have no pivots), we solve for x_1 and x_3 as $x_3 = -3x_4$ and $x_1 = -x_2 - 3x_4$. Therefore, the complete solution becomes:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -x_2 - 3x_4 \\ x_2 \\ -3x_4 \\ x_4 \end{bmatrix} = x_2 \begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \end{bmatrix} + x_4 \begin{bmatrix} -3 \\ 0 \\ -3 \\ 1 \end{bmatrix}$$

The complete solution is a linear combination of the two vectors above because x_2 and x_4, being free variables, can take any value in $\mathbb{R}$. In other words, the complete solution is the span of these two vectors, which is what the null space is. We give our computation of the null space as follows:

$$\mathcal{N}(\boldsymbol{A}) \subset \mathbb{R}^4; \quad \dim\left(\mathcal{N}(\boldsymbol{A})\right) = 2; \quad \text{Basis} = \left\{ \begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} -3 \\ 0 \\ -3 \\ 1 \end{bmatrix} \right\}$$

9.3 Row Space

The set of all possible linear combinations of the rows of a matrix is called its row space, with one little complication. Since our vectors are all column vectors, and the rows of a matrix are definitely not columns, we think of the transpose of the rows as the vectors whose linear combinations make the row space. Equivalently, we may transpose the matrix first and call the column space of the transpose the row space of the original. That is to say, the row space of $\boldsymbol{A}$ is the column space of $\boldsymbol{A}^{\mathsf{T}}$, $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$, which is the notation we will use for the row space. Since the row rank is the same as the column rank, the number of linearly independent rows in $\boldsymbol{A}$ is its rank, which is the same as the dimension of $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$.

9.3.1 Significance of Row Space

If all vectors in a subspace are orthogonal to all vectors in another subspace, then we call these two subspaces orthogonal. Earlier, we established that the row and null spaces of a matrix are orthogonal to each other. In fact, the row and null spaces are orthogonal complements of each other, which means that all the vectors in the containing space that are orthogonal to all the vectors in the null space are in the row space. Again, this complex statement needs some dismantling (and convincing). What this statement says, in mathematical language, is the following:

- A matrix $\boldsymbol{A}$ has a row space $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$ and null space $\mathcal{N}(\boldsymbol{A})$.
- $\boldsymbol{x} \notin \mathcal{N}(\boldsymbol{A})$, $\boldsymbol{y} \in \mathcal{N}(\boldsymbol{A})$, and $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y} = 0 \implies \boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$.

The proof of this assertion is probably beyond the remit of this book on applied linear algebra, and is, therefore, included as an optional box on "**Orthogonal Complementarity**." What it means, however, is important for us to know. Let's first illustrate it using an example.

$$\boldsymbol{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \implies \text{Basis for } \mathcal{C}(\boldsymbol{A}^{\mathsf{T}}) = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \right\}$$

Clearly, $\boldsymbol{A} \in \mathbb{R}^{3\times 3}$ and $\text{rank}(\boldsymbol{A}) = 2$. Therefore, $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}}) \subset \mathbb{R}^3$ with a dimension of 2, which is why we have two basis vectors (the pivot rows of $\boldsymbol{A}$). In this case, $\boldsymbol{A}$ is already in its RREF.

Orthogonal Complementarity


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

In order to compute $\mathcal{N}(\boldsymbol{A})$, we have to find the solutions to $\boldsymbol{Ax} = \boldsymbol{0}$. For our $\boldsymbol{A}$, we can see that all vectors with the first two components being zero and a nonzero third component will satisfy $\boldsymbol{Ax} = \boldsymbol{0}$. We can therefore write:

$$\mathcal{N}(\boldsymbol{A}) \subset \mathbb{R}^3, \text{ of one dimension, with the basis } \left\{ \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$$

Thinking in terms of the coordinate space, we see that the row space is actually the xy-plane and the null space is the z-axis. They are indeed orthogonal complements.

Let's go back to talking about the general case, $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, where $\text{rank}(\boldsymbol{A}) = r$. We know that by definition, all vectors $\boldsymbol{x} \in \mathcal{N}(\boldsymbol{A})$ get transformed to $\boldsymbol{0}$ by $\boldsymbol{Ax} = \boldsymbol{0}$. Because of orthogonal complementarity, we also know that all *orthogonal* vectors are in $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$, the row space of $\boldsymbol{A}$, as illustrated above with our toy example. All the vectors in the row space get transformed to nonzero vectors in $\mathbb{R}^m$. What about the rest of the vectors? After all, most vectors in $\mathbb{R}^n$ are in neither $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$ nor $\mathcal{N}(\boldsymbol{A})$: they are linear combinations of these two sets of vectors. Let's take such a vector $\boldsymbol{x} = \boldsymbol{x}_\parallel + \boldsymbol{x}_\perp$ where $\boldsymbol{x}_\parallel \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$ and $\boldsymbol{x}_\perp \in \mathcal{N}(\boldsymbol{A})$.

$$\boldsymbol{Ax} = \boldsymbol{A}(\boldsymbol{x}_\parallel + \boldsymbol{x}_\perp) = \boldsymbol{Ax}_\parallel + \boldsymbol{Ax}_\perp = \boldsymbol{Ax}_\parallel + \boldsymbol{0} = \boldsymbol{Ax}_\parallel$$

What this equation tells us is that all vectors not in $\mathcal{N}(\boldsymbol{A})$ also end up in $\mathcal{C}(\boldsymbol{A})$ by $\boldsymbol{Ax}$. Moreover, multiple vectors $\boldsymbol{x} \notin \mathcal{N}(\boldsymbol{A})$ get transformed to the same $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$: it is a many-to-one (surjective) mapping.

What is special about the row space is that the mapping from $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$ to $\mathcal{C}(\boldsymbol{A})$ is a one-to-one (injective) mapping. Since it is an important point, let's state it mathematically:

$$\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathsf{T}) \text{ and } \boldsymbol{Ax} = \boldsymbol{b} \implies \text{for a given } \boldsymbol{b}, \text{ the vector } \boldsymbol{x} \text{ is unique.}$$

Proof: One-to-One Mapping from Row Space to Column Space


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

In order to see it, we have to appreciate that $\boldsymbol{b}$ is a linear combination of the linearly independent columns of $\boldsymbol{A}$. For a specific $\boldsymbol{x}$, it is a linear combination with the specified coefficients. As we know, probably from the first chapter, there is only one such linear combination; we cannot get two different linear combinations with the same set of linearly independent vectors and coefficients.

A consequence of $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$ being the orthogonal complement to $\mathcal{N}(\boldsymbol{A})$ is that their dimensions add up to the dimension of the containing space. Or, $n = r + \dim(\mathcal{N}(\boldsymbol{A})) \implies \dim(\mathcal{N}(\boldsymbol{A})) = n - r$. The dimension of the null space of $\boldsymbol{A}$ is called its nullity, and the statement that the rank and nullity add up to the number of columns of $\boldsymbol{A}$ is the famous rank-nullity theorem.

A more formal proof of $\boldsymbol{A}$ being a one-to-one mapping between its row space and column space is given in a box.

CS Connection: Rank-Nullity in Deep Learning


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9.3.2 Recipe to Compute Row Space

It turns out that the computation of the row space because the easiest one. To find the rank and basis of $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$, we will fall back on Gaussian (or Gauss-Jordan) elimination to get to the REF (or RREF) of $\boldsymbol{A}$. Remembering that row operations do not change the row space, a good basis for the row space would be the pivot rows of the RREF (or REF or the original matrix). The dimension of the row space is the number of pivots (also known as rank$(\boldsymbol{A})$), and the containing space is $\mathbb{R}^n$, where n is the number of columns of $\boldsymbol{A}$. The full specification of the row space of $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, where rank$(\boldsymbol{A}) = r$, would therefore

be $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}}) \subseteq \mathbb{R}^n$ with a dimension of r and a basis consisting of the set of r pivot rows.

9.3.3 Worked Example

For the row space of our matrix $\boldsymbol{A}$, we can take the pivot *rows* of $\boldsymbol{R}$ as our basis.

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & 2 & 1 & 9 \\ 1 & 1 & 0 & 3 \end{bmatrix} \xrightarrow{\text{RREF}} \begin{bmatrix} 1 & 1 & 0 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \boldsymbol{R}$$

Note that the row space lives in $\mathbb{R}^4$ because the number of columns of $\boldsymbol{A}$ is 4. It also has a dimension of 2, the same as the rank.

$$\mathcal{C}(\boldsymbol{A}^{\mathsf{T}}) \subset \mathbb{R}^4; \quad \dim\left(\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})\right) = 2; \quad \text{Basis} = \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \\ 3 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 3 \end{bmatrix} \right\}$$

Notice how we have been careful to write the basis as a set of column vectors, although we are talking about the row space. Our vectors are always columns.

9.4 Left Null Space

To complete the picture and bring out the beautiful symmetry of the whole system, we will define one more null space: $\mathcal{N}(\boldsymbol{A}^{\mathsf{T}})$, which is also called the left null space. It lives on the same side as the column space, $\mathcal{C}(\boldsymbol{A})$ (meaning both are subspaces of $\mathbb{R}^m$), and no vector in the input space $\mathbb{R}^n$ (other than the zero vector) can reach this space via the transformation $\boldsymbol{A}$.

9.4.1 Significance of Left Null Space

In order to fully understand the significance of the Left Null Space, $\mathcal{N}(\boldsymbol{A}^{\mathsf{T}})$, we have to wait until we complete our discussions on linear regression in the next chapter. For now, we can think of its intuitive meaning through this lens:

Null Spaces: A Shortcut

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

CS Connection: Left Null Space and Model Errors

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9.4.2 Recipe to Compute Left Null Space

To compute the left null space, we will follow the same procedure as the null space to find the complete solution set of the homogeneous system of linear equations $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{x} = \boldsymbol{0}$. After all, the left null space is $\mathcal{N}(\boldsymbol{A}^{\mathsf{T}})$, which is the null space of $\boldsymbol{A}^{\mathsf{T}}$.

9.4.3 Worked Example

The steps to compute $\mathcal{N}(\boldsymbol{A}^{\mathsf{T}})$ are identical to the ones for $\mathcal{N}(\boldsymbol{A})$, except that we start with $\boldsymbol{A}^{\mathsf{T}}$ instead of $\boldsymbol{A}$, naturally. We will not go through them here, but, as promised earlier, we will share a shortcut for computing null spaces in the box on "**Null Spaces: A Shortcut**," and use the left-null-space computation of this $\boldsymbol{A}$ as an example.

9.5 Synthesis and Review

9.5.1 Summary of the Four Fundamental Spaces

We are now ready to summarize the four fundamental subspaces defined by a matrix. We will present it as a visual diagram first, discuss it in the text here, and then present it again in two tables. It is that important.

In Figure 9.1, we have a pictorial representation of what the four fundamental subspaces mean in terms of the mapping $\boldsymbol{A} : \mathbb{R}^n \mapsto \mathbb{R}^m$

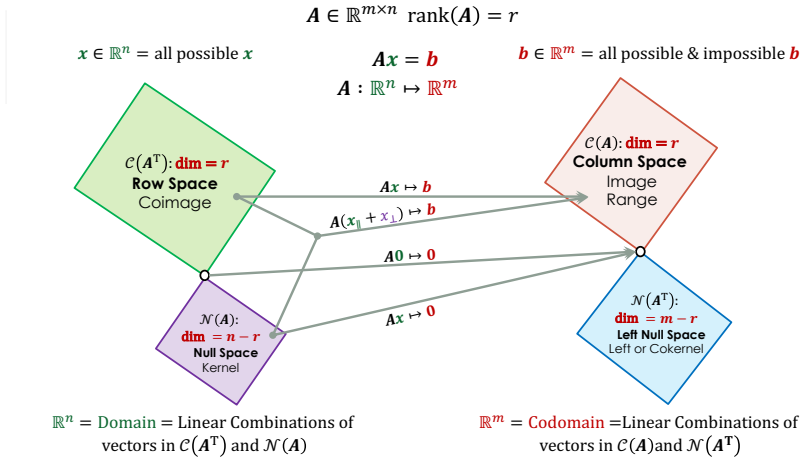


Fig. 9.1 A pictorial representation of the four fundamental subspaces defined by a matrix.

in the system $\boldsymbol{Ax} = \boldsymbol{b}$. They basically carve out different segments of the two vector spaces, $\mathbb{R}^n$ and $\mathbb{R}^m$, with specific properties.

The row space and null space are orthogonal complements. So are the column space and the left null space. All the vectors in the null space ($\mathcal{N}(\boldsymbol{A})$) map to the zero vector $\mathbf{0}$. All other vectors in the row space get mapped to the column space $\mathcal{C}(\boldsymbol{A})$. All other vectors are linear combinations of the vectors in $\mathcal{N}(\boldsymbol{A})$ and $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$. They also go to $\mathcal{C}(\boldsymbol{A})$, as we saw earlier. Since $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$ and $\mathcal{N}(\boldsymbol{A})$ are orthogonal complements, we have nothing left in $\mathbb{R}^n$ to go to the left null space $\mathcal{N}(\boldsymbol{A}^\mathbf{T})$, other than the zero vector $\mathbf{0} \in \mathbb{R}^n$.

As we saw, the geometric view of the fundamental spaces defined by a matrix is closely connected to its rank and the solvability of the system of linear equations it represents. It may be worth our time to look at the shapes of the matrix to see what its fundamental spaces look like. The four subspaces are closely related to the RREF of the matrix, and so is the solvability of the underlying system of linear equations. Keep in mind that while the REF (which is the result of Gaussian elimination) can have different shapes depending on the order in which the row operations are performed, the RREF (the result of Gauss-Jordan) is immutable: a matrix has a unique RREF.

We have presented, in Table 9.2, a comparison of the various possible shapes and ranks of the matrices and the corresponding

Table 9.1 Summary of the four fundamental spaces

Name, Space, Dim.	Basis	Description
Column Space $\mathcal{C}(\boldsymbol{A}) \subseteq \mathbb{R}^m$ $\dim(\mathcal{C}(\boldsymbol{A})) = r$	Pivot columns of $\boldsymbol{A}$	Range (Image) of $\boldsymbol{A} : \mathbb{R}^n \mapsto \mathbb{R}^m$ $\boldsymbol{b} \notin \mathcal{C}(\boldsymbol{A}) \implies$ no solution
Row Space $\mathcal{C}(\boldsymbol{A}^\mathsf{T}) \subseteq \mathbb{R}^n$ $\dim(\mathcal{C}(\boldsymbol{A}^\mathsf{T})) = r$	Pivot rows of $\boldsymbol{R}$ or $\boldsymbol{A}$	Coimage of $\boldsymbol{A} : \mathbb{R}^n \mapsto \mathbb{R}^m$ $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathsf{T}) \implies \boldsymbol{x} \mapsto \boldsymbol{b} \neq \boldsymbol{0}$
Null Space (AKA Kernel) $\mathcal{N}(\boldsymbol{A}) \subseteq \mathbb{R}^n$ $\dim(\mathcal{N}(\boldsymbol{A})) = n - r$	Solution set of $\boldsymbol{Ax} = \boldsymbol{0}$ or $\boldsymbol{Rx} = \boldsymbol{0}$	$\boldsymbol{x} \in \mathcal{N}(\boldsymbol{A}) \implies \boldsymbol{x} \mapsto \boldsymbol{0}$ If $\boldsymbol{Ax} = \boldsymbol{b}$ and $\boldsymbol{x}_\perp \in \mathcal{N}(\boldsymbol{A})$, $\boldsymbol{A}(\boldsymbol{x} + \boldsymbol{x}_\perp) = \boldsymbol{b}$
Left Null Space $\mathcal{N}(\boldsymbol{A}^\mathsf{T}) \subseteq \mathbb{R}^m$ $\dim(\mathcal{N}(\boldsymbol{A}^\mathsf{T})) = m - r$	Solution set of $\boldsymbol{A}^\mathsf{T}\boldsymbol{x} = \boldsymbol{0}$ or $\boldsymbol{R}'\boldsymbol{x} = \boldsymbol{0}$	Unreachable, AKA Cokernel $\boldsymbol{b} \in \mathcal{N}(\boldsymbol{A}^\mathsf{T}) \implies \boldsymbol{Ax} \neq \boldsymbol{b}$

The table shows how the four fundamental subspaces of a matrix are related to the linear equations it represents. $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, $\text{rank}(\boldsymbol{A}) = r$ in $\boldsymbol{Ax} = \boldsymbol{b}$ with $\boldsymbol{A} \xrightarrow{\text{RREF}} \boldsymbol{R}$ and $\boldsymbol{A}^\mathsf{T} \xrightarrow{\text{RREF}} \boldsymbol{R}'$

variations in their fundamental spaces. We should note the following points:

1. The column and row spaces always have the same dimension, and it is equal to the rank of the matrix. In fact, it is another definition for the rank.

$$\dim(\mathcal{C}(\boldsymbol{A})) = \dim(\mathcal{C}(\boldsymbol{A}^\mathsf{T})) = \text{rank}(\boldsymbol{A})$$

2. The dimension of the column space (which is the same as the rank) and the dimension of the null space (which is also called nullity) add up to the dimension of the domain. This is the famous rank-nullity theorem[2].

$$\text{For } \boldsymbol{A} \in \mathbb{R}^{m \times n}, \quad \dim(\mathcal{C}(\boldsymbol{A})) + \dim(\mathcal{N}(\boldsymbol{A})) = n$$

[2]Wikipedia describes it as: "The rank-nullity theorem is a theorem in linear algebra, which asserts that the dimension of the domain of a linear map is the sum of its rank (the dimension of its image) and its nullity (the dimension of its kernel)."

Table 9.2 The fundamental spaces of matrices of different shapes

Matrix Shape	RREF	Observations on the Four Spaces
Square, full-rank $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ $\text{rank}(\boldsymbol{A}) = n$	$\boldsymbol{I}_n$	$\mathcal{C}(\boldsymbol{A}), \mathcal{C}(\boldsymbol{A}^\mathsf{T}) = \mathbb{R}^n$ $\mathcal{N}(\boldsymbol{A}), \mathcal{N}(\boldsymbol{A}^\mathsf{T}) = \{\boldsymbol{0}\}$
"Tall," full-rank $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ $m > n$ $\text{rank}(\boldsymbol{A}) = n$	$\begin{bmatrix} \boldsymbol{I}_n \\ \hline \boldsymbol{0}_{(m-n)\times n} \end{bmatrix}$	$\mathcal{C}(\boldsymbol{A}) \subset \mathbb{R}^m, \dim(\mathcal{C}(\boldsymbol{A})) = n$ $\mathcal{C}(\boldsymbol{A}^\mathsf{T}) = \mathbb{R}^n$ $\mathcal{N}(\boldsymbol{A}) = \{\boldsymbol{0}\}$ $\mathcal{N}(\boldsymbol{A}^\mathsf{T}) \subset \mathbb{R}^m, \dim(\mathcal{N}(\boldsymbol{A}^\mathsf{T})) = m - n$
"Wide," full-rank $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ $m < n$ $\text{rank}(\boldsymbol{A}) = m$	$\begin{bmatrix} \boldsymbol{I}_m \bowtie \boldsymbol{F}_{m\times(n-m)} \end{bmatrix}$	$\mathcal{C}(\boldsymbol{A}) = \mathbb{R}^m$ $\mathcal{C}(\boldsymbol{A}^\mathsf{T}) \subset \mathbb{R}^n, \dim(\mathcal{C}(\boldsymbol{A}^\mathsf{T})) = m$ $\mathcal{N}(\boldsymbol{A}) \subset \mathbb{R}^n, \dim(\mathcal{N}(\boldsymbol{A})) = n - m$ $\mathcal{N}(\boldsymbol{A}^\mathsf{T}) = \{\boldsymbol{0}\}$
Rank-deficient $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ $\text{rank}(\boldsymbol{A}) = r <$ r_m $r_m = \min(m, n)$	$\begin{bmatrix} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r\times(n-r)} \\ \hline \boldsymbol{0}_{(m-r)\times n} \end{bmatrix}$	$\mathcal{C}(\boldsymbol{A}) \subset \mathbb{R}^m, \dim(\mathcal{C}(\boldsymbol{A})) = r$ $\mathcal{C}(\boldsymbol{A}^\mathsf{T}) \subset \mathbb{R}^n, \dim(\mathcal{C}(\boldsymbol{A}^\mathsf{T})) = r$ $\mathcal{N}(\boldsymbol{A}) \subset \mathbb{R}^n, \dim(\mathcal{N}(\boldsymbol{A})) = n - r$ $\mathcal{N}(\boldsymbol{A}^\mathsf{T}) \subset \mathbb{R}^m, \dim(\mathcal{N}(\boldsymbol{A}^\mathsf{T})) = m - r$

CS Connection: The Four Spaces and SVD

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9.5.2 Review: Complete Solution

From the examples worked out, we can see that the null-space computations and the complete solutions of the underlying system of linear equations have a lot in common. It is now time to put these two topics on an equal footing, which also gives us an opportunity to review the process of completely solving a system of linear equations and present it as a step-by-step algorithm.

The computation of the null space of a matrix is, in fact, the same as finding the *special* solutions of the underlying system of linear equations. When we found the *complete* solution earlier in §4.3.1 (page 83) and when we computed the null space above, the procedures may have looked ad hoc. Now that we know all there is to know about the fundamental spaces of a matrix, it is time to finally put to rest the tentativeness in the solving procedure and present it

like an algorithm with unambiguous steps. Let's start by defining the terms.

In $\boldsymbol{Ax} = \boldsymbol{b}$, the *complete* solution is the sum of the *particular* solution and the *special* solutions. We can see an example in Eqn (4.4), where the first vector is the particular solution and the second and third terms making a linear combination of two vectors are the special solutions. The linear combination is, in fact, the null space of $\boldsymbol{A}$. As we saw earlier, if $\boldsymbol{x}_{\parallel}$ is a solution to $\boldsymbol{Ax} = \boldsymbol{b}$, then $\boldsymbol{x}_{\parallel} + \boldsymbol{x}_{\perp}$ also is a solution for any $\boldsymbol{x}_{\perp} \in \mathcal{N}(\boldsymbol{A})$ because, as we established:

$$\boldsymbol{Ax} = \boldsymbol{A}(\boldsymbol{x}_{\parallel} + \boldsymbol{x}_{\perp}) = \boldsymbol{Ax}_{\parallel} + \boldsymbol{Ax}_{\perp} = \boldsymbol{Ax}_{\parallel} + \boldsymbol{0} = \boldsymbol{Ax}_{\parallel}$$

$\boldsymbol{x}_{\parallel}$ is a particular solution (which may be called $\boldsymbol{x}_p$), and the set of $\boldsymbol{x}_{\perp}$ (also known as $\boldsymbol{x}_s$) is the special solution.

Since the special solution, the set of $\boldsymbol{x}_{\perp}$, is a subspace, we can select any full set of linearly independent vectors as the basis to specify it. Let's work through an example to illustrate whatever we stated so far, before listing the algorithm for completely solving a system of linear equations.

An Example

We will reuse the example in Eqn (4.4) (on page 84), where we started with these equations:

$$\begin{aligned} x_1 + x_2 + x_3 + 2x_4 &= 6 \\ 2x_1 + 2x_2 + x_3 + 7x_4 &= 9 \end{aligned}$$

from which we got the augmented matrix:

$$\left[\boldsymbol{A} \mid \boldsymbol{b}\right] = \left[\begin{array}{cccc|c} 1 & 1 & 1 & 2 & 6 \\ 2 & 2 & 1 & 7 & 9 \end{array}\right] \xrightarrow{\text{REF}} \left[\begin{array}{cccc|c} 1 & 1 & 1 & 2 & 6 \\ 0 & 0 & -1 & 3 & -3 \end{array}\right]$$

and ended up with the complete solution:

$$\boldsymbol{x} = \begin{bmatrix} 3 \\ 0 \\ 3 \\ 0 \end{bmatrix} + t_1 \begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \end{bmatrix} + t_2 \begin{bmatrix} -5 \\ 0 \\ 3 \\ 1 \end{bmatrix} \tag{9.3}$$

The null space of the coefficient matrix has the basis vectors appearing as the linear combination in the last two terms above. $\mathcal{N}(\boldsymbol{A})$ is a plane

in $\mathbb{R}^4$, and we can use any two linearly independent vectors on it as its basis. The exact basis vectors we wind up with depend on the actual elimination steps we use, but they all specify the same plane of vectors, and indeed the same subspace. As we can see, in solving the system of equations above, we started with the row echelon form of the augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$. The REF (the output of Gaussian elimination) of a matrix is not unique; it is the RREF (from Gauss-Jordan elimination) that is unique.

Let's solve the system again. This time, we will start by finding the RREF (the output of Gauss-Jordan) because it is unique for any given matrix.

$$\left[\boldsymbol{A} \mid \boldsymbol{b}\right] = \left[\begin{array}{cccc|c} 1 & 1 & 1 & 2 & 6 \\ 2 & 2 & 1 & 7 & 9 \end{array}\right] \xrightarrow{\text{RREF}} \left[\begin{array}{cccc|c} 1 & 1 & 0 & 5 & 3 \\ 0 & 0 & 1 & -3 & 3 \end{array}\right]$$

In order to find a particular solution, we first set values of the free variables to zero, knowing that they are free to take any values. This step, in effect, ignores the free variables for the moment. In the example above, the free variables are x_2 and x_4, corresponding to the columns with no pivots. Ignoring these pivot-less columns, what we see is an augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right] = \left[\boldsymbol{I} \mid \boldsymbol{b}'\right]$, giving us the particular solution[3] as below:

$$\left[\begin{array}{cc|c} 1 & 0 & 3 \\ 0 & 1 & 3 \end{array}\right] \implies \begin{bmatrix} x_1 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} \implies \boldsymbol{x}_p = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 3 \\ 0 \end{bmatrix}$$

The special solutions are from the null space, and we have to solve the homogeneous system of equations $\boldsymbol{Ax} = \boldsymbol{0}$, for which the augmented matrix is $\left[\boldsymbol{A} \mid \boldsymbol{0}\right]$. What we do, in practice, is to ignore the $\boldsymbol{b}$ part of $\boldsymbol{Ax} = \boldsymbol{b}$, or set it to zero, and work with the coefficient part. We then cyclically set one free variable to one, and the rest to zero in the equations specified by the RREF of $\left[\boldsymbol{A} \mid \boldsymbol{0}\right]$:

$$\left[\begin{array}{cccc|c} 1 & 1 & 0 & 5 & 0 \\ 0 & 0 & 1 & -3 & 0 \end{array}\right] \implies \begin{aligned} x_1 + x_2 + 5x_4 &= 0 \\ x_3 - 3x_4 &= 0 \end{aligned}$$

[3]Note that the particular solution obtained using this prescription does not have to be in the row space of the coefficient matrix because we are taking zero values for the free variables.

Setting $x_2 = 1$ and $x_4 = 0$:

$$\begin{aligned} x_1 + x_2 + 5x_4 &= 0 \\ x_3 - 3x_4 &= 0 \end{aligned} \implies \begin{aligned} x_1 + 1 &= 0 \\ x_3 &= 0 \end{aligned}$$

$$\implies \boldsymbol{x}_{\perp,1} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Now, setting $x_2 = 0$ and $x_4 = 1$:

$$\begin{aligned} x_1 + x_2 + 5x_4 &= 0 \\ x_3 - 3x_4 &= 0 \end{aligned} \implies \begin{aligned} x_1 + 5 &= 0 \\ x_3 - 3 &= 0 \end{aligned}$$

$$\implies \boldsymbol{x}_{\perp,2} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -5 \\ 0 \\ 3 \\ 1 \end{bmatrix}$$

Putting it all together, we can write down the complete solution as:

$$\boldsymbol{x} = \begin{bmatrix} 3 \\ 0 \\ 3 \\ 0 \end{bmatrix} + t_1 \begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \end{bmatrix} + t_2 \begin{bmatrix} -5 \\ 0 \\ 3 \\ 1 \end{bmatrix} \tag{9.4}$$

Comparing Eqn (9.4) above to Eqn (9.3), we can see that we got identical solutions in our example. However, in principle, the vectors in the linear combination in the last two terms may differ. Our assertion is that what we may have in Eqn (9.4), if they were different from the ones in Eqn (9.3), would merely be another pair of linearly independent vectors in the same planar subspace specified by the latter.

How do we make sure, though? We know enough linear algebra to answer this question: if we were to place these four vectors in a matrix, we would have only two linearly independent columns, and the rank of the matrix would be two. How do we know the rank? We perform row reduction on it and count the number of pivots.

Squeezing out the last teachable moment from this review of completely solving linear systems, the rank is exactly equal to the number of non-zero rows in the RREF of the coefficient matrix.

The Algorithm

Here is a description of what we did in the example above, but using the most general case. We start with $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, where $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ and $\text{rank}(\boldsymbol{A}) = r \leq \min(m, n)$. We know the shape of the canonical form, RREF:

$$\boldsymbol{A} \xrightarrow{\text{RREF}} \boldsymbol{R} = \left[\begin{array}{c|c} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r\times(n-r)} & \boldsymbol{b}' \\ \hline \boldsymbol{0}_{(m-r)\times n} & \end{array}\right]$$

Note that we use the symbol $\bowtie$ to indicate that the columns of $\boldsymbol{I}_r$ and $\boldsymbol{F}_{r\times(n-r)}$ may be shuffled in; we may not have the columns of $\boldsymbol{F}$ neatly to the right of $\boldsymbol{I}$. With this picture in mind, let's describe the algorithm for completely solving the system of linear equations:

1. Find the RREF through Gauss-Jordan elimination on the augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{b}\right] \to \boldsymbol{R}$.
2. Ignore the free variables by setting them to zero, which is the same as deleting the pivot-less columns in $\boldsymbol{R}$ and the zero rows, giving us $\boldsymbol{I}_r$.
3. Get the particular solution, $\boldsymbol{x}_{\parallel}$, with the r values in $\boldsymbol{b}'$, and zeros for the $n - r$ free variables.
4. For each free variable, with the RREF of the homogeneous augmented matrix $\left[\boldsymbol{A} \mid \boldsymbol{0}\right]$ (which is the same $\boldsymbol{R}$ as above, but with $\boldsymbol{0}$ instead of $\boldsymbol{b}'$ in the augmenting column):
   - Set its value to one, and the values of all others to zero.
   - Solve the resulting equations to get one special solution, $\boldsymbol{x}_{\perp,i}$.
   - Iterate over all free variables.
5. Write down the complete solution:

$$\boldsymbol{x} = \boldsymbol{x}_{\parallel} + \sum_{i=1}^{n-r} t_i \boldsymbol{x}_{\perp,i}$$

6. Know that we have computed the null space as well:

$$\mathcal{N}(\boldsymbol{A}) \subset \mathbb{R}^n, \quad \dim\left(\mathcal{N}(\boldsymbol{A})\right) = n - r, \quad \text{Basis} = \{\boldsymbol{x}_{\perp,i}\}$$

CS Connection: Finding Subspaces in Code


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

9.5.3 Alternative Terminology

In order to see why these fundamental subspaces go by their aliases, we have to look at functions again. In normal algebra, when we have a function $f : \mathbb{R} \to \mathbb{R}$, the totality of the possible values of x (which would be all of $\mathbb{R}$ since x is the so-called independent variable) is called the *domain* and that of $y = f(x)$ is called the *codomain*. Not all of the codomain may be reached, though. For instance, if $f(x) = x^2$, the codomain is $\mathbb{R}$, but the part of the codomain that is reachable is $y \geq 0$. This part is called the *range*. Some people call the range the *image* (because it is what x is reflected into, if we were to guess). Not to be partial to y, they also call the part of the domain that is reflected the *coimage*.

Thinking of $\boldsymbol{A}$ as a mapping or function $\boldsymbol{A} : \mathbb{R}^n \to \mathbb{R}^m$, the domain is $\mathbb{R}^n$ and the codomain is $\mathbb{R}^m$. The column space may be called the range or image, and the row space is the coimage, although we will stay away from such abominations. Not to be outdone, the null spaces also go by names such as kernel (or right kernel) and left kernel (or cokernel). This naming is a bit more mysterious, but everything in the kernel gets mapped to $\mathbf{0}$, and nothing in the domain $\mathbb{R}^n$ ever gets mapped to the left kernel.

CS Connection: The Overloaded "Kernel"


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Regardless of what names they go by, the four fundamental subspaces bring together almost everything we have learned so far. They form the basis of the advanced topics in linear algebra and are therefore critical for furthering our understanding.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

10

Projection, Least Squares and Linear Regression

It was one thing to use computers as a tool, quite another to let them do your thinking for you.

—Tom Clancy



Now that we have learned the four fundamental subspaces defined by a matrix, we can look at one of their most widely used applications in machine learning: linear regression. For this, we will be expanding our notion of projecting one vector onto another. We will be projecting onto entire subspaces instead.

CS Connection: Regression as a Projection

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

10.1 Projection Revisited

In preparation for orthonormalization and the Gram-Schmidt process, we looked at projection earlier in §7.3.1 (page 149). Our approach was to fall back on the trigonometric definition of the dot product using the cosine of the angle between the vectors. While this geometric perspective is useful, we can derive the projection much more elegantly using purely linear algebraic tools.

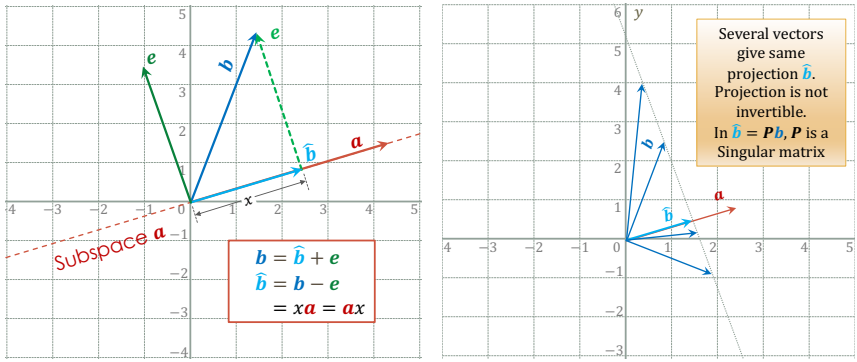


Fig. 10.1 Projection of one vector ($\boldsymbol{b}$ in blue) onto another ($\boldsymbol{a}$ in red). The projection $\hat{\boldsymbol{b}}$ is shown in light blue, and the error vector in green. The right panel shows that many different vectors (blue ones) can all have the same projection. The projection operation is many-to-one, and cannot be inverted. Interactive Visualization

In Figure 10.1, in the left panel, we have a blue vector $\boldsymbol{b}$ being projected onto the red $\boldsymbol{a}$. The projection is another vector, $\hat{\boldsymbol{b}}$, in a brighter shade of blue. Of all the vectors along $\boldsymbol{a}$, why is this the right vector to call the projection? One way to look at it is to imagine that we are shining some light from the top, perpendicular to $\boldsymbol{a}$, in the plane containing $\boldsymbol{a}$ and $\boldsymbol{b}$. The projection then is the shadow cast by the blue vector on the red one. Another way is to think of the difference vector, shown in green as $\boldsymbol{e}$, to be a minimum. The projection is that vector along the direction $\boldsymbol{a}$ which is closest to $\boldsymbol{b}$, where closeness is defined as the distance between the tips of $\boldsymbol{b}$ and its projection $\hat{\boldsymbol{b}}$, which is the same as the norm $\|e\|$. We can call this green vector the error (hence the name $\boldsymbol{e}$), and we are trying to minimize this error.

And when is $\boldsymbol{e}$ the smallest? It is when $\boldsymbol{a}$ and $\boldsymbol{e}$ are orthogonal to each other, or when $\boldsymbol{a} \perp \boldsymbol{e}$. We know the condition for orthogonality:

the dot product $\boldsymbol{a}^\mathsf{T}\boldsymbol{e} = 0$. Since $\hat{\boldsymbol{b}}$ lies along the direction of $\boldsymbol{a}$, we know that it has to be a scalar multiple: $\hat{\boldsymbol{b}} = \boldsymbol{a}\hat{x}$, where we called the scalar $\hat{x}$ (and placed it after $\boldsymbol{a}$ to set up our matrix multiplication). For each value of x, we will get a different candidate for the projection. The right candidate, the one that minimizes the error, is $\hat{\boldsymbol{b}}$, for which $x = \hat{x}$. Now the problem of computing the projection boils down to computing $\hat{x}$.

$$\begin{aligned} \boldsymbol{a}^\mathsf{T}\boldsymbol{e} &= \boldsymbol{a}^\mathsf{T}(\boldsymbol{b} - \hat{\boldsymbol{b}}) = \boldsymbol{a}^\mathsf{T}(\boldsymbol{b} - \boldsymbol{a}\hat{x}) = 0 \\ \boldsymbol{a}^\mathsf{T}\boldsymbol{a}\hat{x} = \boldsymbol{a}^\mathsf{T}\boldsymbol{b} \implies \hat{x} &= \frac{\boldsymbol{a}^\mathsf{T}\boldsymbol{b}}{\boldsymbol{a}^\mathsf{T}\boldsymbol{a}} \\ \hat{\boldsymbol{b}} = \boldsymbol{a}\hat{x} &= \boldsymbol{a}\frac{\boldsymbol{a}^\mathsf{T}\boldsymbol{b}}{\boldsymbol{a}^\mathsf{T}\boldsymbol{a}} = \boldsymbol{a}(\boldsymbol{a}^\mathsf{T}\boldsymbol{a})^{-1}\boldsymbol{a}^\mathsf{T}\boldsymbol{b} \\ \boldsymbol{P} &= \boldsymbol{a}(\boldsymbol{a}^\mathsf{T}\boldsymbol{a})^{-1}\boldsymbol{a}^\mathsf{T} \end{aligned} \tag{10.1}$$

Here, in addition to computing the projection value $\hat{x}$ and the projection vector $\hat{\boldsymbol{b}}$, we have also defined a projection matrix $\boldsymbol{P}$ which we can multiply with any vector to get its projection onto $\boldsymbol{a}$. It is indeed a matrix because $\boldsymbol{a}\boldsymbol{a}^\mathsf{T}$ is a column matrix ($n \times 1$) multiplying a row matrix ($1 \times n$), giving us an $n \times n$ matrix. The factor $(\boldsymbol{a}^\mathsf{T}\boldsymbol{a})^{-1}$ in between is just a scalar, which does not change the shape.

Comparing the derivation of the projection matrix in Eqn (10.1) to the one we did earlier in Eqn (7.9), we can appreciate that they are identical. It is just that the linear algebra way is much more elegant. In this derivation, we took certain facts to be self-evident, such as when two vectors are orthogonal, their dot product is zero. We can indeed prove it, as shown in the box on "**Orthogonality and Dot Product**."

What we just did was project a vector $\boldsymbol{b}$ onto another vector $\boldsymbol{a}$, which means we found the vector closest to $\boldsymbol{b}$ in the one-dimensional subspace spanned by $\boldsymbol{a}$. Before we expand this notion to projecting onto larger subspaces, we should establish exactly why we need to project onto subspaces in the first place.

The motivation, as with most things in linear algebra, is connected to the solvability of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$. Let's take the situation where we have too many equations, which means our coefficient matrix $\boldsymbol{A}$ is "tall." We are particularly interested in such matrices because our datasets in computer science tend to be of that shape.

CS Connection: The Shape of Data

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

In general, "tall" data matrices with real data tend to be full rank because data points, which make up the rows of such matrices, tend to be linearly independent due to measurement errors and statistical fluctuations. The RREF of a "tall" full-rank matrix is $\boldsymbol{I}$ near the top, and zero rows below it. Such matrices, as coefficient matrices in $\boldsymbol{Ax} = \boldsymbol{b}$, have solutions only if the RHS, $\boldsymbol{b}$, is in the column space of $\boldsymbol{A}$, which in general, it will not be.

What do we do in such a situation? We still want to solve the system even when $\boldsymbol{b} \notin \mathcal{C}(\boldsymbol{A})$. In this case, the best we can do is to find the vector $\hat{\boldsymbol{b}}$ in the column space that is closest to $\boldsymbol{b}$, which will be its projection onto $\mathcal{C}(\boldsymbol{A})$.

10.2 Projection to Subspace

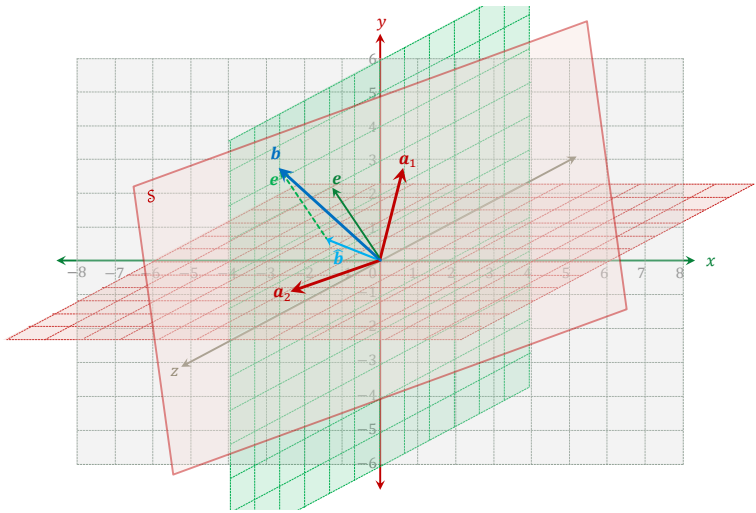


Fig. 10.2 Projecting the blue $\boldsymbol{b}$ onto the subspace that is the span of $\boldsymbol{a}_1$ and $\boldsymbol{a}_2$, the two red vectors. The subspace is a plane, shown in light red. The projection, $\hat{\boldsymbol{b}}$ (in bright blue) of $\boldsymbol{b}$ is in the subspace, and is therefore a linear combination of its basis vectors, $\boldsymbol{a}_1$ and $\boldsymbol{a}_2$, shown in red.

In order to project to a subspace, we need to specify what the subspace is, which we do in linear algebra by specifying the basis. If our basis vectors are $\boldsymbol{a}_1$ and $\boldsymbol{a}_2$, the subspace onto which we are projecting is a plane, as shown in Figure 10.2. Exactly as we did in Eqn (10.1), we want the formulas for $\hat{x}$ (the length of the projection), $\hat{\boldsymbol{b}}$ (the projected vector), and $\boldsymbol{P}$ (the projection matrix). In the case of a plane with two basis vectors, we will have two lengths, $\hat{x}_1$ and $\hat{x}_2$.

Since our projected vector $\hat{\boldsymbol{b}}$ is on the plane spanned by the basis vectors, it is a linear combination of them, and the coefficients of the linear combination are what we call $\hat{x}_1$ and $\hat{x}_2$. Referring to Figure 10.2, where we are projecting the blue $\boldsymbol{b}$ onto the red plane which is the span of $\boldsymbol{a}_1$ and $\boldsymbol{a}_2$, we have the individual components along each basis vector, $\hat{\boldsymbol{b}}_i = \hat{x}_i \boldsymbol{a}_i$. Arranging the basis vectors as the columns of a matrix, we can write our linear combination for $\hat{\boldsymbol{b}}$ as:

$$\boldsymbol{A} = \begin{bmatrix} | & | \\ \boldsymbol{a}_1 & \boldsymbol{a}_2 \\ | & | \end{bmatrix}, \quad \hat{\boldsymbol{b}} = \boldsymbol{A} \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \end{bmatrix} = \boldsymbol{A}\hat{\boldsymbol{x}}$$

which recovers the form of our favorite equation, $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$. All we need to do is to find $\hat{\boldsymbol{x}}$, for which we use the fact that the green error vector $\boldsymbol{e} = \boldsymbol{b} - \hat{\boldsymbol{b}}$ is orthogonal to the red plane in Figure 10.2. When a vector is perpendicular to a subspace, it is perpendicular to every vector in it. In particular, $\boldsymbol{e}$ is orthogonal to every basis vector $\boldsymbol{a}_i$, which means we have $\boldsymbol{a}_i^{\mathsf{T}}\boldsymbol{e} = 0$. We can again write these two equations (for $i = 1$ and 2) as a matrix equation: $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{e} = \boldsymbol{0}$.

$$\begin{aligned} \boldsymbol{A}^{\mathsf{T}}\boldsymbol{e} &= \boldsymbol{0} \\ \boldsymbol{A}^{\mathsf{T}}(\boldsymbol{b} - \hat{\boldsymbol{b}}) &= \boldsymbol{0} \\ \boldsymbol{A}^{\mathsf{T}}\boldsymbol{b} &= \boldsymbol{A}^{\mathsf{T}}\hat{\boldsymbol{b}} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\hat{\boldsymbol{x}} \end{aligned}$$

Recasting the last part of the previous equation in the form $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, we write our equation which will solve all our problems from now on:

$$\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{b} \implies \hat{\boldsymbol{x}} = (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{b} \tag{10.2}$$

What this equation tells us is that even if $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ does not have a solution (because $\boldsymbol{A}$ is full-column-rank with an inconsistent right-hand side), multiplying both sides of it by $\boldsymbol{A}^{\mathsf{T}}$ elegantly maps the problem into a solvable space, at least in an approximate sense.

The Hat Matrix


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

This approximation that gives us the best possible solution $\hat{\boldsymbol{x}}$ is, in fact, identical to the least squares minimization of the sum of squared errors, if we were to do it using standard calculus techniques. A numerical, iterative minimization of the calculus kind may or may not converge to the global minimum, and may have computational complexity issues. In the elegant world of linear algebra, it is only a couple of matrix multiplications and an inversion.

Using our recently acquired knowledge of the four fundamental spaces of $\boldsymbol{A}$, we can immediately see that $\boldsymbol{e} \in \mathcal{N}(\boldsymbol{A}^\mathsf{T})$. And, $\hat{\boldsymbol{b}} \in \mathcal{C}(\boldsymbol{A})$ because it is a linear combination of the columns of $\boldsymbol{A}$. $\boldsymbol{A}$ is guaranteed to be full-rank because its columns span a subspace, and $\boldsymbol{A}\hat{\boldsymbol{x}} = \hat{\boldsymbol{b}}$ is guaranteed to have a unique solution.

The last thing we need to do in this section is to write down the projection matrix $\boldsymbol{P}$ such that $\boldsymbol{P}\boldsymbol{b} = \hat{\boldsymbol{b}}$. Knowing that $\boldsymbol{A}\hat{\boldsymbol{x}} = \hat{\boldsymbol{b}}$ and using the formula for $\hat{\boldsymbol{x}}$ from Eqn (10.2), the projection matrix is:

$$\boldsymbol{P} = \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} \tag{10.3}$$

10.2.1 Properties of the Projection Matrix

The first property of the projection matrix is that applying it multiple times to a vector is the same as applying it once. It makes sense from the meaning of projection—once we project a vector onto a subspace, the projection is already in that subspace. Projecting a vector that is already in a subspace onto that same subspace does not do anything. This property is called idempotence, and the projection

matrix is idempotent, which we can easily verify:

$$\begin{aligned} \boldsymbol{P} &= \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} \\ \implies \boldsymbol{P}^2 &= \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} \\ &= \boldsymbol{A}\left((\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\right)(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} \\ &= \boldsymbol{A}\boldsymbol{I}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} \\ &= \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} \\ &= \boldsymbol{P} \end{aligned}$$

Another property is that $\boldsymbol{P}$ must be symmetric. We can see this geometrically. For any two arbitrary vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ in $\mathbb{R}^m$, projecting them onto the subspace $\mathcal{C}(\boldsymbol{A})$ yields $\hat{\boldsymbol{x}} = \boldsymbol{P}\boldsymbol{x}$ and $\hat{\boldsymbol{y}} = \boldsymbol{P}\boldsymbol{y}$. We can decompose both vectors into their subspace component and their orthogonal error component: $\boldsymbol{x} = \hat{\boldsymbol{x}} + \boldsymbol{e}_x$ and $\boldsymbol{y} = \hat{\boldsymbol{y}} + \boldsymbol{e}_y$.

Let's take the dot product of $\boldsymbol{x}$ with the projection of $\boldsymbol{y}$:

$$\boldsymbol{x}^\mathsf{T}(\boldsymbol{P}\boldsymbol{y}) = \boldsymbol{x}^\mathsf{T}\hat{\boldsymbol{y}} = (\hat{\boldsymbol{x}} + \boldsymbol{e}_x)^\mathsf{T}\hat{\boldsymbol{y}} = \hat{\boldsymbol{x}}^\mathsf{T}\hat{\boldsymbol{y}} + \boldsymbol{e}_x^\mathsf{T}\hat{\boldsymbol{y}}$$

Because $\boldsymbol{e}_x$ is in the left null space $\mathcal{N}(\boldsymbol{A}^\mathsf{T})$ and $\hat{\boldsymbol{y}}$ is in the column space $\mathcal{C}(\boldsymbol{A})$, they are strictly orthogonal. Their dot product is zero, leaving:

$$\boldsymbol{x}^\mathsf{T}\boldsymbol{P}\boldsymbol{y} = \hat{\boldsymbol{x}}^\mathsf{T}\hat{\boldsymbol{y}}$$

Now let's do the reverse—the dot product of the projection of $\boldsymbol{x}$ with $\boldsymbol{y}$:

$$(\boldsymbol{P}\boldsymbol{x})^\mathsf{T}\boldsymbol{y} = \hat{\boldsymbol{x}}^\mathsf{T}(\hat{\boldsymbol{y}} + \boldsymbol{e}_y) = \hat{\boldsymbol{x}}^\mathsf{T}\hat{\boldsymbol{y}} + \hat{\boldsymbol{x}}^\mathsf{T}\boldsymbol{e}_y$$

Again, since $\hat{\boldsymbol{x}} \in \mathcal{C}(\boldsymbol{A})$ and $\boldsymbol{e}_y \in \mathcal{N}(\boldsymbol{A}^\mathsf{T})$, they are orthogonal, leaving:

$$(\boldsymbol{P}\boldsymbol{x})^\mathsf{T}\boldsymbol{y} = \hat{\boldsymbol{x}}^\mathsf{T}\hat{\boldsymbol{y}}$$

Since both expressions evaluate to the exact same scalar $\hat{\boldsymbol{x}}^\mathsf{T}\hat{\boldsymbol{y}}$, we can equate them:

$$\boldsymbol{x}^\mathsf{T}\boldsymbol{P}\boldsymbol{y} = (\boldsymbol{P}\boldsymbol{x})^\mathsf{T}\boldsymbol{y} = \boldsymbol{x}^\mathsf{T}\boldsymbol{P}^\mathsf{T}\boldsymbol{y}$$

Because this equality holds for *any* arbitrary vectors $\boldsymbol{x}$ and $\boldsymbol{y}$, the matrices mapping them must be identical:

$$\boldsymbol{P} = \boldsymbol{P}^\mathsf{T}$$

Let's verify if our general projection matrix $\boldsymbol{P}$ from Eqn (10.3) is indeed symmetric, as it should be:

$$\begin{aligned} \boldsymbol{P}^{\mathsf{T}} &= \left(\boldsymbol{A}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}}\right)^{\mathsf{T}} \\ (1) \qquad &= \left(\boldsymbol{A}^{\mathsf{T}}\right)^{\mathsf{T}}\left((\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\right)^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}} \\ (2) \qquad &= \boldsymbol{A}\left((\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{\mathsf{T}}\right)^{-1}\boldsymbol{A}^{\mathsf{T}} \\ (3) \qquad &= \boldsymbol{A}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}} \\ &= \boldsymbol{P} \end{aligned} \tag{10.4}$$

where we used (1) the product rule of transposes, (2) the fact that the inverse of a transpose is the transpose of the inverse, and (3) the symmetry of the Gram matrix $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$.

10.2.2 Meaning of Projection

The projection operation is a many-to-one mapping, as shown in the right panel of Figure 10.1. Since many vectors like $\boldsymbol{b}$ can be projected onto $\boldsymbol{a}$ to yield the same $\hat{\boldsymbol{b}}$, given $\hat{\boldsymbol{b}}$, we simply cannot figure out which $\boldsymbol{b}$ it came from. Such mappings are *not* injective (i.e., they are not one-to-one), and therefore they cannot be inverted. In other words, the projection operation throws away some information.

CS Connection: Projection as Data Compression


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

In the world of linear algebra, the matrix corresponding to such operations would be a rank-deficient one. When projecting to a subspace defined by one vector, $\boldsymbol{P}$ is a rank-one matrix. For a subspace of dimension two in $\mathbb{R}^n$, it is a rank-two matrix. In general, for any true subspace, $\boldsymbol{P}$ is singular, which is the reason why we cannot apply the product rule for inverses to $(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}$.

$$\boldsymbol{P} = \boldsymbol{A}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}} \neq \boldsymbol{A}\boldsymbol{A}^{-1}(\boldsymbol{A}^{\mathsf{T}})^{-1}\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{I}\boldsymbol{I} = \boldsymbol{I}$$

We should not do this expansion, and the product rule does not apply here because $\boldsymbol{A}^{-1}$ is not defined. After all, the fact that $\boldsymbol{A}$ is not

invertible was the whole point of embarking on this projection trip to begin with.

What happens if we take the full space and try to project onto it? In other words, we try to project a vector onto a "subspace" of dimension n in $\mathbb{R}^n$. In this case, we get a full-rank projection matrix, and the expansion of the inverse above is indeed valid. The projection matrix really is $\boldsymbol{I}$, which is an invertible matrix because every vector gets "projected" directly onto itself. Note that the two properties we were looking for in $\boldsymbol{P}$ are satisfied by $\boldsymbol{I}$: it is idempotent because $\boldsymbol{I}^2 = \boldsymbol{I}$, and of course, $\boldsymbol{I}^\mathsf{T} = \boldsymbol{I}$.

For rank-deficient matrices, $\boldsymbol{P}$ is $\boldsymbol{A}\boldsymbol{A}^{-1}_{\textbf{Left}}$, the left inverse multiplying on the right, almost like an attempt to get as close to $\boldsymbol{I}$ as possible.

10.3 Linear Regression

One of the most interesting applications of the idea of projection is linear regression, where we try to model our data using linear functions. Before going to real data with multiple variables and several observations, let's start with a toy example.

10.3.1 Simple Linear Regression

When we have only one variable y depending linearly on another variable x, we get standard line fitting. This is called simple linear regression, as opposed to multiple linear regression when we have multiple independent variables x_i.

Figure 10.3 shows a data table with just two variables and five data points. We can think of y as a dependent variable, and x as the independent one on which y depends. The dependence is modeled as a line, $y = mx + c$, which is the simple linear regression model. With the data shown in Figure 10.3, we can easily type the numbers into our popular spreadsheet applications and get a "trendline" that gives us the values of $m = 1.2$ and $c = -0.5$. Let's look at how these data points become a system of linear equations, and how their solution then becomes a projection problem.

x	y
1	1
2	2
3	2
4	5

$y = 1.2x - 0.5$
$R^2 = 0.8$

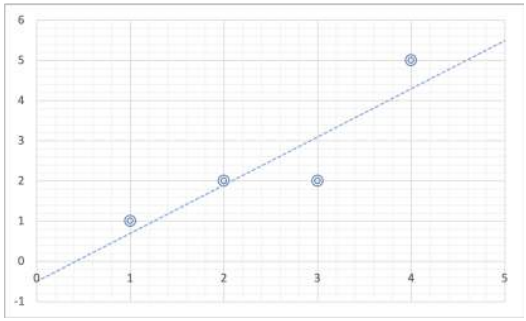


Fig. 10.3 An example of simple linear regression. The data points in the table on the left are plotted in the chart on the right, and a "trendline" is estimated and drawn.

$$
\begin{aligned}
y &= mx + c \\
1 &= m + c \\
2 &= 2m + c \\
2 &= 3m + c \\
5 &= 4m + c
\end{aligned}
\qquad \boldsymbol{Ax} = \boldsymbol{b} \qquad \boldsymbol{A} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \\ 4 & 1 \end{bmatrix} \qquad \boldsymbol{x} = \begin{bmatrix} m \\ c \end{bmatrix} \qquad \boldsymbol{b} = \begin{bmatrix} 1 \\ 2 \\ 2 \\ 5 \end{bmatrix}
$$

Since we are modeling our data as $y = mx + c$ and we have four (x, y) pairs, we get four equations as shown above, which we massage into the $\boldsymbol{Ax} = \boldsymbol{b}$ form. Notice how $\boldsymbol{A}$ has two columns: the first for m, and another one for c that is full of ones. If we had written our model as $y = c + mx$, the column for the intercept c would have been the first one.

As we can see, we have four equations, and if we were to do Gauss-Jordan elimination on $\left[\boldsymbol{A} \mid \boldsymbol{b}\right]$, the third row would indicate an inconsistent equation $0 = 1$. The system is not solvable, which is fine by us at this point in our linear algebra journey. We will compute the best possible solution $\hat{\boldsymbol{x}}$, which will give us our best estimates for

the slope and the intercept as $\hat{m}$ and $\hat{c}$. The steps are shown below.

$$\boldsymbol{A}^{\mathsf{T}} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} = \begin{bmatrix} 30 & 10 \\ 10 & 4 \end{bmatrix} \quad \left|\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\right| = 20$$

$$(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1} = \begin{bmatrix} \frac{1}{5} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{3}{2} \end{bmatrix}$$

$$(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}} = \begin{bmatrix} -\frac{3}{10} & -\frac{1}{10} & \frac{1}{10} & \frac{3}{10} \\ 1 & \frac{1}{2} & 0 & -\frac{1}{2} \end{bmatrix}$$

$$\hat{\boldsymbol{x}} = (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{b} = \begin{bmatrix} \frac{6}{5} \\ -\frac{1}{2} \end{bmatrix} \implies \hat{m} = \frac{6}{5} \text{ and } \hat{c} = -\frac{1}{2}$$

As we can see, our linear regression model becomes $y = \hat{m}x + \hat{c} = 1.2x - 0.5$, which is exactly the same trendline that the spreadsheet application computed in Figure 10.3.

We also have the error vector $\boldsymbol{e} = \boldsymbol{b} - \hat{\boldsymbol{b}}$. $\boldsymbol{b}$ is what we project onto the column space $\mathcal{C}(\boldsymbol{A})$, and $\hat{\boldsymbol{b}} = \boldsymbol{P}\boldsymbol{b}$ is the projection. Let's go ahead and compute $\boldsymbol{P}$ and $\hat{\boldsymbol{b}}$ as well. Using the formula for the projection matrix from Eqn (10.3), we get:

$$\boldsymbol{P} = \boldsymbol{A}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathsf{T}} = \begin{bmatrix} \frac{7}{10} & \frac{2}{5} & \frac{1}{10} & -\frac{1}{5} \\ \frac{2}{5} & \frac{3}{10} & \frac{1}{5} & \frac{1}{10} \\ \frac{1}{10} & \frac{1}{5} & \frac{3}{10} & \frac{2}{5} \\ -\frac{1}{5} & \frac{1}{10} & \frac{2}{5} & \frac{7}{10} \end{bmatrix}$$

$$\hat{\boldsymbol{b}} = \boldsymbol{P}\boldsymbol{b} = \begin{bmatrix} \frac{7}{10} \\ \frac{19}{10} \\ \frac{31}{10} \\ \frac{43}{10} \end{bmatrix} \quad \boldsymbol{e} = \boldsymbol{b} - \hat{\boldsymbol{b}} = \begin{bmatrix} \frac{3}{10} \\ \frac{1}{10} \\ -\frac{11}{10} \\ \frac{7}{10} \end{bmatrix} \quad \|\boldsymbol{e}\|^2 = \frac{9}{5}$$

The square of the norm of the error vector ($\|\boldsymbol{e}\|^2$) represents the unscaled variance in the data that is unexplained by the model, also known as the Sum of Squared Errors (SSE). The total variance is computed independently using its standard formula from statistics,

$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(y_i - \bar{y})^2 = 2.25$. The numerator of this variance calculation is called the Total Sum of Squares (SST). The coefficient of determination, R^2, is the fraction of the variance in the data that is explained by our model, and it is 0.8, just as the trendline from the spreadsheet application reports in Figure 10.3.

10.3.2 Multiple Linear Regression

When we have one dependent variable and one independent variable, it is simple linear regression (SLR). The statistical model in SLR is a line, which means we are saying that the data points all should have been on a line, but for some reason, they wander and fluctuate; therefore, let's look for the best fitting line.

When we have multiple independent variables, we have multiple linear regression (MLR). For two independent variables, the model is a plane, and we are trying to find the best plane that fits the data. For n independent variables, the model is an n-dimensional subspace in the *coordinate* space (*not* a *vector* subspace) that best describes the data.

Height (cm)	Weight (kg)	Hair Len. (cm)	Age (years)	Sex (M/F)
169.0	60.0	10.0	21.2	M
174.0	73.0	12.0	26.8	M
163.0	58.0	22.0	21.0	F
163.0	55.0	18.0	20.0	F
174.0	67.0	15.0	22.3	M
171.0	69.0	3.0	23.9	M
162.5	50.0	40.0	22.5	F
179.5	74.0	7.0	25.7	M
180.0	76.0	5.0	24.2	M
170.0	65.0	8.0	24.6	M
176.0	74.0	7.0	23.1	M
165.0	46.0	20.0	22.6	F
161.0	50.0	23.0	22.2	F
145.0	42.0	40.0	23.1	F
163.0	46.0	27.0	23.9	F
160.0	50.0	25.0	23.0	F
160.0	53.0	27.0	21.0	F
160.0	47.0	30.0	23.0	F
160.0	51.0	30.0	21.0	F
156.0	49.0	25.0	20.5	F
160.0	56.0	28.0	21.6	F
158.0	60.0	40.0	22.4	F
182.0	75.0	5.0	25.3	M
179.0	62.0	8.0	22.2	M
165.0	59.0	5.1	23.1	M
175.0	59.0	12.0	23.1	M
⋮	⋮	⋮	⋮	⋮

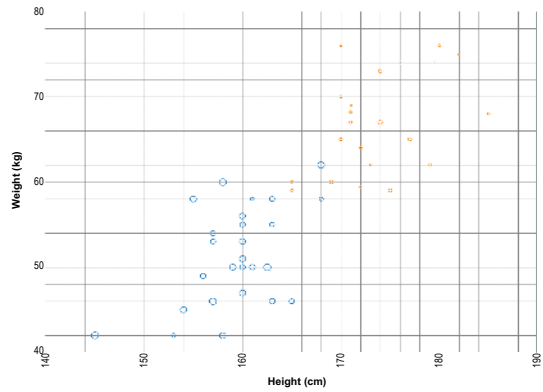


Fig. 10.4 Example of a data matrix and its visualization in multiple linear regression.

In Figure 10.4, we have the first 26 rows of a dataset of 127 observations of weight, height, length of hair, age, and sex, arranged as a matrix. This Young Adult dataset was used for an unrelated research project. The visualization shows Height on the x-axis and

Weight on the y-axis. The size of the point encodes Hair Len., which we can think of as the z-axis coming towards us. The bigger bubbles indicate larger Hair Len. because they are closer to us. Finally, the color indicates the last column, Sex: blue for F and orange for M.

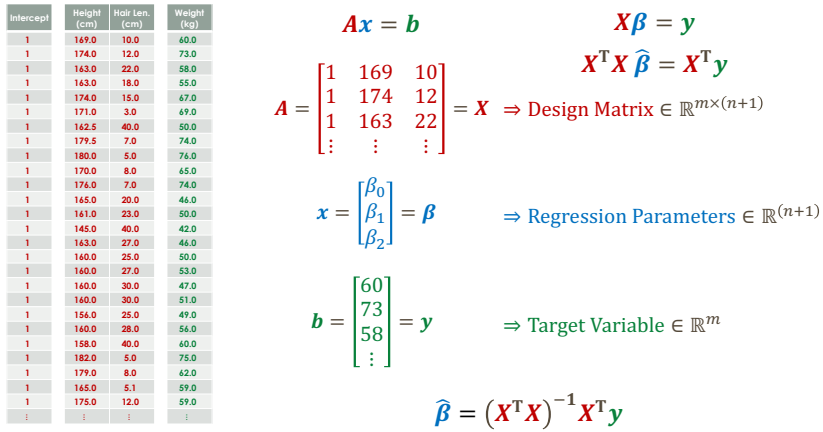


Fig. 10.5 The standard notations used in multiple linear regression. Weight is the dependent (or target or output) variable. Height and Hair Len. are the independent (or predictor or input) variables.

Treating this as a teachable moment, we are going to switch from our standard $\boldsymbol{Ax} = \boldsymbol{b}$ notation to what is generally used in the literature in the context of MLR, as shown in Figure 10.5, color-coded for clarity. Notice that in the case of SLR, we had the model $y = mx + c$, with the intercept introducing a column of ones in our $\boldsymbol{A}$ matrix. In MLR, we are going to keep our intercept as the first parameter, and the column of ones will be the first in our matrix, which we will now call $\boldsymbol{X}$, with the fancy name Design Matrix. Our model is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \quad \text{or} \quad \text{Weight} = \beta_0 + \beta_1 \text{Height} + \beta_2 \text{Hair Len.}$$

Following the same matrix equations, now with the new notations as in Figure 10.5, we get the best estimate for the parameter vector $\hat{\boldsymbol{\beta}}$, so that our model (coming from the 26 data points shown in Figure 10.4 and Figure 10.5) becomes:

$$\begin{aligned} \text{Weight} &= \hat{\beta}_0 + \hat{\beta}_1 \text{Height} + \hat{\beta}_2 \text{Hair Len.} \\ &= -74.06 + 0.814 \text{Height} - 0.151 \text{Hair Len.} \end{aligned}$$

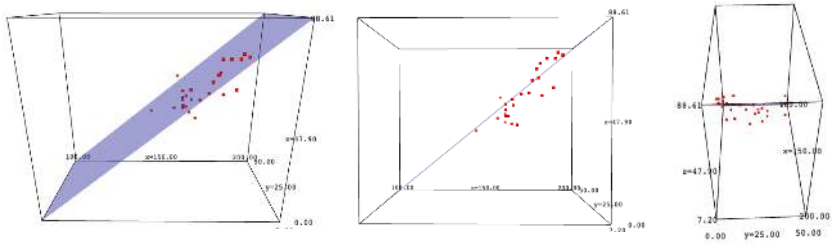


Fig. 10.6 Attempt to visualize a three-dimensional MLR model for Weight. All three panels show the model (which is a plane) and the associated data points, but from different perspectives. The middle one shows the dependency of Weight on Height, and the last one shows that on Hair Len.

Although it is not easy to visualize the model and the points, even in the simple intuitive three-dimensional space, we have attempted to show this model in Figure 10.6. What is perhaps more important is to understand the model in terms of its parameters. We can see that $\hat{\beta}_1 = 0.814$, a positive number indicating that weight increases as height increases. On the other hand, $\hat{\beta}_2 = -0.151$, which tells us that weight decreases as hair length increases. This is consistent with the fact that women tend to have longer hair, and they tend to be smaller and lighter.

As in the case of SLR, we could compute R^2 and analyze whether it is appropriate to have the hair length in the model and so on, which is what we might expect to see in a book on data analytics. This book, however, is written for linear algebra, and this is probably an appropriate point to stop our work on its geometric aspects.

10.4 Summary: Geometric View

This chapter on Projection and Linear Regression brings us to the end of the geometric view of linear algebra. Before we move forward, let's take a moment to compare the algebraic and geometric views we have developed so far.

When we think of a matrix augmented by a vector as a system of linear equations, each row represents an equation. We perform Gaussian elimination, which turns equation manipulation into a deterministic, programmable algorithm. We determine solvability by looking for rows that equate zero to a non-zero constant, which is

a mathematical impossibility. If the system is solvable, we proceed with back substitution and handle any free variables. If it is not solvable, the algebraic view does not give us an immediate recipe for how to proceed. This is what we call the algebraic view.

In the geometric view, we think of the matrix as a collection of column vectors. Solving a system of linear equations in this view is equivalent to asking if the target vector $\boldsymbol{b}$ can be expressed as a linear combination of these columns. The set of all such valid linear combinations is the column space $\mathcal{C}(\boldsymbol{A})$. If $\boldsymbol{b}$ is a member of this space, we can solve the system. If $\boldsymbol{b} \notin \mathcal{C}(\boldsymbol{A})$, the system of equations has no exact solution.

However, our geometric intuition provides a way forward. For the types of systems we are most interested in¯namely tall, full-column-rank matrices¯we saw that any target vector $\boldsymbol{b}$ can be split into a projection vector that lies strictly within $\mathcal{C}(\boldsymbol{A})$, and an orthogonal error vector that belongs to the left null space $\mathcal{N}(\boldsymbol{A}^{\mathsf{T}})$. The recipe for handling an unsolvable system is to swap our unreachable target $\boldsymbol{b}$ for its orthogonal projection in the column space. This guarantees a unique solution, with the unavoidable error elegantly quantified by the norm of the error vector. As we just saw, this fundamental geometric insight forms the exact basis of linear regression, our first machine learning algorithm.

To build more sophisticated algorithms, we must first delve into advanced topics in linear algebra. These new concepts will form the chapters of the next part of this book.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Part IV

Advanced Topics

11

Eigenvalue Decomposition and Diagonalization

Would it save you a lot of time if I just gave up and went mad now?

—Douglas Adams



We have completed the basics of linear algebra. We have perhaps even gone a bit beyond the basics in its algebraic and geometric aspects. Now it is time to switch gears and look at some topics that have an enormous impact in computer science as well as the more classical sciences. Eigenvalues and eigenvectors are the entry point to such topics. Although they appear in the "Advanced Topics" part of this book, eigenvalue decomposition and its associated discussions usually appear toward the end of all undergraduate-level courses on linear algebra.

CS Connection: Why "Eigen" Matters

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

The word "eigen" is German, and it translates to *own* or *characteristic*. We may, therefore, occasionally see textbooks calling eigenvectors *characteristic vectors*, although it is not very common. What is much more common is to call the algebraic expression that gives us the eigenvalues of a matrix its *characteristic polynomial*.

11.1 Fundamentals and Computation

11.1.1 Definitions and Examples

We previously talked about $\boldsymbol{Ax} = \boldsymbol{b}$ as a transformation $\boldsymbol{A} \in \mathbb{R}^{m \times n} : \mathbb{R}^n \rightarrow \mathbb{R}^m$. If we consider square matrices $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, then the mapping is strictly from $\mathbb{R}^n$ to $\mathbb{R}^n$. We can then say that the matrix $\boldsymbol{A}$ is a mapping from a space to itself: it takes vectors in one space and transforms them into other vectors within the *same* space.

Are there vectors in the space that get transformed into a scalar multiple of themselves? If there are, such vectors are called eigenvectors. Writing this statement in symbols, we come up with their definition.

Eigenvectors and eigenvalues

Definition: For $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, a non-zero vector $\boldsymbol{s}$ is an eigenvector if $\boldsymbol{As} = \lambda \boldsymbol{s}$ for some scalar eigenvalue λ.

A few points to note about eigenvalues and eigenvectors:

1. For $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, its eigenvalues do not have to be in $\mathbb{R}$. In other words, just because we have a square matrix over the field of reals ($\boldsymbol{A} \in \mathbb{R}^{n \times n}$), we cannot assume that its eigenvalues are real; it may have complex or imaginary eigenvalues.

2. Similarly, not all real matrices ($\boldsymbol{A} \in \mathbb{R}^{n \times n}$) have real eigenvectors.

3. If $\boldsymbol{s}$ is an eigenvector of $\boldsymbol{A}$ with an eigenvalue λ, so is any scaled version of it ($r\boldsymbol{s}$), and it will have the *same* eigenvalue λ. Proof:
$$\boldsymbol{A}(r\boldsymbol{s}) = r\boldsymbol{As} = r\lambda \boldsymbol{s} = \lambda(r\boldsymbol{s})$$

4. An $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ matrix can have at most n distinct eigenvalues. Correspondingly, it can have at most n linearly independent eigenvectors. While there are infinitely many *scaled* eigenvectors, they represent at most n fundamentally distinct directions in the vector space.

5. Eigenvalue λ can be zero. When $\lambda = 0$, we have $\boldsymbol{A}\boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} \in \mathcal{N}(\boldsymbol{A})$. Note that the eigenvector itself cannot be the zero vector by definition ($\boldsymbol{s} \neq \boldsymbol{0}$).

6. Although presented here in terms of matrices and vectors, the ideas behind eigenvectors came from other fields, such as physics. For this reason, eigenvectors are often conceptually defined as those vectors that do not change their "direction" when $\boldsymbol{A}$ is applied to them.

CS Connection: Markov Chains and Steady States


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

In this book, however, for our own pedantic reasons, we stay away from the geometric notion of the "direction" of vectors to the extent possible and stick to the algebraic definitions.

Before writing down a general method for computing eigenvalues and eigenvectors, let's look at a few intuitive examples to see how they behave in practice.

Permutation Matrix

A permutation matrix is one that shuffles the elements of a vector (or the rows of a matrix, as we saw earlier in §4.3.2, page 88 when dealing with elementary matrices). In $\mathbb{R}^2$, the only possible permutation is

$r_1 \leftrightarrow r_2$ with the following matrix $\boldsymbol{A}$.

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \implies$$
$$\boldsymbol{A} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 1 \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$
$$\boldsymbol{A} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} = -1 \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

As we can see above, we have two eigenvectors for this matrix, with eigenvalues 1 and -1.

Projection Matrix

We came across projection matrices $\boldsymbol{P} \in \mathbb{R}^{n \times n}$, which take any vector $\boldsymbol{x} \in \mathbb{R}^n$ to its projection onto a subspace $\mathcal{S} \subseteq \mathbb{R}^n$. If the vector $\boldsymbol{x}$ that $\boldsymbol{P}$ is acting on is already in the subspace $\mathcal{S}$, we know that its projection is itself. Calling these vectors $\boldsymbol{x}_{\parallel}$, we can write:

$$\boldsymbol{P}\boldsymbol{x}_{\parallel} = \boldsymbol{x}_{\parallel} = 1\boldsymbol{x}_{\parallel}$$

So we have a bunch of eigenvectors $\boldsymbol{x}_{\parallel}$ in $\mathcal{S}$. Furthermore, the eigenvalues for these eigenvectors would be one ($\lambda = 1$).

We also know that if $\boldsymbol{x}$ is orthogonal to the subspace $\mathcal{S}$, the projection will be the zero vector. Calling such vectors $\boldsymbol{x}_{\perp}$, we write:

$$\boldsymbol{P}\boldsymbol{x}_{\perp} = \boldsymbol{0} = 0\boldsymbol{x}_{\perp}$$

So $\boldsymbol{x}_{\perp}$ is an eigenvector with $\lambda = 0$. Note that for any matrix $\boldsymbol{A}$, we can always write $\boldsymbol{A}\boldsymbol{0} = \boldsymbol{0}$, but $\boldsymbol{0}$ itself is *not* an eigenvector by our definition.

Shear Matrix

A shear matrix transforms a square into a parallelogram.[1] Here is an example of a horizontal shear matrix, as shown in Figure 11.1.

$$\boldsymbol{A} = \begin{bmatrix} 1 & 0.5 \\ 0 & 1 \end{bmatrix} \implies$$
$$\boldsymbol{A}\boldsymbol{q}_1 = \boldsymbol{A}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \boldsymbol{A}\boldsymbol{q}_2 = \boldsymbol{A}\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}$$

When a vector is transformed by $\boldsymbol{A}$, its *second* component (y) is not affected. A horizontal shear leaves vectors sitting exactly on the x-axis completely unchanged, and therefore $\boldsymbol{q}_1$ is an eigenvector with eigenvalue $\lambda = 1$.

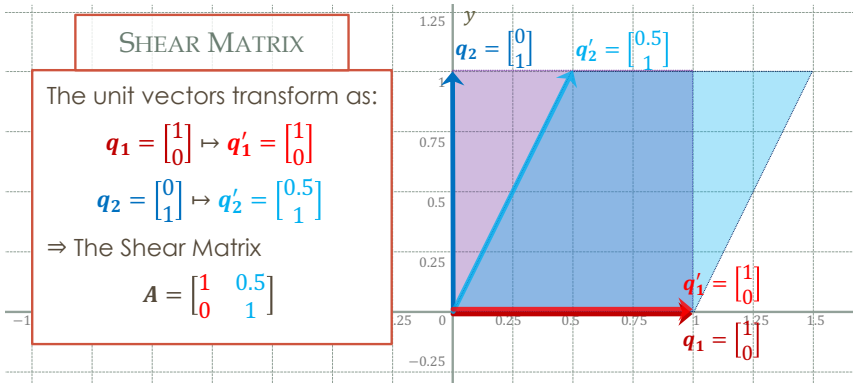


Fig. 11.1 An example shear matrix, showing a square being transformed into a parallelogram. Interactive Visualization

Rotation Matrix

We looked at rotation matrices earlier when talking about orthogonal matrices. In $\mathbb{R}^2$, the rotation matrix is:

$$\boldsymbol{Q}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

[1] Strictly speaking, geometric shapes like squares do not exist in a pure vector space; they belong to coordinate spaces. When we say a square transforms into a parallelogram, we mean the two basis vectors defining the unit square transform into vectors that define a parallelogram. We relax our pedantic exactitude here for intuitive clarity.

For any nontrivial rotation (meaning $\theta \neq k\pi$ for any integer k), we can see that $\boldsymbol{Q}_\theta$ changes the direction of every single vector in $\mathbb{R}^2$. We have no real eigenvectors for this matrix in $\mathbb{R}^2$.

Differentiation

We can think of the set of all continuous functions (of one variable, for instance) as a vector space. It satisfies all the requisite properties. The calculus operation of differentiation is a linear transformation in this space; it satisfies both the homogeneity and additivity properties of linearity.

$$\frac{d}{dx}e^{ax} = ae^{ax} \qquad \implies e^{ax} \text{ is an eigenvector with } \lambda = a$$

$$\frac{d^2}{dx^2}\sin x = -\sin x \qquad \implies \sin x \text{ is an eigenvector with } \lambda = -1$$

11.1.2 Finding Eigenvalues and Eigenvectors

In order to find the eigenvalues and then the eigenvectors, we start from their definitions:

$$\boldsymbol{A}\boldsymbol{s} = \lambda \boldsymbol{s} \implies (\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{s} = \boldsymbol{0}$$

Remembering that $\boldsymbol{s} \neq \boldsymbol{0}$, we can see that $\boldsymbol{A} - \lambda \boldsymbol{I}$ has a non-trivial null space (to which the eigenvector $\boldsymbol{s}$ belongs). Since $\mathcal{N}(\boldsymbol{A} - \lambda \boldsymbol{I})$ has non-zero vectors in it, $\boldsymbol{A} - \lambda \boldsymbol{I}$ is singular, and its determinant has to be zero. This gives us an equation for the eigenvalues:

$$|\boldsymbol{A} - \lambda \boldsymbol{I}| = 0$$

Thus, we get rid of $\boldsymbol{s}$ and end up with a polynomial in λ (when we expand the determinant) equaling zero, from which we can solve for the possible values of λ. This polynomial is called the *characteristic polynomial* of the matrix.

For a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, when we expand the determinant $|\boldsymbol{A} - \lambda \boldsymbol{I}|$ using the Laplace formula in Eqn (3.5), we get a polynomial of order n in λ. This should give us n roots, but not all of them may be real. Once we have the eigenvalues, we can put them back into $(\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{s} = \boldsymbol{0}$ to find the eigenvectors $\boldsymbol{s}$, which is exactly the same as finding the null space of $(\boldsymbol{A} - \lambda \boldsymbol{I})$.

Let's look at the examples from the previous section again to see how we compute the eigenvalues and eigenvectors.

Permutation Matrix

Starting from the permutation matrix in $\mathbb{R}^2$, here are the steps:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad |\boldsymbol{A} - \lambda \boldsymbol{I}| = 0 \implies \begin{vmatrix} -\lambda & 1 \\ 1 & -\lambda \end{vmatrix} = 0$$

$$\text{Expanding the determinant, } \lambda^2 - 1 = 0 \implies \lambda = \pm 1$$

As we saw earlier, we have two eigenvalues, 1 and -1. To find the corresponding eigenvectors, we substitute the λ values into $(\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{s} = \boldsymbol{0}$ and find the null space.

$$\text{With } \lambda = 1, \quad (\boldsymbol{A} - 1\boldsymbol{I})\boldsymbol{s} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\text{With } \lambda = -1, \quad (\boldsymbol{A} + 1\boldsymbol{I})\boldsymbol{s} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Here, to find $\boldsymbol{s}$, we are still using the column-picture of matrix multiplication and figuring out what linear combinations of the columns of $\boldsymbol{A} - \lambda \boldsymbol{I}$ give the zero vector. As we can see, we get two distinct eigenvectors, and they are orthogonal to each other. Note that any scaled version of an eigenvector is still an eigenvector with the same λ. For this reason, we typically normalize them.

The fact that we got real eigenvalues alongside distinct and orthogonal eigenvectors is not an accident. Real symmetric matrices (as our $\boldsymbol{A}$ was, in the case of this permutation matrix) always have real eigenvalues and a full set of orthogonal eigenvectors. They are the best matrices to work with.

Projection Matrix

As a concrete example, let's consider the projection matrix in $\mathbb{R}^3$ that projects vectors onto the xy-plane. Still calling it $\boldsymbol{A}$, what $\boldsymbol{A}$ needs to do is leave the first two components alone and make the last

component zero.

$$\boldsymbol{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad |\boldsymbol{A} - \lambda \boldsymbol{I}| = 0 \implies \begin{vmatrix} 1-\lambda & 0 & 0 \\ 0 & 1-\lambda & 0 \\ 0 & 0 & -\lambda \end{vmatrix} = 0$$

Expanding the determinant, $-\lambda(1-\lambda)^2 = 0 \implies \lambda = 0, 1, 1$

Again, as we saw earlier, we have two eigenvalues, 0 and 1. But note that $\lambda = 1$ is repeated. We state this fact more formally by saying that the *algebraic multiplicity* of the eigenvalue ($\lambda = 1$) is two. Let's go ahead and try to find the corresponding eigenvectors.

$$\text{With } \lambda = 0, \quad (\boldsymbol{A} - 0\boldsymbol{I})\boldsymbol{s} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\text{With } \lambda = 1, \quad (\boldsymbol{A} - 1\boldsymbol{I})\boldsymbol{s} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix} \boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} = \begin{bmatrix} t_1 \\ t_2 \\ 0 \end{bmatrix}$$

where t_1 and t_2 are any real numbers. We see that we have a small issue with the second eigenvalue: its eigenvectors span an entire subspace. This is called the *eigenspace* associated with the eigenvalue ($\lambda = 1$). The dimension of this eigenspace is two, which is known as its *geometric multiplicity*. All eigenvalues have eigenspaces associated with them, with a geometric multiplicity of at least one.

When we have an eigenvalue with a geometric multiplicity greater than one, we must select a full set of linearly independent vectors that span its eigenspace. In other words, we take its basis as the eigenvectors. In this particular case of projecting onto the xy-plane, the perfect basis would be the unit vectors along the x and y directions. Putting it all together, here is the full solution:

$$\lambda_1 = 0, \quad \boldsymbol{s}_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \qquad \lambda_2 = 1, \quad \boldsymbol{s}_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \boldsymbol{s}_3 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Shear Matrix

Moving on to our next example:

$$\boldsymbol{A} = \begin{bmatrix} 1 & 0.5 \\ 0 & 1 \end{bmatrix} \quad |\boldsymbol{A} - \lambda \boldsymbol{I}| = 0 \implies \begin{vmatrix} 1-\lambda & 0.5 \\ 0 & 1-\lambda \end{vmatrix} = 0$$

$$\text{Expanding the determinant, } (1-\lambda)^2 = 0 \implies \lambda = 1$$

As we saw earlier, we have a single eigenvalue of 1, but with an algebraic multiplicity of two. The corresponding eigenvector is found as follows:

$$\text{With } \lambda = 1, \quad (\boldsymbol{A} - 1\boldsymbol{I})\boldsymbol{s} = \begin{bmatrix} 0 & 0.5 \\ 0 & 0 \end{bmatrix} \boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The eigenvalue has an algebraic multiplicity of two, but a geometric multiplicity of only one. We cannot find a full set of (real) eigenvectors, and we are in trouble because for $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, we would like to have n eigenvectors.

Rotation Matrix

We saw geometrically that there were no real eigenvectors for a rotation matrix in $\mathbb{R}^2$. Let's consider a $\frac{\pi}{2}$ rotation, call the matrix $\boldsymbol{A}$, and attempt to compute its eigenvalues and eigenvectors algebraically.

$$\boldsymbol{Q}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} = \begin{bmatrix} \cos\frac{\pi}{2} & -\sin\frac{\pi}{2} \\ \sin\frac{\pi}{2} & \cos\frac{\pi}{2} \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \boldsymbol{A}$$

$$|\boldsymbol{A} - \lambda \boldsymbol{I}| = 0 \implies \begin{vmatrix} -\lambda & -1 \\ 1 & -\lambda \end{vmatrix} = 0$$

$$\text{Expanding the determinant, } \lambda^2 + 1 = 0 \implies \lambda = \pm i$$

We have no real eigenvalues. For the sake of completeness, we can try to find the eigenvectors, although we do not expect to find any in $\mathbb{R}^2$.

$$\text{With } \lambda = i, \quad (\boldsymbol{A} - i\boldsymbol{I})\boldsymbol{s} = \begin{bmatrix} -i & -1 \\ 1 & -i \end{bmatrix} \boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} = \begin{bmatrix} 1 \\ -i \end{bmatrix}$$

$$\text{With } \lambda = -i, \quad (\boldsymbol{A} + i\boldsymbol{I})\boldsymbol{s} = \begin{bmatrix} i & -1 \\ 1 & i \end{bmatrix} \boldsymbol{s} = \boldsymbol{0} \implies \boldsymbol{s} = \begin{bmatrix} 1 \\ i \end{bmatrix}$$

Thus, if we allow ourselves to step into the field of complex numbers, we can find eigenvectors for the rotation matrix where $\boldsymbol{s} \in \mathbb{C}^2$. Physically, the vector that is conserved during a 3D rotation is perpendicular to the plane of rotation, which is why gyroscopes work the way they do. How that fact corresponds to the actual complex eigenvectors of the 2D matrix we computed above, however, is a fairly tortured explanation.

11.2 Mathematical Properties and Geometry

The eigenvalues and eigenvectors provide deep insights into the internal structure of a matrix. They exhibit properties that are directly related to the properties of the matrix itself. Here are some of the most important ones, with proofs where possible. It is highly recommended to verify these properties against the examples we worked out in the previous section.

11.2.1 Eigenvalue Properties

1. **The sum of the eigenvalues equals the trace of the matrix.** For an $n \times n$ matrix $\boldsymbol{A}$:

$$\sum_{i=1}^{n} \lambda_i = \text{trace}(\boldsymbol{A}) = \sum_{i=1}^{n} a_{ii}$$

Proof: Since the characteristic polynomial has roots λ_i, we can write it in factored form:

$$|\boldsymbol{A} - \lambda \boldsymbol{I}| = (-1)^n(\lambda - \lambda_1)(\lambda - \lambda_2)\cdots(\lambda - \lambda_n)$$

We constructed the RHS to have roots at λ_i and to match the coefficient of λ^n in the determinant on the LHS, which expands to give a polynomial in λ where the coefficient of λ^n is $(-1)^n$.

On the LHS, the coefficient of λ^{n-1} is:

$$(-1)^n \sum_{i=1}^{n} a_{ii}$$

On the RHS, the coefficient of λ^{n-1} is:

$$(-1)^n \sum_{i=1}^{n} \lambda_i$$

Since the LHS and RHS coefficients must match exactly, we see that:

$$\sum_{i=1}^{n} \lambda_i = \sum_{i=1}^{n} a_{ii} = \text{trace}(\boldsymbol{A})$$

which proves the property.

Although we proved it here by comparing the coefficients of λ^{n-1}, this property is much easier to prove once we learn matrix similarity in the next chapter. Note that we must include λ_i in the summation as many times as its algebraic multiplicity. Note also that for $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, this property implies that if $\boldsymbol{A}$ has any complex eigenvalues, they must always come in complex conjugate pairs so their imaginary parts cancel out in the sum.

2. **The product of the eigenvalues equals the determinant.** For an $n \times n$ matrix $\boldsymbol{A}$:

$$\prod_{i=1}^{n} \lambda_i = |\boldsymbol{A}|$$

Proof: Again, we start with the factored characteristic polynomial:

$$|\boldsymbol{A} - \lambda \boldsymbol{I}| = (-1)^n (\lambda - \lambda_1)(\lambda - \lambda_2) \cdots (\lambda - \lambda_n)$$

Set $\lambda = 0$ to get:

$$|\boldsymbol{A}| = (-1)^n (-1)^n \lambda_1 \lambda_2 \cdots \lambda_n = \lambda_1 \lambda_2 \cdots \lambda_n = \prod_{i=1}^{n} \lambda_i$$

A critical corollary of this property is that a singular matrix ($|\boldsymbol{A}| = 0$) must have at least one zero eigenvalue.

3. **The eigenvalues of a real, symmetric matrix are real.**

$$\boldsymbol{A} \in \mathbb{R}^{n \times n}, \;\; \boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A} \implies \lambda_i \in \mathbb{R}$$

Proof: To prove this property, we have to step into the somewhat scary field of complex numbers. The standard strategy to prove a number is real is to assume it is complex, and then show that its complex conjugate (where we replace all i with $-i$, denoted by z^*) is equal to itself. If $z = z^*$, it must be purely real.

Following this strategy, let's assume $\lambda \in \mathbb{C}$ is a possibly complex eigenvalue of $\boldsymbol{A}$, and $\boldsymbol{s} \in \mathbb{C}^n$ is its corresponding eigenvector.

$$
\begin{aligned}
&\text{(1) Eigenvalue definition:} && \boldsymbol{A}\boldsymbol{s} = \lambda \boldsymbol{s} \\
&\text{(2) Take complex conjugate:} && \boldsymbol{A}^* \boldsymbol{s}^* = \lambda^* \boldsymbol{s}^* \\
&\text{(3) } \boldsymbol{A} \text{ is real } (\boldsymbol{A}^* = \boldsymbol{A}): && \boldsymbol{A}\boldsymbol{s}^* = \lambda^* \boldsymbol{s}^* \\
&\text{(4) Left-multiply (1) by } \boldsymbol{s}^{*\mathsf{T}}: && \boldsymbol{s}^{*\mathsf{T}}\boldsymbol{A}\boldsymbol{s} = \boldsymbol{s}^{*\mathsf{T}}\lambda \boldsymbol{s} \\
&\text{(5) Rearrange terms:} && \lambda \boldsymbol{s}^{*\mathsf{T}}\boldsymbol{s} = \boldsymbol{s}^{*\mathsf{T}}\boldsymbol{A}\boldsymbol{s} \\
&\text{(6) Transpose rule on RHS:} && \lambda \boldsymbol{s}^{*\mathsf{T}}\boldsymbol{s} = \left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{s}^*\right)^{\mathsf{T}} \boldsymbol{s} \\
&\text{(7) } \boldsymbol{A} \text{ is symmetric } (\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}): && = \left(\boldsymbol{A}\boldsymbol{s}^*\right)^{\mathsf{T}} \boldsymbol{s} \\
&\text{(8) Substitute from (3):} && = \lambda^* \boldsymbol{s}^{*\mathsf{T}}\boldsymbol{s}
\end{aligned}
$$

Finally, combining steps (6) and (8), we get:

$$\lambda \boldsymbol{s}^{*\mathsf{T}}\boldsymbol{s} = \lambda^* \boldsymbol{s}^{*\mathsf{T}}\boldsymbol{s} \implies (\lambda - \lambda^*)\boldsymbol{s}^{*\mathsf{T}}\boldsymbol{s} = 0$$

The term $\boldsymbol{s}^{*\mathsf{T}}\boldsymbol{s}$ is the square of the norm $\|\boldsymbol{s}\|^2$ for a complex vector (as we shall see in the next chapter). Since an eigenvector cannot be the zero vector, its norm cannot be zero. Therefore, we must have:

$$(\lambda - \lambda^*) = 0 \implies \lambda = \lambda^*$$

This proves that λ is strictly real.

4. **The eigenvalues of a real, antisymmetric (skew-symmetric) matrix are purely imaginary.**

$$\boldsymbol{A} \in \mathbb{R}^{n \times n}, \;\; \boldsymbol{A}^{\mathsf{T}} = -\boldsymbol{A} \implies \lambda_i = c_i i, \text{ where } c_i \in \mathbb{R}$$

Proof: The proof is identical to the previous one, except a negative sign appears in step (7) because $\boldsymbol{A}^{\mathsf{T}} = -\boldsymbol{A}$, resulting in the conclusion that $\lambda = -\lambda^*$.

5. **Multiplying a matrix by a scalar multiplies its eigenvalues by the same scalar.**

$$\text{If } \boldsymbol{A}\boldsymbol{s} = \lambda \boldsymbol{s} \implies (\alpha \boldsymbol{A})\boldsymbol{s} = (\alpha \lambda)\boldsymbol{s}$$

Sylvester's Law of Inertia


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
Available in the Full Version of the book.

6. **Adding $\alpha \boldsymbol{I}$ shifts the eigenvalues by α.** If $\boldsymbol{s}$ is an eigenvector of $\boldsymbol{A}$ with eigenvalue λ, we have:

$$(\boldsymbol{A} + \alpha \boldsymbol{I})\boldsymbol{s} = \boldsymbol{A}\boldsymbol{s} + \alpha \boldsymbol{I}\boldsymbol{s} = \lambda \boldsymbol{s} + \alpha \boldsymbol{s} = (\lambda + \alpha)\boldsymbol{s}$$

This proves that $\boldsymbol{s}$ remains an eigenvector of the new shifted matrix, but its corresponding eigenvalue is shifted to $\lambda + \alpha$.

7. **The signs of the eigenvalues match the signs of the pivots.** For real, symmetric matrices, the number of positive eigenvalues is exactly the same as the number of positive pivots found during Gaussian elimination (in the REF, or in $\boldsymbol{U}$ from an $\boldsymbol{LU}$ decomposition). The same holds true for the negative eigenvalues and negative pivots.

11.2.2 Eigenvector Properties

1. **Eigenvectors corresponding to distinct eigenvalues are linearly independent.** [2]

$$\begin{gathered} \text{If } \boldsymbol{A}\boldsymbol{s_i} = \lambda_i \boldsymbol{s_i} \text{ and } \lambda_i \neq \lambda_j \\ \text{then } c_i \boldsymbol{s_i} + c_j \boldsymbol{s_j} = \boldsymbol{0} \implies c_i = c_j = 0 \end{gathered}$$

Proof: We need to prove that if $\boldsymbol{s_i}$ and $\boldsymbol{s_j}$ are eigenvectors of $\boldsymbol{A}$ with distinct eigenvalues λ_i and λ_j, we will not be able to find non-zero scalar weights c_i and c_j that satisfy the linear

[2] We use $\mathbb{R}$ (the real field, $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ and $\boldsymbol{s} \in \mathbb{R}^n$) in the mathematical statement and proof of this property for convenience and because of its relevance to computer science, but the property applies perfectly to $\mathbb{C}$ as well.

combination above.

(1) Linear combination:	$c_i \boldsymbol{s_i} + c_j \boldsymbol{s_j} = \boldsymbol{0}$
(2) Left-multiply by $\boldsymbol{A}$:	$c_i \boldsymbol{A s_i} + c_j \boldsymbol{A s_j} = \boldsymbol{0}$
(3) Substitute eigenvalues:	$c_i \lambda_i \boldsymbol{s_i} + c_j \lambda_j \boldsymbol{s_j} = \boldsymbol{0}$
(4) Multiply (1) by λ_i:	$c_i \lambda_i \boldsymbol{s_i} + c_j \lambda_i \boldsymbol{s_j} = \boldsymbol{0}$
(5) Subtract (3) from (4):	$c_j(\lambda_i - \lambda_j)\boldsymbol{s_j} = \boldsymbol{0}$
(6) Since $\lambda_i \neq \lambda_j$ and $\boldsymbol{s_j} \neq \boldsymbol{0}$:	$c_j = 0$
(7) Substitute $c_j = 0$ into (1):	$c_i = 0$

The converse of this statement is *not* true: if the eigenvalues are *not* distinct, the eigenvectors may still be linearly independent (as seen in the projection matrix example, or the identity matrix $\boldsymbol{I}_n$ which has the eigenvalue 1 repeated n times).

2. **Real symmetric matrices have a full set of orthogonal eigenvectors.**

$$\boldsymbol{A} \in \mathbb{R}^{n \times n}, \;\; \boldsymbol{A^{\mathsf{T}}} = \boldsymbol{A} \text{ and } \lambda_i \neq \lambda_j \implies \boldsymbol{s_i} \perp \boldsymbol{s_j}$$

Proof:

(1) Eigenvalue definition:	$\boldsymbol{A s_i} = \lambda_i \boldsymbol{s_i}$
(2) Dot product with $\boldsymbol{s_j}$:	$(\boldsymbol{A s_i})^{\mathsf{T}} \boldsymbol{s_j} = \lambda_i \boldsymbol{s_i^{\mathsf{T}} s_j}$
(3) Transpose product rule:	$\boldsymbol{s_i^{\mathsf{T}} A^{\mathsf{T}} s_j} = \lambda_i \boldsymbol{s_i^{\mathsf{T}} s_j}$
(4) $\boldsymbol{A}$ is symmetric ($\boldsymbol{A^{\mathsf{T}}} = \boldsymbol{A}$):	$\boldsymbol{s_i^{\mathsf{T}} A s_j} = \lambda_i \boldsymbol{s_i^{\mathsf{T}} s_j}$
(5) $\boldsymbol{s_j}$ is an eigenvector:	$\boldsymbol{s_i^{\mathsf{T}}} \lambda_j \boldsymbol{s_j} = \lambda_i \boldsymbol{s_i^{\mathsf{T}} s_j}$
(6) Factor out scalars:	$\lambda_j \boldsymbol{s_i^{\mathsf{T}} s_j} = \lambda_i \boldsymbol{s_i^{\mathsf{T}} s_j}$
(7) Gather terms:	$(\lambda_j - \lambda_i)\boldsymbol{s_i^{\mathsf{T}} s_j} = 0$
(8) Since $\lambda_j \neq \lambda_i$:	$\boldsymbol{s_i^{\mathsf{T}} s_j} = 0$
(9) Zero dot product implies:	$\boldsymbol{s_i} \perp \boldsymbol{s_j}$

It may happen that the eigenvalues are repeated. For instance, the identity matrix $\boldsymbol{I} \in \mathbb{R}^{n \times n}$ has n repeated eigenvalues of 1. Every vector in $\mathbb{R}^n$ is an eigenvector of $\boldsymbol{I}$. In this case, we can still simply *choose* an orthogonal set of vectors to serve as the full set of eigenvectors for the matrix.

Similarly, when the eigenvectors for a repeated eigenvalue span a multidimensional eigenspace, we can use Gram-Schmidt to construct an orthogonal basis for it.

3. **The eigenvectors of $\boldsymbol{A} + \alpha\boldsymbol{I}$ are the exact same as those of $\boldsymbol{A}$.**
   Proof: As we proved in the eigenvalues section, if $\boldsymbol{s}$ is an eigenvector of $\boldsymbol{A}$, then $(\boldsymbol{A} + \alpha\boldsymbol{I})\boldsymbol{s} = (\lambda + \alpha)\boldsymbol{s}$. The vector $\boldsymbol{s}$ itself remains completely unchanged.

11.2.3 Geometric Interpretation

We might reasonably ask if this is purely an academic exercise. We may not be able to answer this question completely yet, but we can look at a linear transformation and see what the eigenvalue analysis tells us about it geometrically. In the last chapter, we will see how these insights are harnessed in statistical analyses.

Let's start with an example: Given a real, symmetric matrix $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$ (where $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}$), let's find its eigenvalues and eigenvectors, and look at them in the coordinate space $\mathbb{R}^2$.

$$\boldsymbol{A} = \frac{1}{4}\begin{bmatrix} 5 & -\sqrt{3} \\ -\sqrt{3} & 3 \end{bmatrix}$$

$$\lambda_1 = \frac{3}{2}, \quad \boldsymbol{s}_1 = \frac{1}{2}\begin{bmatrix} \sqrt{3} \\ -1 \end{bmatrix} \qquad \lambda_2 = \frac{1}{2}, \quad \boldsymbol{s}_2 = \frac{1}{2}\begin{bmatrix} 1 \\ \sqrt{3} \end{bmatrix}$$

As we can see from Figure 11.2, $\boldsymbol{A}$ takes the first basis vector ($\boldsymbol{q}_1$, shown as a dashed red arrow) to its first column vector ($\boldsymbol{a}_1$, shown as a bright red arrow): $\boldsymbol{q}_1 \mapsto \boldsymbol{a}_1$. The same happens for the second basis vector: $\boldsymbol{q}_2 \mapsto \boldsymbol{a}_2$ (shown in shades of blue). What happens to the basis vectors happens to all vectors; therefore, the entire unit circle in the figure gets mapped into the ellipse as shown.

Although we know about this unit-circle-to-ellipse business, from the matrix $\boldsymbol{A}$ itself, we know very little else. Note that the specific vectors to which the unit vectors transform (in $\boldsymbol{q}_i \mapsto \boldsymbol{a}_i$) are nothing geometrically special; they simply land somewhere on the boundary of the ellipse. What we actually want to know are the defining details of the ellipse, like its size and orientation—which is exactly what the eigenvalues and eigenvectors tell us. The eigenvalues λ_i dictate the lengths of the major and minor axes of the ellipse, and

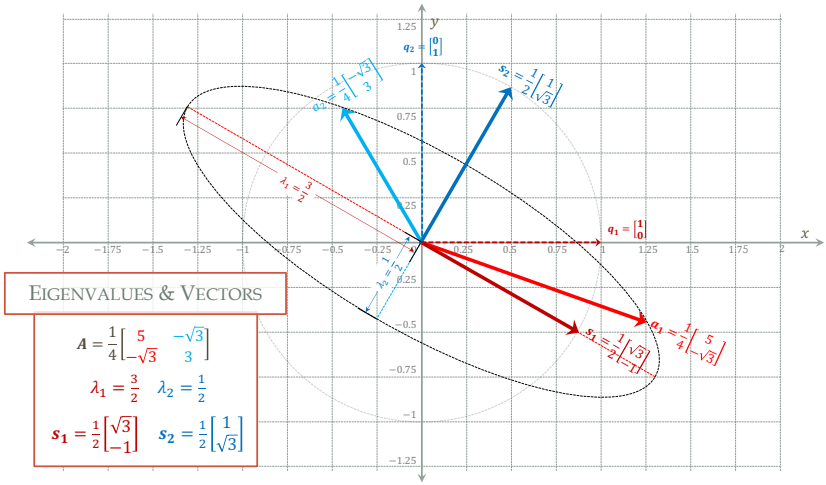


Fig. 11.2 Visualization of eigenvalues and eigenvectors of a real, symmetric matrix: $\boldsymbol{A}$ transforms the unit circle into a rotated ellipse. The eigenvalues specify the lengths of its major and minor axes, and the eigenvectors specify the orientation of the axes. Interactive Visualization

the eigenvectors are the unit vectors pointing along those axes. In Figure 11.2, the eigenvectors ($\boldsymbol{s}_1$ and $\boldsymbol{s}_2$) are shown in darker shades of red and blue, while the corresponding eigenvalues represent the lengths of the axes.

When we move on to higher dimensions, ellipses become ellipsoids or hyperellipsoids, and these axes become their principal axes. The mathematics of eigenanalysis stays exactly the same: we get the directions and lengths of the principal axes.

Note that we cleverly restricted our illustration to a symmetric matrix. Recall that the rotation matrix was a skew-symmetric matrix. Thus, a symmetric matrix can be thought of as a transformation with no inherent rotational component of its own. For such symmetric matrices, Eigenvalue Decomposition (EVD) gives us the exact properties of the ellipse (or ellipsoid) into which the unit circle (or hypersphere) transforms.

Recall also that the Gram matrix ($\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$) is symmetric and full rank. If we zero-center our data (by subtracting the mean of each column from the elements in that column), then $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ is proportional to the covariance matrix of the variables in a dataset. We can then perform eigenanalysis on this covariance matrix (which is a real,

symmetric matrix), yielding deep insights about the directions along which we can decompose the covariance. If we sort these directions by their eigenvalues, we can extract the direction of highest variance, second-highest variance, and so on. We will revisit this idea in much more detail in one of our final topics: Principal Component Analysis (PCA), which is the absolute mainstay of dimensionality reduction in data science.

11.3 Diagonalization and Matrix Powers

We have learned quite a bit about eigenvalues and eigenvectors by now. We might still wonder why at this point. What is the point of learning all this trivia about them? We hinted at its significance in data analytics for dimensionality reduction. We have one more good reason: there is a method to this madness. Once we have the eigenvectors of a matrix, we can *diagonalize* it. And once we diagonalize, we can immediately see how it helps in computing the powers of the matrix. Why would we want to take powers of matrices? Because it is the mathematical basis of modeling time-varying systems.

11.3.1 The Eigenmatrices S and Λ

Suppose $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ has its eigenvalues λ_i and the corresponding eigenvectors $\boldsymbol{s_i}$. Let's construct two matrices, arranging the eigenvectors as columns of the first, and the eigenvalues as the diagonal elements of the second:

$$\boldsymbol{S} = \begin{bmatrix} | & | & \cdots & | \\ \boldsymbol{s}_1 & \boldsymbol{s}_2 & \cdots & \boldsymbol{s}_n \\ | & | & \cdots & | \end{bmatrix} \quad \boldsymbol{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & 0 \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

For simplicity, we may write $\boldsymbol{S} = [\boldsymbol{s_i}]$ and $\boldsymbol{\Lambda} = [\lambda_i]$. With these new matrices, we arrive at the most important algebraic result from this chapter:

$$\boldsymbol{AS} = \boldsymbol{S\Lambda}$$

The LHS is a matrix multiplication, where the product matrix has columns that are the product of $\boldsymbol{A}$ and the corresponding column in $\boldsymbol{S}$. In other words, $\boldsymbol{AS} = \boldsymbol{A}[\boldsymbol{s_i}] = [\boldsymbol{As_i}]$. We think of the

Invertibility vs. Diagonalizability


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

RHS using the column-picture of matrix multiplication again: the columns of $\boldsymbol{S\Lambda} = [\boldsymbol{s_i}]\boldsymbol{\Lambda}$ are the linear combinations of $\boldsymbol{s_i}$ taken with the scaling factors in the columns of $\boldsymbol{\Lambda}$. But the scaling factors are merely λ_i in the ith place. Therefore, the ith column in $\boldsymbol{AS} = \boldsymbol{S\Lambda}$ is the same as $\boldsymbol{As_i} = \lambda_i \boldsymbol{s_i}$, which is simply the now-familiar definition of eigenvalues and eigenvectors.

If we know that $\boldsymbol{S}$ is invertible, we can go one step further by right-multiplying both sides by $\boldsymbol{S^{-1}}$:

$$\text{If } \boldsymbol{S^{-1}} \text{ exists, } \boldsymbol{AS} = \boldsymbol{S\Lambda} \implies \boldsymbol{A} = \boldsymbol{S\Lambda S^{-1}}$$

This is the famous Eigenvalue Decomposition of a real, square, *diagonalizable* matrix. Note that most matrices are diagonalizable over the field of complex numbers.

11.3.2 Powers and Inverses of $\boldsymbol{A}$

As previously advertised, the reason for this decomposition is that it gives us a computationally efficient way to express the powers of a matrix. For a square, diagonalizable matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, we can write:

$$\begin{aligned}
\boldsymbol{A} &= \boldsymbol{S\Lambda S^{-1}} \implies \\
\boldsymbol{A}^2 &= (\boldsymbol{S\Lambda S^{-1}})(\boldsymbol{S\Lambda S^{-1}}) = \boldsymbol{S\Lambda}(\boldsymbol{S^{-1}S})\boldsymbol{\Lambda S^{-1}} \\
&= \boldsymbol{S\Lambda I\Lambda S^{-1}} = \boldsymbol{S\Lambda\Lambda S^{-1}} \\
\boldsymbol{A}^2 &= \boldsymbol{S\Lambda}^2\boldsymbol{S^{-1}}
\end{aligned}$$

Similarly, we can easily see that $\boldsymbol{A}^k = \boldsymbol{S\Lambda}^k\boldsymbol{S^{-1}}$. While this result may look fairly mundane, let's think about a medium-sized matrix, say 100×100, and $k = 60$. Imagine the number of operations required to compute $\boldsymbol{A}^{60}$ natively, which is probably on the order of $60 \times 100^3 = 6 \times 10^7$ (for the 60 full matrix multiplications). But if we have the decomposition, taking the 60th power of $\boldsymbol{\Lambda}$ is trivial: merely 100 scalar exponentiations along the diagonal. The two remaining

matrix multiplications cost about $2 \times 100^3 = 2 \times 10^6$ operations. The overall savings in computational time, therefore, is roughly a factor of 30.

The best modern algorithms for matrix multiplications take about $n^{2.3}$ operations. If we are computing the kth power of $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, it would cost us $kn^{2.3}$ operations. But with diagonalization, it will cost us n exponentiations of the diagonal elements of $\boldsymbol{\Lambda}$ and two multiplications, or a total of $n + 2n^{2.3}$ operations. This implies a massive reduction factor of $\frac{kn^{2.3}}{n+2n^{2.3}} \approx \frac{k}{2}$ for large n.

Since any power of $\boldsymbol{A}$ can be expressed essentially as powers of $\boldsymbol{\Lambda}$, we can make the exact same statement about any polynomial of $\boldsymbol{A}$.

Similarly, since we have a product for $\boldsymbol{A}$, we can compute its inverse using the standard product rule for inverses:

$$\boldsymbol{A^{-1}} = (\boldsymbol{S\Lambda S^{-1}})^{-1} = \left(\boldsymbol{S^{-1}}\right)^{-1} \boldsymbol{\Lambda^{-1} S^{-1}} = \boldsymbol{S\Lambda^{-1} S^{-1}}$$

Since $\boldsymbol{\Lambda}$ is a diagonal matrix with λ_i along the diagonal, its inverse is another diagonal matrix with the elements equal to the reciprocal of λ_i. Therefore, $\boldsymbol{A^{-1}}$ has the exact same eigenvectors $\boldsymbol{s_i}$ as $\boldsymbol{A}$, but with eigenvalues equal to the reciprocals of the original eigenvalues: $\frac{1}{\lambda_i}$.

This can be seen even more directly as follows:

$$\boldsymbol{As_i} = \lambda_i \boldsymbol{s_i} \implies \boldsymbol{s_i} = \lambda_i \boldsymbol{A^{-1} s_i} \implies \boldsymbol{A^{-1} s_i} = \frac{1}{\lambda_i} \boldsymbol{s_i}$$

Enough said.

Since $\boldsymbol{A^{-1}} = \boldsymbol{S\Lambda^{-1} S^{-1}}$, we can see that $\boldsymbol{A}^k = \boldsymbol{S\Lambda}^k \boldsymbol{S^{-1}}$ holds for negative values of k ($k < 0$) as well. Extrapolating even further, through Taylor series expansions, we can compute entities like $e^{\boldsymbol{A}}$ (matrix exponentiation), which are essential for solving systems of differential equations—a topic we consider beyond the scope of this book.

11.3.3 The Eigenbasis

If we have a full set of real eigenvectors for $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, we can use them as a basis for $\mathbb{R}^n$, which we will call the eigenbasis. We can then express any arbitrary state vector $\boldsymbol{x}_0 \in \mathbb{R}^n$ as a linear combination of the eigenbasis vectors, remembering what we learned about changing

the bases of vectors earlier in §7.1.4 (page 141).

$$\boldsymbol{x}_0 = \boldsymbol{s}_1 c_1 + \boldsymbol{s}_2 c_2 + \cdots + \boldsymbol{s}_n c_n = \sum_{i=1}^{n} \boldsymbol{s_i} c_i = \boldsymbol{Sc}$$

where c_i are the coordinates of $\boldsymbol{x}$ in the eigenbasis. (We wrote the scalar after the vector in the summation so that the matrix product is easier to spot).

Once we have the vector $\boldsymbol{x}_0$ expressed in the eigenbasis of $\boldsymbol{A}$, we can drastically simplify the multiplication of the powers of $\boldsymbol{A}$ with $\boldsymbol{x}$:

$$\boldsymbol{A}\boldsymbol{x}_0 = \boldsymbol{A}\sum_{i=1}^{n} \boldsymbol{s_i} c_i = \sum_{i=1}^{n} \boldsymbol{A}\boldsymbol{s_i} c_i = \sum_{i=1}^{n} \lambda_i \boldsymbol{s_i} c_i = \boldsymbol{S\Lambda c}$$

$$\boldsymbol{A}^k\boldsymbol{x}_0 = \sum_{i=1}^{n} \lambda_i^k \boldsymbol{s_i} c_i = \boldsymbol{S\Lambda}^k \boldsymbol{c}$$

11.3.4 Difference Equations and Fibonacci Series

In building mathematical models of systems that evolve in time, we frequently come across situations where the state of the system at any given time step depends directly on the state at the previous step. If all the parameters specifying the state can be written as a vector, we can model the time evolution as $\boldsymbol{x}_{k+1} = \boldsymbol{A}\boldsymbol{x}_k$.

If we know the initial conditions at time step zero ($\boldsymbol{x}_0$), we can write:

$$\boldsymbol{x}_k = \boldsymbol{A}\boldsymbol{x}_{k-1} = \boldsymbol{A}^2\boldsymbol{x}_{k-2} = \cdots = \boldsymbol{A}^k\boldsymbol{x}_0$$

Why does this matter? Let's think of $\boldsymbol{A}$ as the transformation encoding the time evolution of a system, with $\boldsymbol{x}_k$ representing its state at a given step (or iteration, or point in time). Given the state of the system at one step, we evolve it to the next step by multiplying with $\boldsymbol{A}$ to get $\boldsymbol{x}_{k+1} = \boldsymbol{A}\boldsymbol{x}_k$.

Knowing the initial state $\boldsymbol{x}_0$ and, more importantly, the transition matrix $\boldsymbol{A}$, what can we say about the long-term stability of the system? Because we can leverage the eigenbasis expansion we just derived, we can say the following:

$$\lim_{k\to\infty} \boldsymbol{x}_k = \lim_{k\to\infty} \boldsymbol{A}^k\boldsymbol{x}_0 = \sum_{i=1,|\lambda_i|\geq 1}^{n} \lambda_i^k \boldsymbol{s_i} c_i$$

In other words, in the sum that makes up $\boldsymbol{x}_k$, only those eigenvalues (whether they are real or complex) whose absolute value is greater than or equal to 1 matter; they are the only ones that will survive when we take the limit as $k \to \infty$. All components attached to eigenvalues where $|\lambda_i| < 1$ will decay perfectly to zero over time.

As a concrete example of how this idea of the powers of a matrix applies to a real-world problem, let's look at the Fibonacci numbers. This problem may seem academic in nature, but it beautifully demonstrates the kind of thinking that goes into transforming a sequential problem to bring it into the domain of eigenvalues.

The famous Fibonacci sequence appears in nature in unexpected ways, and is heavily used in mathematics and, closer to home, in computer science. The Wikipedia entry has a comprehensive listing of its properties and interesting facts.

To see how we connect eigenvalue computation with Fibonacci numbers, let's start by writing them down. The sequence of numbers is: $0, 1, 1, 2, 3, 5, 8, 13, \cdots$. Each number in the sequence (after the first two) is the sum of the previous two. Calling them $f_0, f_1, f_2, \cdots, f_k, \cdots$, we can formally state that $f_{k+2} = f_{k+1} + f_k$. This is what we call a second-order difference equation because each number depends on the previous two. We do not see any vector or matrix here yet, do we? In order to reveal them, let's build a state vector out of two consecutive Fibonacci numbers:

$$\boldsymbol{x}_k = \begin{bmatrix} f_{k+1} \\ f_k \end{bmatrix} \implies \boldsymbol{x}_{k+1} = \begin{bmatrix} f_{k+2} \\ f_{k+1} \end{bmatrix} = \begin{bmatrix} f_{k+1} + f_k \\ f_{k+1} \end{bmatrix}$$

This setup gives us a chance to write the progression as a matrix equation and connect any state $\boldsymbol{x}_k$ back to the initial state $\boldsymbol{x}_0$:

$$\boldsymbol{x}_{k+1} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} f_{k+1} \\ f_k \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \boldsymbol{x}_k = \boldsymbol{A}\boldsymbol{x}_k = \boldsymbol{A}^{k+1}\boldsymbol{x}_0$$

From the Fibonacci sequence, we can easily see that the numbers are growing, and we may want to find out exactly how fast they are growing. Or perhaps we want a closed-form approximation for the kth Fibonacci number without calculating all the preceding ones. The first question about the growth rate is directly answered by the eigenvalues of $\boldsymbol{A}$, and the second one by the eigenbasis representation of $\boldsymbol{x}_k$.

Let's first compute the eigenvalues.

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad \boldsymbol{A} - \lambda \boldsymbol{I} = \begin{bmatrix} 1-\lambda & 1 \\ 1 & -\lambda \end{bmatrix}$$

$$|\boldsymbol{A} - \lambda \boldsymbol{I}| = 0 \implies -(1-\lambda)\lambda - 1 = 0 \implies \lambda^2 - \lambda - 1 = 0$$

$$\lambda_1 = \frac{1+\sqrt{5}}{2} \approx 1.618, \quad \lambda_2 = \frac{1-\sqrt{5}}{2} \approx -0.618$$

Since $|\lambda_1| > 1$ and $|\lambda_2| < 1$, it is the first eigenvalue that will completely dominate large Fibonacci numbers. In particular, f_{k+1} is going to be about 1.618 times bigger than f_k as $k \to \infty$. This eigenvalue is the famous Golden Ratio.

Let's now try to find a closed-form formula for f_k. We will start with the eigenvectors, express $\boldsymbol{x}_0$ in the eigenbasis, and mathematically evolve it to $\boldsymbol{x}_k$. The eigenvectors of $\boldsymbol{A}$ are the solutions to $(\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{s} = \boldsymbol{0}$.

$$(\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{s} = \boldsymbol{0} \implies \begin{bmatrix} 1-\lambda & 1 \\ 1 & -\lambda \end{bmatrix} \boldsymbol{s} = \boldsymbol{0}$$

$$\implies \boldsymbol{s} = \begin{bmatrix} \lambda \\ 1 \end{bmatrix} \text{ for } \lambda = \lambda_1, \lambda_2$$

The initial state $\boldsymbol{x}_0$ is a linear combination of $\boldsymbol{s}_1$ and $\boldsymbol{s}_2$:

$$\boldsymbol{x}_0 = c_1\boldsymbol{s}_1 + c_2\boldsymbol{s}_2 \implies \begin{bmatrix} 1 \\ 0 \end{bmatrix} = c_1 \begin{bmatrix} \lambda_1 \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} \lambda_2 \\ 1 \end{bmatrix}$$

After a bit of algebra, we find the constants:

$$c_1 = \frac{1}{\sqrt{5}}, \quad c_2 = -\frac{1}{\sqrt{5}}$$

We now know the exact mathematical evolution of $\boldsymbol{x}_k$:

$$\boldsymbol{x}_k = \boldsymbol{A}^k\boldsymbol{x}_0 = \boldsymbol{A}^k(c_1\boldsymbol{s}_1 + c_2\boldsymbol{s}_2) = c_1\lambda_1^k\boldsymbol{s}_1 + c_2\lambda_2^k\boldsymbol{s}_2 = \begin{bmatrix} f_{k+1} \\ f_k \end{bmatrix}$$

We can isolate and read the second element in $\boldsymbol{x}_k$ as our formula for f_k:

$$\begin{bmatrix} f_{k+1} \\ f_k \end{bmatrix} = c_1\lambda_1^k \begin{bmatrix} \lambda_1 \\ 1 \end{bmatrix} + c_2\lambda_2^k \begin{bmatrix} \lambda_2 \\ 1 \end{bmatrix} \implies f_k = c_1\lambda_1^k + c_2\lambda_2^k$$

Knowing that the second term vanishes for large k (because $|\lambda_2| \approx 0.618 < 1$), we finally get an elegant approximation for f_k:

$$f_k \approx c_1 \lambda_1^k = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^k = f_k^{(\text{approx})}$$

The quality of this approximation is demonstrated in Table 11.1, which shows that the formula is stunningly accurate. By the time we reach $k = 11$, the error is practically negligible (about 25 parts in a million).

Table 11.1 Fibonacci numbers (f_k) vs. the eigenvalue approximation ($f_k^{(\text{approx})}$)

k	f_k	$f_k^{(\text{approx})}$	k	f_k	$f_k^{(\text{approx})}$	k	f_k	$f_k^{(\text{approx})}$
0	0	0.45	4	3	3.07	8	21	21.01
1	1	0.72	5	5	4.96	9	34	33.99
2	1	1.17	6	8	8.02	10	55	55.00
3	2	1.89	7	13	12.98	11	89	89.00

Interactive Visualization

11.3.5 Further Applications

The ideas behind eigenvalue decomposition have a multitude of applications, especially in physics and the other physical sciences. In our domain of computer science, the Google PageRank algorithm, described in its own box in the next chapter, is a brilliant success story of this exact line of thinking. Given the advanced scope of this book, we will not exhaustively list applications here, but rather note that a great starting point for exploration is the comprehensive Wikipedia page on the topic.

Appendix: Complex Numbers and Eigenvalue Properties

In our discussion of Eigenvalue Decomposition, we frequently encounter roots that are not real numbers, and we rely on polynomial algebra to prove key matrix properties. This appendix provides a quick primer on these mathematical tools.

A Crash Course in Complex Numbers

The imaginary number i is defined as the mathematical solution to the equation $x^2 = -1$. Because this number cannot exist on the standard real number line, we call it imaginary, and we build the set of complex numbers ($\mathbb{C}$) by combining a real part and an imaginary part . A complex number takes the standard form $z = a + ib$, where $a, b \in \mathbb{R}$. If $b = 0$, the number is purely real, meaning the entire set of real numbers is simply a subset of the complex numbers ($\mathbb{R} \subset \mathbb{C}$).

From a computer science perspective, there is a strict one-to-one correspondence between vectors in $\mathbb{R}^2$ and numbers in $\mathbb{C}$. We can think of $z = a + ib$ as a 2D data structure with coordinates (a, b). The difference is that complex numbers come with richer algebraic semantics. For example, multiplying a complex number by i yields $iz = -b + ia$, which geometrically applies an exact anticlockwise rotation of $90°$ in the complex plane.

The standard operations behave much like vector math, with the addition of the core rule $i^2 = -1$:

- **Addition**: We add the real and imaginary parts independently: $(3 + 2i) + (1 + 4i) = 4 + 6i$.
- **Multiplication**: We apply the standard distributive property and simplify: $(2 + 3i)(1 + 4i) = 2 + 11i - 12 = -10 + 11i$.

CS Connection

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.
CS Connections are available in the Full Version of the book.

Finally, if a complex number lies exactly on the unit circle ($|z| = 1$), its coordinates can be parameterized by an angle θ such that $a = \cos\theta$ and $b = \sin\theta$. Using Euler's formula, we can represent this elegantly as $z = e^{i\theta} = \cos\theta + i\sin\theta$. This formula can be mathematically proven by comparing the power series expansions of the exponential and trigonometric functions, and it acts as a foundational tool in fields like quantum mechanics.

Polynomial Factorization

When analyzing characteristic equations, we frequently expand polynomials and compare them to their factored roots. A fundamental property of polynomials is that if two polynomials evaluate identically for all inputs, their underlying structures are identical. This means we can directly compare the coefficients of like powers on the left-hand side and right-hand side of an equation to deduce unknown values. We can think of this as a combinatorial matching process: we look at all the combinatorial ways to multiply terms from our factors to generate a specific power of λ.

For a general characteristic polynomial, the roots are the eigenvalues λ_i. We can express the determinant expansion as:

$$|\boldsymbol{A} - \lambda \boldsymbol{I}| = \prod_{i=1}^{n} (\lambda_i - \lambda) = c_n \lambda^n + c_{n-1} \lambda^{n-1} + \ldots + c_0$$

By mathematically expanding the factored product on the right, we can compare it to the terms on the left to extract the specific coefficients c_k.

Fundamental Properties of Eigenvalues

Using this strict polynomial equivalence, we can elegantly prove two extremely useful properties of eigenvalues.

1. The sum of the eigenvalues equals the trace of the matrix.

$$\sum_{i=1}^{n} \lambda_i = \text{Trace}(\boldsymbol{A}) = \sum_{i=1}^{n} a_{ii}$$

Proof: We start with the core identity $|\boldsymbol{A} - \lambda \boldsymbol{I}| = \prod_{i=1}^{n} (\lambda_i - \lambda)$ multiplied by $(-1)^n$. If we expand both sides and compare the coefficients specifically for the λ^{n-1} term, we find that the left-hand side evaluates to the trace of $\boldsymbol{A}$ (the sum of the diagonal elements), while the right-hand side evaluates to the sum of the eigenvalue roots.

A direct corollary to this property is that for any real matrix $\boldsymbol{A}$, the trace must be a purely real number. Since the sum of the eigenvalues equals this real trace, any imaginary components among the eigenvalues must perfectly cancel each other out. Therefore, complex eigenvalues must always come in conjugate pairs.

2. The product of the eigenvalues equals the determinant.

$$\prod_{i=1}^{n} \lambda_i = |\boldsymbol{A}|$$

Proof: Taking the exact same identity, we recognize that it is a statement of truth valid for any value of λ. We can completely simplify the equation by setting $\lambda = 0$. The left-hand side collapses to $|\boldsymbol{A} - 0\boldsymbol{I}| = |\boldsymbol{A}|$, and the right-hand side simplifies to the product of the roots $\prod_{i=1}^{n} \lambda_i$, instantly proving the property.

This also gives us a rapid intuition for invertibility: if a matrix is singular (meaning its determinant is zero), we know it must have at least one eigenvalue where $\lambda = 0$. In fact, the total number of zero eigenvalues is exactly equal to the nullity of the matrix (nullity $\boldsymbol{A}$), which is the size of the matrix minus its rank.


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Recap: The Story So Far

Let's think the unthinkable, let's do the undoable. Let us prepare to grapple with the ineffable itself, and see if we may not eff it after all.

—Douglas Adams



As we have seen multiple times, everything in linear algebra is intimately connected to everything else. It is a big and beautiful jigsaw puzzle. Although we chose to learn it in the particular sequence that we did, we could have started our exploration from any one of its corner pieces. Let's look at what we have learned, this time from the perspective of the interplay between the ranks and shapes of matrices and the four fundamental spaces. As we shall see, this summary will also reveal more about the process of solving equations and projecting onto column (or even row) spaces.

Given a matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, we have a mapping from $\mathbb{R}^n$ to $\mathbb{R}^m$, as shown in Figure R.1, where the input space is shown in green and the output space in red. The equation $\boldsymbol{Ax} = \boldsymbol{b}$ expresses that $\boldsymbol{b}$ is a linear combination of the columns of $\boldsymbol{A}$. All such linear combinations lie in $\mathcal{C}(\boldsymbol{A})$, which is a subspace of $\mathbb{R}^m$, the output space shown in red.

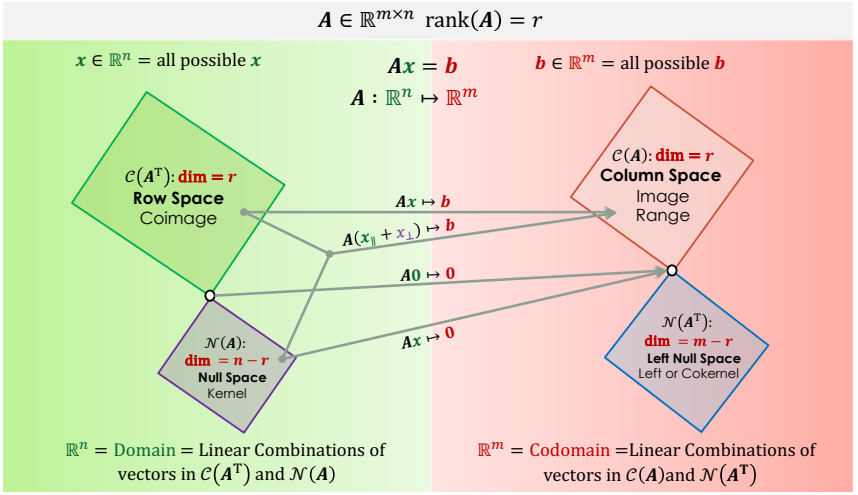


Fig. R.1 Recap of the four fundamental spaces defined by a matrix.

It is true that multiple vectors $\boldsymbol{x} \in \mathbb{R}^n$ can map to the exact same vector $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$. However, the mapping from the row space $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$ to the column space $\mathcal{C}(\boldsymbol{A})$ is strictly one-to-one. In other words, for any vector $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$, there is one and only one vector $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathbf{T})$ such that $\boldsymbol{Ax} = \boldsymbol{b}$. Why is that? Let's prove it, somewhat formally.
Proof: To establish this result, we assume the contrary and derive a contradiction. Suppose there exist two distinct, non-zero vectors $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{C}(\boldsymbol{A}^\mathbf{T})$, with $\boldsymbol{x}_1 \neq \boldsymbol{x}_2$, such that $\boldsymbol{Ax}_1 = \boldsymbol{b}$ and $\boldsymbol{Ax}_2 = \boldsymbol{b}$.

(1) Assumption:	$\boldsymbol{Ax}_1 = \boldsymbol{b}, \quad \boldsymbol{Ax}_2 = \boldsymbol{b}$
(2) Subtracting:	$\boldsymbol{A}(\boldsymbol{x}_1 - \boldsymbol{x}_2) = \mathbf{0}$
(3) Null space definition:	$(\boldsymbol{x}_1 - \boldsymbol{x}_2) \in \mathcal{N}(\boldsymbol{A})$
(4) Since $\boldsymbol{x}_1 \neq \boldsymbol{x}_2$:	$(\boldsymbol{x}_1 - \boldsymbol{x}_2) \neq \mathbf{0}$
(5) Subspace premise:	$\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{C}(\boldsymbol{A}^\mathbf{T})$
(6) Subspace closure of $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$:	$(\boldsymbol{x}_1 - \boldsymbol{x}_2) \in \mathcal{C}(\boldsymbol{A}^\mathbf{T})$
(7) Combining (3) and (6):	$(\boldsymbol{x}_1 - \boldsymbol{x}_2) \in \mathcal{C}(\boldsymbol{A}^\mathbf{T}) \cap \mathcal{N}(\boldsymbol{A})$
(8) Since $\mathcal{C}(\boldsymbol{A}^\mathbf{T}) \perp \mathcal{N}(\boldsymbol{A})$:	$(\boldsymbol{x}_1 - \boldsymbol{x}_2) = \mathbf{0}$

Statement (8) above tells us that since $\mathcal{C}(\boldsymbol{A}^{\mathbf{T}})$ and $\mathcal{N}(\boldsymbol{A})$ are orthogonal subspaces, the only vector they have in common is the zero vector. However, in statement (4), we deduced from our initial assumption that their difference is strictly non-zero. These two statements contradict each other. Therefore, for every $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$ such that $\boldsymbol{Ax} = \boldsymbol{b}$, there is one and only one vector $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^{\mathbf{T}})$ that satisfies the equation. This is another way of saying that the mapping $\mathcal{C}(\boldsymbol{A}^{\mathbf{T}}) \rightarrow \mathcal{C}(\boldsymbol{A})$ is strictly one-to-one.

Intuitively, since we have r linearly independent columns (the pivot columns) that span $\mathcal{C}(\boldsymbol{A})$, any specific non-zero linear combination:

$$\boldsymbol{b} = \sum_{i=1}^{r} x_i \boldsymbol{a}_i$$

must have a unique set of coefficients x_i. If we arrange these coefficients to form a vector $\boldsymbol{x}$, this vector must reside in the row space: $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^{\mathbf{T}})$. If it did not, where else in the input space could the vector $\boldsymbol{x}$ possibly be?

R.1 Full-Rank Square Matrices

When we have a full-rank square matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ where $\text{rank}(\boldsymbol{A}) = n$, the row space is all of $\mathbb{R}^n$. So is the column space. The mapping $\boldsymbol{A} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, which is the exact same as $\boldsymbol{A} : \mathcal{C}(\boldsymbol{A}^{\mathbf{T}}) \rightarrow \mathcal{C}(\boldsymbol{A})$, is strictly one-to-one. If we are given any $\boldsymbol{b}$ in $\boldsymbol{Ax} = \boldsymbol{b}$, we can always find the corresponding $\boldsymbol{x}$ using the inverse mapping $\boldsymbol{A}^{-\mathbf{1}}$, which is guaranteed to exist. In terms of the four fundamental spaces, the picture looks exactly like what is depicted in Figure R.2. Note that both $\mathcal{N}(\boldsymbol{A})$ and $\mathcal{N}(\boldsymbol{A}^{\mathbf{T}})$ contain only the zero vector.

Solving $\boldsymbol{Ax} = \boldsymbol{b}$ is the same as finding the inverse $\boldsymbol{A}^{-\mathbf{1}}$, either of which can be done using Gauss-Jordan elimination. This is what people typically have in mind when they say, "n equations in n unknowns means you can solve." As we shall see again soon, a more precise statement would be, "n independent and consistent equations in n unknowns means we can find a unique solution."

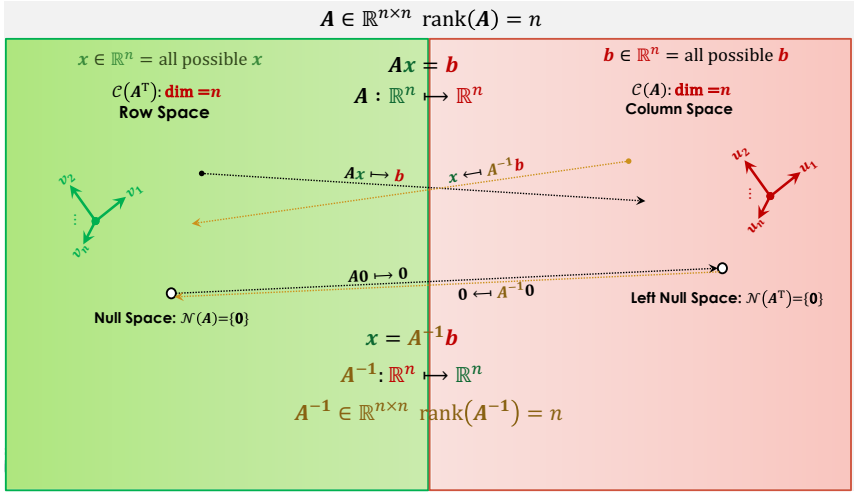


Fig. R.2 Recap of the four fundamental spaces defined by a full-rank, square matrix. Notice that the row and column spaces are, in fact, all of the input and output spaces respectively: $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}}) \equiv \mathbb{R}^n$ and $\mathcal{C}(\boldsymbol{A}) \equiv \mathbb{R}^n$. Also, $\boldsymbol{A}^{-1}$ is a mapping from $\mathcal{C}(\boldsymbol{A})$ back to $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$.

Orthonormal Bases

Also shown in Figure R.2 are the basis vectors for the input and output spaces, $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$. Since $\boldsymbol{A}$ is full rank, the column vectors will suffice as the basis for $\mathcal{C}(\boldsymbol{A})$ (and the columns of $\boldsymbol{A}^{\mathsf{T}}$ for $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$). If we would like them to be normalized, we know how to do it. And, if we would like them to be orthogonal as well, we could perform the Gram-Schmidt process. Let's say the set $\{\boldsymbol{v}_i\}$ is, in fact, such an orthonormal basis. By multiplying them with $\boldsymbol{A}$, we can get some vectors in the column space; but there is no guarantee that we would get $\{\boldsymbol{u}_i\}$. Is there a special basis $\{\boldsymbol{v}_i\}$ such that $\boldsymbol{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i$ (where σ_i is some scaling factor)?

Since the row and column spaces are both $\mathbb{R}^n$, we might as well use the columns of $\boldsymbol{I}_n$ as their basis. If we take $\boldsymbol{v}_i = \boldsymbol{e}_i$ (the standard basis vectors), what do we get upon transforming them using $\boldsymbol{A}$? We get the columns of $\boldsymbol{A}$, which are pretty far from an orthonormal basis. But we do know that if $\boldsymbol{A}$ is diagonalizable, and if its eigenvectors are real and orthogonal, we could use them as the basis, giving us $\boldsymbol{A}\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i$, which are scaled versions of $\boldsymbol{v}_i$.

When can we guarantee that the eigenvectors are real and orthogonal? When $\boldsymbol{A}$ is real and symmetric. Using a different symbol (σ_i instead of λ_i and setting $\boldsymbol{u}_i = \boldsymbol{v}_i$), we can write:

$$\boldsymbol{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{v}_i \implies \{\boldsymbol{A}\boldsymbol{v}_i\} = \{\sigma_i \boldsymbol{u}_i\} \quad \text{or} \quad \boldsymbol{A}\boldsymbol{V} = \boldsymbol{U}\boldsymbol{\Sigma}$$

where $\boldsymbol{V}$ and $\boldsymbol{U}$ are orthonormal basis matrices with $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$ as their respective columns, and $\boldsymbol{\Sigma}$ is the diagonal matrix of scaling factors σ_i.

R.2 Full-Column-Rank, Tall Matrices

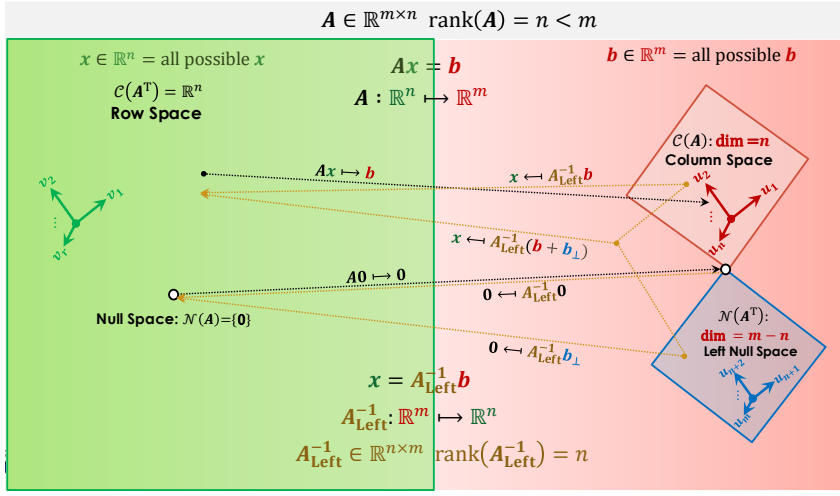


Fig. R.3 Recap of the four fundamental spaces defined by a full-column-rank, tall matrix. Here, the row space is all of the input space: $\mathcal{C}(\boldsymbol{A}^\mathsf{T}) \equiv \mathbb{R}^n$, while the column space is a proper subspace of the output space: $\mathcal{C}(\boldsymbol{A}) \subset \mathbb{R}^m$. $\boldsymbol{A}_{\text{Left}}^{-1}$ is the one-to-one mapping $\mathcal{C}(\boldsymbol{A}) \to \mathcal{C}(\boldsymbol{A}^\mathsf{T})$.

Figure R.3 shows the four fundamental subspaces of a full-column-rank, tall matrix. Notice that the null space $\mathcal{N}(\boldsymbol{A})$ is trivial (meaning it contains only the zero vector), and the row space is all of the input space: $\mathcal{C}(\boldsymbol{A}^\mathsf{T}) = \mathbb{R}^n$. This means that any vector in the input space is mapped to a unique vector in $\mathcal{C}(\boldsymbol{A})$. Why unique? Because in $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, the vector $\boldsymbol{b} \in \mathbb{R}^m$ is a linear combination of the columns of $\boldsymbol{A}$, and any linear combination of linearly independent vectors (with a given set of coefficients) is unique. In other words, given a set of

n linearly independent vectors $\boldsymbol{a}_i$ and scalars x_i, the sum $\sum x_i \boldsymbol{a}_i$ is a unique vector. Of course, this sum is exactly equivalent to $\boldsymbol{Ax}$.

The column space $\mathcal{C}(\boldsymbol{A})$, however, does not cover all of the output space $\mathbb{R}^m$. Therefore, if we take a vector $\boldsymbol{b} \in \mathbb{R}^m$ that is not in the column space ($\boldsymbol{b} \notin \mathcal{C}(\boldsymbol{A})$), it is not a linear combination of the columns of $\boldsymbol{A}$, and there is no solution for $\boldsymbol{x}$ such that $\boldsymbol{Ax} = \boldsymbol{b}$. The system of linear equations is inconsistent. If we perform Gaussian elimination on the augmented matrix $[\boldsymbol{A} \mid \boldsymbol{b}]$, we will get at least one row of the form $0 =$ non-zero, indicating this inconsistency. In short, for $\boldsymbol{Ax} = \boldsymbol{b}$ with a general $\boldsymbol{b} \in \mathbb{R}^m$: no solution exists if $\boldsymbol{b} \notin \mathcal{C}(\boldsymbol{A})$.

We also know that we can find the best approximation to the solution, $\hat{\boldsymbol{x}}$, by projecting $\boldsymbol{b}$ onto $\mathcal{C}(\boldsymbol{A})$ as $\hat{\boldsymbol{b}}$. In this case, $\boldsymbol{A}\hat{\boldsymbol{x}} = \hat{\boldsymbol{b}}$ has a unique solution because $\hat{\boldsymbol{b}} \in \mathcal{C}(\boldsymbol{A})$ by construction. This process is known as least-squares minimization because $\hat{\boldsymbol{b}}$ is the vector in $\mathcal{C}(\boldsymbol{A})$ "closest" to $\boldsymbol{b}$. In other words, the error vector $\boldsymbol{b} - \hat{\boldsymbol{b}}$ has the smallest Euclidean norm.

We now have a recipe for finding the best approximation: project $\boldsymbol{b}$ onto $\mathcal{C}(\boldsymbol{A})$ as $\hat{\boldsymbol{b}} = \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{b}$ and solve $\boldsymbol{A}\hat{\boldsymbol{x}} = \hat{\boldsymbol{b}}$. This yields:

$$\boldsymbol{A}\hat{\boldsymbol{x}} = \hat{\boldsymbol{b}} \implies \boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{b}$$

Notice that this equation states that a linear combination of the columns of $\boldsymbol{A}$ is identical to another linear combination of the same columns. Since linear combinations of linearly independent vectors are unique, we conclude:

$$\hat{\boldsymbol{x}} = (\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{b}$$

We also saw in our earlier discussion that even when $\boldsymbol{Ax} = \boldsymbol{b}$ is not solvable, the equation $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^\mathsf{T}\boldsymbol{b}$ (the normal equation) always has solutions, which leads to the same result:

$$\hat{\boldsymbol{x}} = (\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{b}$$

Finally, recall the left inverse, defined for a full-column-rank tall matrix: $\boldsymbol{A}_{\text{Left}}^{-1} = (\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}$, which satisfies $\boldsymbol{A}_{\text{Left}}^{-1}\boldsymbol{A} = \boldsymbol{I}$. Therefore, when we have $\boldsymbol{Ax} = \boldsymbol{b}$:

$$\boldsymbol{A}_{\text{Left}}^{-1}\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}_{\text{Left}}^{-1}\boldsymbol{b} \implies \hat{\boldsymbol{x}} = \boldsymbol{A}_{\text{Left}}^{-1}\boldsymbol{b} = (\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{b}$$

We use a hat on $\hat{\boldsymbol{x}}$ because the system $\boldsymbol{Ax} = \boldsymbol{b}$ does not have solutions in general unless $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$. What we are calculating is the

best approximation to the solution, which is equivalent to projecting $\boldsymbol{b}$ onto $\mathcal{C}(\boldsymbol{A})$. In summary, for $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ with $\text{rank}(\boldsymbol{A}) = n < m$: $\boldsymbol{A}^{-1}$ is not defined, and generally, no solution exists to $\boldsymbol{Ax} = \boldsymbol{b}$. We do possess a least-squares solution, which represents the best approximation. Keep in mind that eigenanalysis is not possible here because the matrix $\boldsymbol{A}$ is not square.

R.3 Full-Row-Rank, Wide Matrices

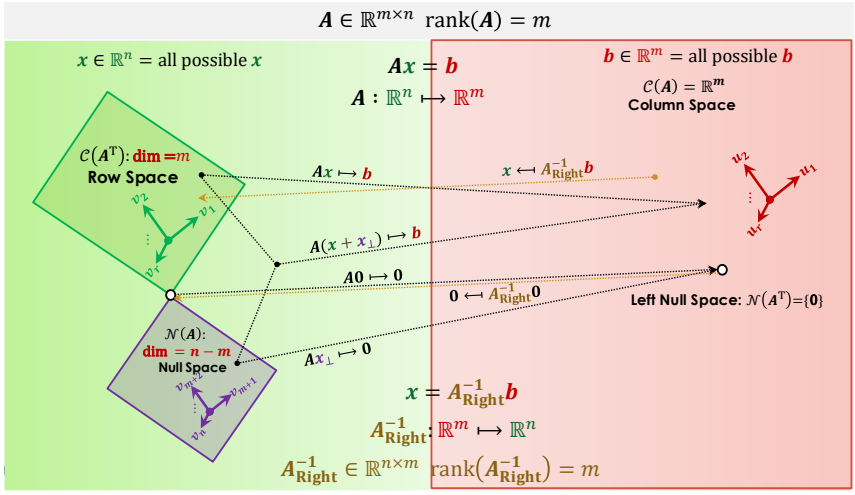


Fig. R.4 Recap of the four fundamental spaces defined by a full-row-rank, wide matrix, showing $\boldsymbol{A}_{\text{Right}}^{-1} : \mathcal{C}(\boldsymbol{A}) \to \mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$. For this matrix, it is the column space that encompasses all of the output space: $\mathcal{C}(\boldsymbol{A}) \equiv \mathbb{R}^m$.

A full-row-rank, wide matrix is indeed the transpose of a full-column-rank, tall matrix. In terms of the four fundamental spaces, as shown in Figure R.4, it is roughly equivalent to swapping the input and output spaces. We might expect to see the right inverse in place of the left inverse, as the mapping from the column space back to the row space. Let's see how it comes about.

For a full-row-rank, wide matrix, its column space is all of the output space: $\mathcal{C}(\boldsymbol{A}) = \mathbb{R}^m$. (See Figure R.4.) The row space is a subspace of the input space: $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}}) \subset \mathbb{R}^n$ and there is a null space $\mathcal{N}(\boldsymbol{A})$, which contains all vectors that are orthogonal to those in the row space.

The mapping $\boldsymbol{A} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is many-to-one: multiple vectors in the input space map to the same output vector. However, the mapping from the row space to the column space, $\boldsymbol{A} : \mathcal{C}(\boldsymbol{A}^\mathsf{T}) \rightarrow \mathcal{C}(\boldsymbol{A})$, is still one-to-one, for the same reasons of uniqueness of linear combinations we expounded on in the previous case. In other words, given any vector $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$ (which is the same as saying $\boldsymbol{b} \in \mathbb{R}^m$ because $\mathcal{C}(\boldsymbol{A}) = \mathbb{R}^m$), there is a unique vector $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$ such that $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.

R.3.1 Why Many-to-One?

We know the answer already: We do not have enough equations to constrain the system to a single solution. If we have one equation ($a_{11}x + a_{12}y = b_1$) in two variables, we have a line and any point (x, y) on the line will map to the same b_1.

We can also see this geometrically. Let's take a vector $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$, which maps to $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$ through $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$. Now take another vector in the input space that is in the null space, $\mathcal{N}(\boldsymbol{A})$. Remembering that the null space is the orthogonal complement of the row space, we call this vector $\boldsymbol{x}_\perp \in \mathcal{N}(\boldsymbol{A})$. We know, by the definition of $\mathcal{N}(\boldsymbol{A})$ as the solution set of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0}$, $\boldsymbol{A}\boldsymbol{x}_\perp = \boldsymbol{0}$. Therefore, $\boldsymbol{A}\boldsymbol{x} + \boldsymbol{A}\boldsymbol{x}_\perp = \boldsymbol{b} + \boldsymbol{0} = \boldsymbol{b}$ or, $\boldsymbol{A}(\boldsymbol{x} + \boldsymbol{x}_\perp) = \boldsymbol{b}$. In other words, the moment we have one vector $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$ such that $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, we can add any vector $\boldsymbol{x}_\perp \in \mathcal{N}(\boldsymbol{A})$ to it and the sum $\boldsymbol{x} + \boldsymbol{x}_\perp$ will still map to the same $\boldsymbol{b}$.

One point to note is that the sum $\boldsymbol{x} + \boldsymbol{x}_\perp$ will always have a norm greater than or equal to that of $\boldsymbol{x}$: $\|\boldsymbol{x} + \boldsymbol{x}_\perp\| \geq \|\boldsymbol{x}\|$ because $\boldsymbol{x}$ and $\boldsymbol{x}_\perp$ form two sides of a right-angled triangle and the sum $\boldsymbol{x} + \boldsymbol{x}_\perp$ is the hypotenuse. In other words, the unique solution $\boldsymbol{x}$ is the *minimum-norm solution* of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.

R.3.2 How to Find the Unique Part?

We actually know the answer to this one as well. When we found the complete solution of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, where we have free variables, we wrote it as $\boldsymbol{x}_p + t_1\boldsymbol{x}_{s1} + t_2\boldsymbol{x}_{s2} + \cdots$, where $\boldsymbol{x}_p$ was the particular solution and $\boldsymbol{x}_{si}$ were the special solutions (of which we had as many as the number of free variables). Now we can see that $\boldsymbol{x}_p$ is, in fact, the part of the solution that lies in the row space. Any linear combination of the $n - r$ special solutions (which form a basis for the null space) would indeed be in the null space, and that is what

is denoted as $\boldsymbol{x}_\perp$ in Figure R.4. We can see how all these different views are coming together beautifully, can't we?

One way of finding the particular solution is to perform Gaussian Elimination on the augmented matrix to locate the pivotless columns, which point to the free variables. We then solve the system after setting the free variables to zero. Now we have r equations and r unknowns, because we have set the $n - r$ free variables to zero. The unique solution to this system is the particular solution $\boldsymbol{x}_p$.

Thinking geometrically, we first notice that the complete solution is the sum $\boldsymbol{x} + \boldsymbol{x}_\perp$. If we project this sum onto the row space $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$, it becomes just $\boldsymbol{x}$, because $\boldsymbol{x}_\perp$ is orthogonal to $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$. When we did the projection, we wrote the matrix that projects onto $\mathcal{C}(\boldsymbol{A})$ as $\boldsymbol{P} = \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T}$. The row space $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$ is, as the symbol indicates, the column space of $\boldsymbol{A}^\mathsf{T}$ and the matrix that would project onto it, $\boldsymbol{P}_{\text{RS}}$ would be just $\boldsymbol{P}$, but with $\boldsymbol{A}$ replaced by $\boldsymbol{A}^\mathsf{T}$. Thus, $\boldsymbol{P}_{\text{RS}} = \boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{A}^\mathsf{T})^{-1}\boldsymbol{A}$ (where we also made use of the fact that the transpose of $\boldsymbol{A}^\mathsf{T}$ is $\boldsymbol{A}$).

Let's say we found a solution $\boldsymbol{x}' = \boldsymbol{x} + \boldsymbol{x}_\perp$ such that $\boldsymbol{A}\boldsymbol{x}' = \boldsymbol{b}$. With $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$ and $\boldsymbol{x}_\perp \in \mathcal{N}(\boldsymbol{A})$, we can see that $\boldsymbol{P}_{\text{RS}}\boldsymbol{x} = \boldsymbol{x}$ and $\boldsymbol{P}_{\text{RS}}\boldsymbol{x}_\perp = \boldsymbol{0}$. Then $\boldsymbol{x} = \boldsymbol{P}_{\text{RS}}\boldsymbol{x}' = \boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{A}^\mathsf{T})^{-1}\boldsymbol{A}\boldsymbol{x}'$. But $\boldsymbol{A}\boldsymbol{x}' = \boldsymbol{b}$, which means we get the minimum-norm solution, the part of $\boldsymbol{x}'$ in $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$, $\boldsymbol{x} = \boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{A}^\mathsf{T})^{-1}\boldsymbol{b}$.

Let's take a step back and remind ourselves what the right inverse is. $\boldsymbol{A}$ is a full-row-rank, wide matrix. Therefore, $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$ is full rank and invertible, which means $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{A}^\mathsf{T})^{-1} = \boldsymbol{I} \implies \boldsymbol{A}_{\textbf{Right}}^{-1} = \boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{A}^\mathsf{T})^{-1}$. Now we see that $\boldsymbol{A}_{\textbf{Right}}^{-1}$ is the mapping that takes any vector $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$ and gives us the unique vector $\boldsymbol{x} \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$ such that $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$. This is shown in Figure R.4.

Notice another interesting fact: The projection matrix, the one that projects to $\mathcal{C}(\boldsymbol{A})$ is:

$$\boldsymbol{P} = \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})^{-1}\boldsymbol{A}^\mathsf{T} = \boldsymbol{A}\boldsymbol{A}_{\textbf{Left}}^{-1}$$

and the one that projects to $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$ is:

$$\boldsymbol{P}_{\text{RS}} = \boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{A}^\mathsf{T})^{-1}\boldsymbol{A} = \boldsymbol{A}_{\textbf{Right}}^{-1}\boldsymbol{A}$$

Beautifully symmetric, isn't it?

As we went through these cases of full-rank matrices, square, tall or wide, we saw how solving the system of linear equations using

Gaussian Elimination is related to the picture of the four fundamental subspaces, and how the mapping between the row space and the column space was one-to-one and could be inverted. We further saw that the left and right inverses were in fact these mappings from $\mathcal{C}(\boldsymbol{A})$ to $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$. In the case of a full-rank, square matrix, they indeed reduce to the double-sided inverse, $\boldsymbol{A}^{-1}$.

In summary, here is what we have seen in the wide matrix case:

- A full-row-rank, wide matrix does not have a standard inverse or admit eigenvalue decomposition. The rank is equal to the number of rows, which is less than the number of columns.
- For any right-hand side vector, the system is always consistent because the column space spans the entire output space.
- The minimum-norm solution is the best choice and can be found by projecting any solution onto the row space.
- Any solution to the system can be written as the sum of a unique particular solution (lying in the row space) and a null-space component (which accounts for the degrees of freedom).
- The particular solution is obtained by setting all free variables to zero and lies entirely in the row space.
- The projection matrix that gives the row-space component is the familiar form, but with the original matrix replaced by its transpose.
- This projection uses the right inverse, and it annihilates the null-space component, since the row space and null space are orthogonal.

R.4 Any General Matrix (Optional Reading)

We have not yet dealt with rank-deficient matrices. Even if the matrix is rank-deficient, the mapping between the row space and the column space is one-to-one and invertible: If $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, then the mapping $\boldsymbol{A} : \mathcal{C}(\boldsymbol{A}^\mathsf{T}) \to \mathcal{C}(\boldsymbol{A})$ is invertible such that there is some pseudo-inverse $\boldsymbol{A}^+ : \mathcal{C}(\boldsymbol{A}) \to \mathcal{C}(\boldsymbol{A}^\mathsf{T})$. But we do not have all the tools necessary to unearth this elusive inverse yet. We need to learn

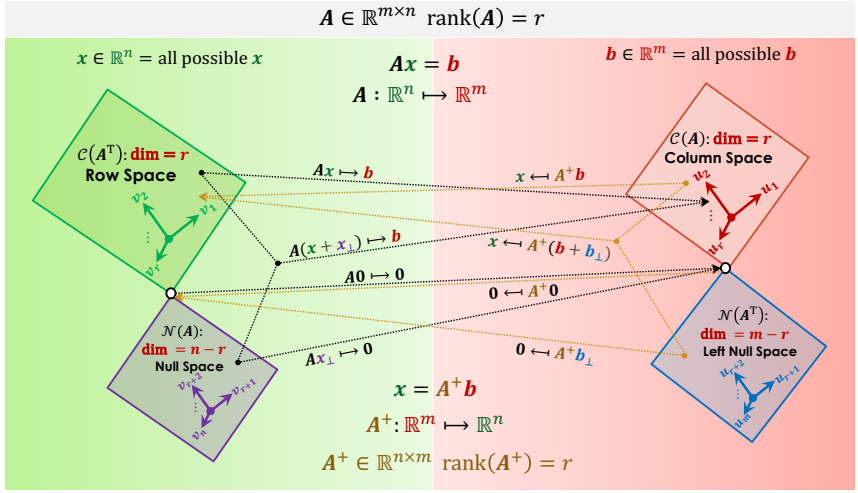


Fig. R.5 The four fundamental spaces defined by a general rank-deficient matrix. Here, the newly defined pseudo-inverse maps $\mathcal{C}(\boldsymbol{A})$ back to $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$.

Singular Value Decomposition, which we will go through in the very last chapter of this book. For now, let's do a quick preview, which may be useful.

In the previous chapter, we learned that eigenanalysis is possible only for square matrices. It works best for symmetric matrices that are diagonalizable. The "thin" version of Singular Value Decomposition, which is closely related to the eigenanalysis of $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ or $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$, works for all matrices because these products are symmetric. What it gives is, in fact, a decomposition of the form:

$$\boldsymbol{A} \in \mathbb{R}^{m\times n} \xrightarrow{\text{SVD}} \boldsymbol{A} = \hat{\boldsymbol{U}}\hat{\boldsymbol{\Sigma}}\hat{\boldsymbol{V}}^\mathsf{T}$$
$$\hat{\boldsymbol{U}} \in \mathbb{R}^{m\times r},\ \hat{\boldsymbol{\Sigma}} \in \mathbb{R}^{r\times r},\ \hat{\boldsymbol{V}} \in \mathbb{R}^{r\times n}$$

$\hat{\boldsymbol{\Sigma}}$ is a diagonal matrix with positive values along the main diagonal, starting from σ_1 all the way to σ_r, where $r = \text{rank}(\boldsymbol{A})$, with the rest of the elements all zero.

In terms of the four fundamental spaces, what SVD is telling us is that there is a special orthonormal basis in the row space, $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_r\}$, that maps to an orthonormal basis in the column space $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_r\}$. The very fact that there are such bases is already remarkable. The fact that these bases can be unearthed is even more so. We will later see how the basis matrices $\hat{\boldsymbol{U}}$ and $\hat{\boldsymbol{V}}$,

as well as the matrix of singular values $\hat{\boldsymbol{\Sigma}}$, come into play. For our purposes here, let's note the following:

$$\hat{\boldsymbol{\Sigma}} = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \end{bmatrix} \implies \hat{\boldsymbol{\Sigma}}^{-\mathbf{1}} = \begin{bmatrix} \frac{1}{\sigma_1} & 0 & \cdots & 0 \\ 0 & \frac{1}{\sigma_2} & \cdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{\sigma_r} \end{bmatrix}$$

With this, we now define the pseudo-inverse $\boldsymbol{A}^{+} = \hat{\boldsymbol{V}}\hat{\boldsymbol{\Sigma}}^{-\mathbf{1}}\hat{\boldsymbol{U}}^{\mathsf{T}}$. Note that $\boldsymbol{A}^{+} \in \mathbb{R}^{n \times m}$.

If we knew that $\boldsymbol{A}\boldsymbol{A}^{+}\boldsymbol{A} = \boldsymbol{A}$, we could do the following:

$$\begin{aligned} \boldsymbol{A}\boldsymbol{A}^{+}\boldsymbol{A}\boldsymbol{x} &= \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \\ \boldsymbol{A}\boldsymbol{A}^{+}(\boldsymbol{A}\boldsymbol{x}) &= \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \\ \boldsymbol{A}\boldsymbol{A}^{+}\boldsymbol{b} &= \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \end{aligned}$$

Since we already have $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, we can say (using the arguments about the uniqueness of linear combinations of linearly independent vectors again) that $\boldsymbol{A}^{+}\boldsymbol{b} = \boldsymbol{x}$, which is the inverse transformation from $\mathcal{C}(\boldsymbol{A})$ to $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$. We have to verify that $\boldsymbol{A}\boldsymbol{A}^{+}\boldsymbol{A} = \boldsymbol{A}$, which is a property of the pseudo-inverse. Furthermore, $\boldsymbol{A}^{+}$ reduces to $\boldsymbol{A}^{-\mathbf{1}}$, $\boldsymbol{A}_{\mathbf{Left}}^{-\mathbf{1}}$ or $\boldsymbol{A}_{\mathbf{Right}}^{-\mathbf{1}}$ for full-rank square, full-column-rank and full-row-rank matrices respectively. As we defined it here, we do not have enough knowledge to prove it yet, but we will prove it in the last chapter, considering the full SVD (rather than the "thin" version we used in our definition of the pseudo-inverse $\boldsymbol{A}^{+}$ above).

The only thing left to do now is to look at the picture in Figure R.5 and marvel at its mathematical beauty, conceptual elegance, and theoretical completeness. In some sense, we have come full circle. We can now appreciate how the algebraic notion of solving equations (using Gauss-Jordan elimination, for instance) is related to the geometric view of the four fundamental spaces, and the more abstract ideas of mappings and their inversions.

12

Special Matrices, Similarity, and Algorithms

The surest way to corrupt a youth is to instruct him to hold in higher esteem those who think alike than those who think differently.

—Friedrich Nietzsche



In Chapter 11, we saw how the properties of eigenvalues and eigenvectors are related to the characteristics of the matrices. We looked at real, symmetric matrices and studied their eigenanalysis to some extent. In this chapter, we will look at some other special matrices and their eigenvalues and vectors. Let's start with another look at symmetric matrices.

12.1 Symmetric and Hermitian Matrices

12.1.1 The Real Case

Real, symmetric matrices ($\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}$) have real eigenvalues and orthogonal eigenvectors, as we proved in painstaking detail in the previous chapter (Item §3, page 217). Here are two theorems on such matrices.

Spectral Theorem

As we (almost) saw, a real symmetric matrix has real, orthonormal (sometimes just referred to as orthogonal) eigenvectors, restricting ourselves to $\mathbb{R}$ again. Using the symbol $\boldsymbol{Q}$ for orthonormal matrices, and knowing that they are invertible with the inverse being the transpose, we can state our eigenvalue decomposition as:

$$\boldsymbol{A} = \boldsymbol{S}\boldsymbol{\Lambda}\boldsymbol{S}^{-1} \iff \boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{-1} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{\mathsf{T}}$$

Knowing that $\boldsymbol{Q}$ is a matrix with columns $\boldsymbol{q}_i$ (which we may write as $\boldsymbol{Q} = [\boldsymbol{q}_i]$), we can expand the product to read:

$$\boldsymbol{A} = \sum_{i=1}^{n} \lambda_i \boldsymbol{q}_i \boldsymbol{q}_i^{\mathsf{T}} \tag{12.1}$$

This expansion is possible because $\boldsymbol{\Lambda}$ has only diagonal entries λ_i and $\boldsymbol{q}_i \perp \boldsymbol{q}_j \implies \boldsymbol{q}_i^{\mathsf{T}}\boldsymbol{q}_j = 0$ if $i \neq j$. Looking at each term in the expansion, we can make the following remarks:

- Each term has a projection matrix $\boldsymbol{q}_i\boldsymbol{q}_i^{\mathsf{T}}$.
- It is a projection onto the eigenspace of the ith eigenvector.
- Each eigenspace is one-dimensional, being the span of just one vector $\boldsymbol{q}_i$.
- Consequently, each projection matrix is a rank-one matrix.

Eqn (12.1) is the spectral theorem, stating that any real, symmetric matrix can be decomposed as a sum of projection matrices of rank one, scaled by the eigenvalue. The eigenvalues are considered the spectrum. Each component in the term is akin to a pure component of the matrix, much like white light has a spectrum of pure primary colors.

CS Connection


Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

Sylvester's Law of Inertia

Now that we are listing theorems, we have another one that goes by the physics-inspired name, the Law of Inertia (attributed to James Joseph Sylvester, not the brawny movie star). We stated it earlier: For real, symmetric matrices, the number of positive eigenvalues is the same as the number of positive pivots. Similarly, the negative pivots and negative eigenvalues are equal in number.

The proof of this theorem (which we will not attempt) involves the concept of similarity of matrices, which is our topic later in this chapter. We will, however, state the theorem more formally.

We have a symmetric matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ so that $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}$, with $\boldsymbol{A}\boldsymbol{s_i} = \lambda_i \boldsymbol{s_i}$. The Reduced Echelon Form (REF) of $\boldsymbol{A}$ is $\boldsymbol{R}$, with the pivot in the ith row being r_i. Then, Sylvester's Law of Inertia states that:

$$\begin{aligned}
\text{Count}(\lambda_i > 0) &= \text{Count}(r_i > 0) \\
\text{Count}(\lambda_i = 0) &= \text{Count}(r_i = 0) \\
\text{Count}(\lambda_i < 0) &= \text{Count}(r_i < 0)
\end{aligned}$$

Note that the REF in this law is the result of Gaussian elimination, done with no scaling of the rows. It is not the RREF from Gauss-Jordan elimination, which makes all pivots one by scaling rows.

CS Connection



Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

CS Connections are available in the Full Version of the book.

12.1.2 The Complex Case

Some of the properties of real, symmetric matrices we stated so far apply also to complex matrices, with one important caveat: We need to redefine what "transpose" means for $\boldsymbol{A} \in \mathbb{C}^{n \times n}$. Let's expand our field to complex numbers and see how the properties and their proofs hold up. But before doing that, we have to state what "transpose" means in $\mathbb{C}$.

Conjugate Transpose and General Symmetry

Earlier, we defined the (Euclidean) norm of $\boldsymbol{x} \in \mathbb{R}^n$ as $\|\boldsymbol{x}\|^2 = \boldsymbol{x}^\mathsf{T}\boldsymbol{x}$. The norm stands for the length or the size of the vector, and we would like it to be real (and positive). In other words, we would like $\|\boldsymbol{x}\| \in \mathbb{R}$ even if $\boldsymbol{x} \in \mathbb{C}^n$.

When x_i is complex (in the form $a + ib$ with $b \neq 0$), x_i^2 is not real, and $\boldsymbol{x}^\mathsf{T}\boldsymbol{x}$ is not real. What is always real and positive is $(a + ib)(a - ib) = a^2 + b^2$. Here, $a - ib$ is the complex conjugate of $a + ib$, which we write as $(a + ib)^* = a - ib$. So we would like to have $x_i^* x_i$ in $\|\boldsymbol{x}\|^2$ rather than x_i^2, which means the right definition of the norm of $\boldsymbol{x} \in \mathbb{C}^n$ is $\|\boldsymbol{x}\|^2 = \boldsymbol{x}^{*\mathsf{T}}\boldsymbol{x}$.

$\boldsymbol{x}^{*\mathsf{T}}$ is the *conjugate transpose*, also known as the *Hermitian transpose*, of $\boldsymbol{x}$, which we will write[1] as $\boldsymbol{x}^\dagger = \boldsymbol{x}^{*\mathsf{T}}$. Note that it is a generalization of $\boldsymbol{x}^\mathsf{T}$: For $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{x}^\dagger = \boldsymbol{x}^\mathsf{T}$. We may come across yet another notation in some texts for the Hermitian transpose as $\boldsymbol{x}^\mathrm{H}$.

Similarly, we can define the complex conjugate transpose (AKA Hermitian transpose) of a matrix $\boldsymbol{A} \in \mathbb{C}^{n\times n}$ as its generalized transpose, $\boldsymbol{A}^\dagger$. When $\boldsymbol{A}^\dagger = \boldsymbol{A}$, we have the generalized version of symmetry, and we call such matrices Hermitian.

Note that the product rule of transposes applies to the Hermitian transposes as well: $(\boldsymbol{AB})^\dagger = \boldsymbol{B}^\dagger\boldsymbol{A}^\dagger$.

Earlier, while discussing orthogonality, we stated that the inverse of an orthogonal matrix (in $\mathbb{R}^{n\times n}$) is its transpose. For $\boldsymbol{A} \in \mathbb{C}^{n\times n}$, the condition would be $\boldsymbol{A}^\dagger = \boldsymbol{A}^{-1}$, and we call such matrices **unitary**. When we look up information, we may come across "unitary" used as a synonym for "orthogonal," which is technically correct because the set of orthogonal matrices is a subset of the set of unitary matrices, just as $\mathbb{R} \subset \mathbb{C}$.

12.1.3 Eigenvalue Properties

With this definition of generalized symmetry of matrices, we are ready to restate some of the properties of eigenvalues and eigenvectors we

[1] Some people write $\boldsymbol{x}^*$ to mean both conjugation on top of transposition. For this reason, a less confusing notation for the conjugate by itself may be an overline $\overline{a + ib} = a - ib$, but it may lead to another contextual confusion: Are we underlining the line above or overlining the variable below? Good or bad, we are going to stick with $*$ for complex conjugate and † for conjugate transpose.

listed for real, symmetric matrices, and extend them to Hermitian matrices. Note, however, that the field of complex numbers, $\mathbb{C}$, is not critical for our use in computer science, except when we look for information on the internet. For instance, we may come across terminology and explanations stated in terms of Hermitian, unitary, complex conjugates, etc. Such usage is common in the research literature as well.

1. The eigenvalues of Hermitian matrices are real:

$$\boldsymbol{A} \in \mathbb{C}^{n\times n},\ \boldsymbol{A}^\dagger = \boldsymbol{A},\ \boldsymbol{A}\boldsymbol{s} = \lambda\boldsymbol{s} \implies \lambda \in \mathbb{R}$$

Proof:

(1) By the definition of eigenvalues:	$\boldsymbol{A}\boldsymbol{s} = \lambda\boldsymbol{s}$
(2) Multiplying on the left with $\boldsymbol{s}^\dagger$:	$\boldsymbol{s}^\dagger\boldsymbol{A}\boldsymbol{s} = \lambda\boldsymbol{s}^\dagger\boldsymbol{s}$
(3) Product rule of Hermitian transposes:	$(\boldsymbol{A}^\dagger\boldsymbol{s})^\dagger\boldsymbol{s} = \lambda\boldsymbol{s}^\dagger\boldsymbol{s}$
(4) Since $\boldsymbol{A}$ is Hermitian:	$(\boldsymbol{A}\boldsymbol{s})^\dagger\boldsymbol{s} = \lambda\boldsymbol{s}^\dagger\boldsymbol{s}$
(5) Using (1):	$(\lambda\boldsymbol{s})^\dagger\boldsymbol{s} = \lambda\boldsymbol{s}^\dagger\boldsymbol{s}$
(6) Since λ is a scalar:	$\lambda^*\boldsymbol{s}^\dagger\boldsymbol{s} = \lambda\boldsymbol{s}^\dagger\boldsymbol{s}$
(7) Since $\boldsymbol{s}^\dagger\boldsymbol{s}$ is never zero:	$\lambda^* = \lambda$

Step (7) says λ is real. Since all eigenvalues are real, the $\boldsymbol{\Lambda}$ matrix (with eigenvalues on the diagonal) is Hermitian as well.

In fact, the proof we gave in §3 (page 217) holds for Hermitian matrices as well, with minor changes. We, however, provided a brand-new proof, now that we are in the happy position of being able to do the same thing in multiple ways.

2. The eigenvectors of Hermitian matrices are orthogonal:

$$\boldsymbol{A} \in \mathbb{C}^{n\times n},\ \boldsymbol{A}^\dagger = \boldsymbol{A},\ \boldsymbol{A}\boldsymbol{s_i} = \lambda_i\boldsymbol{s_i} \quad i \neq j \implies \boldsymbol{s_i} \perp \boldsymbol{s_j}$$

Note that the eigenvectors of a complex matrix are, in general, complex: $\boldsymbol{s_i}, \boldsymbol{s_j} \in \mathbb{C}^n$. The proof is pretty much identical to the one given in the case of real, symmetric matrices (§2, page 220), but with minor changes to call transposes Hermitians.

Here is another, higher-level, matrix-algebra way of proving it:

(1) Eigenvector matrix: $\boldsymbol{A} = \boldsymbol{S}\boldsymbol{\Lambda}\boldsymbol{S}^{-1}$

(2) Taking the Hermitian transpose: $\boldsymbol{A}^\dagger = \boldsymbol{S}^{-1^\dagger}\boldsymbol{\Lambda}^\dagger\boldsymbol{S}^\dagger$

(3) Since $\boldsymbol{A}$ and $\boldsymbol{\Lambda}$ are Hermitian: $\boldsymbol{A} = \boldsymbol{S}^{-1^\dagger}\boldsymbol{\Lambda}\boldsymbol{S}^\dagger$

(4) Equating the RHS of (1) and (3): $\boldsymbol{S}\boldsymbol{\Lambda}\boldsymbol{S}^{-1} = \boldsymbol{S}^{-1^\dagger}\boldsymbol{\Lambda}\boldsymbol{S}^\dagger$

One way this can be true is if $\boldsymbol{S}^\dagger = \boldsymbol{S}^{-1}$ for $\boldsymbol{S} \in \mathbb{C}^{n\times n}$, which is the same as saying $\boldsymbol{S}^\mathsf{T} = \boldsymbol{S}^{-1}$ for $\boldsymbol{S} \in \mathbb{R}^{n\times n}$. If the conjugate transpose of a matrix is its inverse, then the matrix is unitary (the complex equivalent of orthogonal). We have not actually proven it because we are not sure at this point whether the *only way* step (4) can be true is if $\boldsymbol{S}^\dagger = \boldsymbol{S}^{-1}$. Of course it is, but we cannot yet see why.

12.2 Positive Definite and Gram Matrices

We saw that real, symmetric matrices were "good" matrices to work with because they have real eigenvalues and orthogonal eigenvectors. *Positive definite matrices* are even better.

12.2.1 Definitions and Tests

Positive Definite Matrix

Definition: A real, symmetric matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ is positive definite if all its eigenvalues are strictly positive ($\lambda_i > 0$).

If an eigenvalue is equal to zero (which means $\boldsymbol{A}$ is singular), then we call the matrix positive semidefinite. In other words, if we only have $\lambda_i \geq 0$, then all we can say is that $\boldsymbol{A}$ is positive semidefinite.

Similarly, we can define negative definite and negative semidefinite matrices, although they have less common mathematical relevance in our domain.

We can extend the definition to $\boldsymbol{A} \in \mathbb{C}^{n\times n}$ using our definition of the Hermitian transpose, and most of what we learn here will apply

to them as well. However, for our purposes in computer science, we can safely restrict ourselves to the real domain.

1. $\lambda_i > 0$**?** This test follows directly from our definition of positive definite matrices.

2. **Are all pivots** > 0**?** The second test is essentially the same as the first, due to Sylvester's Law connecting the signs of the pivots to the signs of the eigenvalues.

3. $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} > 0$**?** For a positive definite matrix $\boldsymbol{A}$ and for any non-zero vector $\boldsymbol{x}$, the product $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x}$ is strictly greater than zero.

   Test (3) is a powerful one. In fact, some textbooks use this statement as the primary definition of positive definiteness. Let's prove that it is equivalent to Test (1), which is our working definition.

   Proof: $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} > 0 \implies \lambda_i > 0$

   (1) Consider an eigenvector of $\boldsymbol{A}$: $\boldsymbol{A}\boldsymbol{s} = \lambda\boldsymbol{s}$

   (2) Left-multiplying with $\boldsymbol{s}^\mathsf{T}$: $\boldsymbol{s}^\mathsf{T}\boldsymbol{A}\boldsymbol{s} = \lambda\boldsymbol{s}^\mathsf{T}\boldsymbol{s}$

   (3) Since $\boldsymbol{s} \neq \boldsymbol{0}, \boldsymbol{s}^\mathsf{T}\boldsymbol{s} > 0$: $\lambda = \dfrac{\boldsymbol{s}^\mathsf{T}\boldsymbol{A}\boldsymbol{s}}{\boldsymbol{s}^\mathsf{T}\boldsymbol{s}}$

   (4) Since $\boldsymbol{s} \neq \boldsymbol{0}, \boldsymbol{s}^\mathsf{T}\boldsymbol{A}\boldsymbol{s} > 0$: $\lambda > 0$

   Proof: $\lambda_i > 0 \implies \boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} > 0$

   (1) For a real symmetric $\boldsymbol{A}$: $\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\mathsf{T}$

   (2) Multiply with $\boldsymbol{x}^\mathsf{T}$ and $\boldsymbol{x}$: $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{x}^\mathsf{T}\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\mathsf{T}\boldsymbol{x}$

   (3) Calling $\boldsymbol{y} = \boldsymbol{Q}^\mathsf{T}\boldsymbol{x}$: $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{y}^\mathsf{T}\boldsymbol{\Lambda}\boldsymbol{y}$

   (4) Since $\boldsymbol{\Lambda}$ is diagonal $[\lambda_i]$: $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \sum \lambda_i y_i^2$

   (5) Since $\lambda_i > 0$: $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} > 0$

4. **Do all the *leading submatrices* have** $|\boldsymbol{A}_k| > 0$**?** For $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, the leading submatrix (also known as the upper-left submatrix) of dimension $k < n$, denoted $\boldsymbol{A}_k \in \mathbb{R}^{k\times k}$, is a block matrix comprising the first k rows and columns of $\boldsymbol{A}$. For a

positive definite matrix $\boldsymbol{A}$, the determinants of all such leading submatrices $\boldsymbol{A}_k$ must be strictly greater than zero.

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & a_{21} & & & \\ a_{21} & a_{22} & & & \boldsymbol{A}_k \\ \vdots & \cdots & \ddots & & \\ a_{k1} & a_{k2} & \cdots & a_{kk} & \cdots \\ \vdots & & \cdots & & \ddots \end{bmatrix} \tag{12.2}$$

Eqn (12.2) illustrates the definition of a leading submatrix $\boldsymbol{A}_k$.

Proof: Since $\boldsymbol{A}$ is positive definite, we know that $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} > 0$ for *any* non-zero $\boldsymbol{x}$. Let's consider a specific vector $\boldsymbol{x}_{[k,0]}$ which has non-zero entries in the first k elements, with the rest being exactly zero. The product $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{x}_{[k,0]}^\mathsf{T}\boldsymbol{A}_k\boldsymbol{x}_{[k,0]}$ must also be greater than zero. Then, by Test (3), the submatrix $\boldsymbol{A}_k$ is itself positive definite. Because it is positive definite, all of its eigenvalues are positive, and its determinant (which is the product of its eigenvalues) must necessarily be positive.

5. **Can we write $\boldsymbol{B} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A}$?** For a positive definite matrix $\boldsymbol{B}$, we can always find an *invertible* matrix $\boldsymbol{A}$ such that $\boldsymbol{B} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A}$. Although not usually used as a routine test, this property is an important one, and our topic in the last chapter depends on it. Let's therefore prove it here.

   Proof: $\boldsymbol{B} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A} \implies \boldsymbol{B}$ is positive definite.

   $$\boldsymbol{x}^\mathsf{T}\boldsymbol{B}\boldsymbol{x} = \boldsymbol{x}^\mathsf{T}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = (\boldsymbol{A}\boldsymbol{x})^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \|\boldsymbol{A}\boldsymbol{x}\|^2 > 0$$

   Since $\boldsymbol{A}$ is invertible, $\boldsymbol{A}\boldsymbol{x} \neq \boldsymbol{0}$ unless $\boldsymbol{x} = \boldsymbol{0}$.

   Proof:
   $\boldsymbol{B}$ is positive definite $\implies \boldsymbol{B} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ for some invertible $\boldsymbol{A}$.

   (1) Since $\boldsymbol{B}$ is pos. definite: $\boldsymbol{B} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\mathsf{T}$

   (2) Defining $\boldsymbol{\Lambda}^{\frac{1}{2}} = [\sqrt{\lambda_i}]$: $\boldsymbol{B} = \boldsymbol{Q}\boldsymbol{\Lambda}^{\frac{1}{2}}\boldsymbol{\Lambda}^{\frac{1}{2}}\boldsymbol{Q}^\mathsf{T}$

   (Matrix with $\sqrt{\lambda_i}$ on the diagonal) $= \boldsymbol{Q}\boldsymbol{\Lambda}^{\frac{1}{2}}(\boldsymbol{Q}\boldsymbol{\Lambda}^{\frac{1}{2}})^\mathsf{T}$

   (3) Defining $\boldsymbol{A}^\mathsf{T} = \boldsymbol{Q}\boldsymbol{\Lambda}^{\frac{1}{2}}$: $\boldsymbol{B} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A}$

   Since $\boldsymbol{B}$ is positive definite, its eigenvalues are positive, and thus $\boldsymbol{\Lambda}^{\frac{1}{2}}$ is a real, invertible matrix. $\boldsymbol{Q}$ is also invertible because

$\boldsymbol{Q}^{-1} = \boldsymbol{Q}^{\mathsf{T}}$. Therefore, $\boldsymbol{A}$ is the product of invertible matrices and is itself invertible.

Applying the Tests

Let's see how we can apply these five tests to various matrices to determine whether they are positive definite.

- **If $\boldsymbol{A}$ is positive definite, is $\boldsymbol{A}^{-1}$ positive definite?**

  We can apply Test (1): $\boldsymbol{A}$ has strictly positive λ_i. The eigenvalues of $\boldsymbol{A}^{-1}$ are given by $\frac{1}{\lambda_i}$, which are necessarily positive as well. Therefore, $\boldsymbol{A}^{-1}$ is positive definite.

- **If $\boldsymbol{A}$ and $\boldsymbol{B}$ are positive definite, how about $\boldsymbol{A} + \boldsymbol{B}$?**

  We apply Test (3) here: $\boldsymbol{x}^{\mathsf{T}}(\boldsymbol{A} + \boldsymbol{B})\boldsymbol{x} = \boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} + \boldsymbol{x}^{\mathsf{T}}\boldsymbol{B}\boldsymbol{x} > 0$ because both $\boldsymbol{A}$ and $\boldsymbol{B}$ are positive definite. Therefore, the sum is also positive definite.

- **For any full-column-rank $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, is $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ positive definite?**

  Test (3) proves useful again: $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} = (\boldsymbol{A}\boldsymbol{x})^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} = \|\boldsymbol{A}\boldsymbol{x}\|^2 \geq 0$. It is equal to zero only if $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0}$ has non-zero solutions, which would mean $\boldsymbol{A}$ has a non-trivial null space and is rank-deficient. Since we started with the assumption that $\boldsymbol{A}$ is full column rank, $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} > 0$ for all non-zero $\boldsymbol{x}$, meaning $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ is positive definite.

- **For a positive definite $\boldsymbol{A}$, is $\boldsymbol{M}^{-1}\boldsymbol{A}\boldsymbol{M}$ positive definite for *any* invertible $\boldsymbol{M}$?**

  The answer is yes. The reason is that $\boldsymbol{A}$ and $\boldsymbol{M}^{-1}\boldsymbol{A}\boldsymbol{M}$ are *similar matrices*, and similar matrices have the exact same eigenvalues. We shall soon formally define matrix similarity and prove this eigenvalue property.

Quadratic Forms

One of the tests for positive definiteness, namely $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} > 0$ (which we called Test 3), leads to the broader topic of quadratic forms. Although not always directly relevant to core computer science, we will introduce the topic here and list some of its general properties so

that we may recognize it if we encounter it in mathematically oriented literature later on.

For a real, symmetric matrix, we can expand $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x}$ in $\mathbb{R}^2$ as follows:

$$\boldsymbol{A} \in \mathbb{R}^{2\times 2},\ \boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A} \implies \boldsymbol{A} = \begin{bmatrix} a & b \\ b & c \end{bmatrix} \quad \text{and} \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$
$$\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = ax_1^2 + 2bx_1x_2 + cx_2^2 \tag{12.3}$$

As we can see, the product $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x}$ generates a pure quadratic form, with no linear or constant terms. Furthermore, as shown in Test (4), if the determinants of the leading submatrices of $\boldsymbol{A}$ are positive, we must have:

$$a > 0 \quad \text{and} \quad ac - b^2 > 0$$

Let's take a concrete example and see how $a > 0$ and $|\boldsymbol{A}| > 0$ imply that $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} > 0$ for all non-zero $\boldsymbol{x}$. Let:

$$\boldsymbol{A} = \begin{bmatrix} 2 & 6 \\ 6 & c \end{bmatrix} \quad \text{where} \quad |\boldsymbol{A}| = 2c - 36 > 0 \text{ if } c > 18.$$

Let's set $c = 20$. Expanding the quadratic form yields:

$$\begin{aligned} \boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} &= 2x_1^2 + 12x_1x_2 + 20x_2^2 \\ &= 2(x_1^2 + 6x_1x_2 + 9x_2^2) + 2x_2^2 \\ &= 2(x_1 + 3x_2)^2 + 2x_2^2 > 0 \end{aligned}$$

As we can see, if we had set $c < 18$, the final x_2^2 term would become negative, and the whole expression could evaluate to a negative number for certain values of x_1 and x_2 (for instance, setting $x_1 = -3, x_2 = 1$). But if $c > 18$, the expression is a sum of squared terms with positive coefficients, which can never be negative. Hence, the matrix $\boldsymbol{A}$ is positive definite. If $c = 18$, $\boldsymbol{A}$ is positive semidefinite.

Note an incredible algebraic coincidence in the expression $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} = 2(x_1 + 3x_2)^2 + 2x_2^2$. The scalar 2 multiplying the first term is exactly a from the matrix $\boldsymbol{A}$, which is the first pivot. The multiplier of the second term, also 2, is the second pivot. Inside the parentheses of the first term, the factor multiplying x_2 (namely 3) is the exact

multiplier used on the first row during Gaussian elimination to clear out the second row.

The quadratic form is intrinsically connected to pivots and Gaussian elimination. Our discussion with this simple 2×2 example generalizes to $\mathbb{R}^{n \times n}$, a fact that has profound mathematical and algorithmic applications.

This connection between row operations in Gaussian elimination and completing the square in the quadratic form becomes even clearer if we algebraically process the general matrix $\boldsymbol{A}$ from Eqn (12.3):

$$\boldsymbol{A} = \begin{bmatrix} a & b \\ b & c \end{bmatrix} \xrightarrow{r_2 \leftarrow -\frac{b}{a} r_1 + r_2} \begin{bmatrix} a & b \\ 0 & c - \frac{b^2}{a} \end{bmatrix}$$

Now look at the algebraic expansion:

$$\begin{aligned} a\left(x_1 + \frac{b}{a}x_2\right)^2 &= ax_1^2 + 2bx_1x_2 + \frac{b^2}{a}x_2^2 \\ &= \boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} - cx_2^2 + \frac{b^2}{a}x_2^2 \end{aligned}$$

Rearranging to isolate $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x}$:

$$\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = a\left(x_1 + \frac{b}{a}x_2\right)^2 + \left(c - \frac{b^2}{a}\right)x_2^2$$

We can plainly see the pivots (a and $c - \frac{b^2}{a}$) and the row multiplier ($\frac{b}{a}$) exposed directly in the completed-square expression for $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x}$.

Furthermore, this proves that $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} > 0$ for all non-zero $\boldsymbol{x}$ if and only if the second pivot is strictly positive:

$$c > \frac{b^2}{a} \implies ac - b^2 = |\boldsymbol{A}| > 0$$

Positive Definiteness and Symmetry

In our definition of positive definite matrices, we deliberately restricted ourselves to symmetric matrices. Our basic definition made this abundantly clear by stating: "A real, symmetric matrix. . . ". We should note that some texts relax this restriction, considering non-symmetric matrices to be positive definite if they pass the $\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} > 0$ test. While we will stick with our definition because it guarantees

the existence of real eigenvalues and orthogonal eigenvectors (which is what we care about in computer science), purely mathematical frameworks may consider more general definitions over fields like $\mathbb{C}$.

12.2.2 The Gram Matrix

For $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, $\text{rank}(\boldsymbol{A}) = r$, its Gram matrix is defined[2] as $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ and has important applications in machine learning. It also has some interesting properties that we exploit at various points in this book.

Before getting into the properties of Gram matrices, let's revisit some basic properties of ranks. The first one is that the row rank and the column rank of a matrix are the same. Earlier, we defined the rank as the number of pivots and also as the number of linearly independent vectors in its row (or column) space. The number of independent vectors is indeed the dimension of the subspace. To prove the equality of row and column ranks, we will use the latter definition.

We know that elementary row operations do not change the row space of a matrix because they involve linear combinations of the rows. Therefore, $\boldsymbol{A}$ has the same row rank as its RREF because they both have the same row space. As we saw earlier, the shape of the RREF is, in its most general case, as follows:

$$\boldsymbol{A} \xrightarrow{\text{RREF}} \left[\begin{array}{c} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r\times(n-r)} \\ \hline \boldsymbol{0}_{(m-r)\times n} \end{array}\right] = \boldsymbol{R}_1$$

where $\bowtie$ indicates that the $\boldsymbol{I}$ and $\boldsymbol{F}$ matrices have their columns mixed in, not cleanly separated. Clearly, the dimension of the row space of $\boldsymbol{A}$ is r, which is the row rank of both $\boldsymbol{A}$ and $\boldsymbol{R}_1$.

We can go further and get rid of the columns of $\boldsymbol{F}$ using elementary *column* operations, which do not change the column space. Starting from $\boldsymbol{R}_1$, the RREF of $\boldsymbol{A}$, we can perform a series of elementary column operations, without affecting its column space, to reduce the matrix to the following form:

$$\text{RREF} = \left[\begin{array}{c} \boldsymbol{I}_r \bowtie \boldsymbol{F}_{r\times(n-r)} \\ \hline \boldsymbol{0}_{(m-r)\times n} \end{array}\right] \xrightarrow{\text{Col. Ops}} \left[\begin{array}{c|c} \boldsymbol{I}_r & \boldsymbol{0}_{r\times(n-r)} \\ \hline \boldsymbol{0}_{(m-r)\times n} & \end{array}\right] = \boldsymbol{R}_2$$

[2]The author is not quite sure if $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$ is also considered a Gram matrix, although it should be, by symmetry.

This form, obviously, has the same row and column rank of r, and since our elementary operations changed neither, we can say that the row rank of $\boldsymbol{R}_1$ is the same as the column rank of $\boldsymbol{R}_2$, which is r. This claim propagates to the original $\boldsymbol{A}$, and it has the same row and column ranks. Therefore, the row rank of any matrix is the same as its column rank. It should be immediately obvious from this statement that $\text{rank}(\boldsymbol{A}) = \text{rank}\big(\boldsymbol{A}^\mathsf{T}\big)$.

Getting back to $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$, we can see that every vector $\boldsymbol{x} \in \mathcal{N}(\boldsymbol{A})$ is also in the null space of $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$:

$$\begin{aligned} \boldsymbol{A}\boldsymbol{x} = \boldsymbol{0} &\implies \boldsymbol{A}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0} \\ &\implies \boldsymbol{x} \in \mathcal{N}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}) \\ &\implies \mathcal{N}(\boldsymbol{A}) \subseteq \mathcal{N}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}) \end{aligned}$$

There may be more vectors in $\mathcal{N}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})$, which is why we only claim $\mathcal{N}(\boldsymbol{A}) \subseteq \mathcal{N}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})$. Now, coming at it from the other end, if $\boldsymbol{x} \in \mathcal{N}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})$:

$$\begin{aligned} \boldsymbol{A}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0} &\implies \boldsymbol{x}^\mathsf{T}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = 0 \\ &\implies (\boldsymbol{A}\boldsymbol{x})^\mathsf{T}(\boldsymbol{A}\boldsymbol{x}) = 0 \\ &\implies \boldsymbol{A}\boldsymbol{x} = \boldsymbol{0} \end{aligned}$$

Every vector in $\mathcal{N}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})$ is also in $\mathcal{N}(\boldsymbol{A})$. Both these conditions can happen only if $\mathcal{N}(\boldsymbol{A}) = \mathcal{N}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A})$, which means $\boldsymbol{A}$ and $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ have the exact same nullity.

Since $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, its domain is $\mathbb{R}^n$. And since $\boldsymbol{A}^\mathsf{T}\boldsymbol{A} \in \mathbb{R}^{n\times n}$, its domain also is $\mathbb{R}^n$. Therefore, by the rank-nullity theorem, we have $\text{rank}(\boldsymbol{A}) = \text{rank}\big(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\big)$ because:

$$\begin{aligned} \text{rank}(\boldsymbol{A}) &= \dim(\mathbb{R}^n) - \text{nullity}(\boldsymbol{A}) \\ \text{rank}\big(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\big) &= \dim(\mathbb{R}^n) - \text{nullity}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}) \\ \text{And} \quad \text{nullity}(\boldsymbol{A}) = \text{nullity}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}) &\implies \text{rank}(\boldsymbol{A}) = \text{rank}\big(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\big) \end{aligned}$$

It is also easy enough to prove that $\boldsymbol{A}$ and $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ have the same row space, by looking at the product $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ as the linear combinations of the rows of $\boldsymbol{A}$ and as the linear combinations of the columns of $\boldsymbol{A}^\mathsf{T}$. At this stage in our linear algebra journey, we are in the happy position of being able to prove a lemma in multiple ways, and we can afford to pick and choose.

Summarizing, $\boldsymbol{A}$ and $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ have the same row and null spaces. $\boldsymbol{A}^\mathsf{T}$ and $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$ have the same column space and left null space. All four of them have the exact same rank.

For a full-column-rank matrix ($\boldsymbol{A} \in \mathbb{R}^{m\times n}, \text{rank}(\boldsymbol{A}) = n$), the Gram matrix $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ is a full-rank, square matrix with the same rank. It is also much smaller. We can, therefore, test the linear independence of the n column vectors in $\boldsymbol{A}$ by looking at the invertibility (or, equivalently, the determinant) of the Gram matrix. In data science, as we shall see in the last chapter of this book, the Gram matrix is the covariance matrix of a zero-centered dataset.

12.3 Markov Matrices and State Transitions

A square matrix is called a Markov matrix if all its entries are non-negative and the sum of each column vector is equal to one. It is also known as a left stochastic matrix[3]. "Stochastic," by the way, is a fancy word meaning probabilistic. Markov matrices are also known as probability, transition, or stochastic matrices.

Markov Matrix

Definition: $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ is a Markov matrix if

$$0 \leq a_{ij} \leq 1 \quad \text{and} \quad \sum_{i=1}^{n} a_{ij} = 1$$

Markov matrices usually describe the transition probabilities between states in a stochastic mathematical model.

12.3.1 Properties of Markov Matrices

All the following properties of Markov matrices follow from the fact that the columns add up to one. In other words, if we were to add up

[3]The right stochastic matrix, on the other hand, would be one in which the rows add up to one. Since our vectors are all column vectors, it is the left stochastic matrix that we will focus on. But we should keep in mind that the rows of a matrix are, at times, considered "row vectors." For such a row vector, a matrix would multiply it on the right, and we can think of the so-called right eigenvectors.

all the *rows* of a Markov matrix, we would get a row of ones because each *column* adds up to one.

1. Markov matrices have one eigenvalue exactly equal to one.
2. The product of two Markov matrices is another Markov matrix.
3. All eigenvalues of a Markov matrix are less than or equal to one in absolute value: $|\lambda_i| \leq 1$.

Let's try proving these properties one by one. Since its columns add up to one, a Markov matrix always has one eigenvalue equal to one. *Proof*:

(1) Since $\boldsymbol{A}$ is a Markov matrix: $\displaystyle\sum_{i=1}^{n} a_{ij} = 1$

(2) Therefore: $\displaystyle a_{jj} = 1 - \sum_{\substack{i=1 \\ i \neq j}}^{n} a_{ij}$

(3) The diagonal element in $\boldsymbol{A} - \boldsymbol{I}$: $\displaystyle(\boldsymbol{A} - \boldsymbol{I})_{jj} = 1 - \sum_{\substack{i=1 \\ i \neq j}}^{n} a_{ij}$

(4) Sum of columns in $\boldsymbol{A} - \boldsymbol{I}$: $(\boldsymbol{A} - \boldsymbol{I})_{jj} = 0$

(5) Therefore: $|\boldsymbol{A} - \boldsymbol{I}| = 0$

Step (5) above says that $|\boldsymbol{A} - \lambda \boldsymbol{I}| = 0$ with $\lambda = 1$, which means one is an eigenvalue of any Markov matrix.

Let's prove it again using a slightly more sophisticated technique. Let's construct a column vector of all ones, and call it $\boldsymbol{u} \in \mathbb{R}^n$. Taking the sum of the rows of the Markov matrix $\boldsymbol{A}$ is the same as multiplying on the left with $\boldsymbol{u}^\mathsf{T}$ (which is a single-row matrix of all ones). We know that the product $\boldsymbol{u}^\mathsf{T}\boldsymbol{A} = \boldsymbol{u}^\mathsf{T}$ because the columns of $\boldsymbol{A}$ add up to one. Taking the transpose yields $\boldsymbol{A}^\mathsf{T}\boldsymbol{u} = \boldsymbol{u}$, which says that $\boldsymbol{A}^\mathsf{T}$ has an eigenvalue of one (with $\boldsymbol{u}$ as the eigenvector, though that is not important for us). Now, the eigenvalues of a matrix and its transpose are the same (even though the eigenvectors may be different) because the characteristic polynomial, being a determinant, does not change when we take the transpose. So we can see that $\boldsymbol{A}$ has at least one eigenvalue equal to one.

The second property is that the product of two Markov matrices is another Markov matrix. If $\boldsymbol{A}$ and $\boldsymbol{B}$ are Markov, then $\boldsymbol{AB}$ cannot

have any negative entries because matrix multiplication¯being strictly addition and multiplication of positive elements¯cannot introduce a negative sign. We also know that each of the columns of $\boldsymbol{A}$ and $\boldsymbol{B}$ adds up to one, which means $\boldsymbol{u}^\mathsf{T}\boldsymbol{A} = \boldsymbol{u}^\mathsf{T}$ and $\boldsymbol{u}^\mathsf{T}\boldsymbol{B} = \boldsymbol{u}^\mathsf{T}$. Consider $\boldsymbol{u}^\mathsf{T}\boldsymbol{A}\boldsymbol{B} = \boldsymbol{u}^\mathsf{T}\boldsymbol{B} = \boldsymbol{u}^\mathsf{T}$. Therefore, the columns of $\boldsymbol{A}\boldsymbol{B}$ also add up to one, meaning the product of two Markov matrices is another Markov matrix.

The third property is that the absolute values of the eigenvalues of a Markov matrix are less than or equal to one: $|\lambda_i| \leq 1$. This follows logically from the product property. $\boldsymbol{A}^n$ is a Markov matrix. If one of the eigenvalues was greater than one, then as $k \to \infty$, $\boldsymbol{A}^k$ could not be a Markov matrix because its elements would have to grow exponentially.

12.3.2 Steady State

In order to make this cryptic proof of the third property a bit more accessible, let's work out an example. Let's say we are studying human migration patterns across the globe, and we know the yearly migration probabilities as shown in Table 12.1 through some unspecified demographic studies. We know nothing else, except perhaps that the birth and death rates are close enough to each other for us to assume they cancel out. One reasonable question to ask would be about the steady state: If we wait long enough, do the populations stabilize?

Note that the numbers in each column add up to 100% because people either stay or leave. The numbers in each row, on the other hand, do not. Asia-Pacific and Africa lose people to the Americas and Europe.

Once we have probabilities like those in Table 12.1, the first thing to do is put the values in matrices, now that we know enough linear algebra:

$$\boldsymbol{A} = \begin{bmatrix} 0.80 & 0.04 & 0.05 & 0.05 \\ 0.10 & 0.90 & 0.07 & 0.08 \\ 0.03 & 0.01 & 0.75 & 0.02 \\ 0.07 & 0.05 & 0.13 & 0.85 \end{bmatrix}, \quad \boldsymbol{x}_0 = \begin{bmatrix} 4.68 \\ 1.20 \\ 1.34 \\ 0.75 \end{bmatrix}, \quad \boldsymbol{x}_{k+1} = \boldsymbol{A}\boldsymbol{x}_k$$

where we put the initial populations in a vector $\boldsymbol{x}_0$. As we can see, $\boldsymbol{A}$ is a Markov matrix. It describes how the population evolves over time. The populations for year k evolve to that of year $k+1$ via $\boldsymbol{A}\boldsymbol{x}_k$,

Table 12.1 Migration probabilities

Destination ↓ Source →	Asia-Pacific	Americas	Africa	Europe
Asia-Pacific	80%	4%	5%	5%
Americas	10%	90%	7%	8%
Africa	3%	1%	75%	2%
Europe	7%	5%	13%	85%
Population (billions)	4.68	1.20	1.34	0.75

which is identical to what we did in the case of Fibonacci numbers in §**??** (page **??**).

As we learned there, the long-term evolution of the populations is fully described by the eigenvalues λ_i of $\boldsymbol{A}$. If $|\lambda_i| > 1$, we will have a growing system. If $|\lambda_i| < 1$, we will have a system tending to zero. If we have an eigenvalue $|\lambda_i| = 1$, we will have a steady state. And we know that $\boldsymbol{A}$ definitely has an eigenvalue equal to one.

Knowing that $\boldsymbol{x} = \boldsymbol{x}_k$ will stabilize and reach an equilibrium value, we can implement an iterative method to compute it: First, initialize $\boldsymbol{x} \leftarrow \boldsymbol{x}_0$. Then, iterate until convergence: $\boldsymbol{x} \leftarrow \boldsymbol{A}\boldsymbol{x}$. Running this in a numeric program, we get:

$$\boldsymbol{x}^{\mathsf{T}} = \begin{bmatrix} 0.7262 & 1.844 & 0.2549 & 1.175 \end{bmatrix}$$

The eigen-way of doing it would be to find the eigenbasis, find their linear combination to form $\boldsymbol{x}_0$:

$$\boldsymbol{x}_0 = \sum_{i=1}^{n} c_i \boldsymbol{s_i}$$

and then say that (with $\lambda_1 = 1$ and all other $|\lambda_i| < 1$):

$$\boldsymbol{x}_k = \boldsymbol{A}^k \sum_{i=1}^{n} c_i \boldsymbol{s_i} = \sum_{i=1}^{n} c_i \boldsymbol{A}^k \boldsymbol{s_i} = \sum_{i=1}^{n} c_i \lambda_i^k \boldsymbol{s_i} = c_1 \lambda_1^k \boldsymbol{s}_1 = c_1 \boldsymbol{s}_1$$

Therefore, we need to know only c_1. But this still looks like a pain to compute because, in order to find c_1, we have to find all the other c_i, which means we have to find all the eigenvectors first.

Luckily for us, there is a shortcut. As $\boldsymbol{x}$ evolves from $\boldsymbol{x}_0$ to $\boldsymbol{x}_k$, the sum of its components is the total population, which we are assuming to be a constant. Therefore, in $\boldsymbol{x}_k = c_1 \boldsymbol{s}_1$, the components of $\boldsymbol{x}_k$ and $c_1 \boldsymbol{s}_1$ should add up to the same number, which is the total population.

In other words,

$$c_1 = \frac{\sum x_{0i}}{\sum s_{1i}}$$

where the summation is over the components of $\boldsymbol{x}_0$ and $\boldsymbol{s}_1$. So we only need to know one eigenvector, $\boldsymbol{s}_1$, corresponding to the dominant eigenvalue $\lambda = 1$, in order to compute the limiting value of $\boldsymbol{x} = \boldsymbol{x}_k$ as $k \to \infty$. As expected, this computation will yield the exact same answer:

$$\boldsymbol{x}^{\mathsf{T}} = \begin{bmatrix} 0.7262 & 1.844 & 0.2549 & 1.175 \end{bmatrix}$$

12.3.3 Portal to a Big Field

Markov matrices are the starting point for related topics in mathematical modeling, some of which are heavily used in machine learning and data mining. For instance, Hidden Markov Models (HMMs) are used in text mining for part-of-speech tagging. In our short introduction here, we focused only on the basics, ignoring many subtleties.

A quick flip through the pages of this book will reveal that permutation matrices are, in fact, Markov matrices. What are their steady states? They have none; they have oscillating solutions. Other, more complicated matrices may have multiple steady states among which the solutions oscillate.

The Google PageRank algorithm is a multi-billion dollar success story built on Markov matrices, which, as we see in the upcoming box, does not take much more than our discussion here to fully understand.

12.4 Matrix Similarity and Canonical Forms

12.4.1 Similarity Transformations

We consider a matrix $\boldsymbol{A}$ *similar* to another one $\boldsymbol{B}$ if we can write $\boldsymbol{A} = \boldsymbol{M}^{-1}\boldsymbol{B}\boldsymbol{M}$ for some invertible matrix $\boldsymbol{M}$. We denote similarity as $\boldsymbol{A} \sim \boldsymbol{B}$. As we can immediately see, $\boldsymbol{A}$ is similar to $\boldsymbol{\Lambda}$, its eigenvalue matrix, when we have a full set of eigenvectors (and therefore the eigenvector matrix $\boldsymbol{S}$ is invertible) because then $\boldsymbol{A} = \boldsymbol{S}\boldsymbol{\Lambda}\boldsymbol{S}^{-1}$. Note that we are talking about any square matrix $\boldsymbol{A}$ now, not necessarily symmetric matrices.

While this definition of matrix similarity may look strange at first, there are good reasons behind it: Similar matrices share several key properties. Here is a list, with proof wherever necessary:

1. For invertible matrices $\boldsymbol{A}$ and $\boldsymbol{B}$, $\boldsymbol{AB} \sim \boldsymbol{BA}$.

   Proof:

   $$\boldsymbol{AB} \sim \boldsymbol{MABM^{-1}} \sim \boldsymbol{BABB^{-1}} \sim \boldsymbol{BA} \quad \text{with} \quad \boldsymbol{M} = \boldsymbol{B}$$

2. If $\boldsymbol{A} \sim \boldsymbol{B}$, they have the same characteristic polynomials.

   Proof: Let's use $p_{\boldsymbol{A}}(\lambda)$ to denote the characteristic polynomial of $\boldsymbol{A}$. Since $\boldsymbol{A} \sim \boldsymbol{B}$, we know $\boldsymbol{A} = \boldsymbol{M^{-1}BM}$.
   (1) Characteristic Polynomial:

   $$p_{\boldsymbol{A}}(\lambda) = |\boldsymbol{A} - \lambda \boldsymbol{I}|$$

   (2) Since $\boldsymbol{A} = \boldsymbol{M^{-1}BM}$:

   $$p_{\boldsymbol{A}}(\lambda) = \left|\boldsymbol{M^{-1}BM} - \lambda \boldsymbol{I}\right|$$

   (3) Since $\boldsymbol{M^{-1}M} = \boldsymbol{I}$:

   $$p_{\boldsymbol{A}}(\lambda) = \left|\boldsymbol{M^{-1}BM} - \boldsymbol{M^{-1}}\lambda \boldsymbol{IM}\right|$$

   (4) Factorizing $\boldsymbol{M}$ and $\boldsymbol{M^{-1}}$:

   $$p_{\boldsymbol{A}}(\lambda) = \left|\boldsymbol{M^{-1}}(\boldsymbol{B} - \lambda \boldsymbol{I})\boldsymbol{M}\right|$$

   (5) Since $|\boldsymbol{AB}| = |\boldsymbol{A}|\,|\boldsymbol{B}|$:

   $$p_{\boldsymbol{A}}(\lambda) = \left|\boldsymbol{M^{-1}}\right| |\boldsymbol{B} - \lambda \boldsymbol{I}|\,|\boldsymbol{M}|$$

   (6) Since $|\boldsymbol{A}|\,|\boldsymbol{A^{-1}}| = |\boldsymbol{I}| = 1$:

   $$p_{\boldsymbol{A}}(\lambda) = |\boldsymbol{B} - \lambda \boldsymbol{I}| = p_{\boldsymbol{B}}(\lambda)$$

   Step (6) above shows that both $\boldsymbol{A}$ and the similar matrix $\boldsymbol{B}$ have the same characteristic polynomial. Note that, as a consequence of the characteristic polynomials being the same, the algebraic multiplicities of the eigenvalues (how many times each one is repeated) are also the exact same for $\boldsymbol{A}$ and $\boldsymbol{B}$.

3. If $\boldsymbol{A} \sim \boldsymbol{B}$, they have the same eigenvalues, but not necessarily the same eigenvectors.

Proof: We already showed that $\boldsymbol{A}$ and $\boldsymbol{B}$ have the same characteristic polynomial. Therefore they have the same eigenvalues (which are the roots of the said polynomial). But here is another proof.

(1) Definition of similarity:	$\boldsymbol{A} = \boldsymbol{M}^{-1}\boldsymbol{B}\boldsymbol{M}$
(2) Left-multiplying by $\boldsymbol{M}$:	$\boldsymbol{M}\boldsymbol{A} = \boldsymbol{B}\boldsymbol{M}$
(3) Right-multiplying by $\boldsymbol{s}$:	$\boldsymbol{M}\boldsymbol{A}\boldsymbol{s} = \boldsymbol{B}\boldsymbol{M}\boldsymbol{s}$
(4) Since $\boldsymbol{s}$ is eigenvector of $\boldsymbol{A}$:	$\boldsymbol{M}\lambda\boldsymbol{s} = \boldsymbol{B}\boldsymbol{M}\boldsymbol{s}$
(5) Commuting λ:	$\lambda(\boldsymbol{M}\boldsymbol{s}) = \boldsymbol{B}(\boldsymbol{M}\boldsymbol{s})$
(6) LHS $\leftrightarrow$ RHS:	$\boldsymbol{B}(\boldsymbol{M}\boldsymbol{s}) = \lambda(\boldsymbol{M}\boldsymbol{s})$

Step (6) says that $\boldsymbol{M}\boldsymbol{s}$ is an eigenvector of $\boldsymbol{B}$ with the same eigenvalue λ, by the definition of eigenvalues and eigenvectors.

4. Combining the previous property with the first one, we can see that $\boldsymbol{A}\boldsymbol{B}$ and $\boldsymbol{B}\boldsymbol{A}$ have the same eigenvalues.

5. If $\boldsymbol{A} \sim \boldsymbol{B}$ and $\boldsymbol{A}$ is positive definite, so is $\boldsymbol{B}$.
   Proof: The determinant is the product of eigenvalues.

6. $\boldsymbol{A} \sim \boldsymbol{B} \implies |\boldsymbol{A}| = |\boldsymbol{B}|$.
   Proof: The determinant is the product of eigenvalues.

7. $\boldsymbol{A} \sim \boldsymbol{B} \implies \text{trace}(\boldsymbol{A}) = \text{trace}(\boldsymbol{B})$.
   Proof: The trace is the sum of eigenvalues.

8. Similar matrices have the same rank.
   Proof: The number of (positive and negative) pivots is the same as the number of (positive and negative) eigenvalues, through Sylvester's Law of Inertia. And the rank is the number of non-zero pivots.

Equivalence Relation

The properties listed above are the ones that are useful for us in linear algebra. However, more basic than those, similarity as a relation has some fundamental properties that make it an *equivalence relation*. Here is a formal statement of what that means.

For $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C} \in \mathbb{R}^{n \times n}$, we can easily show that similarity as a relation is:

Reflexive: $\boldsymbol{A} \sim \boldsymbol{A}$

Proof: $\boldsymbol{A} = \boldsymbol{I}\boldsymbol{A}\boldsymbol{I}^{-1}$

Symmetric: $\boldsymbol{A} \sim \boldsymbol{B} \implies \boldsymbol{B} \sim \boldsymbol{A}$

Proof: If $\boldsymbol{A} = \boldsymbol{M}\boldsymbol{B}\boldsymbol{M}^{-1}$, isolating $\boldsymbol{B}$ yields:

$$\boldsymbol{B} = \boldsymbol{M}^{-1}\boldsymbol{A}\boldsymbol{M} = \boldsymbol{M}'\boldsymbol{A}\boldsymbol{M}'^{-1}$$

Transitive: If $\boldsymbol{A} \sim \boldsymbol{B}$ and $\boldsymbol{B} \sim \boldsymbol{C}$, then $\boldsymbol{A} \sim \boldsymbol{C}$

Proof: Given $\boldsymbol{A} = \boldsymbol{M}\boldsymbol{B}\boldsymbol{M}^{-1}$ and $\boldsymbol{B} = \boldsymbol{N}\boldsymbol{C}\boldsymbol{N}^{-1}$:

$$\begin{aligned}\boldsymbol{A} = \boldsymbol{M}\boldsymbol{B}\boldsymbol{M}^{-1} &= \boldsymbol{M}(\boldsymbol{N}\boldsymbol{C}\boldsymbol{N}^{-1})\boldsymbol{M}^{-1} \\ &= (\boldsymbol{M}\boldsymbol{N})\boldsymbol{C}(\boldsymbol{M}\boldsymbol{N})^{-1} \\ &= \boldsymbol{M}'\boldsymbol{C}\boldsymbol{M}'^{-1}\end{aligned}$$

When a relation has these three fundamental properties, it is called an equivalence relation. And matrix similarity is indeed an equivalence relation.

Diagonalizability

Since $\boldsymbol{A}$ is similar to its eigenvalue matrix $\boldsymbol{\Lambda}$, if $\boldsymbol{A}$ is diagonalizable, so are all the matrices similar to it. In fact, this statement would make a good, albeit incomplete, definition of similarity.

Similarity

Definition: A matrix is similar to another matrix if they diagonalize to the same diagonal matrix.

As we can see, the similarity relation puts diagonalizable matrices into distinct families. In $\mathbb{R}^{n \times n}$, we have an infinity of such mutually exclusive families. All matrices with the same set of eigenvalues belong to the same family.

When we said "the same diagonal matrix" in the definition of similarity above, we were being slightly imprecise: We should have

specified that shuffling the order of the eigenvalues is okay. We are really looking for the same *set* of eigenvalues, regardless of order.

However, this definition leaves something unspecified: What happens if a matrix is not diagonalizable? Does it belong to no family? Is it similar to none? Is it an orphan? It is in this context that *Jordan Normal Forms* come in to help. Since it is such an important topic, we will promote it to a section of its own.

12.4.2 Jordan Normal Forms

As we remember, in order for a matrix to be diagonalizable, its eigenvector matrix $\boldsymbol{S}$ needs to be invertible, so that we can go from $\boldsymbol{AS} = \boldsymbol{\Lambda S}$ (which is always true) to $\boldsymbol{A} = \boldsymbol{S\Lambda S^{-1}}$. For $\boldsymbol{S}$ to be invertible, we need its columns to be linearly independent, which means we need a full set of eigenvectors. In other words, for $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, we need n linearly independent eigenvectors for it to be diagonalizable.

We also saw (in §11.2.2, page 219) that the eigenvectors corresponding to distinct eigenvalues are linearly independent. We can, therefore, say that a matrix with no repeated (AKA degenerate) eigenvalues is diagonalizable. The converse is not true though: If a matrix does have repeated eigenvalues, it does not mean that it cannot be diagonalized. The identity matrix in $\mathbb{R}^n$, for instance, has the eigenvalue one repeated n times, but is perfectly diagonalizable. In fact, it is already in the diagonal form. As we shall see shortly, the right statement about diagonalizability is that if the algebraic multiplicity of an eigenvalue is greater than its geometric multiplicity, the matrix cannot be diagonalized.

We looked at such an example earlier, namely the shear matrix, which was not diagonalizable. A general form of such a shear matrix $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$ is shown below. It has a twice-repeated eigenvalue of λ because the product of the eigenvalues is $|\boldsymbol{A}| = \lambda^2$ and their sum is $\text{trace}(\boldsymbol{A}) = 2\lambda$, which can be easily verified by solving its characteristic polynomial[4]. But it has only one eigenvector $\boldsymbol{s}$.

$$\boldsymbol{A} = \begin{bmatrix} \lambda & a \\ 0 & \lambda \end{bmatrix} \implies \lambda_1 = \lambda_2 = \lambda, \quad \boldsymbol{s} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \tag{12.4}$$

[4]The characteristic equation is $|\boldsymbol{A} - \lambda'\boldsymbol{I}| = (\lambda - \lambda')^2 = 0$, with λ' as the dummy variable in the polynomial because we already used λ as the diagonal elements of the shear matrix.

In fancier language, the eigenvalue λ has an algebraic multiplicity of two, and a geometric multiplicity of one. It is when the geometric multiplicity is smaller than the algebraic multiplicity that we have a matrix that cannot be diagonalized. Note that the geometric multiplicity can never be greater than the algebraic multiplicity.

The closest we can get to a diagonal matrix for this matrix $\boldsymbol{A}$ in Eqn (12.4) is when we have the repeated eigenvalues along the diagonal, and a one in the position of a. This matrix is called the *Jordan normal* (or *canonical*) *form* of $\boldsymbol{A}$. It is defined as a block matrix in terms of what are known as Jordan blocks.

Jordan Normal Form

Definition: A square matrix ($\boldsymbol{J}$) made up of *Jordan blocks* is called a Jordan normal form (JNF) of a matrix $\boldsymbol{A}$ if $\boldsymbol{A} \sim \boldsymbol{J}$.

Jordan Block

Definition: A Jordan block of size k and value λ, denoted $J_k(\lambda)$, is a square matrix with the value λ repeated along its main diagonal and ones along the *superdiagonal* with zeros everywhere else. Here are some examples of Jordan blocks:

$$J_1(\lambda) = \begin{bmatrix}\lambda\end{bmatrix} \quad J_2(\lambda_1) = \begin{bmatrix}\lambda_1 & 1 \\ 0 & \lambda_1\end{bmatrix} \quad J_3(7) = \begin{bmatrix}7 & 1 & 0 \\ 0 & 7 & 1 \\ 0 & 0 & 7\end{bmatrix} \tag{12.5}$$

As we can see, superdiagonal means the diagonal above the main diagonal, so to speak. In the matrix $\boldsymbol{J}$, each eigenvector has a Jordan block of its own, and one block has only one eigenvalue. If we have repeated eigenvalues, but linearly independent eigenvectors, we get multiple Jordan blocks. For instance, for the identity matrix in $\mathbb{R}^{n\times n}$, we have n Jordan blocks, $J_1(1)$, each with $\lambda = 1$. In the examples above in Eqn (12.5), the first one corresponds to a good eigenvalue with an associated eigenvector. The second one has the eigenvalue repeated twice, but with only one eigenvector, much like our shear matrix. The third one is for an eigenvalue with an algebraic multiplicity of three and geometric multiplicity of two, to use the right terms.

Table 12.2 Jordan Normal Forms $\boldsymbol{J}$ of $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$ with $\boldsymbol{A}\boldsymbol{s_i} = \lambda_i \boldsymbol{s_i}$

	Comments on $\boldsymbol{A}$	Multiplicities Algeb.	Geom.	Jordan Normal Form	Examples of Similar Matrices
1	Most matrices $\text{rank}(\boldsymbol{A}) = 2$	$1, 1$	$1, 1$	$\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$	Any $\boldsymbol{A}$ with trace and $\lvert\boldsymbol{A}\rvert$ equal to sum and product of λ_i
2	$\lambda_2 = 0$, $\boldsymbol{s}_2 \in \mathcal{N}(\boldsymbol{A})$ $\text{rank}(\boldsymbol{A}) = 1$	$1, 1$	$1, 1$	$\begin{bmatrix} \lambda_1 & 0 \\ 0 & 0 \end{bmatrix}$	$\begin{bmatrix} \lambda_1 - t_1 & t_2 \\ \frac{\lambda_1 t_1 - t_1^2}{t_2} & t_1 \end{bmatrix}$
3	Repeated λ and $\boldsymbol{s}$ $\text{rank}(\boldsymbol{A}) = 2$	2	2	$\begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}$	Only $\boldsymbol{J} = \boldsymbol{A}$
4	$\lambda_1 = \lambda_2 = 0$ $\boldsymbol{s}_1, \boldsymbol{s}_2 \in \mathcal{N}(\boldsymbol{A})$ $\text{rank}(\boldsymbol{A}) = 0$	2	2	$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$	Only $\boldsymbol{J} = \boldsymbol{A}$
5	Repeated λ, one $\boldsymbol{s}$ $\text{rank}(\boldsymbol{A}) = 2$	2	1	$\begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}$	$\begin{bmatrix} 2\lambda - t_1 & t_2 \\ \frac{-\lambda^2 - 2\lambda t_1 - t_1^2}{t_2} & t_1 \end{bmatrix}$

In the last column, $t_1, t_2 \in \mathbb{R}$ are any numbers that will generate an example of a similar matrix for the corresponding row. They are constructed such that the sum and product of the eigenvalues come out right. Note that the Jordan normal forms in all rows except the fifth one have two Jordan blocks each.

With these definitions of Jordan normal forms and blocks, we can state Jordan's Theorem.

Jordan's Theorem

Every square matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ with $k \leq n$ linearly independent eigenvectors $\boldsymbol{s_i}, 1 \leq i \leq k$ and the associated eigenvalues $\lambda_i, 1 \leq i \leq k$, which are not necessarily linearly independent, is similar to a Jordan matrix $\boldsymbol{J}$ made up of Jordan blocks along its diagonal.

To start with something simple before generalizing and complicating life, let's look at $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$ with eigenvalues λ_1 and λ_2 and the corresponding eigenvectors $\boldsymbol{s}_1$ and $\boldsymbol{s}_2$. Table 12.2 tabulates the various possibilities. Let's go over each of the rows, and generalize it from $\mathbb{R}^{2\times 2}$ to $\mathbb{R}^{n\times n}$.

1. The first row is the good case, where we have distinct eigenvalues and linearly independent eigenvectors. The Jordan Normal Form (JNF) is the same as $\boldsymbol{\Lambda}$. Each λ_i is in a Jordan block $J_1(\lambda_i)$ of its own.

2. When one of the two eigenvalues is zero, the matrix is singular, and one of the eigenvectors is the basis of its null space. Qualitatively though, this case is not different from the first row. In $\mathbb{R}^{n\times n}$, we will have JNF = $\boldsymbol{\Lambda}$.

3. When the eigenvalues are repeated (meaning algebraic multiplicity is two), but we have two linearly independent eigenvectors. Again, JNF = $\boldsymbol{\Lambda}$. However, we have no other similar matrices: $\boldsymbol{MJM^{-1}} = \lambda \boldsymbol{MIM^{-1}} = \lambda \boldsymbol{I} = \boldsymbol{J}$.

4. This row shows a rank-zero matrix. Although troublesome, it is also qualitatively similar to the previous row.

5. In the last row, we have the geometric multiplicity smaller than the algebraic one, and we have a single $J_2(\lambda)$ block.

In order to show the Jordan form in all its gory detail, we have a general Jordan matrix made up of a large number of Jordan blocks in Eqn (12.6) below:

$$\boldsymbol{J} = \begin{bmatrix} \boxed{\lambda_1}^{J_1(\lambda_1)} & & & & & & & & & \\ 0 & \boxed{\lambda_1}^{J_1(\lambda_1)} & & & & & & & & \\ 0 & 0 & \boxed{\lambda_1}^{J_1(\lambda_1)} & & & & & & & \\ 0 & 0 & 0 & \boxed{\begin{matrix}\lambda_2 & 1 & \\ 0 & \lambda_2 & 1 \\ 0 & 0 & \lambda_2\end{matrix}}^{J_3(\lambda_2)} & & & & & \\ 0 & 0 & 0 & 0 & 0 & 0 & \boxed{\lambda_3}^{J_1(\lambda_3)} & & & \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \boxed{\begin{matrix}1 & \\ \lambda_4 & 1 \\ 0 & \lambda_4\end{matrix}}^{J_2(\lambda_4)} & & \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \ddots \end{bmatrix} \tag{12.6}$$

Each Jordan block is highlighted in a colored box with its label $J_k(\lambda)$. Here is what this Jordan matrix $\boldsymbol{J}$ tells us about the original matrix $\boldsymbol{A}$ (of which $\boldsymbol{J}$ is the Jordan Normal Form):

- The eigenvalues of $\boldsymbol{J}$ and $\boldsymbol{A}$ are the same: λ_i.
- The algebraic multiplicity of any eigenvalue is the sum of the *sizes* of the Jordan blocks associated with it.
- Its geometric multiplicity is the *number* of Jordan blocks associated with it.
- $\boldsymbol{A}$ can be diagonalized if and only if each Jordan block in $\boldsymbol{J}$ is of size one. In other words, $\boldsymbol{J}$ needs to be purely diagonal for $\boldsymbol{A}$ to be diagonalizable.

Table 12.3 lists the algebraic and geometrid multiplicities of the eigenvalues shown in Eqn (12.6). Since $\boldsymbol{J}$ has some elements in the superdiagonal, $\boldsymbol{A}$ is not diagonalizable. The closest $\boldsymbol{A}$ can get to a diagonal matrix is indeed its Jordan normal form $\boldsymbol{J}$.

Table 12.3 Multiplicities of eigenvalues from Jordan blocks

Eigenvalue	Multiplicities	
λ_i	Algebraic	Geometric
λ_1	3	3
λ_2	3	1
λ_3	1	1
λ_4	2	1

Much like the other topics in this chapter, the Jordan canonical form is also a portal, this time to advanced theoretical explorations in linear algebra, perhaps more relevant to mathematicians than computer scientists.

12.5 Computational Algorithms

We already learned three named algorithms, namely,

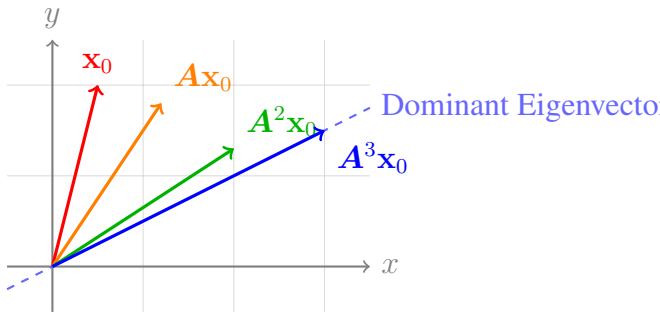
1. Gaussian Elimination, also known as $\boldsymbol{PLU}$ or just $\boldsymbol{LU}$ decomposition.
2. Gauss-Jordan Elimination, for matrix inversion.
3. Gram-Schmidt Orthonormalization, which gave us $\boldsymbol{QR}$ decomposition.

They are neatly summarized in Table 8.1 and recapped in the associated text. We came across one more algorithm earlier in this chapter (see §12.3.2, page 260), where we computed one eigenvector corresponding to $\lambda = 1$. The method is called the Power Iteration Algorithm, and can, in fact, find the largest eigenvalue/eigenvector pair.

12.5.1 Power Iteration

We saw power iteration in our Markov matrix, migration patterns, example. In general, the power iteration algorithm returns the dominant eigenvalue (which is the one with the largest absolute value).

The process is simple: We start with a random initial vector, apply the matrix to it, renormalize it and iterate.



Input: $\boldsymbol{A} \in \mathbb{R}^{n \times n}$
Output: Dominant $\lambda \in \mathbb{C}$

1: *Start with a random* $\boldsymbol{s}$
2: **repeat**
3: *Normalize it:* $\boldsymbol{s} \leftarrow \frac{\boldsymbol{s}}{\|\boldsymbol{s}\|}$
4: $\boldsymbol{s} \leftarrow \boldsymbol{A}\boldsymbol{s}$
5: **until** *Convergence*
6: **return** $\boldsymbol{s}$ *is the eigenvector for the largest* λ

Once we have $\boldsymbol{s}$, we can calculate λ as:

$$\lambda = \frac{\boldsymbol{s}^\mathsf{T}\boldsymbol{A}\boldsymbol{s}}{\boldsymbol{s}^\mathsf{T}\boldsymbol{s}} \quad \text{(Because } \boldsymbol{A}\boldsymbol{s} = \lambda\boldsymbol{s}\text{)}$$

which, by the way, is called the Rayleigh quotient.

Limitations

The power iteration algorithm has a couple of limitations[5].

1. If we start with a bad guess for the initial vector, the power iteration algorithm may not converge to the dominant eigenvalue.

2. For complex eigenvalues, we have to start with a complex initial vector. Otherwise, the algorithm may oscillate between the two conjugate eigenvalue pairs.

[5]From Wiki University.

12.5.2 QR Algorithm

A general numerical method to compute all eigenvalues and eigenvectors of a matrix is the $\boldsymbol{QR}$ algorithm, based on the Gram-Schmidt process.

Input: $\boldsymbol{A} \in \mathbb{R}^{n \times n}$
Output: $\lambda_i \in \mathbb{C}$

repeat
  Perform the Gram-Schmidt process to get $\boldsymbol{QR}$
  We get: $\boldsymbol{A} = \boldsymbol{QR}$
  Consider: $\boldsymbol{RQ} = \boldsymbol{Q^{-1}QRQ} = \boldsymbol{Q^{-1}AQ}$
  $\implies$ $\boldsymbol{RQ}$ *and* $\boldsymbol{A}$ *are similar and have the same eigenvalues*
  Therefore, set $\boldsymbol{A} \leftarrow \boldsymbol{RQ}$
until *Convergence*
return *The diagonal elements of* $\boldsymbol{R}$ *as the eigenvalues*

Convergence is obtained when $\boldsymbol{A}$ becomes close enough to a triangular matrix. At that point, the eigenvalues are its diagonal elements. Once we have the eigenvalues, we can compute the eigenvectors as the null space of $\boldsymbol{A} - \lambda \boldsymbol{I}$ using elimination algorithms.

12.5.3 Cholesky Decomposition

A matrix factorization technique that has several applications is the Cholesky Decomposition, which says that any positive definite matrix $\boldsymbol{A}$ can be written as the product of a lower triangular matrix and its transpose. We are, once again, dealing with real matrices, but this decomposition applies to complex matrices as well.

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$$

The algorithm to perform this factorization is written in terms of block matrices. We first partition $\boldsymbol{A}$ and $\boldsymbol{L}$ into blocks as shown in Eqn (12.7). We keep the element in the first row, first column as one block, and partition the rest of each matrix into a column vector ($\boldsymbol{a}_{21}, \boldsymbol{l}_{21} \in \mathbb{R}^{n-1}$), its transpose, and a smaller matrix like $\boldsymbol{A}_{22}, \boldsymbol{L}_{22} \in \mathbb{R}^{(n-1)\times(n-1)}$.

$$\boldsymbol{A} = \left[\begin{array}{c|c} a_{11} & \boldsymbol{a}_{21}^{\mathsf{T}} \\ \hline \boldsymbol{a}_{21} & \boldsymbol{A}_{22} \end{array}\right] \quad \boldsymbol{L} = \left[\begin{array}{c|c} l_{11} & \boldsymbol{0}^{\mathsf{T}} \\ \hline \boldsymbol{l}_{21} & \boldsymbol{L}_{22} \end{array}\right] \tag{12.7}$$

Note that we used the fact that $\boldsymbol{A}$ is symmetric in dividing up the top row of $\boldsymbol{A}$ as $[a_{11}\ \boldsymbol{a}_{21}^{\mathsf{T}}]$. Similarly, we could write the first row of $\boldsymbol{L}$ as $[l_{11}\ \boldsymbol{0}^{\mathsf{T}}]$ because it is lower triangular. And, we do not have to worry about the second factor $\boldsymbol{L}^{\mathsf{T}}$ at all because it is just the transpose of $\boldsymbol{L}$.

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}} \implies \left[\begin{array}{c|c} a_{11} & \boldsymbol{a}_{21}^{\mathsf{T}} \\ \hline \boldsymbol{a}_{21} & \boldsymbol{A}_{22} \end{array}\right] = \left[\begin{array}{c|c} l_{11} & \boldsymbol{0}^{\mathsf{T}} \\ \hline \boldsymbol{l}_{21} & \boldsymbol{L}_{22} \end{array}\right] \left[\begin{array}{c|c} l_{11} & \boldsymbol{l}_{21}^{\mathsf{T}} \\ \hline \boldsymbol{0} & \boldsymbol{L}_{22}^{\mathsf{T}} \end{array}\right]$$

Expanding the matrix multiplication $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$ using the block matrices, and comparing the corresponding elements in $\boldsymbol{A}$ and $\boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$:

$$\begin{aligned} \left[\begin{array}{c|c} a_{11} & \boldsymbol{a}_{21}^{\mathsf{T}} \\ \hline \boldsymbol{a}_{21} & \boldsymbol{A}_{22} \end{array}\right] &= \left[\begin{array}{c|c} l_{11}^2 & l_{11}\boldsymbol{l}_{21}^{\mathsf{T}} \\ \hline l_{11}\boldsymbol{l}_{21} & \boldsymbol{l}_{21}\boldsymbol{l}_{21}^{\mathsf{T}} + \boldsymbol{L}_{22}\boldsymbol{L}_{22}^{\mathsf{T}} \end{array}\right] \\ a_{11} = l_{11}^2 &\implies l_{11} = \sqrt{a_{11}} \\ \boldsymbol{a}_{21} = l_{11}\boldsymbol{l}_{21} &\implies \boldsymbol{l}_{21} = \frac{\boldsymbol{a}_{21}}{l_{11}} = \frac{\boldsymbol{a}_{21}}{\sqrt{a_{11}}} \\ \boldsymbol{A}_{22} = \boldsymbol{l}_{21}\boldsymbol{l}_{21}^{\mathsf{T}} + \boldsymbol{L}_{22}\boldsymbol{L}_{22}^{\mathsf{T}} &\implies \boldsymbol{A}_{22} - \boldsymbol{l}_{21}\boldsymbol{l}_{21}^{\mathsf{T}} = \boldsymbol{L}_{22}\boldsymbol{L}_{22}^{\mathsf{T}} \end{aligned} \tag{12.8}$$

The second and third last lines in Eqn (12.8) tell us the elements of $\boldsymbol{L}$. Note that we decide to go with the positive square root for l_{11}. The very last line tells us that once we get l_{11} and $\boldsymbol{l}_{21}$, the problem reduces to computing the Cholesky decomposition of a smaller matrix $\boldsymbol{A}' = \boldsymbol{A}_{22} - \boldsymbol{l}_{21}\boldsymbol{l}_{21}^{\mathsf{T}}$.

Before we write it down as a formal algorithm, the only thing left to do is to ensure that $\boldsymbol{A}'$ is positive definite. Otherwise, we are not allowed to assume that we can find $\boldsymbol{A}' = \boldsymbol{L}'\boldsymbol{L}'^{\mathsf{T}}$. In the fourth test for positive definiteness (on 251), we proved that the upper-left submatrices of a positive definite matrix were also positive definite. If we look closely at the proof, we can see that we did not need to confine ourselves to "upper-left": Any submatrix sharing the main diagonal is positive definite. Therefore, if $\boldsymbol{A}$ in our Cholesky factorization is positive definite, so is $\boldsymbol{A}_{22}$. We also saw that sums of positive definite matrices are positive definite. Extending it to nontrivial (meaning, non-zero) differences, we can see that $\boldsymbol{A}' = \boldsymbol{A}_{22} - \boldsymbol{l}_{21}\boldsymbol{l}_{21}^{\mathsf{T}}$ is positive definite[6].

[6]To be very precise, $\boldsymbol{l}_{21}\boldsymbol{l}_{21}^{\mathsf{T}}$ is a rank-one matrix, and is only positive semidefinite. But the difference is still positive definite.

Looking at Eqn (12.8), we can translate it to an algorithm[7] as below:

Input: $\boldsymbol{A} \in \mathbb{R}^{n \times n}$
Output: $\boldsymbol{L} \in \mathbb{R}^{n \times n}$
1: *Initialize* $\boldsymbol{L}$ *with zeros*
2: **repeat**
3: *Divide* $\boldsymbol{A}$ *into block matrices as in Eqn* (12.7)
4: $l_{11} \leftarrow \sqrt{a_{11}}$
5: $\boldsymbol{l}_{21} \leftarrow \frac{\boldsymbol{a}_{21}}{l_{11}}$
6: $\boldsymbol{A} \leftarrow \boldsymbol{A}_{22} - \boldsymbol{l}_{21}\boldsymbol{l}_{21}^{\mathsf{T}}$
7: **until** $\boldsymbol{A}_{22}$ *becomes a scalar*
8: **return** *The matrix* $\boldsymbol{L}$

Applications of Cholesky Decomposition

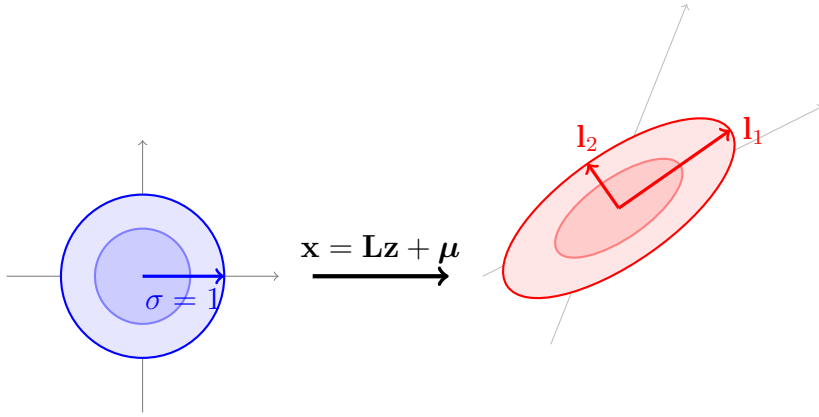
1. The $\boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$ decomposition is faster in solving systems of linear equations than Gauss-Jordan by about a factor of two. The catch is that it applies only when the coefficient matrix $\boldsymbol{A}$ is positive definite.

2. In simulation programs, we usually face the problem of having to work with multivariate normal distributions, where we need to simulate correlated, normally distributed variables, which are specified by their covariance matrix ($\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$) with means ($\boldsymbol{\mu} \in \mathbb{R}^{n}$).

   The standard way of accomplishing this task is to draw the required number (n) of random variables for a standard normal distribution (with $\sigma = 1, \mu = 0$) and multiply this random vector ($\boldsymbol{x} \in \mathbb{R}^{n}$) by the lower triangular matrix $\boldsymbol{L}$ and shift it by $\boldsymbol{\mu}$, where $\boldsymbol{L}$ comes from the Cholesky Decomposition of $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$.

   The required vector of random numbers would then be $\boldsymbol{L}\boldsymbol{x} + \boldsymbol{\mu}$, which would have the right covariance matrix ($\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$) and means ($\boldsymbol{\mu} \in \mathbb{R}^{n}$).

[7] This description and the algorithm listed are based on the discussion in this video.



$\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{L}\mathbf{L}^T)$

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

13

Singular Value Decomposition

What is best in mathematics deserves not merely to be learnt as a task, but to be assimilated as a part of daily thought, and brought again and again before the mind with ever-renewed encouragement.

—Bertrand Russell



Singular Value Decomposition (SVD) has come into prominence in the last couple of decades because of its direct applicability in computer science algorithms, especially when dealing with large volumes of data. It finds applications in, for instance, data compression, dimensionality reduction, principal component analysis, and more.

From a mathematical perspective, SVD is a topic that embodies pure elegance. It brings a large part of what we discussed in this book (eigenanalysis, vector spaces and bases, orthogonality, the four fundamental subspaces, etc.) together into one cohesive whole. In many ways, therefore, Singular Value Decomposition is the perfect closing chapter for an introductory book on linear algebra.

13.1 The Mechanics of SVD: Intuition and Computation

13.1.1 What SVD Does

In eigenvalue decomposition (EVD), we took a square matrix $\boldsymbol{A}$ and wrote it as:

$$\boldsymbol{AS} = \boldsymbol{S\Lambda}$$

where $\boldsymbol{\Lambda}$ is a matrix with eigenvalues on the diagonal, and $\boldsymbol{S}$ is the matrix where we have the eigenvectors as columns. For real, symmetric matrices, we saw that $\boldsymbol{S}$ was orthogonal, meaning its transpose is the same as its inverse, and we wrote:

$$\boldsymbol{A} = \boldsymbol{Q\Lambda Q^{\mathsf{T}}}$$

Since an orthogonal matrix represents a rotation, we can interpret this decomposition geometrically. In $\boldsymbol{Ax} = \boldsymbol{Q\Lambda Q^{\mathsf{T}}x}$, what $\boldsymbol{A}$ does to $\boldsymbol{x}$ (how it transforms $\boldsymbol{x}$) is the same as a rotation ($\boldsymbol{Q^{\mathsf{T}}}$), followed by a scaling ($\boldsymbol{\Lambda}$), and then another rotation ($\boldsymbol{Q}$) by the same angle(s) in the opposite direction.

This interpretation of eigenvalue decomposition (EVD), which we called the Spectral Theorem, works only for real, symmetric matrices. In fact, the computation of eigenvalues and eigenvectors can be done only for square matrices. Given that the data matrices we deal with are often far from square, these constraints severely restrict the applicability of EVD in computer science. What SVD does is decompose *any* matrix into a rotation, followed by a scaling, followed by another rotation, viewing $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ as a transformation taking a vector $\boldsymbol{x}$ from $\mathbb{R}^n$ to $\mathbb{R}^m$.

To reiterate, when we apply SVD, $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ can be any matrix. It does not have to be symmetric, nor even square. Although it does not need to be real either, we will again restrict ourselves to real matrices. To assign symbols to these verbose statements:

$$\begin{gathered}\boldsymbol{A} \in \mathbb{R}^{m\times n} \xrightarrow{\text{SVD}} \boldsymbol{A} = \boldsymbol{U\Sigma V^{\mathsf{T}}} \\ \boldsymbol{U} \in \mathbb{R}^{m\times m},\ \boldsymbol{\Sigma} \in \mathbb{R}^{m\times n},\ \boldsymbol{V} \in \mathbb{R}^{n\times n}\end{gathered} \tag{13.1}$$

$\boldsymbol{\Sigma}$ is a diagonal matrix (or as diagonal as a non-square matrix can be), with strictly positive values along the main diagonal, starting from σ_1 all the way to σ_r, where $r = \text{rank}(\boldsymbol{A})$, with the rest of the elements being all zero. Note also that the action of a diagonal matrix on a

matrix $\boldsymbol{X}$ by multiplying it on the *left* ($\boldsymbol{\Sigma X}$) is to scale the ith row of $\boldsymbol{X}$ by σ_i.

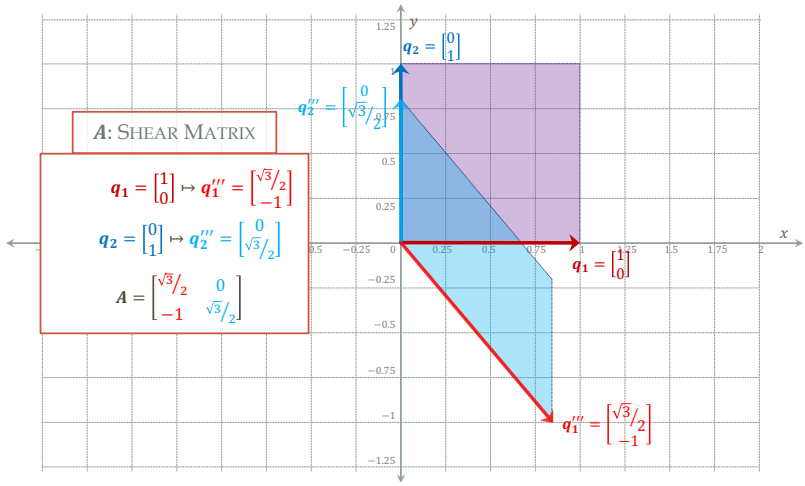


Fig. 13.1 The shear matrix under SVD analysis.

Before learning how SVD does its magic, let's take a look at an example. For ease of visualization, we will work with a square matrix so that we can draw the vectors and their transformations in $\mathbb{R}^2$.

$$\boldsymbol{A} = \begin{bmatrix} \frac{\sqrt{3}}{2} & 0 \\ -1 & \frac{\sqrt{3}}{2} \end{bmatrix} \xrightarrow{\text{SVD}} \boldsymbol{A} = \boldsymbol{U\Sigma V^{\mathsf{T}}}$$

$$\boldsymbol{U} = \begin{bmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{bmatrix} \quad \boldsymbol{\Sigma} = \begin{bmatrix} \frac{3}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \quad \boldsymbol{V} = \begin{bmatrix} \frac{\sqrt{3}}{2} & \frac{1}{2} \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix}$$

A few points to note about these matrices:

- We do not yet know how we mathematically derived the decomposition $\boldsymbol{A} = \boldsymbol{U\Sigma V^{\mathsf{T}}}$, but we can verify its validity by multiplying them out. In fact, what we did to come up with the "decomposition" above was start from the $\boldsymbol{U}, \boldsymbol{\Sigma}$, and $\boldsymbol{V}$ matrices and take their product to get $\boldsymbol{A}$.
- $\boldsymbol{U}$ is indeed a rotation, as described in Figure 7.2, with the angle of rotation $\theta = -\frac{\pi}{3}$, which is a clockwise rotation of 60°.
- Similarly, $\boldsymbol{V}$ is a rotation $\theta = -\frac{\pi}{6}$, or clockwise 30°. Therefore, $\boldsymbol{V^{\mathsf{T}}}$ (being the same as $\boldsymbol{V^{-1}}$) is an anti-clockwise rotation of 30°.

- Most importantly, as shown in Figure 13.1, the matrix $\boldsymbol{A}$ is a vertical shear matrix: It shears the x-component down (proportional to the y-component), and leaves the y-component alone, but for some scaling. As we learned earlier (in §11.1.1, page 211 and the explanation on page 215), shear matrices do not have a full set of eigenvectors and cannot be diagonalized. But they do have a singular value decomposition, as do all matrices.

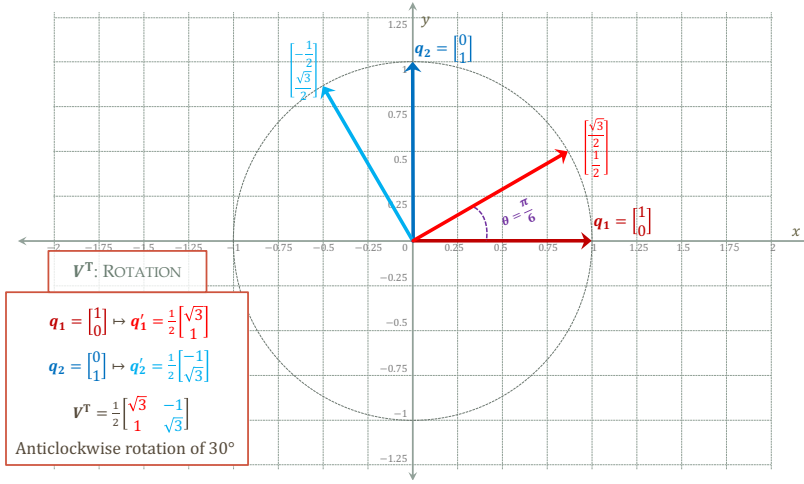
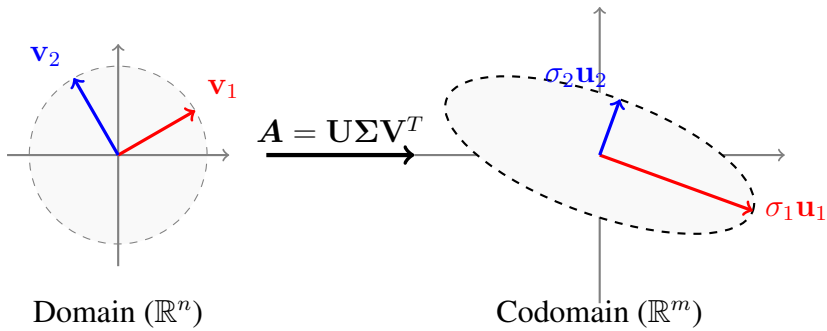


Fig. 13.2 The first rotation by $\boldsymbol{V}^{\mathsf{T}}$, an anti-clockwise rotation of 30°.



Figures 13.2 to 13.5 show the actions of these three matrices alongside a visual summary. The transformation of a vector $\boldsymbol{x}$ by $\boldsymbol{A}\boldsymbol{x}$ is broken down into $\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathsf{T}}\boldsymbol{x}$.

The first multiplication by $\boldsymbol{V}^{\mathsf{T}}$ is a rotation, shown in Figure 13.2. It does not change the size of any vector $\boldsymbol{x}$, nor the size of the basis vectors $\boldsymbol{q}_1$ and $\boldsymbol{q}_2$ shown in red and blue. In their new, rotated

positions, they are shown in lighter red and blue. The unit vectors all have their tips on the unit circle, as shown in Figure 13.2. Because of the rotation (of 30°) by $\boldsymbol{V}^{\mathsf{T}}$, all the vectors originally in the first quadrant are now bounded between the light red and blue vectors $\boldsymbol{q}_1'$ and $\boldsymbol{q}_2'$.

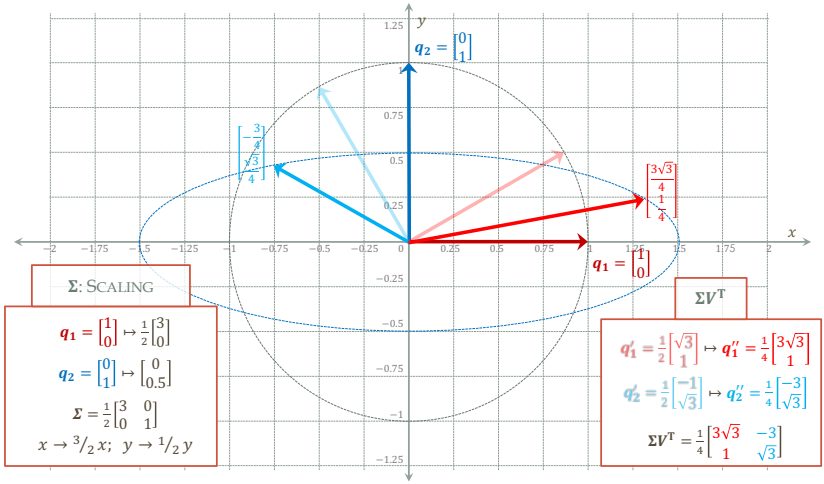


Fig. 13.3 The second scaling by $\boldsymbol{\Sigma}$, x-components by 1.5 and y by 0.5.

We then apply the scaling $\boldsymbol{\Sigma}$ to the product $\boldsymbol{V}^{\mathsf{T}}\boldsymbol{x}$, which scales along the *original* (*not* the rotated) unit vectors $\boldsymbol{q}_1$ and $\boldsymbol{q}_2$. In Figure 13.3, we can see how the rotated unit vectors (now shown in translucent red and blue) get transformed into their new versions. What $\boldsymbol{\Sigma}$ does to the unit vectors $\boldsymbol{q}_i$, it does to all vectors. Therefore, the unit circle gets elongated along the x-direction and squashed along the y-direction.

What we mean by this statement is that all vectors whose tips are on the unit circle get transformed such that their tips end up on the resulting ellipse. As part of this transformation, the rotated unit vectors—the translucent red and blue vectors $\boldsymbol{q}_1'$ and $\boldsymbol{q}_2'$—get transformed to $\boldsymbol{q}_1''$ and $\boldsymbol{q}_2''$ (in brighter colors) on the ellipse. In other words, the effect of the first two transformations, the product $\boldsymbol{\Sigma}\boldsymbol{V}^{\mathsf{T}}$, is to move all the vectors in the first quadrant of the unit circle to the arc of the ellipse between the bright red and blue vectors $\boldsymbol{q}_1''$ and $\boldsymbol{q}_2''$.

Notice that the transformed ellipse in Figure 13.3 has its primary axes perfectly aligned with the x and y directions.

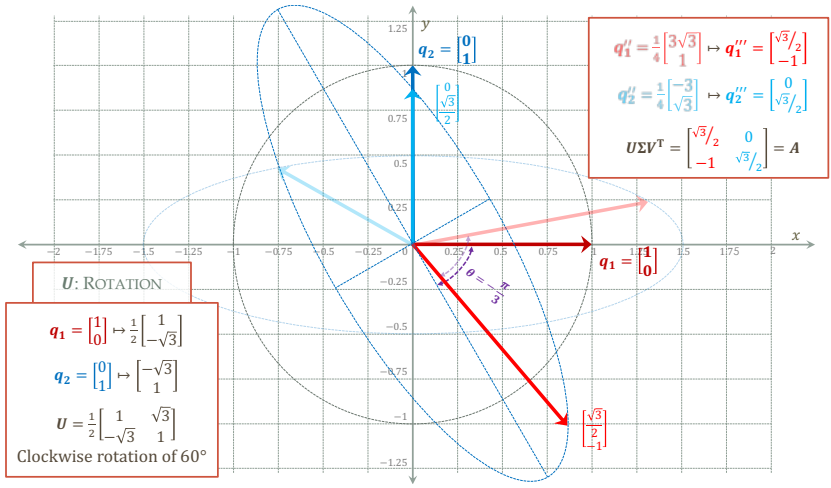


Fig. 13.4 The third rotation by $\boldsymbol{U}$, a clockwise rotation of 60°.

The last step, shown in Figure 13.4, is the rotation by $\boldsymbol{U}$. Based on our earlier calculation, it is a *clockwise* rotation of 60°. It rotates the entire ellipse through that angle. Remembering that the axes of the ellipse after the scaling (in Figure 13.3) were aligned with the original unit vectors, we can clearly see how the whole geometry gets rotated.

The $\boldsymbol{\Sigma V^T}$-transformed versions of the unit vectors, now shown in translucent red and blue in Figure 13.4 as $\boldsymbol{q}_1''$ and $\boldsymbol{q}_2''$, get rotated to the bright red vector with its tip at $(\frac{\sqrt{3}}{2}, -1)$ and the bright blue vector with its tip at $(0, \frac{\sqrt{3}}{2})$. This is exactly what the shear matrix $\boldsymbol{A}$ does, as originally illustrated in Figure 13.1.

Now we are in a position to summarize what $\boldsymbol{A}$ does, either through the direct path $\boldsymbol{Ax}$ or by the three steps $\boldsymbol{U\Sigma V^T x}$, as shown in Figure 13.5.

We may be tempted to think, from the figures, that one rotation and one scaling might be enough to do everything that $\boldsymbol{A}$ does. We should, however, note that the original first quadrant of the unit circle in Figure 13.2 is mapped to the arc of the ellipse between the light red and blue vectors in Figure 13.3.

One rotation and one scaling *could* give us this mapping, but the resulting ellipse would be forced to have its axes along the unit vectors. Or, by applying a scaling and then a rotation, we could get

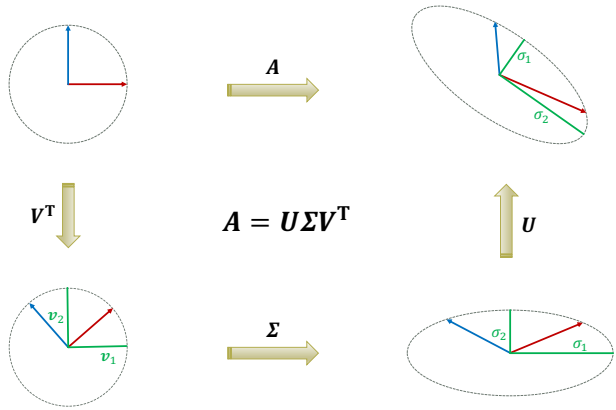


Fig. 13.5 The geometry of SVD, summarized.

to the correctly angled ellipse as in Figure 13.4, but the points from the first quadrant of the unit circle would incorrectly map to the first quadrant of the ellipse. We really do need two separate rotations and a scaling to achieve the exact transformation.

The story in the general case of $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, of course, is much more complicated, but the fundamental idea is exactly the same. We use our intuitions from $\mathbb{R}^2$ in order to harness the power of SVD in higher dimensions.

13.1.2 How SVD Works

From our discussions on the four fundamental subspaces defined by a matrix $\boldsymbol{A}$, we know that its row space $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$ gets mapped to its column space $\mathcal{C}(\boldsymbol{A})$. Furthermore, the mapping is one-to-one and onto, which means a vector $\boldsymbol{v} \in \mathcal{C}(\boldsymbol{A}^\mathbf{T})$ gets mapped to a unique vector $\boldsymbol{u} \in \mathcal{C}(\boldsymbol{A})$. Both $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$ and $\mathcal{C}(\boldsymbol{A})$ have the same dimension, which is the rank of $\boldsymbol{A}$, r. In other words, we should be able to find r orthogonal vectors in $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$ to span the row space; we could, for instance, apply the Gram-Schmidt algorithm to do it.

Let's say that these r orthonormal basis vectors in $\mathcal{C}(\boldsymbol{A}^\mathbf{T})$ are $\boldsymbol{v}_i$. When applying the transformation of $\boldsymbol{A}$ to them, we get some vectors in $\mathcal{C}(\boldsymbol{A})$: $\boldsymbol{A}\boldsymbol{v}_i = \boldsymbol{u}_i$. We know that these are r *unique* vectors $\boldsymbol{u}_i$ because $\boldsymbol{A} : \mathcal{C}(\boldsymbol{A}^\mathbf{T}) \rightarrow \mathcal{C}(\boldsymbol{A})$ is a one-to-one mapping. However, we have no reason to think that they are orthogonal or normalized.

We can insist that they be normalized by factoring out their norms: $\boldsymbol{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i$ with $\|\boldsymbol{u}_i\| = 1$.

The vectors $\boldsymbol{u}_i$ are unique because each of them is a different linear combination of the columns of $\boldsymbol{A}$ with the coefficients specified in $\boldsymbol{v}_i$, and the vectors $\boldsymbol{v}_i$ are linearly independent. Taking this as one of the last teachable moments in this textbook, let's harp on this point a bit more.

Since $\boldsymbol{v}_i \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$, we know that $\boldsymbol{A}\boldsymbol{v}_i \neq \boldsymbol{0}$. And since the vectors $\boldsymbol{v}_i$ form a basis for $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$, they are linearly independent, which means, at the very least, the components of one of them cannot all be the same as that of another. By the column picture of matrix multiplication, $\boldsymbol{u}_i = \boldsymbol{A}\boldsymbol{v}_i$ is a linear combination of the columns of $\boldsymbol{A}$ with different coefficients that are components of $\boldsymbol{v}_i$. And, from way back in the first or second chapter, we know that two different linear combinations $\sum s_i \boldsymbol{v}_i$ and $\sum t_i \boldsymbol{v}_i$ cannot be the same: Linear combinations are unique.

What we are demanding in the Singular Value Decomposition (SVD) of $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, where $\text{rank}(\boldsymbol{A}) = r \leq \min(m, n)$, is something remarkable. Our goal is to find a *special* orthonormal basis in the row space of $\boldsymbol{A}$ that gets mapped to an orthonormal basis in its column space. We want a basis $\boldsymbol{V}$ for $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$, and a basis $\boldsymbol{U}$ for $\mathcal{C}(\boldsymbol{A})$, with a *special* requirement: each $\boldsymbol{v}_i \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$ gets mapped to a $\boldsymbol{u}_i \in \mathcal{C}(\boldsymbol{A})$, with a possible scaling factor σ_i.

The remarkable fact behind SVD is that we will *always* be able to find such bases, and we shall shortly see why and how. We will have a mathematical proof and a recipe to find them.

Since the rank of the matrix is r, we can find only r linearly independent vectors in $\mathcal{C}(\boldsymbol{A})$ and $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$. We will worry about the rest of the column vectors in $\boldsymbol{U}$ and $\boldsymbol{V}$ later.

Let's write down all we have so far:

$$\hat{\boldsymbol{U}} = \begin{bmatrix} | & | & \cdots & | \\ \boldsymbol{u}_1 & \boldsymbol{u}_2 & \cdots & \boldsymbol{u}_r \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{m\times r} = [\boldsymbol{u}_i]$$

$$\hat{\boldsymbol{V}} = \begin{bmatrix} | & | & \cdots & | \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_r \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{n\times r} = [\boldsymbol{v}_i]$$

We are putting a hat on the $\boldsymbol{U}$ and $\boldsymbol{V}$ matrices because they are smaller, more economical ones for now. They are not the full-sized versions from Eqn (13.1).

Each $\boldsymbol{v}_i \in \mathcal{C}(\boldsymbol{A}^\mathsf{T})$ is going to be mapped to a corresponding $\boldsymbol{u}_i \in \mathcal{C}(\boldsymbol{A})$, as per our requirement. We then have the following, where we have arranged $\sigma_i, 1 \leq i \leq r$ as the diagonal elements in $\hat{\boldsymbol{\Sigma}} \in \mathbb{R}^{r \times r}$. Remember that we require the columns of $\hat{\boldsymbol{V}}$ and $\hat{\boldsymbol{U}}$ to be orthonormal.

$$\boldsymbol{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i \implies \boldsymbol{A}\hat{\boldsymbol{V}} = \hat{\boldsymbol{U}}\hat{\boldsymbol{\Sigma}}$$

Remembering that for a matrix with orthonormal columns, the product of its transpose with itself is the identity matrix. For instance, the product $\hat{\boldsymbol{U}}^\mathsf{T}\hat{\boldsymbol{U}}$ for $\hat{\boldsymbol{U}} \in \mathbb{R}^{m \times r}$ is $\boldsymbol{I}_r \in \mathbb{R}^{r \times r}$ because the elements in the product are the dot products of the columns of $\hat{\boldsymbol{U}}$, which are either zero or one. We should not, however, call $\hat{\boldsymbol{U}}$ an orthogonal matrix because it is not square. The same argument applies to $\hat{\boldsymbol{V}}^\mathsf{T}\hat{\boldsymbol{V}}$ as well.

We can now do a bit of matrix algebra magic to compute $\hat{\boldsymbol{U}}$ and $\hat{\boldsymbol{V}}$:

(1) Orthonormal requirement: $\boldsymbol{A}\hat{\boldsymbol{V}} = \hat{\boldsymbol{U}}\hat{\boldsymbol{\Sigma}}$

(2) Taking the transpose of (1): $\hat{\boldsymbol{V}}^\mathsf{T}\boldsymbol{A}^\mathsf{T} = \hat{\boldsymbol{\Sigma}}^\mathsf{T}\hat{\boldsymbol{U}}^\mathsf{T}$

(3) Multiplying (2) and (1): $\hat{\boldsymbol{V}}^\mathsf{T}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\hat{\boldsymbol{V}} = \hat{\boldsymbol{\Sigma}}^\mathsf{T}\hat{\boldsymbol{U}}^\mathsf{T}\hat{\boldsymbol{U}}\hat{\boldsymbol{\Sigma}}$

(4) Since $\hat{\boldsymbol{U}}^\mathsf{T}\hat{\boldsymbol{U}} = \boldsymbol{I}$: $\hat{\boldsymbol{V}}^\mathsf{T}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\hat{\boldsymbol{V}} = \hat{\boldsymbol{\Sigma}}^\mathsf{T}\hat{\boldsymbol{\Sigma}}$

(5) Since $\hat{\boldsymbol{\Sigma}}$ is diagonal: $\hat{\boldsymbol{V}}^\mathsf{T}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\hat{\boldsymbol{V}} = \hat{\boldsymbol{\Sigma}}^2$

(6) Left/right multiply by $\hat{\boldsymbol{V}}, \hat{\boldsymbol{V}}^\mathsf{T}$: $\hat{\boldsymbol{V}}\hat{\boldsymbol{V}}^\mathsf{T}\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\hat{\boldsymbol{V}}\hat{\boldsymbol{V}}^\mathsf{T} = \hat{\boldsymbol{V}}\hat{\boldsymbol{\Sigma}}^2\hat{\boldsymbol{V}}^\mathsf{T}$

(7) If $\hat{\boldsymbol{V}}\hat{\boldsymbol{V}}^\mathsf{T} = \boldsymbol{I}$: $\boldsymbol{A}^\mathsf{T}\boldsymbol{A} = \hat{\boldsymbol{V}}\hat{\boldsymbol{\Sigma}}^2\hat{\boldsymbol{V}}^\mathsf{T}$

In statement (7), we specify a conditional: If $\hat{\boldsymbol{V}}\hat{\boldsymbol{V}}^\mathsf{T} = \boldsymbol{I}$, then we get an equation involving $\hat{\boldsymbol{V}}$ and $\hat{\boldsymbol{\Sigma}}^2$, where we can identify an eigenvalue equation, much like the Spectral Theorem, $\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\mathsf{T}$. It then follows that the singular value decomposition is the eigenvalue decomposition of the product $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ and the singular values are the positive square roots of its eigenvalues.

The condition $\hat{\boldsymbol{V}}\hat{\boldsymbol{V}}^\mathsf{T} = \boldsymbol{I}$ is a problem, though. We proved that $\hat{\boldsymbol{V}}^\mathsf{T}\hat{\boldsymbol{V}} = \boldsymbol{I}$. Can we also show that $\hat{\boldsymbol{V}}\hat{\boldsymbol{V}}^\mathsf{T} = \boldsymbol{I}$? It turns out that

we cannot, for one very good reason: It is not true. One way to get around this difficulty is to consider only full-column-rank matrices $\boldsymbol{A}$, in which case we have $\text{rank}(\boldsymbol{A}) = r = n$ and a square matrix $\boldsymbol{V} \in \mathbb{R}^{n \times n}$. We will drop the hat on $\boldsymbol{V}$ because it is now the basis for the whole of the domain of $\boldsymbol{A}$, namely $\mathbb{R}^n$.

What we now have are the matrices as shown in Figure 13.6. Following the same logic, we can show that

$$\boldsymbol{A}\boldsymbol{A}^\mathsf{T} = \hat{\boldsymbol{U}}\hat{\boldsymbol{\Sigma}}^2\hat{\boldsymbol{U}}^\mathsf{T}$$

with the proviso that $\boldsymbol{A}$ be a full-row-rank matrix in this case.

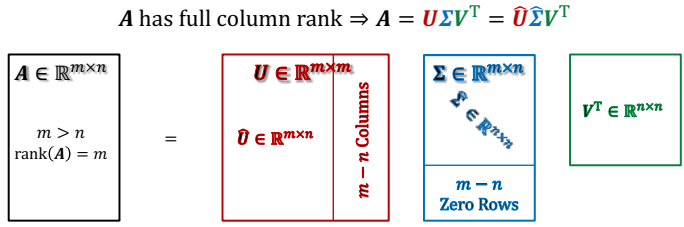


Fig. 13.6 The shapes of $\boldsymbol{U}, \boldsymbol{\Sigma}$ and $\boldsymbol{V}$ in SVD when $\boldsymbol{A}$ is a full-column-rank matrix.

As for $\hat{\boldsymbol{\Sigma}}^2 = [\sigma_i^2]$, it is the square of a diagonal matrix. It is, therefore, simply another matrix with the squared elements along the diagonal.

Comparing these results with our eigenvalue equations in the Spectral Theorem (§12.1.1, page 246), we can see that these are the exact eigenvalue equations for $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ and $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$. Therefore, we conclude that the *left-singular vectors* $\boldsymbol{u}_i$ are the eigenvectors of $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$ and the *right-singular vectors* $\boldsymbol{v}_i$ are those of $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$. And the *singular values* are the square root of the eigenvalues of either $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ or $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$. As we saw in one of the exercises in the previous chapter, the non-zero eigenvalues of $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ are identically the same as those of $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$.

Although we derived the formulas for the singular vectors using the provisos of the matrix $\boldsymbol{A}$ being full-column or full-row rank, they are indeed valid for all matrices. The right approach would have been to complete the orthonormal basis on the domain side (from $\hat{\boldsymbol{V}}$, which is the basis for the row space) to all of $\mathbb{R}^n$ (by adding the basis for the null space as well). Similarly, we should have completed the basis on the output space ($\mathbb{R}^m$) by including the basis vectors of the left null space so that we can drop the hats on both $\boldsymbol{V}$ and $\boldsymbol{U}$ and get the

picture shown in Figure 13.7. It is perhaps wise to repeat these wordy statements with more mathematical precision as in the following list, where we will use the symbol $\mathcal{B}$ to denote a basis.

- $\hat{\boldsymbol{V}}$ is a basis for the row space of $\boldsymbol{A}$: $\hat{\boldsymbol{V}} = \mathcal{B}\{\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})\}$
- $\boldsymbol{V}$ is a basis for the domain, which is the union of the bases for the row and null spaces of $\boldsymbol{A}$:

$$\boldsymbol{V} = \mathcal{B}\{\mathbb{R}^n\} = \mathcal{B}\{\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})\} \cup \mathcal{B}\{\mathcal{N}(\boldsymbol{A})\}$$

- $\hat{\boldsymbol{U}}$ is a basis for the column space of $\boldsymbol{A}$: $\hat{\boldsymbol{U}} = \mathcal{B}\{\mathcal{C}(\boldsymbol{A})\}$
- $\boldsymbol{U}$ is a basis for the output space (AKA co-domain), which is the union of the bases for the column and the left null spaces of $\boldsymbol{A}$:

$$\boldsymbol{U} = \mathcal{B}\{\mathbb{R}^m\} = \mathcal{B}\{\mathcal{C}(\boldsymbol{A})\} \cup \mathcal{B}\{\mathcal{N}(\boldsymbol{A}^{\mathsf{T}})\}$$

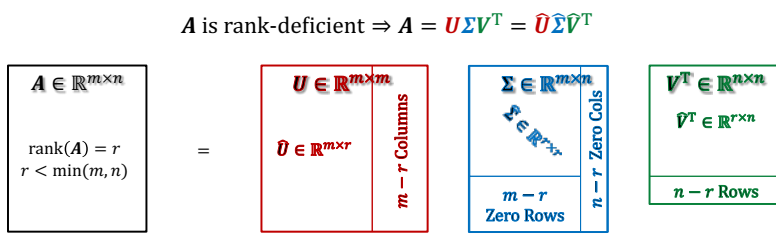


Fig. 13.7 The shapes of $\boldsymbol{U}$, $\boldsymbol{\Sigma}$ and $\boldsymbol{V}$ in SVD when $\boldsymbol{A}$ is a general, rank-deficient matrix.

Now we can confidently state $\boldsymbol{V}^{-1} = \boldsymbol{V}^{\mathsf{T}}$ and $\boldsymbol{U}^{-1} = \boldsymbol{U}^{\mathsf{T}}$. The matrix algebra we performed earlier can be repeated without the hats and everything is perfect.

We have one more way of arriving at this realization, by noting that since these Gram matrices, $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ and $\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}$, have the same rank as $\boldsymbol{A}$ (which is r), we have r positive eigenvalues and as many orthogonal eigenvectors. The rest of the eigenvalues ($n - r$ for $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ and $m - r$ for $\boldsymbol{A}\boldsymbol{A}^{\mathsf{T}}$) are zeros.

Remembering that the eigenvectors corresponding to zero eigenvalues are, in fact, the basis for the null space of the matrix, we can complete the $\boldsymbol{U}, \boldsymbol{\Sigma}$ and $\boldsymbol{V}$ (the unhatted ones) matrices. To get the final answer of SVD, we pad $\boldsymbol{U}$ with the basis of the left null space

$\mathcal{N}(\boldsymbol{A}^\mathsf{T})$ and $\boldsymbol{V}$ with the basis of the null space $\mathcal{N}(\boldsymbol{A})$, which are really the eigenvectors of $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$ and $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ corresponding to zero. They are, indeed, the missing left and right singular vectors we are seeking.

And the $\boldsymbol{\Sigma}$ matrix gets the correct zero eigenvalues to pad from the $(r+1)^{\text{th}}$ to the nth diagonal position, resulting in Figure 13.7 and the following equations:

$$\begin{aligned}
\boldsymbol{U} &= \begin{bmatrix} | & | & \cdots & | \\ \boldsymbol{u}_1 & \boldsymbol{u}_2 & \cdots & \boldsymbol{u}_m \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{m\times m} = [\boldsymbol{u}_i] \\
\boldsymbol{V} &= \begin{bmatrix} | & | & \cdots & | \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_n \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{n\times n} = [\boldsymbol{v}_i] \\
\boldsymbol{\Sigma} &= \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & \vdots & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \cdots & \vdots \\ 0 & 0 & \cdots & \sigma_r & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{m\times n} = [\sigma_i]
\end{aligned} \tag{13.2}$$

The $\boldsymbol{\Sigma}$ matrix is arranged such that $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ so that the first singular value is the most important one. The singular vectors are eigenvectors of $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ and $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$:

$$\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\boldsymbol{V} = \boldsymbol{V}\boldsymbol{\Sigma}^2 \quad \text{and} \quad \boldsymbol{A}\boldsymbol{A}^\mathsf{T}\boldsymbol{U} = \boldsymbol{U}\boldsymbol{\Sigma}^2 \tag{13.3}$$

We can summarize the singular value decomposition of $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\mathsf{T}$ (where $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ and $\text{rank}(\boldsymbol{A}) = r$) in a few bullet points:

1. The matrix $\boldsymbol{V} \in \mathbb{R}^{n\times n}$ is the right[1] singular matrix.
   (a) It is an orthonormal basis for $\mathbb{R}^n$, the domain of the transformation $\boldsymbol{A} : \mathbb{R}^n \to \mathbb{R}^m$.
   (b) The columns of $\boldsymbol{V}$ ($\boldsymbol{v}_i$) are the *right* singular vectors.
   (c) $\boldsymbol{v}_i, 0 < i \leq r$ are eigenvectors of $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$.
   (d) The first r of them span the *row* space of $\boldsymbol{A}, \mathcal{C}(\boldsymbol{A}^\mathsf{T})$.

[1] To remember whether $\boldsymbol{U}$ or $\boldsymbol{V}$ is left or right, note that the left singular matrix $\boldsymbol{U}$ appears on the left in $\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\mathsf{T}$ and $\boldsymbol{V}$ on the right.

(e) The rest $(n - r)$ span its null space, $\mathcal{N}(\boldsymbol{A})$.

(f) $\boldsymbol{v}_i, r < i \leq n$ are found (if needed) by computing the null space.

2. $\boldsymbol{U} \in \mathbb{R}^{m \times m}$ is the left singular matrix.

   (a) It is an orthonormal basis for $\mathbb{R}^m$, the output space (AKA the co-domain) of the transformation $\boldsymbol{A} : \mathbb{R}^n \to \mathbb{R}^m$.

   (b) Its columns, $\boldsymbol{u}_i$, are the *left* singular vectors.

   (c) $\boldsymbol{u}_i, 0 < i \leq r$ are the eigenvectors of $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$.

   (d) Again, the first r of them form a basis for the *column* space $\mathcal{C}(\boldsymbol{A})$.

   (e) The rest $(m-r)$ of them span the left null space, $\mathcal{N}(\boldsymbol{A}^\mathsf{T})$.

   (f) $\boldsymbol{u}_i, r < i \leq m$ are found (if needed) by computing the left null space.

3. The diagonal matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{m \times n}$ (the same size as $\boldsymbol{A}$) contains the r singular values.

   (a) The first r singular values, $\sigma_i, 0 < i \leq r$ are strictly positive.

   (b) They are the non-negative square roots of the eigenvalues of $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$ (for tall matrices) or $\boldsymbol{A}\boldsymbol{A}^\mathsf{T}$ (for wide matrices).

   (c) The rest of the singular values, $\sigma_i, r < i \leq \min(m, n)$, are all exactly zeros.

   (d) By convention, the singular values are arranged in descending order:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

These bullet points are illustrated in Figure 13.7, the full-blown SVD. Since only the first r singular values are non-zero (and also because the basis vectors $\boldsymbol{v}_i$ of the null space map to $\boldsymbol{0} \in \mathbb{R}^m$), we can ignore the last $(m - r)$ columns of $\boldsymbol{U}$ and the $(n - r)$ columns of $\boldsymbol{V}$.

We can also take the leading $r \times r$ part of $\boldsymbol{\Sigma}$ to come up with a "thin" or "economical" version of SVD. Note that this version does not involve any approximations; it is merely a choice of ignoring the zeros that do not fundamentally matter anyway.

We also have the following properties for singular values:

- For square matrices, the product of the singular values is the absolute value of the determinant. For instance, for $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$, $\sigma_1\sigma_2 = |\lambda_1\lambda_2| = ||\boldsymbol{A}||$.
- For $\boldsymbol{A} \in \mathbb{R}^{2\times 2}$, $\sigma_1 \geq |\lambda_1| \geq |\lambda_2| \geq \sigma_2$.

13.2 Applications: Low-Rank Approximation and Data Compression

13.2.1 Data Compression

Remember that the Spectral Theorem (§12.1.1, page 246) allowed us to write:

$$\boldsymbol{A} = \sum_{i=1}^{n} \lambda_i \boldsymbol{q}_i \boldsymbol{q}_i^{\mathsf{T}}$$

from the eigenvalue decomposition (EVD) of a real, symmetric $\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{\mathsf{T}} \in \mathbb{R}^{n\times n}$.

In SVD, we have $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathsf{T}} \in \mathbb{R}^{m\times n}$ (for *any* matrix). This product can also be expanded as a sum of rank-one matrices:

$$\boldsymbol{A} = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^{\mathsf{T}} \tag{13.4}$$

where we used the fact from Eqn (13.2) that the diagonal matrix $\boldsymbol{\Sigma}$ has only the first $r = \text{rank}(\boldsymbol{A})$ non-zero elements, and therefore, the sum runs from 1 only to r, not to m or n, the matrix dimensions.

It is perhaps important enough to reiterate that Eqn (13.4), by itself, is not an approximation just because we are summing only up to r, which is to say, we are using the "economical" (hatted) SVD matrices. Even if we were to use the full matrices, the multiplication $\boldsymbol{\Sigma}\boldsymbol{V}^{\mathsf{T}}$ would have resulted in a matrix ($\in \mathbb{R}^{m\times n}$) with the last $n - r$ rows zero because only the first r singular values ($\sigma_i, 0 < i \leq r$) are non-zero.

Each term in the summation in Eqn (13.4), $\boldsymbol{A}_i = \boldsymbol{u}_i \boldsymbol{v}_i^{\mathsf{T}}$, is a rank-one matrix because it is a linear combination of one row matrix $\boldsymbol{v}_i^{\mathsf{T}}$, by the row-picture of matrix multiplication. The first of them, $\boldsymbol{A}_1 = \boldsymbol{u}_1 \boldsymbol{v}_1^{\mathsf{T}}$, is the most important one because σ_1 is the largest among the singular values. We can therefore see that $\boldsymbol{A}_1$ is the best rank-one approximation of the original matrix $\boldsymbol{A}$.

Let's say $\boldsymbol{A}$ is a megapixel grayscale image of size 1000×1000. It takes a million bytes to store it. $\boldsymbol{A}_1$, on the other hand, takes up only 2001 bytes: 1 for (σ_1) + 1000 for ($\boldsymbol{u}_1$) + 1000 for ($\boldsymbol{v}_1$). If $\boldsymbol{A}_1$ is not good enough, we may include up to k such rank-one matrices at a storage cost of $2001k$, which is smaller than a million for $k < 499$. Typically, the first few tens of σ_i would be enough to keep most of the visual information in $\boldsymbol{A}$.

In general, in order to store up to k rank-one approximations of $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, we need $k(m+n+1)$ units of memory, which could be significantly smaller than mn. The Singular Value Decomposition, therefore, can be the basis of a powerful data compression algorithm. Although we have more powerful modern techniques than the plain old SVD for image compression, it is a technique that still inspires some researchers[2].

13.2.2 Principal Component Analysis (PCA)

In data science, SVD is commonly used for dimensionality reduction to uncover the low-dimensional patterns in the data. Although it is called Principal Component Analysis (PCA), it is essentially SVD using different jargon and with one extra step. We will stick with our current notation to describe PCA.

Let's say $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ is a data matrix, such as the Young Adult dataset we used in Figure 10.4 while studying multiple linear regression. Here, we have an observation as a row in the matrix, and variables along which the observations are made in the columns. We have m observations along n variables ($x_j, 1 \leq j \leq n$), and as we saw multiple times, $m \gg n$.

The first step in PCA is to compute the mean (as in average) of each column, and subtract it from all elements in that column.

$$\mu_j = \frac{1}{m} \sum_{i=1}^{m} a_{ij} = E[x_j]; \quad a_{ij} \leftarrow a_{ij} - \mu_j$$

Thus, we get the so-called zero-centered data in our $\boldsymbol{A}$ matrix (which we will call $\boldsymbol{A}_0$ to avoid confusion), where each column has a mean

[2] A quick search revealed this article from 2017.

of 0. Now, the Gram matrix $\boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0$ becomes a much smaller matrix in terms of the number of elements because $\boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0 \in \mathbb{R}^{n\times n}$ and $n \ll m$.

It also becomes proportional to the covariance matrix of the data. To see how this magic happens, we only need to expand the product. Calling $\boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0$ by a new name, $\boldsymbol{C} \in \mathbb{R}^{n\times n} = [c_{ij}] = \boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0$, we can write:

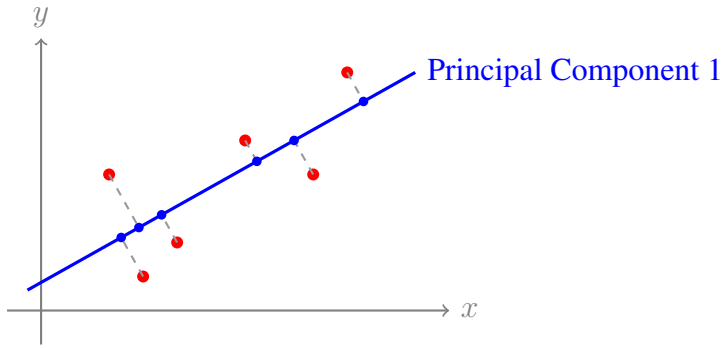
$$c_{ij} = \boldsymbol{a}_{i(0)}^\mathsf{T}\boldsymbol{a}_{j(0)} = \sum_{k=1}^{m}(a_{ki} - \mu_i)(a_{kj} - \mu_j)$$

which is proportional to the covariance between the ith and j^{th} variables. For example, if we set $i = j$ to look at the diagonal elements of $\boldsymbol{C}$, we get:

$$\sum_{k=1}^{m}(a_{ki} - \mu_i)^2 = m\,\mathrm{Var}(x_i)$$

which is the variance of the ith variable x_i (times the number of observations m).

Taking the SVD of $\boldsymbol{A}_0$ is mathematically identical to finding the eigenvalues and eigenvectors of $\boldsymbol{C}$, the elements of which are proportional to the covariances of the variables in the data. Therefore, as we saw in §11.2.3 (page 221), what PCA does is find directions along which the variance is the highest, second highest, and so on.



In other words, it gives us a hierarchical coordinate system, in which the first direction corresponds to the highest variance in the data, the next one the second highest, and so on. Note that it derives these directions from the data directly, without assuming any kind of underlying probability distribution for the variables.

Once we have the SVD of $\boldsymbol{A}_0$, we can look at the right singular vectors $\boldsymbol{v}_i$, which are the eigenvectors of $\boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0$ (as in Eqn (13.3))

and write:

$$\boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0\boldsymbol{V} = \boldsymbol{V}\boldsymbol{\Sigma}^2 \tag{13.5}$$

We know that the singular values are sorted so that σ_1 is the largest. Using the insights about what eigenanalysis does (from §11.2.3, page 221), we make the following observations:

1. The eigenvectors of the Gram matrix $\boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0$ represent the directions in its row space along which the variances in the data are sorted descending. Remembering that $\boldsymbol{A}_0^\mathsf{T}\boldsymbol{A}_0$ and $\boldsymbol{A}_0$ have the same row space from §12.2.2 (page 256), the eigenvectors in the columns of $\boldsymbol{V}$ are the directions in the data space along which we have the sorted variances as well.

2. The matrix $\boldsymbol{T} = \boldsymbol{A}_0\boldsymbol{V}$ holds the *principal components* of the data (the first of which is a linear combination of the columns of $\boldsymbol{A}_0$ with the largest variance, the second the second largest, and so on).

3. $\boldsymbol{V}$, whose columns specify the coefficients required in the linear combinations, is called the *loading* matrix.

4. The elements of $\boldsymbol{\Sigma}$ tell us how much variance each principal component holds. If we want, for instance, to capture 90% of the variance in the data, we start with the first component, and look at σ_1^2. If it is less than 90% of the total sum of squares, we keep adding the next component until the cumulative sum of σ_i^2 crosses the required threshold.

5. The first k principal components are then used as engineered features in our machine learning algorithm. Typically, $k \ll n$, resulting in significant dimensionality reduction. Although its description looks different, what we are doing here is effectively taking the first k terms in the summation in Eqn (13.4).

6. Dimensionality reduction has the added advantage of smoothing out the noise in the data. Much like fitting a line or a polynomial to a bunch of (x, y) data points may result in a better fit than the collection of the points themselves (which may be subject to statistical or measurement fluctuations), a lower-rank approximation of the data matrix may result in a more robust model.

We discussed the whole PCA topic as though it was a sub-topic of SVD. For historical reasons, however, PCA may appear with different jargon and notations in other sources, with no reference to SVD. The mathematics behind it is exactly the same as our discourse here. It has to be, for mathematical truth is singular.

A Simulated Example

As a well-established and widely used technique, PCA is available in all modern statistical applications. In order to make its discussion clear, we will use R and create a toy example using simulation. We are going to simulate a multivariate normal distribution, centered at $\boldsymbol{\mu}$ with a covariance matrix $\boldsymbol{S}$, generating 1000 tuples (x, y).

$$\boldsymbol{\mu} = \begin{bmatrix} 2.5 \\ 2.5 \end{bmatrix} \quad \boldsymbol{S} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Since we set the parameters for the simulation, we already know what to expect: We started with the covariance matrix $\boldsymbol{S}$, and we expect the bivariate normal distribution to show up as an ellipse, centered at $(2.5, 2.5)$, with the major and minor axes aligned along the eigenvectors $\boldsymbol{s}_1$ and $\boldsymbol{s}_2$ of $\boldsymbol{S}$. The lengths of the axes are going to be proportional to the eigenvalues λ_i, as we established in §11.2.3 (page 221).

Running the eigenvalue decomposition on $\boldsymbol{S}$ in R, we get:

```
eigen() decomposition
$values
[1] 3 1

$vectors
           [,1]       [,2]
[1,] 0.7071068 -0.7071068
[2,] 0.7071068  0.7071068
```

In this output, `$values` are our eigenvalues λ_i, which says the variances along the major and minor axes of the generated data in Figure 13.8, left panel, are 3 and 1. Therefore, the lengths of the axes (σ_i) are the square roots, namely about 1.732 and 1, which is what we see in Figure 13.8 on the left (except that we scaled the standard

deviations by a factor of two so that 95% of the points are within the ellipse).

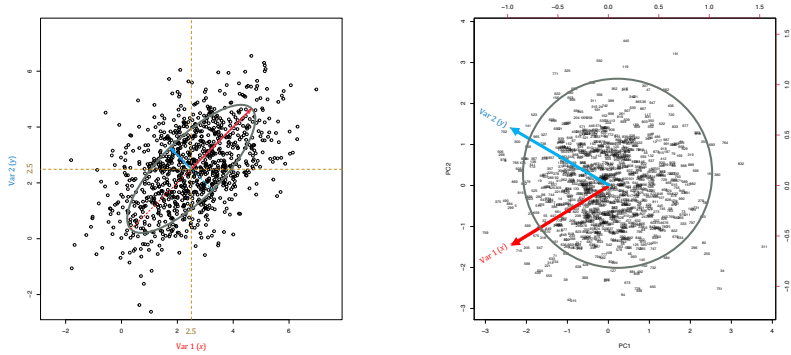


Fig. 13.8 Left: Example of simulated (x, y) pairs, showing the elliptical shape, the directions and sizes of the major and minor axes. Right: The "biplot" from the PCA function in R.

Once we have the 1000 generated rows of simulated data, we run PCA on it, and look at what it can reveal.

```
Standard deviations (1, .., p=2):
[1] 1.718043 1.018423

Rotation (n x k) = (2 x 2):
            PC1        PC2
[1,] -0.7043863 -0.7098168
[2,] -0.7098168  0.7043863
```

From the PCA output, we see the standard deviations σ_1 and σ_2 are close to what we specified in our true covariance matrix $\boldsymbol{S}$, as revealed by the eigenanalysis on it. Ideally, the values should have been $\sqrt{3}$ and $\sqrt{1}$, but we got 1.718 and 1.018—close enough for a random sample.

The first principal component is a linear combination of x and y with coefficients -0.704 and -0.710. Note that SVD vectors are unique in magnitude and direction up to a sign flip; their signs are not fixed.

The right panel in Figure 13.8 is the so-called "biplot" of the analysis, which shows a scatter plot between the first and second principal components, as well as the loading of the original variables.

PCA on Iris Dataset


Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

The first thing to note is that `PC1` and `PC2` are now uncorrelated standard normal distributions, as indicated by the circular spread in the biplot rather than the skewed ellipse in the raw data.

The directions shown in red and blue are to be understood as follows: The first variable x "loads" `PC1` with a weight of -0.704 and `PC2` with -0.710 (which form the first right singular vector, $\boldsymbol{v}_1$). It is shown as the red arrow in the biplot, but with some scaling so that its length corresponds to the weight. The second variable y loads `PC1` and `PC2` at -0.710 and 0.704 (the second right singular vector, $\boldsymbol{v}_2$), shown as the blue arrow, again with some scaling. Since both principal components load each variable with similar weights, the lengths of the red and blue arrows are similar.

13.3 Theoretical Connections: Inverses and Subspaces

13.3.1 The Pseudo-Inverse

We talked about the left and right inverses in §5.3.3 (page 109). Eqn (5.1), for instance, shows how we define the left inverse of a "tall," full-rank matrix. The SVD method gives us another way to define an inverse for *any* matrix, called the pseudo-inverse, $\boldsymbol{A}^+$.

Let's first define what we are looking for.

Pseudo-Inverse

Definition: A matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ has an associated pseudo-inverse $\boldsymbol{A}^+$ if the following four criteria are met:

1. $\boldsymbol{A}\boldsymbol{A}^+\boldsymbol{A} = \boldsymbol{A}$: Note that $\boldsymbol{A}\boldsymbol{A}^+$ does not have to be $\boldsymbol{I}$.
2. $\boldsymbol{A}^+\boldsymbol{A}\boldsymbol{A}^+ = \boldsymbol{A}^+$: The product $\boldsymbol{A}^+\boldsymbol{A}$ does not have to be $\boldsymbol{I}$ either.

Pesudo-Inverse vs. Other Inverses

Get the **Full Edition** of **LA4CS** with **Summaries**, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Available in the Full Version of the book.

3. $(\boldsymbol{A}^+\boldsymbol{A})^\mathsf{T} = \boldsymbol{A}^+\boldsymbol{A}$: Like the Gram matrix, $\boldsymbol{A}^+\boldsymbol{A}$ needs to be symmetric.
4. $(\boldsymbol{A}\boldsymbol{A}^+)^\mathsf{T} = \boldsymbol{A}\boldsymbol{A}^+$: The other product, $\boldsymbol{A}\boldsymbol{A}^+$, should be symmetric too.

With the SVD of $\boldsymbol{A}$, we can come up with an $\boldsymbol{A}^+$ that satisfies the four criteria:

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\mathsf{T} \implies \boldsymbol{A}^+ = \boldsymbol{V}\boldsymbol{\Sigma}^+\boldsymbol{U}^\mathsf{T} \tag{13.6}$$

where $\boldsymbol{\Sigma}^+$ is a diagonal matrix containing the reciprocals of σ_i when $\sigma_i \neq 0$ and zero when $\sigma_i = 0$. In practice, since floating-point zero comparison is always troublesome in computing, we will use a small lower bound threshold for σ_i.

Note that $\boldsymbol{\Sigma}^+$ has the same size as $\boldsymbol{A}^\mathsf{T}$, while $\boldsymbol{\Sigma}$ has the same size as $\boldsymbol{A}$. In other words, for $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, $\boldsymbol{\Sigma}^+ \in \mathbb{R}^{n\times m}$.

At this stage in our linear algebra learning, it should be trivial for us to verify that this definition of $\boldsymbol{A}^+$ does satisfy the four criteria. We only need to note that $\boldsymbol{U}$ and $\boldsymbol{V}$ are square, full-rank, orthogonal matrices, with the associated properties $\boldsymbol{U}\boldsymbol{U}^\mathsf{T} = \boldsymbol{U}^\mathsf{T}\boldsymbol{U} = \boldsymbol{I}_m$ and $\boldsymbol{V}\boldsymbol{V}^\mathsf{T} = \boldsymbol{V}^\mathsf{T}\boldsymbol{V} = \boldsymbol{I}_n$. Remember that $\boldsymbol{U}$ and $\boldsymbol{V}$ are, in fact, the complete bases for $\mathbb{R}^m$ and $\mathbb{R}^n$, respectively.

It is possible to rewrite the pseudo-inverse using the "thin" SVD, where we have the hatted matrices. The shapes of these matrices are specified in Eqn (13.2) and shown again pictorially in Figure 13.6. This shows the typical case in data science, where we have a full-column-rank "tall" matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}, m > n, \text{rank}(\boldsymbol{A}) = n$. All the columns in $\boldsymbol{A}$ are linearly independent.

$\boldsymbol{U}$ contains the left singular vectors, which form a complete basis for the output space $\mathbb{R}^m$. The first n of these vectors are a basis for the column space $\mathcal{C}(\boldsymbol{A})$. The remaining $m - n$ vectors are the basis for the left null space $\mathcal{N}(\boldsymbol{A}^\mathsf{T})$. We have n singular values, in descending order, along the main diagonal of $\boldsymbol{\Sigma}$. Since $\boldsymbol{\Sigma}$ has m rows (the same

shape as $\boldsymbol{A}$), we have $m - n$ zero rows. $\boldsymbol{V} \in \mathbb{R}^{n \times n}$ holds a full basis for $\mathbb{R}^n$. There are no other dimensions left anywhere to account for.

Figure 13.7 shows the most general case of a rank-deficient matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}, \text{rank}(\boldsymbol{A}) = r < \min(m, n)$. The left and right singular matrices ($\boldsymbol{U}$ and $\boldsymbol{V}$) are both square matrices forming full bases for $\mathbb{R}^m$ and $\mathbb{R}^n$, respectively. The first r columns of both contain the bases for the column space and the row space of $\boldsymbol{A}$, namely $\mathcal{C}(\boldsymbol{A})$ and $\mathcal{C}(\boldsymbol{A}^\mathsf{T})$.

The rest of the columns are the bases for the null spaces: The last $m - r$ columns in $\boldsymbol{U}$ span the left null space $\mathcal{N}(\boldsymbol{A}^\mathsf{T})$, and the last $n - r$ columns of $\boldsymbol{V}$ span the null space $\mathcal{N}(\boldsymbol{A})$. As for the singular values in $\boldsymbol{\Sigma}$, only the first r of them are non-zero; the rest are all zeros.

As we can see in Figure 13.7, the columns (and rows) to the right of (and below) the thin lines in $\boldsymbol{U}, \boldsymbol{\Sigma}$, and $\boldsymbol{V}^\mathsf{T}$ are there to account for the null spaces. They do not affect the product at all because the zeros in $\boldsymbol{\Sigma}$ will cancel them out anyway. We can, therefore, write an economical SVD for $\boldsymbol{A}$ by keeping only the columns (and rows) to the left of (and above) the thin lines in $\boldsymbol{U}, \boldsymbol{\Sigma}$, and $\boldsymbol{V}^\mathsf{T}$. We call these smaller matrices $\hat{\boldsymbol{U}}, \hat{\boldsymbol{\Sigma}}$, and $\hat{\boldsymbol{V}}^\mathsf{T}$. We can then write:

$$\boldsymbol{A} = \hat{\boldsymbol{U}}\hat{\boldsymbol{\Sigma}}\hat{\boldsymbol{V}}^\mathsf{T} \quad \text{where } \hat{\boldsymbol{U}} \in \mathbb{R}^{m \times r},\ \hat{\boldsymbol{\Sigma}} \in \mathbb{R}^{r \times r},\ \hat{\boldsymbol{V}} \in \mathbb{R}^{n \times r}$$

Since we defined $\boldsymbol{\Sigma}^+$ appearing in Eqn (13.6) as having non-zero values only in the leading $r \times r$ submatrix, we can see that only the first r columns of $\boldsymbol{U}$ and $\boldsymbol{V}$ have any bearing on $\boldsymbol{A}^+$. Therefore, we can write the pseudo-inverse as:

$$\boldsymbol{A}^+ = \hat{\boldsymbol{V}}\hat{\boldsymbol{\Sigma}}^{-1}\hat{\boldsymbol{U}}^\mathsf{T} \tag{13.7}$$

Note that $\hat{\boldsymbol{\Sigma}}^+$ is indeed equivalent to $\hat{\boldsymbol{\Sigma}}^{-1}$ because $\hat{\boldsymbol{\Sigma}}$ is a full-rank, diagonal matrix, and its true inverse is simply another matrix with the reciprocals of the diagonal elements along its diagonal:

$$\hat{\boldsymbol{\Sigma}}^{-1} = \begin{bmatrix} \frac{1}{\sigma_1} & 0 & \cdots & 0 \\ 0 & \frac{1}{\sigma_2} & \cdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{\sigma_r} \end{bmatrix} \in \mathbb{R}^{r \times r} = \begin{bmatrix} \frac{1}{\sigma_i} \end{bmatrix}$$

Since we are working with the "economical" version $\hat{\boldsymbol{\Sigma}}$, there are no divide-by-zero errors to worry about when computing its inverse: all the singular values σ_i inside it are strictly non-zero.

Note that in Eqn (13.6), the full matrices of singular vectors on the right ($\boldsymbol{U}$ and $\boldsymbol{V}$) are all square and invertible. We are justified in taking their inverses (with $\boldsymbol{U^{-1}} = \boldsymbol{U^{\mathsf{T}}}$ and $\boldsymbol{V^{-1}} = \boldsymbol{V^{\mathsf{T}}}$). But their overall product ($\boldsymbol{A^{+}}$) is not a square matrix, and we are not justified in calling it a true inverse, which is exactly why we call it a pseudo-inverse.

Because $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{A^{+}} \in \mathbb{R}^{n\times m}$, both $\boldsymbol{AA^{+}}$ and $\boldsymbol{A^{+}A}$ are valid matrix multiplications. The pseudo-inverse naturally reduces to the left-inverse for full-column-rank matrices, and the right-inverse for full-row-rank matrices. And of course, for a full-rank square matrix, it is the plain old inverse. It is indeed worth our time to verify these statements, which is done in a box in this chapter.

The first criterion in the definition of $\boldsymbol{A^{+}}$ says $\boldsymbol{AA^{+}A} = \boldsymbol{A}$, which means $\boldsymbol{AA^{+}Ax} = \boldsymbol{Ax}$ or $\boldsymbol{AA^{+}b} = \boldsymbol{b}$ in our favorite equation $\boldsymbol{Ax} = \boldsymbol{b}$. Let's consolidate:

$$\begin{aligned}\boldsymbol{AA^{+}Ax} &= \boldsymbol{Ax}\\ \boldsymbol{AA^{+}}(\boldsymbol{Ax}) &= \boldsymbol{Ax}\\ \boldsymbol{AA^{+}b} &= \boldsymbol{b}\end{aligned}$$

What is this telling us? Let's say $\boldsymbol{A^{+}b} = \boldsymbol{y}$. Then substituting that into the previous equation yields $\boldsymbol{Ay} = \boldsymbol{b}$.

Earlier, in §R.4, page 242, we argued that this, in conjunction with $\boldsymbol{Ax} = \boldsymbol{b}$, implied that $\boldsymbol{x} = \boldsymbol{y}$ because of the uniqueness of linear combinations. Here is another argument: Since we have Eqn (13.7), we can see that $\boldsymbol{A^{+}b}$ is in the row space of $\boldsymbol{A}$ because $\boldsymbol{A^{+}b} = \hat{\boldsymbol{V}}\hat{\boldsymbol{\Sigma}}^{-1}\hat{\boldsymbol{U}}^{\mathsf{T}}\boldsymbol{b}$ is a linear combination of the columns of $\hat{\boldsymbol{V}}$.

Since $\hat{\boldsymbol{V}}$ is a basis for $\mathcal{C}(\boldsymbol{A^{\mathsf{T}}})$, $\boldsymbol{A^{+}b} \in \mathcal{C}(\boldsymbol{A^{\mathsf{T}}})$. In other words, for a non-zero $\boldsymbol{b}$, $\boldsymbol{y}$ is in $\mathcal{C}(\boldsymbol{A^{\mathsf{T}}})$. This is to say, for a given $\boldsymbol{b} \in \mathcal{C}(\boldsymbol{A})$, we have $\boldsymbol{A^{+}b} = \boldsymbol{y}$ with $\boldsymbol{y} \in \mathcal{C}(\boldsymbol{A^{\mathsf{T}}})$. Thus, we have the inverse mapping $\boldsymbol{A^{+}} : \mathcal{C}(\boldsymbol{A}) \to \mathcal{C}(\boldsymbol{A^{\mathsf{T}}})$, leading directly to our closing discussion.

13.3.2 Fundamental Spaces and SVD

Singular Value Decomposition and the subsequent definition of the pseudo-inverse complete the beautiful symmetry of the fundamental subspaces we started in Chapter 9. The matrix $\boldsymbol{A}$ represents a linear mapping from $\mathbb{R}^n$ to $\mathbb{R}^m$: It takes vectors $\boldsymbol{x} \in \mathbb{R}^n$ and maps them to vectors $\boldsymbol{b} \in \mathbb{R}^m$, denoted as $\boldsymbol{A} : \mathbb{R}^n \to \mathbb{R}^m$. Restricted to the row

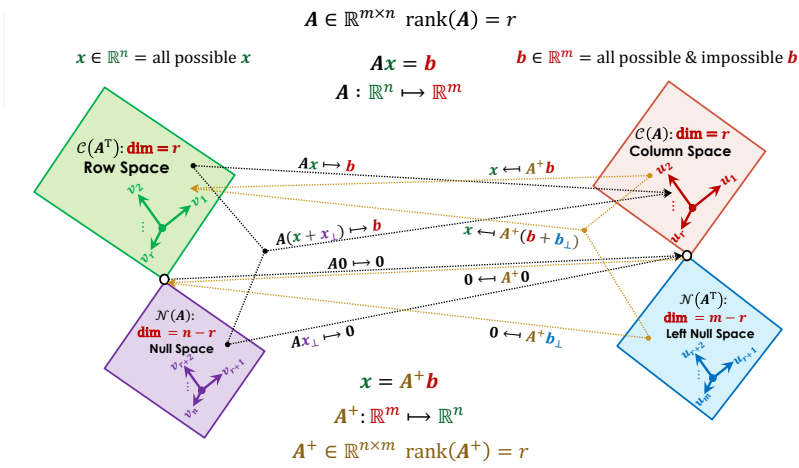


Fig. 13.9 The elegant symmetry of the four fundamental subspaces, completed by the pseudo-inverse A^+.

space and column space of A (from $\mathcal{C}(A^\mathsf{T})$ to $\mathcal{C}(A)$), this mapping is strictly one-to-one and onto.

The pseudo-inverse A^+ performs the exact reverse mapping: $A^+ : \mathbb{R}^m \to \mathbb{R}^n$. As Figure 13.9 beautifully illustrates, this inverse operation occurs in a completely symmetric fashion. It acts as a one-to-one mapping from $\mathcal{C}(A)$ back to $\mathcal{C}(A^\mathsf{T})$, while elegantly mapping the respective null spaces to their zero counterparts. Instead of overcomplicating the geometry with text, it is wiser to let the diagram speak its own thousand words. Let us pause to appreciate its exquisite completeness.

If we can allow ourselves to be mesmerized by the deep geometric symmetry of this framework and the clarity of the mathematical path that brought us here, we have surely earned the right to call ourselves mathematicians or computer scientists. This unified picture is the perfect, natural milestone to bring this journey through linear algebra to its triumphant conclusion.

Get the **Full Edition** of **LA4CS** with
Summaries, **Exercises** and **Solutions**
Only $7.95. Scan, Click or Tap to buy.

Summing Up. . .

What we want is to see the child in pursuit of knowledge,
and not knowledge in pursuit of the child.
—George Bernard Shaw



We have now come to the end of our journey through the wonderful world of linear algebra. Although we have covered a tremendous amount of material, we have still only scratched the surface of this vast and profound branch of mathematics. Linear algebra is a deeply established field, and depending on the domain in which it is applied, it can take on entirely different forms. Studied as a graduate-level pure mathematics course, it may focus heavily on abstract algebraic structures and generalized fields. When applied to quantum mechanics, it transforms into a completely different beast—one featuring alternative notations, infinite-dimensional spaces, and vectors that are actually functions—where the focus shifts to the physical interpretation of eigenvalues and eigenvectors.

Even within our own field of computer science, diving into the literature will expose you to a variety of notations and perspectives,

heavily influenced by the authors' backgrounds and their specific applications. Despite this vastness and diversity, my hope is that we have covered enough ground together to give you the tools to dissect the intricacies of any of these applications or styles of discourse. To be sure, future literature may seem unfamiliar, and the applications may look daunting. But remember: their difficulty is merely a matter of deciphering new notation and jargon, not a fundamental lack of the underlying mathematics. You already have the foundational concepts.

More importantly, I hope this book has kindled your interest in this particularly beautiful field of mathematics. I hope you look forward to exploring it further, continuing to learn, and applying these insights to enrich your professional life as a computer scientist.

Glossary

The symbols, terms, and abbreviations most commonly used in this book are listed and described below for easy reference.

For a more comprehensive reference, we can also consult the Online Glossary of Linear Algebra terms and definitions on our companion website.

Scalar: Scalars are written in lowercase letters, *e.g.*, s, similar to the notation for the elements of vectors and matrices.

Vectors: Bold lowercase letters represent vectors, *e.g.*, $\boldsymbol{x}$, with elements x_i. Vectors are always column matrices, and are written with square brackets when needed. Note, however, that **SageMath** writes vectors as $(x_1, x_2, \cdots, x_m)$ (using parentheses with commas between the elements), and we may use that notation as well, albeit rarely.

Matrices: Bold capital letters represent matrices, *e.g.*, $\boldsymbol{A}$. For the elements of $\boldsymbol{A}$, the corresponding lowercase letter, a_{ij} is used. When explicitly writing out the elements of the matrix, square brackets are used.

Elements of Matrices: The elements of matrices, being scalars, are written using lowercase letters. When we write $\boldsymbol{A} = \big[a_{ij}\big]$, we mean that $\boldsymbol{A}$ is a matrix with a general element a_{ij} in the ith row and j^{th} column. We also write $\boldsymbol{A} = [\boldsymbol{a}_j]$ to indicate that $\boldsymbol{A}$ is composed of column vectors, with $\boldsymbol{a}_i$ as the jth column.

Although used only a couple of times in the book, we also write $\boldsymbol{A} = [\boldsymbol{a}_i^{\mathsf{T}}]$ to denote the matrix $\boldsymbol{A}$ consisting of row vectors, with $\boldsymbol{a}_i^{\mathsf{T}}$ as the ith row.

Identity and Zero: The identity matrix is represented by $\boldsymbol{I}$ (or $\boldsymbol{I}_n$ to specify an $n \times n$ size). The zero vector is denoted by a bold $\mathbf{0}$ to distinguish it from the scalar zero.

Determinant: Our favorite symbol for determinant is a vertical line: The determinant of $\boldsymbol{A}$ is $|\boldsymbol{A}|$.

Transpose: The symbol T represents transpose. $\boldsymbol{A} \in \mathbb{R}^{m \times n} \implies \boldsymbol{A}^{\mathsf{T}} \in \mathbb{R}^{n \times m}$.

Hermitian Transpose: The complex conjugate transpose of $\boldsymbol{A}$ is indicated as $\boldsymbol{A}^{\dagger}$, although it is used very sparingly in this book.

Inverses: The standard inverse of a matrix $\boldsymbol{A}$ is written as $\boldsymbol{A}^{-1}$ (or $\boldsymbol{A}^{-1}$). The pseudo-inverse is written as $\boldsymbol{A}^{+}$. Left and right inverses, when explicitly distinguished, are written as $\boldsymbol{A}_{\textbf{Left}}^{-1}$ and $\boldsymbol{A}_{\textbf{Right}}^{-1}$ respectively.

Rank and Trace: The rank of a matrix $\boldsymbol{A}$ is written as $\text{rank}(\boldsymbol{A})$. The trace, which is the sum of the diagonal elements of a square matrix, is written as $\text{trace}(\boldsymbol{A})$ or $\text{Tr}(\boldsymbol{A})$.

Fields: Our matrices and vectors are almost always over the field of reals, which is represented as $\mathbb{R}$. We will write $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{x} \in \mathbb{R}^{m}$. Although we may not use it, it is possible to have matrices and vectors over other fields and rings, such as integers ($\mathbb{Z}$), rationals ($\mathbb{Q}$) or complex ($\mathbb{C}$).

Spaces: We use calligraphic symbols such as $\mathcal{S}$ for spaces. In particular, the column space of $\boldsymbol{A}$ is $\mathcal{C}(\boldsymbol{A})$, row space $\mathcal{C}(\boldsymbol{A}^{\mathsf{T}})$, null space $\mathcal{N}(\boldsymbol{A})$ and its left null space is $\mathcal{N}(\boldsymbol{A}^{\mathsf{T}})$.

Inner Product and Orthogonality: The inner (or dot) product of two vectors is typically written using matrix multiplication as $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y}$, or occasionally with a dot as $\boldsymbol{x} \cdot \boldsymbol{y}$. When two vectors are orthogonal, we write $\boldsymbol{x} \perp \boldsymbol{y}$.

Eigenvalues and Singular Values: Eigenvalues are represented by the lowercase Greek letter lambda (λ). Singular values are

represented by the lowercase Greek letter sigma (σ), and the diagonal matrix of singular values is written as $\boldsymbol{\Sigma}$.

Math Symbols: In definitions and equations, we will use common mathematical symbols such as:

- $\forall$: For all or for any. $\forall\, \boldsymbol{x}$ means for any vector $\boldsymbol{x}$.
- $\in$: Is a member of the set. $s \in \mathbb{R}$ says that the scalar s is a member of the set of reals.
- $\implies$: Implies. $i, j \in \mathbb{Z} \implies \frac{i}{j} \in \mathbb{Q}$.
- $\subset, \subseteq$: Subset of. $\mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R} \subseteq \mathbb{C}$.

Nonstandard Notation: We use the symbol $\bowtie$ in a manner not seen elsewhere: We write $\boldsymbol{A} = [\boldsymbol{I} \bowtie \boldsymbol{F}]$, for instance, to indicate $\boldsymbol{A}$ is a matrix composed of the columns of the identity matrix and the matrix $\boldsymbol{F}$, but the columns are not necessarily in the order in which they appear in the constituent matrices. They may be "shuffled."

Norm: Double vertical lines indicate the norm (usually the Euclidean norm, usually of a vector). $\|\boldsymbol{x}\|$ is the norm of the vector $\boldsymbol{x}$.

Credits

Resources

This book is written explicitly as a textbook to support a corresponding course at Singapore Management University. For each weekly session of our course, which corresponds to a chapter in the book, we will have a curated list of videos either as preparatory, or for review and problem solving.

Linear Algebra is a well-established branch of mathematics, and we have an abundance of online resources to draw from. Here are some more resources:

MIT Open Courseware 18.06SC: Available on YouTube for free, these excellent lectures by Gilbert Strang and associated recitations are a great resource for our course. Many of the suggested readings (especially the problem-solving kind) in this book are drawn from this resource.

3Blue1Brown: Another excellent YouTube channel on mathematics, provided for free by Grant Sanderson, is a must-watch for students of applied mathematics and computer science. We have listed several videos from its playlist Essence of Linear Algebra in our chapters.

Gilbert Strang *"Linear Algebra and its Applications"*: This is the book version of the lectures in 18.06SC, and may be useful for sample problems and as lecture notes. However, lecture notes, problem sets and lecture transcripts from the book are all available online at MIT Open Courseware.

Philip Klein *"Coding the Matrix: Linear Algebra through Computer Science Applications"*: A very comprehensive and well-known work, this book teaches Linear Algebra from a computer science perspective. Some of the labs in our course are inspired by or based on the topics in this book, which has an associated website with a lot of information.

Books

In addition to this textbook, *"Linear Algebra for Computer Science"*, here are some other books that we can freely download and learn from:

Jim Hefferson *"Linear Algebra"*: This book uses **SageMath** and has tutorials and labs that can be downloaded for more practice from the author's website.

Robert Beezer *"A First Course in Linear Algebra"*: Another downloadable book with associated web resources on **SageMath**. In particular, it has an on-line tutorial that can be used as a reference for **SageMath**.

Stephen Boyd *"Introduction to Applied Linear Algebra"*: This well-known book takes a pragmatic approach to teaching Linear Algebra. Commonly referred to by the acronym (*VMLS*) of its subtitle (*"Vectors, Matrices, and Least Squares"*), this book is also freely downloadable and recommended for computer science students.

Copyright

A legal disclaimer: The websites and resources listed above are governed by their own copyright and other policies. By listing and referring to them, we do not imply any affiliation with or endorsement from them or their authors.

ABOUT LA4CS

An enjoyable and readable textbook on mathematics, **LA4CS** introduces the essential concepts and practice of Linear Algebra to the undergraduate student of computer science.

The focus of this book is on the elegance and beauty of the numerical techniques and algorithms originating from Linear Algebra. As a practical handbook for computer and data scientists, **LA4CS** restricts itself mostly to real fields and tractable discourses, rather than deep and theoretical mathematics.

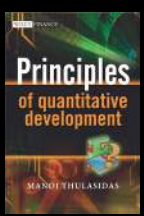
Its companion website, **LA4CS.com**, features extra information, downloadable material, and links to the video lectures based on this textbook.

ABOUT THE AUTH0R



Manoj Thulasidas, is an **Associate Professor of Computer Science** (Education) who teaches Data Analytics and Linear Algebra to undergraduate students of computer science and information systems at **Singapore Management University**.





His other works include ***The Unreal Universe***, an inquiry into the philosophical underpinnings of physics (from his career as a **physicist** at **CERN** in Geneva) and ***Principles of Quantitative Development***, a practitioner's guide to the lucrative profession of **quantitative finance** (from his experiences as a **quant** in Singapore).

Buy it

Scan or Tap

ASIAN BOOKS
ISBN 978-981-18-2045-8
9 789811 820458